



coreAssembler

User Guide

coreAssembler — Product Code: 3533-0

Copyright Notice and Proprietary Information

© 2024 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>
All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.
www.synopsys.com

Contents

Revision History.....11

Preface.....21

 About This User Guide.....21

 Related Documents.....21

 Manual Overview.....22

 Typographical and Symbol Conventions.....22

 Synopsys Common Licensing (SCL).....23

 Synopsys Statement on Inclusivity and Diversity.....23

 Customer Support.....24

Chapter 1 Introduction.....25

 1.1. Design Reuse with Synopsys coreTools.....26

 1.1.1. coreAssembler Process Activities.....26

 1.2. Key Terms and Concepts.....28

 1.3. coreAssembler Features.....30

Chapter 2 The coreAssembler Environment.....31

 2.1. coreKits and Workspaces.....32

 2.2. Activity-Based Design Flow.....33

 2.3. User Interfaces.....37

 2.3.1. Command Line Mode.....37

 2.3.2. Batch Mode.....37

 2.3.3. GUI Mode.....37

 2.4. Online Help Tools.....40

 2.5. coreAssembler Spreadsheets.....41

 2.6. Basic Operations.....42

 2.6.1. Invoking coreAssembler.....42

 2.6.2. Installing coreKits.....45

 2.6.3. Component Update Checking.....45

 2.6.4. Workspace Operations.....48

 2.6.5. Quitting coreAssembler.....51

2.6.6. Working with the Activity List.....	51
2.6.7. Generating Reports and Optional Views.....	51
Chapter 3 Creating the RTL View of a Subsystem.....	53
3.1. Specify Subsystem.....	54
3.1.1. Add Components Tab.....	54
3.1.2. Configure Components Tab.....	64
3.1.3. Configure Interfaces Tab.....	64
3.1.4. Memory Map Tab.....	64
3.1.5. Reporting and Header File Generation.....	66
3.1.6. Completing the Specify Subsystem Definition.....	68
3.1.7. Reporting and Header File Generation.....	68
3.2. Complete Connections.....	72
3.2.1. Manually Connect Schematic.....	73
3.2.2. Hierarchical Connection Support.....	74
3.2.3. Complete Connections Accelerator Keys.....	74
3.3. Generate Subsystem RTL.....	75
3.3.1. User-Defined Netlisting.....	78
3.3.2. User-Defined Packages for Components.....	79
Chapter 4 Creating the Gate-Level Netlist for a Subsystem.....	81
4.1. Specifying Target Technology (TechSetup).....	82
4.1.1. Logical Library Setup.....	82
4.1.2. Physical Library Setup.....	83
4.1.3. Set Tool Installation Root.....	83
4.2. Technology Cell Binding.....	85
4.2.1. Tech Cell Mapping Use Case.....	86
4.3. Initializing Subsystem Constraints.....	89
4.3.1. Manual Clock Definition.....	89
4.3.2. Propagation Control.....	89
4.3.3. Constraint Initialization Flow.....	91
4.3.4. Constraint Propagation.....	91
4.4. Specifying Clocks.....	93
4.5. Specifying Operating Conditions and Wireloads.....	95
4.6. Specifying Power Intent.....	97
4.7. Specifying Timing Exceptions.....	99
4.8. Specifying Port Constraints.....	100

4.9. Specifying Synthesis Methodology.....	103
4.9.1. General Synthesis Methodology Attributes.....	103
4.9.2. Advanced Synthesis Methodology Attributes.....	104
4.10. Specifying Test Methodology.....	107
4.10.1. General Test Methodology Attributes.....	107
4.10.2. Scan Insertion.....	107
4.10.3. Auto Fix.....	110
4.10.4. Test Wrappers.....	111
4.10.5. Wrapper Integration.....	112
4.10.6. Built-in Self Test.....	113
4.10.7. Testpoint Insertion.....	115
4.11. Synthesizing Subsystems.....	117
Chapter 5 Performing ASIC Synthesis.....	119
5.1. Performing ASIC Synthesis.....	120
5.1.1. Synthesis and Integration Methodologies.....	120
5.1.2. Choosing Synthesis Activities in the GUI.....	123
5.1.3. Specifying ASIC Synthesis Execution Options.....	149
5.1.4. Viewing Licenses for Implementation Tools.....	150
5.1.5. Selecting Synthesis Reports.....	151
5.1.6. Launching an ASIC Synthesis Run.....	154
5.1.7. Checking Synthesis Results.....	161
5.1.8. Refining Your Synthesis Intent.....	162
5.2. Generating Test Vectors.....	164
Chapter 6 Verifying Subsystems and Components.....	171
6.1. Overview.....	172
6.1.1. Individual Component Verification.....	172
6.1.2. Testbench Workspace Creation.....	172
6.1.3. Features.....	173
6.1.4. Limitations and Notes.....	173
6.2. Verifying an Individual Component	174
6.3. Verifying a Subsystem	175
6.4. Static Checking Activity.....	176
6.4.1. Configuration.....	176
6.4.2. Execution Options.....	177
6.4.3. Goals.....	178

6.4.4. Clock Domains.....	179
6.5. RTL Static Checking.....	180
6.5.1. Setup.....	184
6.5.2. Execution Options.....	187
6.6. Formal Verification Activity.....	189
6.6.1. Executing the Formal Verification Activity.....	189
6.6.2. UPF Support in Formality.....	192
6.6.3. Checking Formal Verification Results.....	193
6.7. Static Timing Analysis Activity.....	194
6.7.1. Checking Static Timing Analysis Results.....	201
6.8. Constraint Analysis Activity.....	202
6.9. Explore RTL Architecture Activity.....	204
6.10. Timing Constraints Manager.....	209
6.10.1. Constraints Verification.....	209
6.10.2. Constraints Verification (CV) Activity.....	212
6.10.3. Running the TCM Flow.....	213
6.10.4. Constraints Verification Directory Tree Structure.....	213
6.11. Create Testbench Workspace Activity (Subsystem Workspace).....	215
6.11.1. Testbench Strategy and Language.....	216
6.11.2. Test Coverage (UVM RAL Method).....	216
6.11.3. UVM RAL Subsystem Ping Test.....	217
6.11.4. Testbench Workspace Creation Methodology Overview.....	218
6.11.5. Create Testbench GUI Functionality.....	218
6.12. Simulate Subsystem Testbench.....	222
6.12.1. Simulator Setup.....	222
6.12.2. Testbench Definition.....	222
6.12.3. Execution Options.....	222
6.12.4. Generating the Simulation run.scr Script.....	223
6.13. Integrating the Subsystem into Your Design.....	224
6.13.1. Connecting to Surrounding Logic.....	224
6.13.2. Verifying Functionality.....	224
6.13.3. Verifying Timing.....	224
6.14. Batch Commands.....	225
6.15. Licensing and Keys.....	226
Chapter 7 Exporting a Subsystem for Reuse.....	227

7.1. Export Subsystem.....	228
7.1.1. Create Subsystem Template.....	228
7.1.2. Export IP-XACT Compliant XML.....	228
7.1.3. Build Subsystem coreKit	228
Chapter 8 Unpackaged Component Import.....	231
8.1. Locating a Design for Import.....	232
8.1.1. Importing the Component Data.....	232
8.1.2. Importing Soft IP.....	232
8.1.3. Importing Hard IP.....	232
8.1.4. Importing IP-XACT Components and Designs.....	233
8.1.5. Importing Synthesis Intent.....	233
8.2. Interfacing your Imported Component to Other Subsystem Components.....	234
8.2.1. Attaching an Available Interface.....	234
8.2.2. Port/Parameter Association.....	234
8.2.3. Loading of Undefined Interfaces.....	235
8.2.4. Adding Attached Interfaces.....	235
8.3. Recommendations and Limitations.....	236
8.3.1. One-time Use Model.....	236
8.3.2. Limited Configuration Capability.....	236
8.3.3. Connecting an Imported Core.....	236
Chapter 9 Advanced User Modes.....	237
9.1. Command Line Interface.....	238
9.1.1. Accessing the Command Line Interface.....	238
9.1.2. Setting a Design Configuration Parameter.....	238
9.1.3. Setting an Activity Parameter.....	238
9.1.4. Reporting Activity Parameters.....	239
9.1.5. Setting the Value of a Synthesis Strategy Parameter.....	239
9.1.6. Abbreviating Commands.....	239
9.1.7. Re-Executing the coreTool Design Flow.....	240
9.2. Batch Mode.....	241
9.2.1. Batch Command File Examples.....	241
9.2.2. Automatically Generating a Batch File.....	241
9.2.3. Batch File Options.....	242
9.2.4. Executing a Batch Command File.....	242
9.2.5. Batch Mode coreKit Installation.....	242

9.3. Editing Tool Setup and User Preferences.....	244
9.3.1. Setup Variables.....	244
9.3.2. User Preferences.....	245
9.4. Working with Verification IP.....	259
9.5. Working with Hard Macros.....	260
9.6. Dynamically Adding Files to the HDL File List.....	261
9.7. Signal Naming Control in Generated Netlist.....	262
9.8. Replace Component Mapping File Format.....	263
9.8.1. Memory Map Propagation.....	264
9.9. Dumping Debug Information.....	265
9.10. Documentation Generation.....	266
9.11. Table Driven Data Import/Export.....	267
9.11.1. General Table Formatting.....	267
9.11.2. Table Commands.....	268
Chapter 10 Customizing a Technology Library.....	275
10.1. Library Customization Attributes.....	277
10.2. Cell Customization Attributes.....	278
Chapter 11 Using IP-XACT Components and Designs.....	279
11.1. IP-XACT Overview.....	280
11.2. IP-XACT Schema Versions.....	281
11.3. List of Features.....	282
11.3.1. IP-XACT Packaging Features.....	282
11.3.2. coreAssembler.....	283
11.4. User Interface.....	284
11.4.1. Creating a IP-XACT Library.....	284
11.4.2. Adding an IP-XACT Library to Your Search Path (coreAssembler).....	284
11.4.3. Importing an IP-XACT Component (coreAssembler).....	284
11.4.4. Importing a IP-XACT Design (coreAssembler).....	285
11.4.5. Exporting an IP-XACT Component (coreAssembler, coreConsultant).....	285
11.4.6. Exporting a IP-XACT Design (coreAssembler only).....	285
11.4.7. Legacy HDL Translation to IP-XACT.....	286
11.4.8. Using IP-XACT Generators and Generator Chains.....	286
11.4.9. The TGI (Tight Generator Interface).....	286
11.5. Batch Mode Interface.....	288
11.5.1. Importing an IP-XACT Component.....	288

- 11.5.2. Importing IP-XACT Designs or Generator Chains.....288
- 11.5.3. Exporting an IP-XACT Component.....288
- 11.5.4. Exporting an IP-XACT Design (coreAssembler only).....289
- 11.5.5. IP-XACT Generators.....290
- 11.6. Licensing and Keys.....291
- 11.7. IP-XACT XML Implementation.....292
 - 11.7.1. Synopsys IP-XACT Constructs.....292
 - 11.7.2. Unsupported IP-XACT Constructs in coreTools.....292
 - 11.7.3. IP-XACT Examples.....292
- 11.8. Glossary of IP-XACT Terminology.....293

Revision History

The following table provides a summary of changes made to this User Guide.



Note

Links and references to section, table, figure, and page numbers in this table are only assured to be valid for the version in which the change is made.

Version	Date	Description
W-2024.09-1	October 2024	Updated: ■ Table 6-19 on page 204 with "Utilization Ratio" activity parameter details.
W-2024.09	September 2024	Updated: □ Table 4-3 on page 85 with "Invert connection", "Pins Connected to High or Low" and "CTL Model File" fields. □ Table 4-4 on page 87 with a new use case. □ Figure 6-2 on page 182 □ Table 4-4 on page 87 with additional example. □ Table 6-5 on page 184 with algorithm for calculating Default Engineering Council File Version. □ Table 9-2 on page 245 with Include Doc attributes GUI details. Added: □ Figure 4-1 on page 86
V-2024.06-2	June 2024	Updated version
V-2024.06	June 2024	Updated: □ ScanCompressionUsage and EnableTestMaxManagerFlow strategy parameter descriptions in Table 5-47 on page 128 □ Table 4-20 on page 80 Removed: ■ Removed the remote option for Runstyle and LintGoals activity parameter in Table 6-69 on page 169

Version	Date	Description
		■ Added: □ “Timing Constraints Manager” on page 209 □ SeqCompressionInputs and SeqCompressionOutputs strategy parameters in Table 5-12 on page 139
V-2024.03-1	May 2024	Updated date and version
V-2024.03	March 2024	■ Changed instances of DesignWare Cores to Synopsys IP to align with corporate naming conventions. ■ Added EnableDistributedCodec Strategy parameter in Table 5-47 on page 128
V-2023.12-1	January 2024	Updated date and version
V-2023.12	December 2023	■ Added: □ include_defines_in_fileset and include_undefined_macros_in_ipxact preference parameters in Table 9-89 on page 223 □ EnableBottomUpFlow activity parameter in Table 6-69 on page 171 □ ReportAnalysisCoverageCheckType activity parameter in Table 6-78 on page 184 □ EnableTestMaxManagerFlow and EnableTPI strategy parameters in Table 5-47 on page 128 □ “Performing ASIC Synthesis using QIK Reference Methodology with TestMAX Manager” on page 149 □ GenerateRMReports and DisableCongestionReport strategy parameters in Table 5-45 on page 124 and Table 5-57 on page 136 □ ReportQOR and ReportQORHistogram strategy parameters in Table 5-57 on page 136 ■ Removed: □ “Performing FPGA Synthesis” section in “Performing ASIC Synthesis” on page 107 □ Synplify Library Setup (FPGA) section in “Creating the Gate-Level Netlist for a Subsystem” on page 75 □ FPGA keywords applicable in the document ■ Updated: □ “Synthesizing Subsystems” on page 106 to remove content related to “Performing FPGA Synthesis” □ “Performing ASIC Synthesis” on page 107 □ Table 2-4 on page 32 to remove the “Perform FPGA Synthesis” activity

Version	Date	Description
		□ “RTL Static Checking” on page 167 with bottom-up flow and RTLStaticCheckingAsSubblock design attribute and description □ “Add New Component” on page 52 with prime profile groups
V-2023.09-1	September 2023	Updated version
V-2023.09	September 2023	Updated “Add New Component” on page 52 with prime profile application during component instantiation ■ Added: □ “Performing ASIC synthesis using QIK Reference Methodology for DesignWare Interface IP” on page 147 □ EnableQIK, FinalFCRMFlowStep, TechnologySideFileSettings, and TechnologyRMScripts parameters in Table 5-46 on page 125 □ QIK parameters in Table 5-50 on page 131, Table 5-51 on page 131, Table 5-52 on page 132, Table 5-53 on page 132, Table 5-54 on page 134
U-2023.06-1	June 2023	Updated version
U-2023.06	June 2023	■ Added: □ “RTL Static Checking” on page 167 to include VC SpyGlass support □ “Working with Fusion Compiler Strategies” on page 109 □ “Performing ASIC Synthesis using Design Compiler” on page 141, “Performing ASIC Synthesis using Fusion Compiler” on page 144, “Hierarchical (Bottom-Up) Synthesis using Design Compiler” on page 143, and “Hierarchical (Bottom-Up) Synthesis using Fusion Compiler” on page 147 □ retain_static_port_range tool preference option in Table 9-91 on page 221 ■ Updated: □ “Enabling Physical Synthesis” on page 110 with a note. □ “Launching an ASIC Synthesis Run” on page 141
U-2022.12-2	February 2023	■ Added: □ retain_multidim_ports GUI Preference in Table 9-91 on page 221 □ SetTechnologyAfterFloorplan strategy parameter in Table 5-46 on page 124
U-2022.12	December 2022	■ Added □ ExploreFlowRunStyle parameter in Table 6-87 on page 189 □ AlternateStrategiesRunStyle parameter in Table 6-74 on page 174 □ DPXRunStyle parameter in Table 6-75 on page 175 □ DCRM DFT Parameters in Table 5-41 on page 116

Version	Date	Description
		❑ FCRM DFT Parameters in Table 5-47 on page 127 ❑ UseSynplifyElite and UseSynplifyBase parameters in Table 5-65 on page 157 ❑ PropagateUpwardOnly parameter in Table 4-22 on page 80 ❑ FCRM power parameters in Table 5-49 on page 129 ■ Updated parameter type in table headings and column in all tables
T-2022.06-SP2	September 2022	■ Updated: ❑ “Edit Search Path” on page 55 ❑ CTLModels parameter description in the Fusion_Compiler_Reference_Methodology strategy in Table 5-46 on page 124 ■ Added: ❑ CTLModels DCRM strategy parameter in Table 5-38 on page 113 ❑ EnableHierarchicalSynthesis in Table 5-49 on page 129 ❑ NDM Library physical synthesis DCRM strategy parameters in Table 5-42 on page 117
T-2022.06-SP1	July 2022	■ Added: ❑ DPX Flow in Table 6-68 on page 158 ❑ DFTSynthesisOptimization DCRM strategy parameter in Table 5-38 on page 112
T-2022.06	June 2022	Added the CTLModels parameter in the Fusion_Compiler_Reference_Methodology strategy in Table 5-46 on page 124.
S-2021.09-SP4	April 2022	■ Added: ❑ OverrideSearchPath, OverrideLinkLibraryMapping, and OverrideDefaultOperatingCondition parameters for the Static Timing Analysis interface in Table 6-72 on page 164
S-2021.09-SP3	March 2022	■ Added: ❑ SelfGating parameter in Table 5-43 on page 120 and DcSelfGatingReport parameter in Table 5-54 on page 134 ❑ Hierarchical Synthesis parameters in Table 5-44 on page 122
S-2021.09-SP2	January 2022	■ Updated TetraMax to TestMax and LinkToTetraMax to LinkToATPG in the document ■ Updated OptimizationFlow parameter description in Table 5-38 on page 112

Version	Date	Description
		■ Added: □ CongestionOptimizationFactor parameter to the Design_Compiler_Reference_Methodology strategy in Table 5-42 on page 117 □ EnableECOFlow parameter to Table 6-71 on page 162 □ FixECOTimingArgs parameter to Table 6-71 on page 162 □ ReducedEffortOptimizationFlow parameter to Table 5-38 on page 112
S-2021.09-SP1	October 2021	■ Added EnablePortSideConstraint parameter, MinRoutingLayer parameter, and MaxRoutingLayer parameter to the Design_Compiler_Reference_Methodology strategy in Table 5-42 on page 117 ■ Added EnablePortSideConstraint parameter to Fusion_Compiler_Reference_Methodology strategy in Table 5-46 on page 124 ■ Added MinRoutingLayer parameter and MaxRoutingLayer parameter to the Fusion_Compiler_Reference_Methodology strategy in Table 5-46 on page 125 ■ Added GenerateIgnoredLayersReport parameter to Table 5-54 on page 134
S-2021.09	September 2021	■ Added the UtilizationRatio parameter to the Fusion_Compiler_Reference_Methodology strategy in Table 5-46 on page 124 and Design_Compiler_Reference_Methodology strategy in Table 5-42 on page 117 ■ Added the TechnologyNode parameter to the Fusion_Compiler_Reference_Methodology strategy in Table 5-46 on page 124 and Design_Compiler_Reference_Methodology strategy in Table 5-38 on page 112 ■ Added write_absolute_dependency_path preference attribute to the GUI Preferences in Table 9-84 on page 203
R-2020.12-SP5	July 2021	■ Added the EnableLibraryConfigurationFlow parameter to the Basic settings for the Fusion Compiler Reference Methodology in Table 5-46 on page 124 ■ Added Control Files parameters to the Constraint Analysis activity in Table 6-76 on page 168
R-2020.12-SP4-1	June 30, 2021	■ No technical changes
R-2020.12-SP4	June 2021	■ Added the UseTechFile parameter to the Fusion_Compiler_Reference_Methodology strategy in Table 5-46 on page 124 ■ Added the GenerateAreaReport to the synthesis reports in Table 5-54 on page 134

Version	Date	Description
		■ Added InsertDft, DFTFlowType, and UseTechFile parameters to the Explore RTL Architecture activity in Table 6-78 on page 170 ■ Documented preferences for Schematics in Table 9-84 on page 203 ■ Added accessHandle information to the IP-XACT preferences in Table 9-84 on page 203
R-2020.12-SP3-1	June 2021	■ Added QoR strategy and effort level to the Fusion_Compiler_Reference_Methodology parameters in Table 5-46 on page 124 ■ Added parameters for the Multi Vth synthesis constraint for the Design Compiler tool in Table 5-42 on page 117 and for the Fusion Compiler tool in Table 5-46 on page 125 ■ Added the ReadConstraints and WriteICC2Files parameters to the DCTCL_quick_map_strategy in Table 5-52 on page 130 ■ Added alternate strategies parameters to the Formal Verification activity in Table 6-67 on page 157
R-2020.12-SP3	April 2021	■ Added the TransitionDelayCoverageGoal parameter to Table 5-58 on page 142
R-2020.12-SP2	March 2021	■ Clarified the contents of workspaces and added Figure 2-2 on page 28 ■ Added “Select Items” on page 54 for the Schematic > Select Items menu ■ In “Technology Cell Binding” on page 77, updated the bind_cell_mapping command to bind_tech_cell_mapping ■ Added “Tech Cell Mapping Use Case” on page 77 ■ Added automotive synthesis control in “Design_Compiler_Reference_Methodology” on page 112 and “Fusion_Compiler_Reference_Methodology” on page 123 ■ Added Automotive Reports in Table 5-54 on page 134
R-2020.12-SP1	January 2021	■ Defined the term TechSetup in Table 1-2 on page 24 and in section “Specifying Target Technology (TechSetup)” on page 74. ■ Added “Explore RTL Architecture Activity” on page 170
R-2020.12	December 2020	■ Updated “Logical Library Setup” on page 74 to clarify Fusion Compiler library details ■ Added new activity in “Technology Cell Binding” on page 77 ■ Added “Fusion_Compiler_Reference_Methodology” on page 123

Version	Date	Description
		■ In “Choosing a Synthesis Strategy” on page 111, removed the following sections: □ 5.1.2.1.2 DCTCL_high_effort_incr □ 5.1.2.1.3 DCTCL_area_recovery □ 5.1.2.1.4 DCTCL_design_rule_fix
Q-2020.03-SP1	April 2020	■ Updated description of the Rename command in “Rename” on page 54 ■ Added the ExportConnectionCommentsInline parameter in Table 3-15 on page 67
P-2019.06-SP4	December 2019	■ Added the ReportClockGateSavings report option in Table 6-73 on page 165
P-2019.06-SP3	October 2019	■ Added Design Compiler NXT to synthesis options in Table 5-53 on page 132 ■ Added fan-in/fan-out report and DFT coverage estimate report options in Table 5-54 on page 134 ■ Added TestMAX™ ATPG option in Table 5-57 on page 140
P-2019.06-SP1	July 2019	■ Added the Implementation Context parameter in Table 4-22 on page 80
P-2019.06	June 2019	■ Added the DelimitBitVector parameter to Table 3-15 on page 67 ■ Removed support for VMM testbenches in the following sections: □ “Create Testbench Workspace Activity (Subsystem Workspace)” on page 175 □ “Simulate Subsystem Testbench” on page 182
O-2018.09-SP4	April 2019	■ Added the SDC Generation option in Table 5-38 on page 112 ■ Updated descriptions of GatingAndFunction and GatingOrFunction in Table 5-43 on page 120 ■ In the Assembly preferences in Table 9-84 on page 203: □ Updated a GUI label □ Added a preference for controlling auto connection of interfaces
O-2018.09-SP2	December 2018	Updated Table 5-53 on page 132 Enhanced “Editing Tool Setup and User Preferences” on page 201 and added Table 9-84 on page 203 to explain preferences
O-2018.09-SP1	October 2018	Removed HAPS entry from Table 4-18 on page 76
O-2018.09	September 2018	Documented the “Constraint Analysis Activity” on page 168
N-2017.12-SP4	June 2018	Added macro prefixes to “Setting Workspace Options” on page 42 and “Configure Components Tab” on page 57

Version	Date	Description
		Added Write ICC2 Files parameter in Table 5-38 on page 112
N-2017.12-SP3	April 2018	Added At Speed Test parameters to Table 5-58 on page 142
N-2017.12-SP2	March 2018	Added SpyGlass to Table 1-1 on page 23, “coreAssembler Features” on page 25, and Table 2-4 on page 30 Added the “Static Checking Activity” on page 152
N-2017.12-SP1	January 2018	Added the Extend MW Layers parameter to Table 5-42 on page 117 Added the Use the scan compression Protocol parameter to Table 5-58 on page 142
N-2017.12	December 2017	Version change
M-2017.06-SP3	October 2017	Added the Optimization flow parameter to Table 5-38 on page 112
M-2017.06-SP2	September 2017	Version change
M-2017.06-SP1	July 2017	Version change
M-2017.06	June 2017	Added support for Golden UPF Flow; see Table 5-40 on page 114, Table 6-66 on page 156, and Table 6-70 on page 161
L-2016.09-SP4	April 2017	Added Table 6-82 on page 182 to show Simulator Setup parameters for the Verify Subsystem activity
L-2016.09-SP3	March 2017	Version change
L-2016.09-SP2	December 2016	Version change
L-2016.09-SP1	October 2016	Added the UseNetsForOpenOutputPins parameter to the Generate Subsystem RTL Parameters in Table 3-15 on page 67 Added the AddPiConstraints parameter to the TMAX Reference Methodology strategy in Table 5-58 on page 142
L-2016.09	September 2016	Added the TetraMAX Reference Methodology strategy; Table 5-58 on page 142
K-2015.06-SP7	July 2016	Added scan-related attributes; p. 97
K-2015.06-SP6	June 2016	Added UVM RAL Subsystem Ping Test; p. 178
K-2015.06-SP5-2	April 29, 2016	Version change
K-2015.06-SP5-1	April 2016	Added the “Use assign for initialization” field in Table 3-15 on page 67
K-2015.06-SP5	March 2016	Added Table 3-15 on page 67 Reduced the synthesis strategies available and adjusted associated text; p. 109 - p. 139. Updated Figure 5-6 on page 103.
K-2015.06-SP3	November 2015	Added link to information about setting the component search path to help when adding a new component; p. 50.

Version	Date	Description
K-2015.06	June 2015	Added definitions for prefixed components, non-prefixed components, and shared components; p. 24. Added details about instantiating components in batch mode; p 50. Added the duplicate_component command; p. 52., Added the EnableSPG parameter; p. 117. Updated the “Names” row in Table 9-89.
J-2015.03	March 2015	Added Specify Power Intent activity, p. 86
J-2014.12	December 2014	IP-XACT updates, Table 1-2, and p. 84, p. 234, p. 234, p. 242.
I-2014.03-SP2	July 2014	Updated SwitchingActivityFile attribute to take multiple files: Table 6-71.
I-2014.03	March 2014	Added UVM/RAL description to Verify chapter, p. 176. Change to Generate Subsystem RTL -- unconnected inputs may be left open, p. 67.
I-2013.09-SP2	January 2014	Clarification of VMM-only functionality p. 153
I-2013.09	September 2013	Removed support for VMT simulation (Directed Tests/Verilog), p. 175; Added support for UVM RAL register testing, p. 176
H-2013.03-SP2	July 2013	Target Language support added to create_workspace, p. 43
H-2013.03-SP1	April 2013	Added StaticTimingAuxScript to Specify Synthesis Methodology, p. 92; and to Static Timing Analysis activity “Analysis Setup” tab, p. 162
H-2013.03	March 2013	Added UVM variant option to Component RAL views, p. 61. Added ReportRegisterInfo and ReportSyncPaths reports, p. 165. Updated copyright information and date/version.
G-2012.06-SP3	January 2013	Updated Reference Methodology Synthesis information, p. 112; Added DC Explorer option to DCTCL_quick_map_strategy, p. 129. Updated Date and Version information.
G-2012.06-SP2	October 2012	Fixed broken link to Licensing Information on www.synopsys.com
G-2012.06	June 2012	Added Table-driven Data Import/Export. p. 220 Enhanced support for System Verilog interfaces. Propagation of conditional pins. Supported SDC version updated to 1.9.
F-2011.12-SP2	April 2012	Added Physical Synthesis “FloorplanFileDataFormat” “PhysicalConstraintFile” strategy parameters, and also hid “TclPhysicalConstraintFile” “DefPhysicalConstraintFile” parameters. p. 117
F-2011.12	December 2011	New default synthesis strategy Design_Compiler_Reference_Methodology (renamed from DCTCL_one_pass_compile_ultra) p. 112
F-2011.06-SP2	October 2011	Added Physical Synthesis “WriteMilkywayCell” strategy parameter. p. 117
F-2011.06	June 2011	New Specify Subsystem activity merges Add Subsystem Components, Configure Components, Memory Map Specification into multi-tabbed single activity, pp. 48-64; Memory Map propagation added, p. 60; Legacy HDL translation to IP-XACT, p. 239.
E-2010.09-SP2	January 2011	Removed Physical Synthesis controls and psyn_shell.

Version	Date	Description
E-2010.09	September 2010	System Verilog output language (Generate Subsystem RTL), p. 67; Separate FPGA synthesis activity, p. 146; HAPS board selection - FPGA synthesis, p. 75; Place & Route controls - FPGA synthesis strategy, p. 147; Area Estimation reporting, p. 63;
D-2010.03-SP2	July 2010	Static Timing Analysis -- SuppressDuplicateBusPaths added, p. 165;
D-2010.03-SP1	April 2010	Filename prefixing added, p. 57; remove_connection command allows removing partial connections, p. 66
D-2010.03	March 2010	Support for IEEE p1685 (IP-XACT 1.5), p. 233; Complete Connections activity updates, p. 64; Change Connections (show all connections), p. 55; create_connection/connect_interface -hier update, p. 66; Port naming %d and %D, p. 55; build_debug_tarfile enhanced, p. 218; Synplify (FPGA synthesis) uses DC DWBB library, p. 146
C-2009.06-SP2	January 2010	New IP-XACT Preferences. p. 202; New Add Subsystem Comp menu item--"Mark All Unconnected Interfaces as Unused".p. 56; "Complete Connections activity "hex value," "Remove Glue Logic" button. p. 64; Specify VLVN data in Generate Subsystem RTL activity. p. 67;
C-2009.06-SP1	October 2009	Workspace options has changed. p. 42; Report viewing and generation.p. 45; Optional views generation.p. 45; Export interface options, p. 55; Complete Connections dialog is update, Glue and Default buttons added. p. 64;
C-2009.06-2	July 2009	DcTimingExceptionReport default=0, p. 134; StaticTimingAnalysis-ReportExceptions default=1, p. 165
C-2009.06	June 2009	Interface loading simplified, p. 193; Documentation generation, p. 219; Static Timing Analysis interface, p. 161; UPF support in Formality, p. 159; Constraint propagation control, p. 80; Component replacement mapping, p. 53,217; Batch script enhancements, p. 200; Manual connections schematic, p. 66

Preface

This chapter contains the following sections:

- [“About This User Guide”](#) on page 21
- [“Related Documents”](#) on page 21
- [“Manual Overview”](#) on page 22
- [“Typographical and Symbol Conventions”](#) on page 22
- [“Synopsys Common Licensing \(SCL\)”](#) on page 23

About This User Guide

This document explains how to use coreAssembler in graphic and batch modes to connect reusable configured cores into synthesizable subsystems. The user guide is written for IC designers who are using synthesizable cores. Readers should be familiar with:

- IC design flow, including the concept of integrating previously designed blocks into a larger design.
- A hardware description language (either VHDL or Verilog).
- Synthesis with Synopsys Design Compiler.
- Design verification, configurable test benches, and VHDL/Verilog simulation tools.

Related Documents

This document is part of the coreTools document set. Information about coreTools is available from the following sources and from the online help system on the Help menu of each tool:

- **General Documents**

- Guide to coreTools Documentation*

- coreTools Release Notes*

- **Working with Individual IP Components**

- coreConsultant User Guide*

- **Tutorials**

- Using Synopsys Library IP in coreAssembler*

For thorough coverage of the theory and practice of design-reuse:

- Keating, Michael and Bricaud, Pierre. *Reuse Methodology Manual, 3rd Edition*. Kluwer Academic Publishers, 2002.

You may also find useful the documentation for the following related Synopsys products: Synopsys IP, Design Compiler, DC-Topographical, PrimeTime, and Power Compiler. For more information, refer to:

- Documentation on the Web available through SolvNetPlus:
<http://solvnetplus.synopsys.com> (a Synopsys username and password are required)
- Documentation for Synopsys IP components, located using “Search for IP”:
<https://www.synopsys.com/designware-ip.html>

Manual Overview

This guide contains the following information:

- [Chapter 1, “Introduction”](#) defines key terms and concepts; introduces the coreAssembler activity flow.
- [Chapter 2, “The coreAssembler Environment”](#) explains installing coreKits, creating workspaces, the activity-based design flow, user interfaces, online help, coreAssembler spreadsheets, and basic operations.
- [Chapter 3, “Creating the RTL View of a Subsystem”](#) describes the activities involved in assembling, configuring, connecting, and generating an RTL view of a subsystem.
- [Chapter 4, “Creating the Gate-Level Netlist for a Subsystem”](#) describes the activities involved in configuring, connecting and building a gate-level view of a subsystem.
- [Chapter 5, “Performing ASIC Synthesis”](#) describes how to select a synthesis strategy, synthesize the subsystem, and analyze the synthesis results.
- [Chapter 6, “Verifying Subsystems and Components”](#) describes the activities involved in functional and formal verification of subsystem views generated in the tool. Also, it describes the activities you use to create a testbench workspace, instantiate a DUT and VIP components and perform a simulation.
- [Chapter 7, “Exporting a Subsystem for Reuse”](#) describes how to package and export a subsystem coreKit.
- [Chapter 8, “Unpackaged Component Import”](#) describes how to import and use simple components which have not been packaged into coreKits in a coreAssembler subsystem.
- [Chapter 9, “Advanced User Modes”](#) covers batch (command line) mode and tool customization.
- [Chapter 10, “Customizing a Technology Library”](#) covers customization techniques for various technology libraries.
- [Chapter 11, “Using IP-XACT Components and Designs”](#) covers how to import and export IP-XACT-compliant components and designs into coreAssembler, and how to use IP-XACT generators and generator chains.

Typographical and Symbol Conventions

The following conventions are used throughout this document:

Table 1 Documentation Conventions

Convention	Description and Example
%	Represents the UNIX prompt.
Bold	User input (text entered by the user). % \$SYNOPSYS/bin/coreAssembler
Monospace	System-generated text (prompts, messages, files, reports). No Mismatches: 66 Vectors processed: 66 Possible"
Italic or Italic	Variables for which you supply a specific value. As a command line example: % setenv <i>SYNOPSYS prod_dir</i> In body text: In the previous example, <i>prod_dir</i> is the directory where your product must be installed.
(Vertical rule)	Choice among alternatives, as in the following syntax example: -effort_level low medium high
[] (Square brackets)	Enclose optional parameters: <i>pin1 [pin2 ... pinN]</i> In this example, you must enter at least one pin name (<i>pin1</i>), but others are optional (<i>[pin2 ... pinN]</i>).
TopMenu > SubMenu	Pulldown menu paths, such as: File > Save As ...

Synopsys Common Licensing (SCL)

You can find general SCL information on the following page:

<http://www.synopsys.com/keys>

License Information -- Synopsys Common Licensing Quickstart:

<https://www.synopsys.com/support/licensing-installation-computeplatforms/licensing.html>

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive

language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Customer Support

Synopsys provides various methods for contacting Customer Support, as follows:

- For fastest response, enter a case through SolvNetPlus:
 - a. Open the SolvNetPlus web page using the following URL:
<https://solvnetplus.synopsys.com/>
 - b. Click the Cases menu and then click Create a New Case (below the list of cases).
 - c. Complete the mandatory fields that are marked with an asterisk and click Save.
Make sure to include the following:
 - Product L1: coreTools
 - Product L2: coreAssembler
 - d. After creating the case, attach any debug files you created.
For more information about general usage, refer to the following article in SolvNetPlus:
<https://solvnetplus.synopsys.com/s/article/SolvNetPlus-Usage-Help-Resources>
- Or, send an e-mail message to support_center@synopsys.com (your e-mail will be queued and then, on a first-come, first-served basis, manually routed to the correct support engineer):
- Include the Product L1, Product L2, process, and version number in your e-mail so it can be routed correctly.
- Attach any debug files you created.
- Or, telephone your local support center:
 - North America:
Call 1-800-245-8005 from 7 AM to 5:30 PM Pacific time, Monday through Friday.
 - All other countries:
<https://www.synopsys.com/support/global-support-centers.html>

1

Introduction

The coreAssembler tool simplifies the process of assembling a subsystem of components by automating the configuration and interconnection steps, providing an automated path to implementation for the entire subsystem, and providing a starting point for verification.

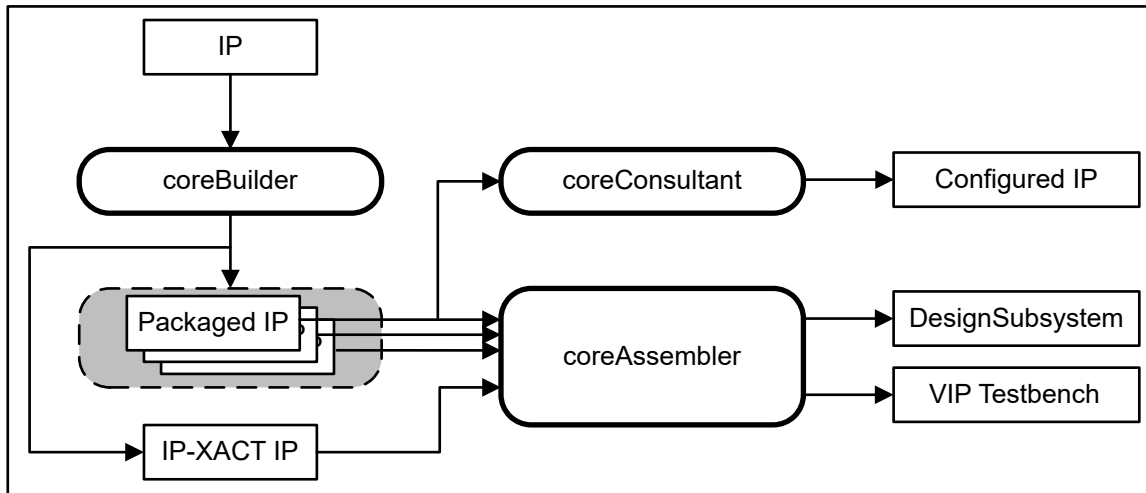
This chapter contains the following sections:

- [“Design Reuse with Synopsys coreTools”](#) on page 26
- [“Key Terms and Concepts”](#) on page 28
- [“coreAssembler Features”](#) on page 30

1.1 Design Reuse with Synopsys coreTools

The coreTools set includes coreBuilder, coreConsultant, and coreAssembler.

Figure 1-1 coreTools Products in IP Workflows



Core developers use the coreBuilder tool to package the design itself, along with the core developer's detailed knowledge of the design structure and functionality, in coreKit format.

Core integrators use coreConsultant to work with individual coreKits created by the coreBuilder tool. coreConsultant automates the process of configuring, synthesizing, and verifying a core as a stand-alone component.

Core integrators also use coreAssembler to assemble (RTL and netlist assembly) packaged cores into subsystems that are in turn integrated into chip-level applications. The entire subsystem can be synthesized and verified as a whole in coreAssembler.

1.1.1 coreAssembler Process Activities

After you have created a workspace for a subsystem, coreAssembler steps you through the assembly, configuration, simulation, and synthesis of the cores in the subsystem:

1. To assemble a subsystem, click the activity [“Specify Subsystem”](#) on page 54
 - a. [“Add Components Tab”](#) on page 54 lets you add instances and hierarchical components, while coreAssembler makes automatic interface connections between components.
 - b. [“Configure Interfaces Tab”](#) on page 64 lets you change interface parameters.
 - c. [“Memory Map Tab”](#) on page 64 lets you specify a subsystem memory map using auto-initialization and manual spreadsheet entries.
 - d. [“Configure Components Tab”](#) on page 64 is where you specify the values for component configuration parameters (such as bus width or number of ALUs in a processor). coreAssembler accepts legal input only and enforces any parameter dependencies specified by the core developers.

Once you have finished specifying your subsystem, coreAssembler writes out the appropriate component HDL source files (RTL format) and updates all other configuration-dependent files specified by the core developer.

2. “[Complete Connections](#)” on page 72 where you connect unconnected signals.
3. “[Generate Subsystem RTL](#)” on page 75 generates RTL files for the subsystem-level design after top-level connections are made. These files may be in encrypted form, as specified by the core developer or your license feature.
4. “[Verifying an Individual Component](#)” on page 174 and “[Verifying a Subsystem](#)” on page 175 describe how to verify the DUT from within the tool if the verification functionality was packaged with the coreKits used in the subsystem.
5. “[Performing ASIC Synthesis](#)” on page 119 steps you through setting up the technology library, prompts you to enter design constraints, and gives you the option of overriding the synthesis intent specified by core developers. coreAssembler then checks to confirm that your synthesis specifications are complete and consistent. Prior to synthesizing the configured subsystem, coreAssembler prompts you for a synthesis strategy. Then, coreAssembler drives the required implementation tool to execute your selected strategy. [Table 1-1](#) lists the tools that coreAssembler currently supports.

Table 1-1 Supported Implementation and Verification Tools

Tool	Function
Design Compiler®	A selection of strategies for RTL-to-gates synthesis and incremental synthesis. coreAssembler interacts with Design Compiler in Tcl mode (dc_shell-t).
DC Explorer	Synthesis using DC-Explorer, provided that version G-2012.06-SP2 or newer DC-Explorer is available and you use the DCTCL_quick_map_strategy.
DC Topographical	A selection of physical synthesis strategies that implement both RTL-to-placed-gates and gates-to-placed-gates methodologies.
PrimeTime®	Static Timing Analysis, Timing model generation.
SpyGlass®	Static design checks using SpyGlass
Formality®	Formal verification.
RTL Architect™	Enables early, physically aware RTL exploration to achieve a better starting point for RTL synthesis and better convergence during physical implementation.
VCS®/Vera®	Simulation model verification.

The coreAssembler tool generates an optimized set of commands to drive the implementation tool based on the core configuration, cell-set characteristics of your technology library, and the features available in your current version of the selected implementation tool. You can select to execute the selected strategy immediately or just generate the scripts for later use.

6. Once you have verified the functionality and timing of your particular configuration of the subsystem as a stand-alone entity, you can integrate the subsystem into your application and verify at the chip level.

1.2 Key Terms and Concepts

The following table defines the key terms and concepts used in coreAssembler.

Table 1-2 Key Terms and Concepts

Term	Definition
Application Design	Overall chip-level design into which a subsystem is integrated. An application design includes one or more subsystems, plus custom, application-specific logic, hard macros, and memory.
Component	An instance of a core in the subsystem or a hierarchical cell.
Configuration Intent	The range of values allowed for each parameter associated with a reusable core. Using coreBuilder, a core developer specifies how the core may be configured, and also specifies parameter dependencies so when the core is added to a subsystem coreAssembler will accept only legal combinations of parameter settings from the core integrator.
Core	A reusable design created and configured by a core developer and incorporated into chip-level applications by one or more core integrators.
Core Developer	The creator of a reusable core. Using the coreBuilder application, the core developer applies attributes to a core design and packages the core and its attributes into a coreKit so that core integrators can easily implement the core in system-on-a-chip applications.
Core Integrator	The person who uses coreAssembler to incorporate reusable cores into a larger design called a subsystem.
coreKit	The design package created by the coreBuilder application to facilitate IP reuse. Each coreKit includes the core itself, the synthesis methodology specified by the core developer, interfaces for subsystem connection, and optional items, such as verification environment files and core-specific documentation.
Design Context	Aspects of the subsystem's target environment that affect the synthesis of the subsystem. The core integrator specifies the design context before coreAssembler generates a synthesis strategy for the subsystem. Design context includes information such as technology libraries, wire load model, parent wire load model, and operating conditions.
Hierarchical Cell	A cell (design instance) representing a level of hierarchy within a subsystem. Also referred to as a hierarchical component.
IEEE P1685	Refers to the IEEE standardized version of the IP-XACT XML standard. The most recent version is P1685-2014.
Intellectual Property (IP)	In the broad context of design reuse, IP refers to any reusable design. The scope of this manual is mainly soft IP because coreAssembler focuses primarily on the reuse of soft cores (user-configurable, technology-independent blocks of synthesizable RTL code). Hard IP support is available if the hard core includes an RTL wrapper.
Interface	A set of ports and parameters that defines a connection point to a component. Interfaces are "provided" by one component and "consumed" by one or more other components. For example, a bus IP would provide a master interface while each master component in the subsystem would consume this interface.

Table 1-2 Key Terms and Concepts (continued)

Term	Definition
IP-XACT	An XML-based standard for describing IP and IP packaging, as well as plug-in to IP tools and flows, to enable more efficient and cost-effective integration of IP from multiple sources and vendors.
Non-prefixed component	A non-prefixed component does not have a prefix applied to all modules within the component when instantiated (for more about prefixes, see "Prefixed component"). Only non-prefixed leaf components or hierarchical components containing non-prefixed components can be shared. See also: "Shared component" .
Prefixed component	A prefixed component has a prefix (the instance name) automatically applied to all modules within the component. Only IP packaged as a coreKit can support prefixing, and then only if prefixing is enabled when the IP is packaged in coreBuilder. You can tell if a component is prefixed by generating and reviewing the Subsystem Components table within the Subsystem Contents report. Components that have prefixing enabled are automatically prefixed when instantiated in coreAssembler unless the -noprefix option is used. See also: "Non-prefixed component" .
Shared component	Shared components are ones which utilize a single (or common) definition in the coreAssembler workspace. Source code is only written for a single instance of a group of shared components. As a result, all instances in a shared group must have the same values for any configurable `defines in Verilog or configurable constants in VHDL. Verilog parameters and VHDL generics may have different values per instance as those values are passed in through the instance.
SPIRIT	(Structure for Packaging, Integrating, and Re-using IP within Tool flows) Legacy term defining the former consortium group coordinating the development and standardization of IP-XACT, which is now controlled by Accellera.
Subsystem	The highest level of RTL automatically generated by coreAssembler. The subsystem contains reusable cores connected using coreAssembler.
Synthesis Intent	Attributes the core developer applies to the top-level design, ports, and subdesigns of a core. coreAssembler uses these attributes to drive the synthesis process. Synthesis intent determines how coreAssembler generates synthesis constraints, strategies, and directives.
TechSetup	The collective settings that are made in the Specify Target Technology activity to specify the technology libraries for ASIC synthesis.

1.3 coreAssembler Features

The following lists the key features of coreAssembler:

- Block-based, hierarchical subsystem RTL and netlist assembly
- Importing and exporting of IP-XACT compliant components and subassemblies
- Grouping/ungrouping of multiple components into new level of subsystem hierarchy
- Automatic interconnection of components with predefined compatible interfaces
- Add glue logic, and make any necessary manual connections (and, nand, or, nor, buffer, inv, xor, mux)
- Automatic propagation of system configuration values to individual components to confirm consistent configuration throughout the subsystem
- Automatic generation of hierarchical subsystem RTL and configured component RTL
- Automated script generation for synthesis of the entire subsystem using Design Compiler or DC Graphical
- Automated links for DFT integration and test pattern generation using TestMAX
- Automated links to Formality for formal design verification
- Automated links to Spyglass for static design checks
- Ability to customize activity flow on a workspace basis and restrict generation of unneeded files
- Automatic generation of separate testbench workspace with subsystem DUT instantiated
- Address map initialization
- Table driven data Import/Export: (basic attribute, memory map, subsystem configuration, subsystem connections)
- Propagation of conditional pins
- SystemVerilog support: interface, connections to non-SV interfaces, testbench top-level ports, monitor connections.

The following points lists the features available when using Synopsys AHB, APB and AXI bus fabric in coreAssembler:

- Automatic connection of Master/Slave Verification IP and optional bus monitors
- Register write/read testing

2

The coreAssembler Environment

Before using coreAssembler to connect, configure, and synthesize reusable cores, understand the concepts about the coreAssembler work environment.

This chapter contains the following sections:

- [“coreKits and Workspaces”](#) on page 32
- [“Activity-Based Design Flow”](#) on page 33
- [“User Interfaces”](#) on page 37
- [“Online Help Tools”](#) on page 40
- [“coreAssembler Spreadsheets”](#) on page 41
- [“Basic Operations”](#) on page 42

2.1 coreKits and Workspaces

The primary inputs to coreAssembler are coreKits provided by core developers, IP-XACT packaged IP, or unpackaged IP. If using coreKits, they must first be installed using coreConsultant or coreAssembler.



Note

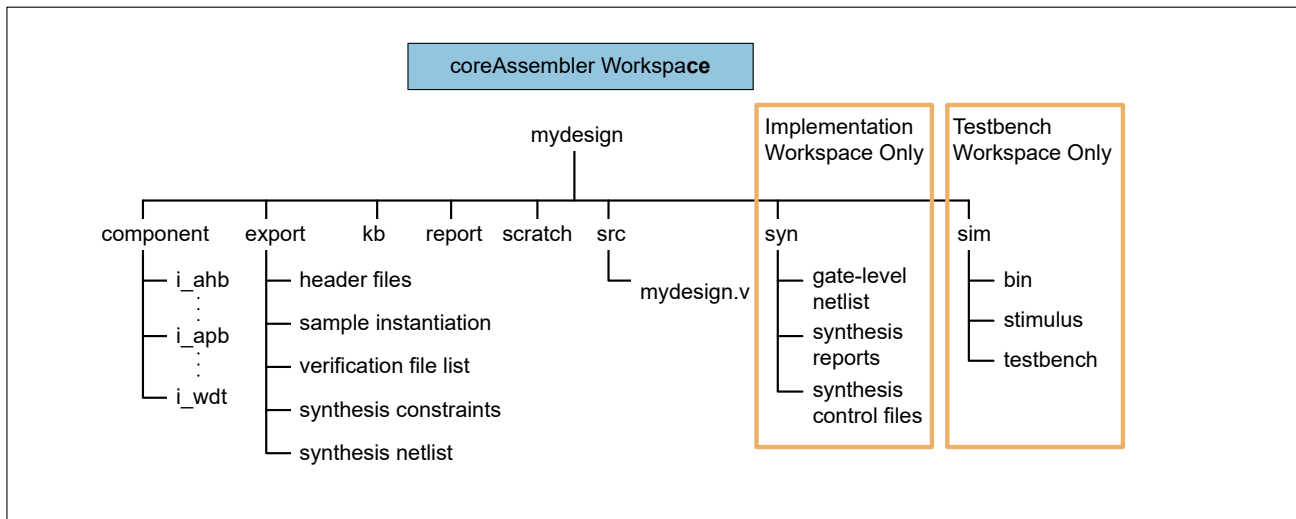
IP provided by Synopsys comes pre-installed in the library.

A workspace is your working version of a subsystem, where you connect, configure, simulate, and synthesize your subsystem. You can create, copy, and modify workspaces to experiment with different design alternatives. The exact contents of each workspace depends on the types of IP used, your flow sequence, your unique subsystem design, and component configurations.

Workspaces contain directories, files, and symbolic links to IP component installations (in \$DESIGNWARE_HOME for Synopsys Library IP).

Figure 2-1 shows an example of a coreAssembler workspace. Some of the directories shown are created after you perform the related activities (for example, the export and syn (synthesis) directories).

Figure 2-1 coreAssembler Workspace Tree



Note

IP-specific directories and files are not shown here, but they may also reside in your workspace. For information about IP-specific objects, see the documentation for your IP product.

For information on managing workspace changes and linked resources, see “[Workspace Operations](#)” on page 48.

2.2 Activity-Based Design Flow

coreAssembler uses an activity-based flow to guide you through the required implementation tasks for a subsystem. Activities that depend on other activities are disabled until you complete all prerequisite activities. For example, verification activities are disabled until you specify your configuration and assemble the subsystem. You can customize the available activities, such as removing synthesis activities, or component verification, if not needed.

The coreAssembler activities are organized into a hierarchy of view generation and view verification processes. The different views of a core are listed in [Table 2-1](#).

Table 2-1 coreAssembler Views of a Subsystem

View	Purpose	Outputs
RTL View	Defines the subsystem at the RTL level in either Verilog, System Verilog or VHDL. Subsystem components are defined by the core integrator and then automatically connected by coreAssembler.	■ export/src/<design>.v[hd] ■ export/src/<design>.sv ■ export/<design>_params.v[hd] ■ export/<design>.lst ■ export/README ■ components/ <component>/src/*.v[hd]
Gate-Level View	A translation of the RTL view into a technology-specific netlist which can include physical placement information.	■ export/<design>.db ■ export/<design>.v[hd] ■ export/<design>.sdc
GTECH View	A translation of the RTL view into a generic technology. This view is used for RTL simulation of encrypted source code by non-Synopsys simulators which are unable to read the encrypted RTL. Note: This view is provided only at the component level. Generation of this view is not built into coreAssembler, but is provided by all Synopsys DesignWare IP.	■ gtech/syn/final<design>.v[hd]
Exported Subsystem View	Create reusable views of the subsystem. A subsystem template can be created to provide a starting point for new designs, or a coreKit can be created for use in coreAssembler or coreConsultant.	■ export/<design>.template ■ export/coreKit/ package_subsystem-<design>.tcl ■ export/coreKit/ package_subsystem-<design>.log ■ export/coreKit/ interface_definitions-<design>.tcl ■ export/coreKit/<design>.coreKit ■ SPIRIT/<design>/...

Table 2-1 coreAssembler Views of a Subsystem (continued)

View	Purpose	Outputs
		■ batch.tcl ■ index.html
Testbench View	Create a separate workspace with reusable views of the testbench environment. The testbench workspace view is required in order to generate tests for the Simulate Subsystem Testbench activity.	■ export/.descriptions/<design>_* ■ export/<design>_comp.v[hd] ■ export/<design>_params.v[hd] ■ export/<design>.lst ■ export/README ■ SPIRIT/<design>/... ■ index.html

The specific set of activities for a subsystem are a combination of the default (built-in) coreAssembler activities listed by view in [Table 2-2](#), plus core-specific activities defined by the core developer. For example, the core developer creates core-specific verification activities and package them into the coreKits.

Table 2-2 coreAssembler Default Activities

Activity	Function
Create RTL	
Specify Subsystem	This activity presents the following tabbed dialog views to allow you to create and configure your subsystem: ■ Add Components: Place component instances in your subsystem and auto-connect defined interfaces ■ Memory Map: Create/modify/validate memory maps for component register implementations ■ Configure Interfaces: Create and configure interface definitions ■ Configure Components: Specify the values for configuration parameters
Complete Connections	Connect component pins and subsystem ports manually until the subsystem design is complete
Generate Subsystem RTL	Generate HDL source code for top-level design of the subsystem
Create Gate-Level Netlist (available after first core is added)	
Specify Target Technology	Gets variable settings from DesignCompiler and extracts technology-specific information from the specified technology libraries
Technology Cell Binding	Implements the technology cell mapping functionality
Initialize Subsystem Constraints	Allows you to change the default constraints used for synthesis

Table 2-2 coreAssembler Default Activities (continued)

Activity	Function
Specify Clocks	Define clocks and all properties associated with clocks, such as waveform and latency
Specify Operating Conditions & Wireloads	Specify information defining the context in which the core is to be used (operating conditions and wire loads)
Specify Timing Exceptions	Specify the timing exceptions in the subsystem design. Synthesis generates a report_timing_exceptions report which includes false path information
Specify Port Constraints	View and (if necessary) modify developer-specified port attributes, most of which control constraint generation
Specify Synthesis Methodology	View and (if necessary) modify developer-specified values for attributes that describe design characteristics and control the synthesis process
Specify Test Methodology	Set/review Design for Test attributes on designs
Perform ASIC Synthesis	Select synthesis strategy/options, execute ASIC synthesis, and analyze results
Generate Test Vectors	Set/review ATPG strategy, setup and execution options to create test vectors using TestMAX
Create Component GTECH Simulation Model (available after first core is added)	
Generate GTECH Model	Generate a GTECH model (applies only to Synopsys DesignWare IP)
Verify Component (available after first core is added)	
Setup and Run Simulations	Set up and run a standalone simulation of an individual component. Applies only to components packaged with unit-level test environments
Verify Subsystem (available after first core is added)	
Static Checking	Perform static checking of the subsystem and its components
Formal Verification	Generate and (optionally) run formal verification scripts
Static Timing Analysis	Generate and (optionally) run static timing analysis scripts
Create Testbench Workspace	Generate a Device Under Test of the current subsystem, create a Testbench workspace, instantiate the DUT and (optionally) add Verification IP
Export Subsystem	
Build Subsystem coreKit	Run coreBuilder and package the subsystem as a coreKit
Create Subsystem Template	Create a template file in the ./export directory of the current workspace

Table 2-3 coreAssembler Testbench Activities

Activity	Function
Create Testbench	
Specify Testbench	This activity presents the following tabbed dialog views to allow you to create and configure your subsystem: ■ Add Components: Place testbench component (DUT and verification models) in your testbench and auto-connect defined interfaces ■ Configure Interfaces: Create and configure interface definitions ■ Memory Map: Create/modify/validate memory maps for component register implementations. ■ Configure Components: Specify the values for configuration parameters and generate testbench HDL for all verification components.
Complete Connections	Connect component pins and verification ports manually until the testbench creation is complete
Generate Testbench HDL	Generate HDL source code for top-level testbench for a subsystem DUT
Verify Subsystem (available after first core is added)	
Simulate Subsystem Testbench	Generate a set of verification ping-tests, and run the simulation on the DUT using the connected verification IP. Only available for DesignWare components

2.3 User Interfaces

coreAssembler provides the following user interfaces:

- [“Command Line Mode”](#)
- [“Batch Mode”](#)
- [“GUI Mode”](#)

2.3.1 Command Line Mode

Command line execution is available through the coreAssembler console in GUI mode or by invoking coreAssembler in shell mode. For more, see [Advanced User Modes](#) on page 237.

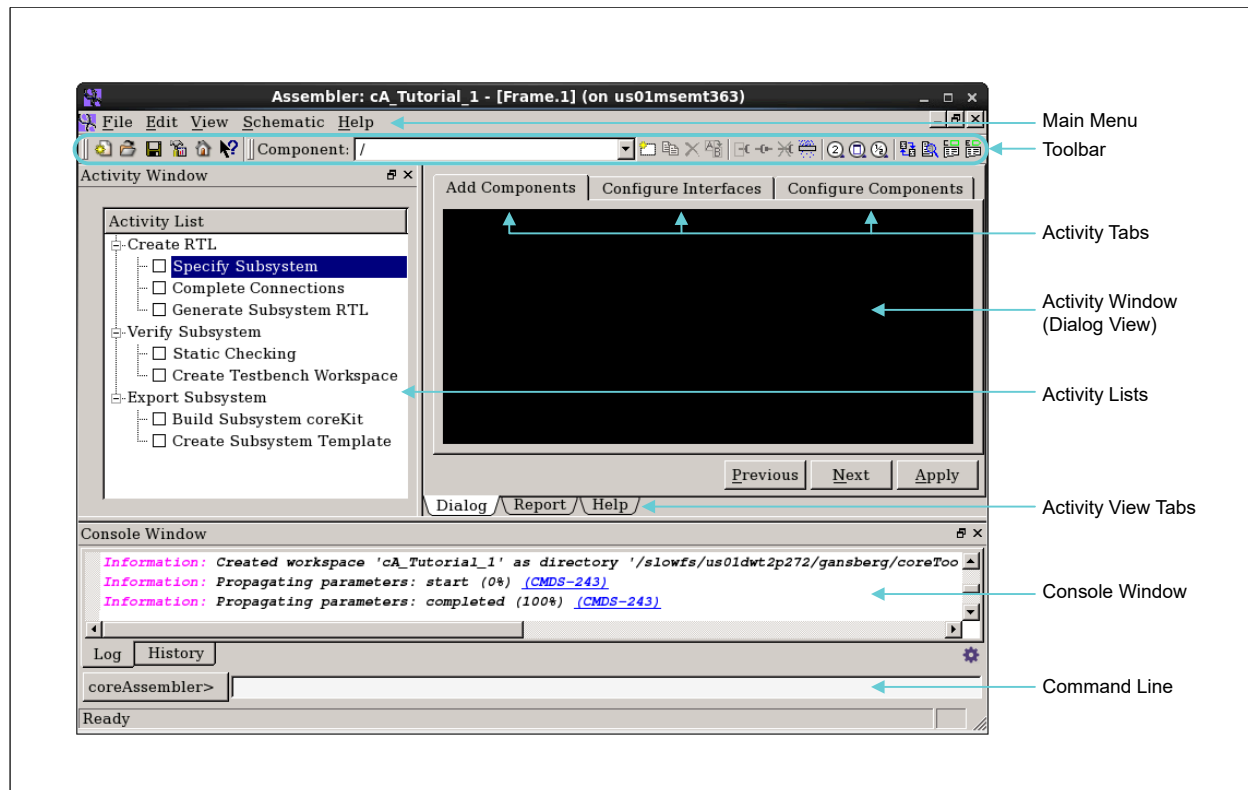
2.3.2 Batch Mode

Batch mode allows you to execute a series of coreAssembler commands in a batch file. To write a batch file that re-creates the current state of your workspace, use the `write_batch_script` command or the **File > Write Batch Script** menu item. For more, see [“Batch Mode”](#) on page 241.

2.3.3 GUI Mode

GUI mode is recommended for new users. In GUI mode, you interact with coreAssembler through the console pane, the activity list, and the activity view for the individual activities, as shown in [Figure 2-2](#).

Figure 2-2 coreAssembler GUI Features



■ Main Menu

This menu bar has workspace, setup, editing, viewing, and help options.

■ Toolbar

This row contains viewing and command shortcut icons. The “Component” field lets you change the view to a different component or level of hierarchy.

■ Console Window

Contains three tabs. The Log tab contains the sequential log of all coreAssembler commands and their respective output. The History tab contains a sequential list of all coreAssembler commands executed directly in the console. The Errors/Warnings tab contains a sequential list of all errors messages and warnings and supports linking from a specific message into the Log tab.

■ Command Line

Enter coreAssembler commands, source batch scripts and TCL scripts, and get help on commands or “man” pages.

■ Activity List

Shows all activities available in coreAssembler in a list view. Completion status of each activity is indicated by a check mark in the box associated with each activity. Clicking on a label opens the Activity View Pane for that activity.

■ Activity Dialog Tabs

For the Specify Subsystem activity, these tabs are sub-activities that allow you to move back and forth between editing tasks with the main activity. NOTE: The “Apply” button is not used to apply each of the tabbed sub-activities, but only used when ALL tabbed information is complete, and to finish the main (in this case, Specify Subsystem) activity.

■ Activity Window

If there is a current activity (an open workspace), This pane shows an activity view (schematic view, list view, dialog box, spreadsheet, report, or help). This pane contains three Activity View tabs:

Table 2-4 Activity Window View Tabs

Tab Label	Description
Dialog	A tab showing the “Dialog” view for data entry relevant to the current activity. This view displays a schematic diagram, component and interface list view, dialog box, or spreadsheet entry.
Report	A tab for the “Report” view, which contains report generation control, view generation control, and information displays for the current activity. See “Generating Reports and Optional Views” on page 51.
Help	A tab for the “Help” view, which shows Help information for the current activity.

For information on how to use coreAssembler in GUI mode to generate RTL and gate-level views, see [Creating the RTL View of a Subsystem](#) on page 53 and [Creating the Gate-Level Netlist for a Subsystem](#) on page 81, respectively. For information on formal verification of your subsystem and verification of individual components, see [Verifying Subsystems and Components](#) on page 171.

2.4 Online Help Tools

Several types of online help are available through coreAssembler:

■ “What’s This?” Quick Help

When you position your pointer over a GUI item and right-click, coreAssembler displays a menu with the “What’s This?” option. Clicking “What’s This?” displays a brief help message for the selected item. “What’s This?” can also be accessed by left-clicking the Question Pointer on the toolbar and then left-clicking on a GUI item.

■ The Toolbar Help Menu

Click the toolbar Help button to show a list of help topics. When you click a topic, the corresponding help information appears.

■ Activity View Help Tab

The Activity Window has a Help tab that displays a context-specific help page, with links to the online command reference and other appropriate references. The Activity View Help tab has several toolbar buttons at the top which can be used to go back to the previous page, to go forward to the next page, to refresh the current page, to find text, or to return to the coreTools home page (from left to right). You can use the buttons on the upper right to open a separate window with the same page in it. Opening the page in a separate window can be useful, as it allows you to see the information in the Help tab and in the Dialog tab at the same time.

■ Help on coreAssembler Commands

The Synopsys coreTools Online Command Reference Index is available through the Help menu. It contains a collection of man pages for all coreAssembler commands, attributes, variables, and item types.

You can also access coreAssembler command help by entering one of the following commands at the coreAssembler prompt in the console pane:

```
coreAssembler> help [cmd] [-verbose]
coreAssembler> [cmd] -help
coreAssembler> man [cmd]
```

2.5 coreAssembler Spreadsheets

In GUI mode, coreAssembler sometimes presents spreadsheets so you can review and modify data associated with the current activity. In terms of operation, coreAssembler spreadsheets resemble the spreadsheets used in popular database and financial applications. Left-click to select a row, column, or cell. Right-click to access a menu of options including copy, cut, paste, help, and column fill operations. [Table 2-5](#) describes the basic spreadsheet operations.

Table 2-5 Common Spreadsheet Operations

Operation	How To Perform The Operation
Resize a column	Use your left mouse button to grab the column separator in the column title bar. Move it left or right.
Enter a value in a cell	Click in a cell and type in text or select a value from the combo box menu. coreAssembler does not save the value until you click the OK button. If you click the Cancel button, coreAssembler does not save your edits.
Select multiple cells in a column	To select all the cells in a column, click your left mouse button on the title bar of the column. To select several (but not all) cells, use your left mouse button to select the first cell and drag down to the last cell you wish to select. Alternately, press Shift, and use the left mouse button to select the last cell. CTRL-L is the toggle control for selecting/deselecting cells.
Cut	To cut the contents of one or more cells, select the cells, click the right mouse button, and select Cut. The keyboard shortcut for Cut is CTRL-X.
Copy	To copy the contents of one or more cells, select the cells, click the right mouse button, and select Copy. The keyboard shortcut for Copy is CTRL-C.
Paste	To paste the content of a previous cut or copy operation into one or more cells, select the cells into which you want to paste the data, click the right mouse button and select Paste. The size of the paste must match the size of the corresponding cut/copy. The keyboard shortcut for Paste is CTRL-V.
Fill a column	To fill one or more consecutive cells in a column with the current contents of a cell, select the cell containing the content, plus one or more cells below it. Click the right mouse button, and select Fill Down.
Edit Formula	Some spreadsheet cells support formulas. When you double-click these cells, a button with an equal sign appears. Click the button to select the desired formula and enter required formula data.

2.6 Basic Operations

Before you begin working with cores in a coreAssembler workspace, it is important to be familiar with the following basic operations:

- [“Invoking coreAssembler”](#)
- [“Installing coreKits”](#)
- [“Workspace Operations”](#)
- [“Quitting coreAssembler”](#)
- [“Working with the Activity List”](#)

2.6.1 Invoking coreAssembler

To invoke coreAssembler in GUI mode, enter this command at the UNIX prompt:

```
% <install_dir>/bin/coreAssembler
```

To invoke coreAssembler from the command line, the complete syntax is:

```
coreAssembler [Workspace_name] [-shell] \ [-x "command_string"] [-f cmd_file]
```

Argument	Description
<i>Workspace_name</i>	Loads the workspace <i>Workspace_name</i> upon startup.
-shell	Selects shell mode instead of GUI mode. For more information about shell mode, refer to “Advanced User Modes” on page 237.
-x “ <i>command_string</i> ”	Executes <i>command_string</i> upon startup. <i>command_string</i> is one or more coreAssembler commands, separated by semicolons (;).
-f <i>cmd_file</i>	Executes the commands in a specified command file (<i>cmd_file</i>) in batch mode upon startup. <i>cmd_file</i> contains one or more coreAssembler commands. For details, see “Advanced User Modes” on page 237.

2.6.1.1 Examples

To invoke coreAssembler and automatically load the *Subsystem1* workspace, enter the following command:

```
% coreAssembler path/Subsystem1
```

To invoke coreAssembler in shell mode and execute the batch command file named *MySubsystemBatch.tcl*, enter the following command:

```
% coreAssembler -shell -f MySubsystemBatch.tcl
```

2.6.1.2 The coreAssembler Main Window

The coreAssembler main window ([Figure 2-2](#) on page 38) appears when you invoke coreAssembler in GUI mode. The console pane, activity list pane, activity view pane, and command line entry field serve as your interfaces to the coreAssembler tool.

You can perform operations by selecting menu options from the toolbar or by entering coreAssembler commands at the `coreAssembler>` prompt at the bottom of the main window. [Table 2-6](#) on page 43 defines the menu options available on the toolbar.

Table 2-6 Toolbar Menu Options

Menu Item	Description	Command Line
File > Install coreKit	Install a coreKit.	batch_install
File > New Workspace	Create a new workspace.	create_workspace
File > Open Workspace	Open an existing workspace.	open_workspace
File > Save Workspace	Save the currently loaded workspace.	save_workspace
File > Save Workspace As	Save the currently loaded workspace into a new directory.	duplicate_workspace
File > Close Workspace	Close a workspace.	close_workspace
File > Generate Reports	Allows selection and generation of activity (reports are not automatically generated)	generate_reports
File > Generate Optional Views	Allows selection and generation of alternate views (alternate views are not automatically generated)	generate_views
File > Write Batch Script	Create a batch script to re-create the current state of your workspace.	write_batch_script
File > Build Debug Tar-file	Generate a tarfile containing the batch script, .workspace, logfiles, environment, tool versions and DesignWare Home.	Not applicable
File > Write IP-XACT File	Writes a IP-XACT-compliant file for the subsystem as a design (top), or as a component.	write_ipxact_component write_ipxact_design write_ipxact_designConfiguration
File > Read IP-XACT File	Reads a IP-XACT-compliant file description of a design or generator chain.	read_ipxact_file
File > Remove IP-XACT File	Removes the specified IP-XACT file and related objects defined in the file.	remove_ipxact_file
File > Tables... > Read Attribute Table > Write Attribute Table > Read Pin Connection Table > Write Pin Connection Table > Read Subsystem Table > Write Subsystem Table	Submenu of read/write table commands allowing table-driven data entry.	read_attribute_table write_attribute_table read_pin_connection_table write_pin_connection_table read_subsystem_table write_subsystem_table
File > Exit	Exit coreAssembler.	exit or quit
Edit > Tool Installation Roots	Specify tool installation directories. Also, specify use of 64-bit tool binaries.	set_tool_root

Table 2-6 Toolbar Menu Options (continued)

Menu Item	Description	Command Line
Edit > Preferences	Specify GUI preferences and some verification, execution, synthesis, workspace, library updates, and IP-XACT options. For more about setting preferences, see “Editing Tool Setup and User Preferences” on page 244.	Not applicable
Edit > Workspace Options	Set workspace options for the current workspace.	set_workspace_options
View > Home Page	Show the coreAssembler Home Page in the Activity View pane.	Not applicable.
View > Reports	View a list of the available reports for the current subsystem.	Not applicable.
View > Dialog Window	Show or hide the schematic/dialog/report window(check indicates this window is visible).	Not applicable.
View > Console Window	Show or hide the Console pane and the command line in the Main Window (check indicates Console is visible). Function key F3 toggles displaying the Console Window.	Not applicable.
View > Activity Window	Show or hide the Activity pane in the Main Window (check indicates Activity pane is visible).	Not applicable.
View > Application Tool Bar	Show or hide the application toolbar (check indicates toolbar is visible).	Not applicable.
Schematic > [various subsystem specification commands]	The Schematic menu is available when the Add Components view for the Specify Subsystem activity is selected. This menu provides commands for creating and configuring the subsystem. See page 51 for a description of Schematic menu items.	Not applicable
Help > About	Views the coreAssembler main page in the View pane. This page contains coreAssembler version and information.	Not applicable
Help > Check for IP Updates	Performs a manual check of the components in your subsystem against your DESIGNWARE_HOME library and the Synopsys website. Also shows STARs added/fixed for the components in newer versions.	Not applicable
Help > Show IP Update Report	Displays the current IP Update Check report (located at <workspace>/report/IPUpdateCheck.html).	Not applicable
Help > coreAssembler Tool Help	Open a Help Window to view a selected help item. Includes: Error Messages, the coreAssembler User Guide, the Command Reference, Release Notes, and Application Notes.	man or help

Table 2-6 Toolbar Menu Options (continued)

Menu Item	Description	Command Line
Help > Help for component <name>	Open a Help Window to view help on a subsystem component. An entry is for each design instance.	Not applicable

2.6.2 Installing coreKits

Synopsys ships its cores already installed, so this installation procedure is not required for Synopsys IIP cores. The following procedure is necessary only if you are using non-Synopsys cores. Custom cores must be packaged as *.corekit files in order to be installed and loaded into coreAssembler.

Use the following procedure to install a coreKit, unless a coreKit provider supplies special installation instructions for that core.

1. Extract the coreKit file from the delivery medium and install it on your computer by following the instructions provided by the core developer. The coreKit delivery mechanism can be CD-ROM or electronic delivery through FTP.
2. Invoke coreAssembler and load the coreKit using the **File > Install coreKit** menu command on the coreAssembler toolbar to load the coreKit. coreAssembler displays a dialog box in which you select a coreKit file and the directory where you want it installed.
3. In the Packaged coreKit File field, navigate to the directory where the packaged coreKits have been placed and select the desired coreKit file.
4. In the Installation Directory field navigate to the directory where you want the coreKit installed.
5. Click OK to continue the installation.

coreAssembler writes all files associated with the coreKit into your selected coreKit installation directory, using a directory structure specified by the core developer.

2.6.3 Component Update Checking

coreAssembler provides a way to check for Synopsys Library synthesizable and verification component updates as well as existing STARS for the Synopsys IP used in a coreAssembler workspace.

The first time you complete the Specify Subsystem activity, or open a workspace where automatic IP update checking has not been initialized, you are prompted to enable automatic IP update checking and specify a frequency in days for how often to perform the check. Subsequently, the update check happens automatically when the specified time period has elapsed and either of the following occurs:

- You open a workspace where the **Specify Subsystem** activity is completed
- You complete the **Specify Subsystem** activity during a session.

At the conclusion of the update check, the update check report is shown only if:

- There is a newer version of one or more of the components.
- You have added a new component that has not been checked previously.
- There are STARS that have not been previously reported for the one or more of the components.

In other words, if nothing relevant has changed, the report is not shown.



Note

If you specify automatic update checking in coreConsultant, those settings apply to coreAssembler also. They are saved in the .synopsys_rt_prefs.tcl file in your home directory. Changes to this feature in one tool affect both tools.

You can also manually check for Synopsys IP updates at any time using the **Help > Check for IP Updates** menu item.

The results of both the manual and automated checks are displayed in a top-level HTML report window.



Note

Both automatic and manual IP update checks do not update or change your design in any way. Update checks only generate a report letting you know of newer versions of components or new STARS. It is then entirely up to you whether to update your DESIGNWARE_HOME library or subsystem design workspace.

2.6.3.1 Features

- Opt in or out of the automatic checking for used IP updates at any time
- Checks the Synopsys website for IP updates and STARS for used IP
- Checks your DESIGNWARE_HOME library for newer versions of IP
- Links you to the Synopsys download site
- Differentiates between previously reported STARS and new STARS for your IP
- Allows you to review report from the previous update check

2.6.3.2 Synopsys IP Update Tool Preference

If you previously disabled automatic update checking and wish to enable it, you use **Edit > Preferences > Synopsys IP Updates** dialog to do this. (For more about preferences, see “[Editing Tool Setup and User Preferences](#)” on page 244.)

Table 2-7 Synopsys IP Update Tool Preferences

Text Item	Description
Enable automatic Synopsys IP update Checks	If enabled, automatic update checking is preformed whenever: ■ You complete Specify Subsystem activity ■ The workspace is opened and the frequency has elapsed If disabled, no automatic checks are performed, and you are asked if you want to inform Synopsys about opting out. If disabled, you can still perform a manual IP update check anytime using the Help > Check For IP Updates menu item.
Synopsys IP update check frequency in days	Specifies the interval between automatic update checks. If set to 0, a check is made every time the workspace is opened. If set to > 0, that interval passes between checks. If set to -1, you have not executed the Configure Check dialog.

Table 2-7 Synopsys IP Update Tool Preferences (continued)

Text Item	Description
Inform Synopsys about opting out of automatic IP update checks	If you disable automatic IP update checking, this option allows you to notify Synopsys. If checked, Synopsys is notified “anonymously” that a single user opted out of automatic checking. If unchecked, Synopsys is not notified.

2.6.3.3 Manual IP Update Checks

At any time, you can perform a manual IP update check (exactly the same check as performed automatically) by executing the **Help > Check For IP Updates**. This option is useful when you move between phases of a project and want to verify you are using the current IP, or want to view any STARS for the components used in your subsystem design.

2.6.3.4 IP Update Check Report Format

The generated report is an HTML file named IPUpdateCheck.html, and is saved in the “report” directory within your workspace. [Table 2-8](#) describes the report format.

Table 2-8 IP Update Check Report Format

Report Heading	Description
Component Name	Synopsys component name, such as “DW_ahb_uart”
Instance Name	Name of the instance used in your workspace such as “i_uart”
Instantiated Version	Version number of the instance in your workspace.
DesignWare Home Version	Version number of the component in your DESIGNWARE_HOME library.
Latest Available Version	The current version available for download from Synopsys.
Previously Reported Stars	Lists STARS that have appeared in the previous update checks.
New Stars	Lists any new STARS on the Synopsys website since the last report.
IP Status	The following are status text for this field: No Update Available -- your component is current. Download Latest IP -- a newer version of the component is available on the Synopsys website. This status text is also a link that opens your preferred browser to the download area on the Synopsys website. (Set your preferred browser using the Edit > Preferences menu item. For more about preferences, see “Editing Tool Setup and User Preferences” on page 244.) Replace Instantiated IP -- a newer version of the component is available in your DESIGNWARE_HOME library. Use the Replace Component menu item in the Schematic or Tree View window.

To view an existing report, use the **Help > Show IP Update Report** menu item. This menu item is only available if a previous report has been generated.

If, for any reason, you want to save a report (for specification purpose), be sure to copy it from the workspace “reports” directory. If you do not, the next time a report is generated, it overwrites the existing report.

2.6.4 Workspace Operations

You can perform these workspace operations using the coreAssembler menu items, or commands.

2.6.4.1 Setting Workspace Options

You can customize the activity flow during a coreAssembler session. By streamlining your flow, you can improve coreAssembler performance and save disk space. The following coreAssembler tasks can be individually enabled or disabled:

- Generate reports as activities complete — To increase performance, reports are not automatically generated as an activity completes. Use this option to automatically generate activity reports.
- Generate IP-XACT files — Disabled by default, you can set this option to allow generation of IP-XACT files. (set through File > Generate Optional Views)
- Include synthesis-related activities — Include activities in the activity list which relate to generation of a gate-level implementation (synthesis) of the component or subsystem. Default: enabled.
- Prefix generated files — If all of the instantiated components support file prefixing, and this option is checked, component files are prefixed with the same prefix as the designs (for example, the component instance name).
- Prefix macros — If all of the instantiated components support macro prefixing, and this option is checked, ‘define <variable> statements are prefixed with the same prefix as the designs (for example, the component instance name). To manually set macro prefixes, use the prefix_macros command. If you discover that the macro prefixes are unwanted, you can remove them using the following file: <workspace>/scratch/<workspace_name>_unprefix.vh.
- Write Register Abstraction Layer (RAL) file — Writes a RAL file for each component. (set through File > Generate Optional Views) Default: disabled
- Write batch script after each assembly-related activity — Write a batch script for recreating the subsystem as each activity in the 'Create RTL' group completes. The batch script is written to /export/batch.tcl. Default: disabled.
- Include component-specific unit-level test activities — Include activities associated with unit-level testing of Synopsys implementation IP. This includes GTECH model generation and individual component simulation. Default: enabled.
- Write remap information and memory map header file — Writes remap information and memory map header files. This option only available in coreAssembler.

To customize these options for all new workspaces created within coreAssembler, use the **Edit > Preferences > Workspace Options** dialog box. Each newly created workspace inherits the options you have specified. These global options are remembered between coreAssembler sessions. For more about preferences, see [“Editing Tool Setup and User Preferences”](#) on page 244

To change these options on an existing workspace, open the workspace and choose Edit > Workspace Options. Check the boxes according to the options you want and OK the dialog.



Note

This menu item does not change the global setting for coreAssembler.

2.6.4.2 Creating a Workspace

Think of your workspace as a private area for developing your own configuration of the subsystem and all the cores contained in it. You can create multiple workspaces for different projects (or for experimentation with various design alternatives).

When you create a workspace, coreAssembler creates both a directory structure for your work and a knowledge database for the workspace. The knowledge database stores the incremental state of your workspace. As you continue through the project design flow, coreAssembler saves your cumulative changes in this workspace knowledge database.

In command line mode, use the `create_workspace` command to create a workspace. See the coreAssembler online reference for details.

To create a workspace in GUI mode:

1. Select **File > New Workspace** on the coreAssembler toolbar or click the “Create a new subsystem (workspace)” link in the coreTools home page.
2. Enter your specifications in the “Create a new Workspace” dialog.
3. Choose whether this new workspace is for a “Implementation” subsystem or a “Testbench” verification environment. For information on creating a testbench workspace, see [“Create Testbench Workspace Activity \(Subsystem Workspace\)”](#) on page 215.
4. Optionally, choose a “Target Language” for your workspace. If specified, newly created identifiers (port and instance names) will be validated against the target language only instead of all languages (Verilog, System Verilog, VHDL), as is the case by default. For example, with a target language of 'Verilog', it is possible to create a port named 'entity' or have different ports named 'A' and 'a' on the same module.
5. Then click OK to create your workspace.

You can also create a new workspace using a pre-defined template by checking the “Using a Template?” box in the dialog and specifying the path to a template file. In this context, a template is a subsystem starting point that consists of one or more partially configured components. The components and configuration values in a template might be “locked” by the creator of the template so they cannot be changed when the template is used by someone else. The template is intended to be used as a starting point in defining a specific implementation of a general subsystem type. For details of template creation, see [“Create Subsystem Template”](#) on page 228.

When coreAssembler finishes creating and setting up your workspace, it automatically displays the Activity List for the workspace. At this point, you can either continue your work by initiating the first activity in the Activity List or save and close your workspace and continue at a later time. For instructions on completing the system specification activities, see [“Creating the RTL View of a Subsystem”](#) on page 53 and [“Creating the Gate-Level Netlist for a Subsystem”](#) on page 81

2.6.4.3 Saving a Workspace

coreAssembler maintains the current state of your workspace (your current configuration of a subsystem) in a knowledge database within the workspace. You can save this information at any time in the design flow. Once you’ve done so, you can close the workspace and quit coreAssembler. You can then restart coreAssembler, re-open your workspace, and continue where you left off.

To save your current workspace, select **File > Save Workspace** from the coreAssembler toolbar.

In command line mode, use the coreAssembler `save_workspace` command to save your workspace, for example:

```
coreAssembler> save_workspace
```

Each time you successfully complete an activity, coreAssembler automatically saves the workspace to disk.

2.6.4.4 Saving a Workspace As

You can also save your current workspace to a different workspace name by selecting **File > Save Workspace As**. When you are using the **Save Workspace As** command, you can require that linked installation files be copied to the duplicate workspace.

2.6.4.5 Closing a Workspace

To close a workspace, select **File > Close Workspace**. In command line mode, use the coreAssembler `close_workspace` command to close your workspace, for example:

```
coreAssembler> close_workspace (Close without saving.)
coreAssembler> close_workspace -save (Save if needed, then close.)
```

2.6.4.6 Opening a Workspace

You can open an existing workspace by specifying the workspace name on the command line when you invoke coreAssembler. For example:

```
% coreAssembler root_directory/Subsystem1
```

You can also open an existing workspace by selecting **File > Open Workspace** or by clicking “Open...” in the coreTools home page. In the dialog box, select the directory containing the workspace, then click OK. Valid workspaces appear in the directory browser as file folders labeled “cA”.

In command line mode, use the coreAssembler `open_workspace` command to open a workspace. For example, to open the workspace `subsystem1`, enter the command:

```
coreAssembler> open_workspace root_directory/subsystem1
```

2.6.4.7 Moving a Workspace

If you want to move your workspace, or move referenced components, you might need to update your workspace to reflect the changes.

- You can use the UNIX “mv” command to move the workspace directory as long as you are on the same network, and the workspace name is not changed.
- If you move the location of your installed IP, you must update workspace links. You can run the “`update_workspace_links -moved_install <dir>`” command on the workspace to resolve broken symbolic links to the new install area. For example, if you move the DesignWare Home tree, the new path is the `$DESIGNWARE_HOME/iip/component/version` directory.
- To replace the links in a workspace with the actual objects (for portability), use the “`update_workspace_links -copy`” command. This is useful for backup and archive purposes, but creates a much larger workspace tree.

The `update_workspace_links` command is used to modify the symbolic links within a workspace.

```
update_workspace_links [-copy] [-moved_install <dir>] [-component <comp>]
[workspace]
```

The `-copy` option copies all files linked to the workspace. This option removes all workspace symbolic links into the installation area. All links from the workspace to the installation area are replaced with copies of the actual files/directories.

The `-moved_install` option is used to indicate that the installation area for the given workspace has been moved from its original location to the directory specified by `<dir>`. Workspace links to the prior installation area are modified to point into `<dir>` instead.

For a full command description, enter from the coreAssembler command line:

```
prompt> man update workspace_links
```

To remove symbolic links from workspace X and replace them with the linked objects:

```
prompt> update_workspace_links -copy X
```

2.6.5 Quitting coreAssembler

To quit coreAssembler, select **File > Exit**. If you have changed your workspace since last saving it, coreAssembler prompts you to save the workspace, as described in [Closing a Workspace](#) on page 50.

To quit coreAssembler in command line mode, use the coreAssembler quit command:

```
coreAssembler> quit
```

2.6.6 Working with the Activity List

When coreAssembler finishes creating and setting up your workspace, it automatically displays the Activity List for the workspace. Click the item label on the Activity List to initiate that activity. The coreAssembler tool displays the specific Activity View where you select or enter data associated with the activity. After entering data for an activity, click the Apply button. This completes the activity and shifts to the Report tab of the corresponding activity.

If you have customized workspace options, not all activities are visible. See [“Setting Workspace Options”](#) on page 48 for workspace activity customization.

When you close your workspace, coreAssembler closes the associated Activity List. Similarly, when you re-open your workspace, coreAssembler re-displays the associated Activity List.

For instructions on completing the system specification activities, see [“Creating the RTL View of a Subsystem”](#) on page 53 and [“Creating the Gate-Level Netlist for a Subsystem”](#) on page 81.

2.6.7 Generating Reports and Optional Views

When you complete an activity, the Report tab at the bottom of the View Pane shows you the reports and views that are available for that activity. To enhance performance, these reports and views are not automatically generated, but complete control is provided through the activity summary page:

Table 2-9 The Activity Summary Page Controls

Control name	Description
generate report	Click the link text to generate the specific report and view the report.
generate view	Click the link text to generate specific optional views (such as IP-XACT)
regenerate report	Discards the previous report, regenerates the report, and views that report
<report_name>	Click the report name (previously generated) to view the specific report
File-->Generate Reports	Performs the same behavior as the File > Generate Reports menu item

Table 2-9 The Activity Summary Page Controls (continued)

Control name	Description
File-->Generate Optional Views	Performs the same behavior as the File > Generate Optional Views menu item
View-->Reports	Performs the same behavior as the View > Reports menu item. Presents a dialog box containing an “index” of all of the generated reports.

2.6.7.1 Generating Multiple Reports

The **File > Generate Reports** menu item presents a dialog box that allows you to generate multiple activity reports from a single action. The “Delete Old Reports” option removes existing reports before regenerating the reports that you choose. There is a check box for each report you can generate, but if the item is grayed out, that report cannot yet be generated (the corresponding activity might not be complete). Click the OK button in the Generate Reports dialog box to generate the selected reports.



Note

- Old reports are not automatically removed when you make design changes, so you can check the box corresponding to “Delete Old Reports” first to avoid viewing stale data.
- Reports for synthesis and verification activities are generated by the corresponding tools, and not handled by the Generate Reports dialog.

2.6.7.2 Generating Multiple Views

The **File > Generate Optional Views** menu items allows you to generate multiple activity design views (such as IP-XACT files) from a single action. There is a check box for each view you can generate, but if the item is grayed out, that view cannot yet be generated (the corresponding activity might not be complete). Click the OK button in the Generate Optional Views dialog box to generate the selected views.



Note

Old views are not automatically removed when you make design changes, so regenerate your optional views after making design changes.

3

Creating the RTL View of a Subsystem

The RTL View defines the subsystem at the RTL level in either Verilog, SystemVerilog, or VHDL. You configure the subsystem components and they are then automatically connected by coreAssembler.

This chapter contains the following sections:

- [“Specify Subsystem”](#) on page 54
- [“Complete Connections”](#) on page 72
- [“Generate Subsystem RTL”](#) on page 75

3.1 Specify Subsystem

In this activity, you define your subsystem. This includes instantiation of components, connecting and configuring interfaces, specifying configuration values, editing existing subsystem components and specifying a subsystem memory map. When you select **Specify Subsystem** in the Activity List, the Activity Window displays the **Add Components** tab and schematic view of the subsystem and shows you the following Activity Dialog Tabs:

- **“Add Components Tab”:**

Select a component to add to the subsystem including a hierarchical component and glue logic, edit the search path for components, export the interfaces, change existing interface connections, set an interface to “unused,” and rename or duplicate a component in the subsystem.

- **“Configure Interfaces Tab”:**

Specify value for interface configuration parameters. These values are automatically propagated to the connected components.

- **“Memory Map Tab”:**

Specify base addresses and required address ranges for each slave in the subsystem.

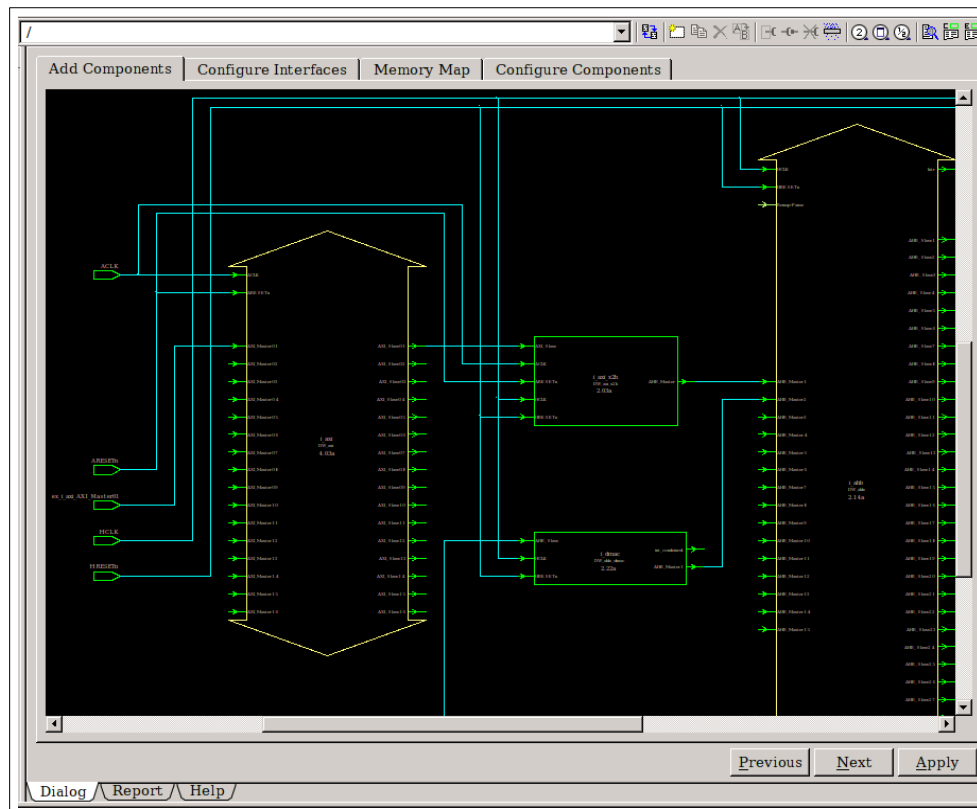
- **“Configure Components Tab”:**

Specify the values for parameters.

3.1.1 Add Components Tab

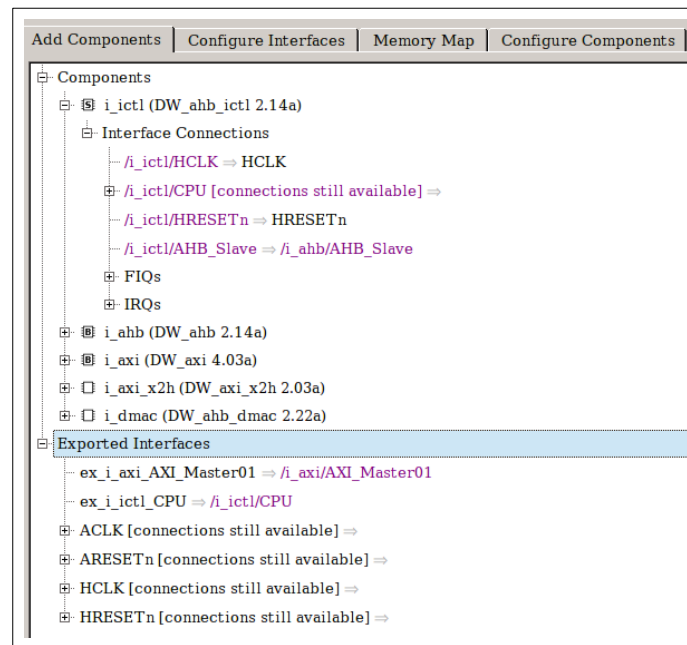
You can access the menu of options for the **Specify Subsystem** activity by selecting **Schematic** from the toolbar menu or by right clicking in the schematic pane. The activity menu selections are discussed individually in the following sections.

Figure 3-1 Schematic View of Subsystem

**Note**

The schematic view shows interface-to-interface connections, not pin-to-pin-connections.

A tree view (textual) showing subsystem contents is also available. To access this view, select **Schematic > Inspect > Toggle View**. This tree view also shows interface-to-interface connections, not pin-to-pin connections. There is a top-level entry in the tree for each component and for exported interfaces. Components that are incomplete or contain errors are shown in red. The icon which appears in front of the instance name of each component indicates the component type. Supported types are bus, master, slave, and monitor.

Figure 3-2 Tree View of a Schematic

You can expand a component entry by clicking on the “+” sign. When expanded, the tree view shows two entries: the component Interface Parameters, and the Interface Connections, listing each component interface and its current connections. For interfaces with multiple connections, the **Schematic > Move > Up/Down** menu items can be used to change component ordering.

As you instantiate components, their respective interfaces are automatically connected if these connections are not ambiguous. You must either explicitly connect ambiguous interfaces (through the Change Connections dialog) or mark the interface unused. Interfaces that require a connection are shown in red. Other interfaces default to the color of the symbol.

The following sections describe each of the available commands for the Add Components dialog view. Note that each of these features is available by right-clicking on an entry in the subsystem tree, right clicking in the schematic pane, or from the main **Schematic** menu.

Many of these features also have a quick-access icon above the schematic/tree views.

3.1.1.1 Add New Component

When you choose **Schematic > Add New Component**, the Add Component Instance to Subsystem dialog appears. The list of available components shows all the installed coreKits and IP-XACT components in the search path. If an installed component you want to use is not in the list, you need to add its location to the search path. For information about editing the search path, see [“Edit Search Path”](#) on page 61.

To add one or more components, click the “Component Name(s)” field; an instance name appears, which you can use or type over to create a custom instance name. Continue clicking on the Component Name(s) field for components you want to add. Then click OK to add the components to the subsystem and make all possible automatic (that is, non-ambiguous) connections. Clicking the “Clear Names” button clears all of the Component Names field so that you can start over entering components. To add multiple instances of the same component, type in the successive instance names in the “Component Name(s)” field separated by a comma or space.

To instantiate components in batch or command line mode, use the following command:

```
instantiate_component -name <instance name> <component name>
```

This command supports a large number of options; for more about these options, see the man page.

If there are ambiguous connections, coreAssembler automatically favors a selected component, instead of posting the Change Connections dialog box. If no component is selected, or multiple ambiguous components are selected, the Change Connections dialog appears to the user to specify how the interfaces are connected. For example, if you have two DW_ahb components and you select one before adding AHB slaves, the slaves are added to the selected AHB instead of displaying the Change Connections dialog box.

You can set the default instance name shown in this field with the TCL procedure `generate_instance_name` which takes the name of the selected component as its input argument. This procedure is typically defined in the system-wide or user setup file `.synopsys_rt.setup`. For example:

```
proc generate_instance_name {compName} {return 'i_$compName'}
```

You can specify the way components are displayed in the Add Component Instance to Subsystem dialog box by using the **Edit > Preferences** menu item, and clicking on the “Schematic” item. In the “Add component dialog view type.” field, you can choose Hierarchical VLVN (vendor/library/name/version) in Flat VLVN or Flat by component name only. The default is Hierarchical VLVN mode.

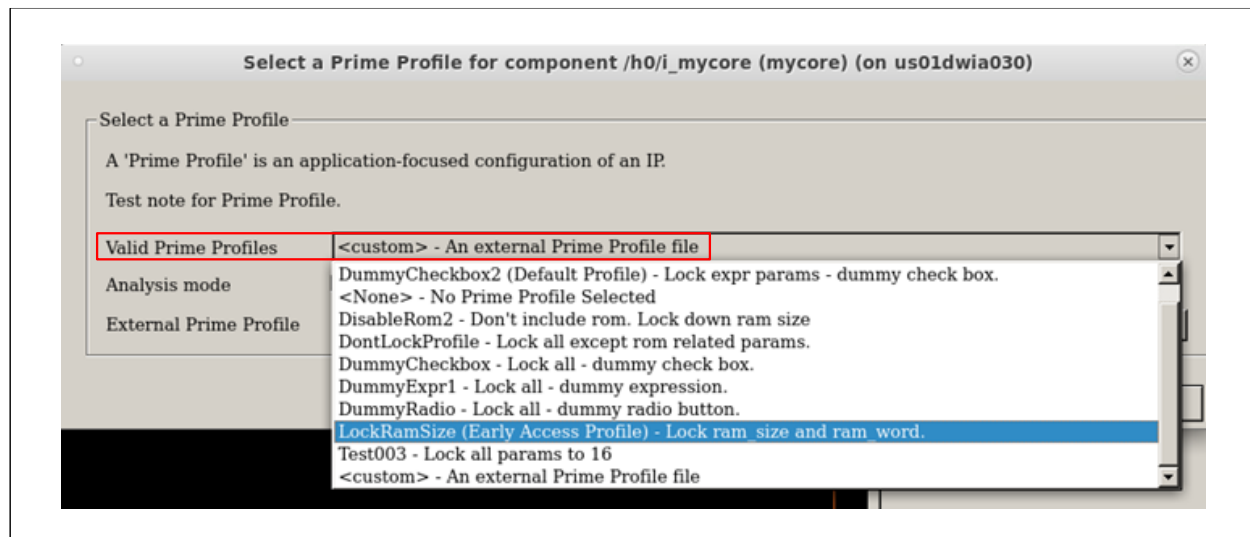


Note

Glue Logic: If you need to add glue logic to your subsystem it can be done using the Add Components dialog or it can be added within the Complete Connections activity using the 'Glue Logic...' button. See “[Complete Connections](#)” on page 72.

If a component has packaged Prime Profiles in its coreKit, adding a new component brings up a Prime Profile selection pop-up. The drop-down menu as shown in [Figure 3-3](#) can be used to select the packaged profiles. Selecting <custom> will enable specifying an external Prime Profile. The analysis mode can also be enabled.

Figure 3-3 Prime Profile Drop-Down Menu



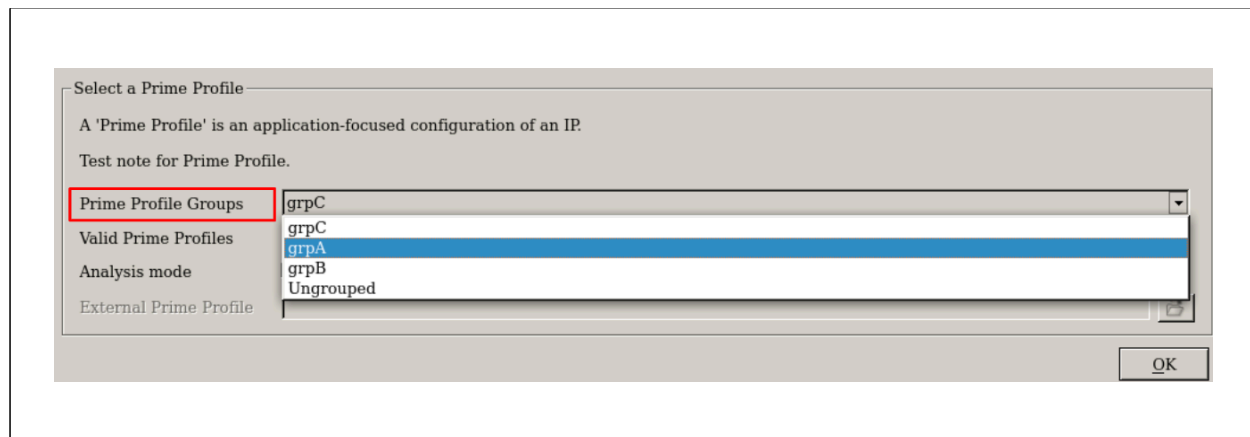
For the command line, the Prime Profiles can be applied with the `instantiate_component` command using `-prime_profile <Prime Profile Tag>` (for packaged Prime Profiles) or `-prime_profile </path/to/Profile.pp.gz>` (for external Prime Profiles). For enabling analysis mode on the Prime Profile application, the `-enable_analysis_mode` switch can be used. If the components have default Prime Profiles with valid licenses, they are applied implicitly if the `-prime_profile` option is not specified.

A component may allow a `<None>` option to appear in the Prime Profile selection drop down. This allows users to skip the Prime Profile application. If default Prime Profiles exist, the `<None>` option appears after them.

Prime Profile Groups facilitate ease of use during GUI led profile selection. A group selection drop-down appears if the profiles were created using the `-group` option. Selecting a group will restrict the packaged prime profiles being displayed. The default group is decided based on the first available default profile or alphabetical order if no defaults exist.

Figure 3-4 shows the Prime Profile Group option in the GUI.

Figure 3-4 Prime Profile Groups



3.1.1.2 New Hierarchical Component

When assembling a large design, the number of components and connections at the top-level can become overwhelming. It is also sometimes easier to design a multi-layer subsystem by creating blocks for each of the layers, and connecting them at a top level. Hierarchical components can help with this process.

To create a hierarchical component (grouping of components), choose the **Schematic > Add Hierarchical Level** item from the design popup menu. Enter the names for the hierarchical design and instance. When you OK this dialog choice, the schematic view changes to the context of the new hierarchical component.

You can navigate into and out of hierarchical components using the **Enter Cell** and **Exit Cell** menu items, or the hierarchy context pulldown box just left of the navigation icons.

If you are using command-line mode, or running a batch script to create your design, the following command is used to create a new hierarchical component:

```
create_hierarchical_component <component_path> <design_name>
```

where component path is a fully qualified hierarchical path, and design_name is optional, if you want a different name than the instance name. This command names the resulting hierarchical component using the “full” component path.

3.1.1.3 Grouping and Ungrouping Components

Another way to create component hierarchy is to use the **Schematic > Group Components** menu item. This menu choice groups all selected components into a new hierarchical component; you provide the name. In this way you can “unclutter” your design by creating sub-blocks of components. Interface connections, manual connections and configuration are preserved. Manual connections are restored each time the Complete Connections activity is entered.

Connections from the “moved” components to those components still in the current level result in exported interfaces from the new hierarchical cell. Manual connections to components outside of the hierarchical cell result in exported pins being created.

If you are using command-line mode or a batch script, the following command is used to group components:

```
group_components -name <heirarchical_name> <list_of_components>
```

To ungroup a grouped hierarchical component (or Hierarchical Level), select the hierarchical component and choose the **Schematic > Ungroup Components** menu item. Ungrouping preserves interface connections, manual connections, and configuration information.

If you are using command-line mode, use this command to ungroup components:

```
ungroup_component <hierarchical_name>
```



Note

The group_components and ungroup_component commands can be used in batch mode, but never appear in a generated batch script. This is because the generated batch script (write_batch_script command) recreates the current design state, not the steps used to get to the current design state.

3.1.1.4 Changing Views

Components represented using the IP-XACT XML specification can contain multiple views which can have different HDL file sets within them. Selection of a desired view therefore impacts the files provided to implementation and verification tools. To change a component view, select the component and then use the **Schematic > Change View** menu item. A dialog lets you choose from available views for that component.

From the command line or in batch mode, use the following command:

```
set_component_view [-instance <instance name>] <view name>
```

The coreAssembler tool default uses the initial 'current' view for a given component. The following command returns the currently selected view for a component:

```
get_component_view [-instance <instance name>]
```

The currently selected view is also shown as a text entry within the corresponding schematic symbol.

3.1.1.5 Import Component

If you want to use a component that has not been packaged using the coreBuilder tool, choose the **Schematic > Import Component** menu item. The Import Component dialog appears. For more details on importing unpackaged components, see “[Unpackaged Component Import](#)” on page 231.

For details on adding IP-XACT packaged components to your component search path, see “[Using IP-XACT Components and Designs](#)” on page 279.

3.1.1.6 Duplicate Component Feature

This command is used to create an additional instance of the specified component. All interface and configuration information is transferred to the additional instance. First, select the component to duplicate and then choose the **Schematic > Duplicate Component** menu item. Accept the default instance name or type in a new name and OK the dialog box.

From the command line or in batch mode, use the following command:

```
duplicate_component <component> <new component> [-share]
```

This command duplicates the instance indicated by <component> and gives it the name <new component>. If <component> is a non-prefixed leaf component, it will be automatically shared. If <component> is a non-prefixed hierarchical component and none of the contained components (recursively) are prefixed, then the hierarchical component can be shared by specifying the -share option. Otherwise, the new hierarchical component <new component> will be a duplicated copy of the original hierarchical component.

3.1.1.7 Remove Item Feature

When you choose the **Schematic > Remove Item**, a confirm box appears. Click the OK button in the box to remove the selected component or exported interface.

3.1.1.8 Replace Component

This command is used to replace one component with another. Although it is typically used to update a given component to a newer version, the components do not need to be exactly the same. The command attempts to preserve all connectivity and configuration information where this is possible. If information cannot be applied to the new component, you are queried (in GUI mode only) regarding how to correlate interface and configuration parameters between the old and new components.

If updating to a newer version of the same component, you are prompted about replacing all instances of the component in the current workspace.

The Replace Component dialog box uses the same component view as the Add Component Instance to Subsystem dialog. As such, you can specify how to view the component list through **Edit > Preferences**. For more information, see “[Add New Component](#)” on page 56.

3.1.1.8.1 Mapping Between Replaced Components

At times you need to replace a component with one that has interfaces that do not map exactly to the original component. Replace Component (replace_component command) supports the entry of mapping information. The mapping information is used to resolve differences between the source and target components.

The mapping information can be provided in three ways:

- By specifying the mapping information directly
- Through the specification of the name of a mapping file
- With the ComponentMappingData attribute which can be set on a design

The last two methods enable component swapping without user intervention, even for components which are not 100% consistent in terms of their interfaces, top-level ports, and configuration parameters.

For information on the mapping file format, see [“Replace Component Mapping File Format”](#) on page 263.

3.1.1.9 Attach Interfaces

The Attach Interfaces command brings up a dialog which shows all current interfaces for the selected component. You can attach or remove all interfaces in this dialog. See [“Unpackaged Component Import”](#) on page 231 for more details.

3.1.1.10 Rename

You can rename a selected component instance or a selected exported interface by selecting **Schematic > Rename**, which brings up the Rename Component or Exported Interface dialog. This command does not work for hierarchical components.

3.1.1.11 Enter and Exit a Cell

To navigate into or out of a hierarchical level (group) use the **Schematic > Enter Cell** and **Exit Cell** menu items. Command operations that use incomplete hierarchical paths/names reference this level of hierarchy (full hierarchical path naming works from any level of hierarchy).

3.1.1.12 Edit Search Path

When you choose **Schematic > Edit Search Path...**, the Edit Component Search Path dialog appears. In this dialog you can change an existing search path or add additional paths. The default search path is \$DESIGNWARE_HOME/iip/latest if the DESIGNWARE_HOME environment variable is set.

If you are instantiating IP-XACT packaged components, this search path is used to find referenced IP-XACT components, bus definitions, abstraction definitions, and designs. For details, see [“Using IP-XACT Components and Designs”](#) on page 279.

The Search Path entries are searched for the installed coreKits directly in the given search path directory. For the IP-XACT files, the directory is searched recursively and any valid IP-XACT *.xml files will be available for use.

3.1.1.13 Load Interface Definitions

When you choose **Schematic > Load Interface Definitions...**, the Load Interface Definitions dialog appears. This dialog lets you specify a path to one or more Interface Definitions files. Clicking OK then executes the file, thus loading the definition.

3.1.1.14 Select Items

For a convenient way to locate and select design items, choose **Schematic > Select Items...**, which displays the Schematic Cell and Interface Selection Tool. This dialog presents a collapsible view of exported interfaces and cells. It includes filters to reduce the set of items in the list.

You can click, Shift-click, and Ctrl-click on cells and exported interfaces and then choose from the following options in the dialog:

- **Select Items:** Selects the chosen cells and exported interfaces
- **Zoom To Fit Items:** Fills the schematic view with the chosen items
- **Clear Selection:** Unselects items that were previously selected

Filters and options are applied when you click either OK or Apply. Clicking Apply allows the dialog to remain displayed.

3.1.1.15 Change Connection

If no component in the subsystem is selected, choosing **Schematic > Change Connection** from the menu bar brings up the Connect Interfaces tree for exported interfaces. If a component in the subsystem is selected, choosing **Schematic > Change Connection** from the menu bar brings up the Connect Interfaces tree for that component.

An alternative way to bring up the Connect Interfaces dialog for a component is to select that component in the schematic and then right click the mouse. This brings up the popup Schematic menu from which you can choose Change Connection to access to Connect Interfaces tree for the selected component.

The Connect Interfaces dialog is interface-specific and shows all current interface connections for the selected interface. You can change all previously specified interface connections in this dialog. A check-box is available to view all possible interface connections, instead of just those at the current level of hierarchy. This enables making interface connections across hierarchical boundaries without manually exporting interfaces up to a common hierarchical level.

3.1.1.16 Export Interface

If you want to make connections to an interface from outside the subsystem or outside of a hierarchical component, you can “export” the interface, which creates ports for each interface port and makes the appropriate connections to the interface being exported. When exporting an interface within a hierarchical component, the interface appears as a pin/port on the hierarchical symbol, and is either automatically connected to the higher-level of hierarchy if an interface “match” is found, or can manually be connected.

Schematic > Export Interface brings up the Export Interface Instance from Subsystem dialog. This dialog is specific to the selected interface. A default name for the new exported interface is given, which you can change. Click **OK** to complete the export process. The names default to match the interface port names unless you change the “Port Name Format” option. The format string defaults to %e_%p to be consistent with prior versions of the tools. The format string supports the following specifiers:

%c	cell instance name
%p	interface instance port name
%e	exported interface name
%i	interface instance name
%d, %D	port directionality (i, o, b for %d) (I, O, B for %D)

This control is also available in batch mode using the -format option to the export_interface command.

3.1.1.17 Export All Unconnected Interfaces

All interfaces must be either directly connected, exported, or set unused, before your subsystem is validated. A quick way to handle interface that have external connections is to use the **Schematic > Export All Unconnected Interfaces** menu item. These interfaces are all automatically exported using default naming. If later you want to change to an internal connection, you can do so with the **Schematic > Change Connection** menu item.

3.1.1.18 Set Unused

Schematic > Set Unused sets the selected interfaces to Unused. An interface should be marked unused when it is not required for a connection. All interfaces must either be connected to another interface, exported, or explicitly marked unused. An unused interface can be re-used by selecting **Schematic > Change Connection**.

Schematic > Mark All Unconnected Interfaces As Unused applies the “Set Unused” operation to all remaining unconnected interfaces in the design.

3.1.1.19 Detach Interface

Used to remove an interface previously created using “Attach Interface.”

3.1.1.20 Port Associations

Schematic > Port Associations brings up the Associate and Setup Interface Ports dialog. This dialog is interface-specific and is enabled only when you have selected an exported interface or an interface attached to an imported component. It shows all current port associations (links, directionality, number of bits, status, options, parameters, bus alignment, and defaults) for the selected interface. You can change all previously specified port associations in this dialog. The primary use of this button is to update top-level port names for exported interfaces. See “[Unpackaged Component Import](#)” for more details.

3.1.1.21 Filter

The **Schematic > Filter** command can be used to reduce clutter in the schematic representation of the subsystem by hiding certain connections and cells. Connections can still be seen in the subsystem tree view or by bringing up the component inspector using **Schematic > Inspect**. A check box allows filtering schematic components without interfaces.

3.1.1.22 Propagate / Remove Memory Map

The **Propagate Memory Map** command propagates the memory maps visible from an exported slave interface to that slave. The **Remove Memory Map** command remove a memory map.

3.1.1.23 Inspect

The **Schematic > Inspect** command allows you to toggle between the schematic view and the tree view of a subsystem, inspect the configuration of a single component, look at the entire list of components, or look at the list of exported interfaces. Selecting a component in the tree view also highlights it in the schematic view.

3.1.1.24 Open/Close All (Tree View Only)

The tree view of your design expands and collapses to show more or less detail. The **Popup > Open All**, **Close All** and **Close All Components** menu items are useful to quickly collapse or expand the tree views.

3.1.1.25 Moving View Items (Tree View Only)

The tree view lists components and interfaces in a logical order. You can re-arrange the tree display and order these interfaces and components any way you want. Select a tree entry and use the **Popup > Move > Up** to move it up the tree, and use **Move > Down** to move the entry down the tree.

You cannot re-order the Interface Parameters list.

3.1.1.26 Zooming Schematic Magnification (Schematic View Only)

The **Schematic > Zoom > [type]** item lets you **Zoom in 2x**, **Zoom out 2x**, or **Zoom full** to see the entire design.

3.1.2 Configure Components Tab

The Configure Components tab is component-specific and provides a hierarchical tree menu for all components in your subsystem, which expands/collapses to reveal/hide configuration dialogs. You need to review and update component parameters as needed for the proper configuration of the components in your subsystem. Some component configuration parameters are set automatically based on values of interface parameters. These parameters appear disabled in the component configuration dialogs. For details of specific component configuration requirements, see the component databook or switch to the Help tab to see a table of parameter descriptions.

The RTL for each of the components is generated at the completion of the Specify Subsystem activity. (Top-level RTL is generated later during “[Generate Subsystem RTL](#)” on page 75.)

When writing out the RTL, you can choose to automatically add file and macro prefixes for all leaf-level components (that originated in coreKits) such that the prefix is the leaf-level instance name. This option must be set prior to completing the Specify Subsystem activity (for more information, see “[Setting Workspace Options](#)” on page 48). Note that this option is only enabled if all components in the subsystem support file name prefixing.

3.1.3 Configure Interfaces Tab

The Configure Interfaces Tab allows you to view and configure values for interface parameters. These values control design specification such as address width, data bus width, interrupt polarity, and presence of special signals such as Burst or Lock. The interface configuration must be specified before completing the Specify Subsystem activity in order to properly create component-to-component connections (needed for component-level RTL generation).

Interface configuration values are specified using a tree control where the tree entries are connected 'provider' interfaces. Selecting an interface shows a spreadsheet containing all relevant parameters. The first column shows the parameters that can be set on the provider interfaces and the remaining columns (one per connected consumer interface) shows the parameters that can be set on the indicated consumer interface.

3.1.4 Memory Map Tab

The Memory Map dialog contains a table for specifying the base address and required address range for each slave connected to a common bus fabric. There is a single column for the address range and one or more columns for the base address depending on the remap modes supported by the bus fabric to which the slaves are connected.

Upon completion of the activity, you can generate multiple reports and memory map related header files within the “Report” tab. These are covered in the sections following this one on the actual specification of the memory map.

The dialog box show a spreadsheet for each of the slaves.

Figure 3-5 Memory Map Dialog

Name	Interface	Address Range	BaseAddress (Default)	BaseAddress (Normal)	BaseAddress (Boot)
/i_ahb/AHB_Slave					
	/i_ahb/Arbiter	0x1000		0x14000	
	/i_dmac/AHB_Slave	0x1000		0x15000	
	/i_ictl/AHB_Slave	0x1000		0x16000	
	/i_apb_sys/i_apb/AHB_Slave	0x4000		0x10000	
	/ex_i_ahb_AHB_Slave	0x1000		0x17000	
/i_apb_sys/i_apb/APB_Slave					
	/i_apb_sys/i_timers/APB_Slave	0x1000	0x10000		
	/i_apb_sys/i_uart_1/APB_Slave	0x1000	0x11000		
	/i_apb_sys/i_uart_2/APB_Slave	0x1000	0x12000		
	/APB_Slave3	0x1000	0x13000		
/i_axi/AXI_Slave					
	/i_axi_x2h/AXI_Slave	0x8000		0x10000	

The recommended method to create a valid memory map is:

1. Click the Initialize All button at the top of the dialog.
2. Enter a Base Address value for the entire map in the Get Starting BaseAddress dialog, and then OK the dialog. The system populates the map with auto-generated values.
3. Modify any values in the map as necessary to meet your design needs.
4. Click the Validate All button to check that you still have a valid map (that no fields overlap).

3.1.4.1 Memory Map Spreadsheet Description

The Memory Map dialog contains a separate spreadsheet for each set of slaves containing memory maps and connected to a common bus. For most slaves, only the base address is specified. However, for exported interfaces, bus bridges, and IP-XACT components with user-defined address ranges, the 'address range' is also specified. This indicates the range of addresses required for this component, starting at the specified base address. If the bus to which the components are connected supports multiple address modes, then there is a separate column for specifying the base address in each mode. For example, the Synopsys AHB bus fabric component supports both 'normal' mode addressing and 'boot' mode addressing. Therefore two base addresses can be specified for slaves connected to this bus, when boot mode addressing is enabled (that is, remap support is enabled on the bus component). For IP-XACT components, this corresponds to the remap states defined within a mirrored slave interface.

Certain components can be defined to have disjoint blocks of memory. In that case the same component can appear in the spreadsheet multiple times. In this case the name of the interface shown in the first column includes information to indicate which address block it applies to. An example of this is the Synopsys memory controller which contains a block of memory for its internal registers and a separate block representing the actual memory accessed through the controller.

Exported interfaces do not have a memory map by default, so one is created automatically.

The dialog contains the "Initialize All" button. Clicking this button invokes the automatic address map initialization code that exists in the previous version (Z-2007.06) of coreAssembler. You are asked to provide the "BaseAddress" for the Memory Map. This functionality works for any subsystem (that is, no restriction about bus type).

The spreadsheet sorts first by the 'Bus' column and then by one of the “Base Address” columns. The **Memory Map** dialog also contains several buttons:

- **Initialize All** - clicking this button invokes automatic address map initialization. The ‘Get Starting BaseAddress’ dialog prompts you for a Memory Map BaseAddress
- **Sort** - enabled when exactly one “Base Address” column is selected, causing the table to be sorted based on the values of the base address in that column.
- **Validate All** - causes the base address and address ranges to be validated. Checking is performed to make sure that for any given bus there are no slaves with overlapping memory regions. In addition, for instances of Synopsys IP DW_axi, DW_ahb, and DW_apb components, additional checking is done to ensure that the specified start address is on a legal boundary. If this check can be expanded to all buses of these types that is done as well. Validation is also run automatically when the Apply button for the activity is selected. Robust error messages let you know when violations occur.

In certain cases the base address can be calculated from other base addresses (and associated address ranges). For example, when you have a simple AHB/APB subsystem, then the base address and address range of the APB bridge on the AHB can be calculated from the base addresses and ranges of the slaves on the slave side of the APB bus. In these cases the base address columns are filled in automatically. This applies to most bridge components provided that there is no feedback loop within the connectivity of the bus fabric components. If there is a path from the master side of the bridge back to the slave side, the base address and address range values for the slave side of the bridge needs to be provided, although auto-initialization might be possible.

Start and end address values and other address related parameters such as **# of regions**, and slave visibility modes specified (or inferred from information) in this activity are automatically propagated to instances of Synopsys IP components DW_axi, DW_ahb, and DW_apb.

The batch interface for this activity is enabled with the following command:

`set_base_address` - this command can be used to set the base address of a given slave. For command syntax, see the coreAssembler “man” page.

3.1.5 Reporting and Header File Generation

A number of different header files and reports can be manually generated upon completion of the Specify Subsystem activity. Links to generate and view all of the optional header files and reports are available from the “Report” view. Control over generating memory map header files is also available in the **File > Generate Optional Views** dialog.

3.1.5.1 Header File Generation

Header files are generated in C and Verilog formats for the register map for each bus (IP-XACT channel) for each remap state supported by that bus. For example, a simple subsystem containing the following:

```
DW_ahb ' i_ahb (configured to support remapping)
DW_ahb_ictl ' i_ictl
DW_apb ' i_apb
DW_apb_uart ' i_uart
DW_apb_timers ' i_timers
```

Causes the following files to be generated in the export/headers directory within the workspace:

```
i_ahb_normal_map.v, i_ahb_normal_map.vhd, i_ahb_normal_map.h
```

```
i_ahb_boot_map.v, i_ahb_boot_map.vhd, i_ahb_boot_map.h
i_apb_map.v, i_apb_map.vhd, i_apb_map.h
```

The header file content is such as the following:

```
/* Header file for i_ahb in 'normal' mode */
`ifndef i_ahb_normal_map_v
    `define i_ahb_normal_map_v
    `define i_ictl_BaseAddress 32'h0x1000
    `define i_ictl_AddressRange 32'h0x100
    `define i_apb_BaseAddress 32'h0x1100
    `define i_apb_AddressRange 32'h0x100
`endif /* i_ahb_normal_map_v */
```

If the component contains more than one addressable slave interface, the interface name is also included within the name of the ``define` (for example, `i_ictl_AHB_Slave_BaseAddress`). Similarly, if a memory map contains multiple regions, then the name of the region is appended to the name of the ``define`.

Details about specific registers are available from within the export directory of the specific component (for example `<workspace>/components/i_uart/export`). The detailed files for a specific component contain information about individual registers and register fields, including size, offset, reset value, and so on.

3.1.5.2 Memory Map Report Generation

Two primary reports can be generated from the “Reports” tab. The first is the **base address** report. It reflects the information specified within the Memory Map dialog. The report contains a table for each bus (IP-XACT channel) in the subsystem. Within a given table there is a row for each disjoint memory region of each slave. For each row, the component instance name, slave interface name, memory map name, base address (per remap mode) and address range are specified. If a given table contains one or more slaves containing multiple disjoint address regions (blocks or banks of memory with unrelated base addresses), then the table also includes a column containing the hierarchical name of the address block associated with the given row.

The second report is the **register details** report. This report can be generated for each component containing a memory map within the subsystem, provided the memory map is accessible through a connected slave interface. The register details report for a given component contains a separate table for each unique address block defined for the component. For each table, header information indicates the remap states and associated base addresses for the address map. The table contain a row for each register defined in the address block, sorted by offset value. Columns in the table contain the address offset, register name, description, size, reset value, and access type. If a given register has fields defined for it, the register name is a link to a report containing field level information (same columns) for the given register.

The previous details apply to coreAssembler, but the same reports can be generated in coreConsultant and coreBuilder as well. In those tools, base address information is not reported, and in coreBuilder, columns are added to indicate under which conditions conditional registers appear.

3.1.5.3 Memory Map Propagation

To help you generate a unified memory map for top-level exported interfaces seen as slaves from outside of the subsystem, the **Propagate Memory Map** item in the schematic menu is available when a valid exported interface is selected. A corresponding command is also available:

```
propagate_memory_map -interface <interface_name> -map <memory_map_name>
```

The generated memory map contains an address block for each address block seen in each memory map visible by tracing interface connections from the exported interface. Register offsets within each address block are unchanged. Base addresses of the blocks themselves are adjusted based on the start address of the slave containing the memory map. It is not legal to propagate a memory map to an exported interface that is already associated with a memory map.

Existing (propagated) memory maps can be removed using **Remove Memory Map** item in the schematic menu (valid when an exported interface is selected that has previously been associated with a memory map), or using the following command:

```
remove_memory_map -interface <interface name>
```

This removes the associated memory map and the link from the interface to the memory map.

3.1.6 Completing the Specify Subsystem Definition

Once you have successfully added all components and configured components/interfaces, and specified the memory map, click the **Apply** button to complete the **Specify Subsystem** activity. By default, this activity does not complete unless the subsystem is valid (but you can set a preference to save an invalid subsystem). A valid subsystem has each interface connected, exported, or explicitly marked unused. In addition, any requirements as to number of interface connections must be met. Finally, parameter values on interfaces must be consistent with any read-only values for associated component parameter values. There can also be custom checks defined using the `add_verify_subsystem_hook` command. See the man page for `add_verify_subsystem_hook` for details.

3.1.7 Reporting and Header File Generation

A number of different header files and reports can be manually generated upon completion of the **Specify Subsystem** activity. Links to generate and view all of the optional header files and reports are available from the “Report” view. Control over generating memory map header files and RAL files is also available in the **File > Generate Optional Views** dialog. The following reports are available:

Table 3-1 Reports and Optional Views

Report Name	Description
Activity Specific Reports:	
Subsystem Contents	This report documents each component instantiated in the subsystem and the interface level connectivity of those components.
AXI Slave Visibility	This report documents the AXI slave visibility in the subsystem.
Memory Map	This report documents the address ranges assigned to each slave in the subsystem.
Component Source Code	This report documents the source files of all leaf level components in the subsystem.
Component Configuration	This report documents component-level configuration choices and includes parameter descriptions and default values.

Table 3-1 Reports and Optional Views (continued)

Report Name	Description
Component Registers	This report documents component-level registers and register fields and documents key characteristics of the registers and fields such as offset address, size, access modes, and so on.
I/O	This report include a symbolic view of the component (or subsystem) and tables documenting the ports of the component (or subsystem) and key attributes of those ports such as size, directionality, and so on.
Area Estimate	This report documents estimated area requirements for the given configuration of the current components. Area estimation is based on technology and configuration.
Activity Specific Optional Views:	
Memory Map Headers	Generate header files for each bus fabric component defining the start address and range of each slave visible from the bus fabric component. Separate files are generated for each remap mode supported by the bus fabric.
Component Register Headers	Generate header files containing definitions of key values from the memory map of each component.
Component RAL	Generate RAL files for each component. You can optionally generate the UVM variant of the RAL files using the write_ral_file command with the -uvm option.

3.1.7.1 Header File Generation

Header files are generated in C and Verilog formats for the register map for each bus (IP-XACT channel) for each remap state supported by that bus. For example, a simple subsystem containing the following:

```
DW_ahb ' i_ahb (configured to support remapping)
DW_ahb_ictl ' i_ictl
DW_apb ' i_apb
DW_apb_uart ' i_uart
DW_apb_timers ' i_timers
```

Causes the following files to be generated in the export/headers directory within the workspace:

```
i_ahb_normal_map.v, i_ahb_normal_map.vhd, i_ahb_normal_map.h
i_ahb_boot_map.v, i_ahb_boot_map.vhd, i_ahb_boot_map.h
i_apb_map.v, i_apb_map.vhd, i_apb_map.h
```

The header file content such as the following:

```
/* Header file for i_ahb in 'normal' mode */
`ifndef i_ahb_normal_map_v
`define i_ahb_normal_map_v
`define i_ictl_BaseAddress 32'h0x1000
`define i_ictl_AddressRange 32'h0x100
```

```

`define i_apb_BaseAddress 32'h0x1100
`define i_apb_AddressRange 32'h0x100
`endif /* i_ahb_normal_map_v */

```

If the component contains more than one addressable slave interface, then the interface name is also included within the name of the ``define` (for example, `i_ictl_AHB_Slave_BaseAddress`). Similarly, if a memory map contains multiple regions, then the name of the region is appended to the name of the ``define`.

Details about specific registers are available from within the export directory of the specific component (for example `<workspace>/components/i_uart/export`). The detailed files for a specific component contain information about individual registers and register fields, including size, offset, reset value, and so on.

3.1.7.2 Memory Map Report Generation

Two primary reports can be generated from the “Reports” tab. The first is the **base address** report. It reflects the information specified within the Memory Map dialog. The report contains a table for each bus (IP-XACT channel) in the subsystem. Within a given table there is a row for each disjoint memory region of each slave. For each row, the component instance name, slave interface name, memory map name, base address (per remap mode) and address range are specified. If a given table contains one or more slaves containing multiple disjoint address regions (blocks or banks of memory with unrelated base addresses), then the table also includes a column containing the hierarchical name of the address block associated with the given row.

The second report is the **register details** report. This report can be generated for each component containing a memory map within the subsystem, provided the memory map is accessible through a connected slave interface. The register details report for a given component contains a separate table for each unique address block defined for the component. For each table, header information indicates the remap states and associated base addresses for the address map. The table contains a row for each register defined in the address block, sorted by offset value. Columns in the table contain the address offset, register name, description, size, reset value, and access type. If a given register has fields defined for it, the register name is a link to a report containing field level information (same columns) for the given register.

The previous details apply to coreAssembler, but the same reports can be generated in coreConsultant and coreBuilder as well. In those tools, base address information is not reported, and in coreBuilder, columns are added to indicate under which conditions conditional registers appear.

3.1.7.3 Area Estimate Report

After you have completed the **Specify Subsystem** activity, you can generate an Area Estimate report. This report is generated by collecting area data from the various IP in your design, if available. For information on how this data is added to a component in coreBuilder, see “Area Estimation Modeling” in the *coreBuilder User Guide*.

The Area Estimate report contains a report for each component in the design subsystem. In addition, if at least one technology label is common across all design components, a subsystem-level report is generated. This report includes links to individual component reports, and provides a subsystem-level value for each common technology label. [Table 3-3](#) shows a typical Area Estimate report:

Table 3-2 Area Estimate Report

Area	Technology	Match Weight Percentage	Matching Parameters
10072	090	18.9	2/5
11111	090	17.0	2/4
8000	090	15.4	1/4

Table 3-3 Area Estimate Detail Report — “Area: 8000, Technology: 090, Match Percentage: 15.4”

Parameter	Weight	Estimation Value	Actual Value
P1	0	1	2
P2	7	7	1
P3		2	0

If no area estimation numbers are contained in the components, the report is empty, and a message informs you of this lack of data.

3.2 Complete Connections

You invoke this activity to connect any remaining pins that were not automatically connected as part of an interface. Unconnected input pins can be connected to unconnected output pins, tied off to a constant value, or exported from the subsystem (that is, connected to an automatically created input port of the subsystem). Unconnected output pins can be connected to an existing input pin, explicitly marked as unconnected (open), or exported from the subsystem.

The text field box that is below each spreadsheet contains the Descriptions (if any) of the currently selected pin and any connections it might have. If a signal needs to be inverted, use “Glue” logic to make the connection, as described later.

The **Filters** field for both Sources and Loads allows you to limit the signals/ports that are shown. You can check whether to:

- Show connected ports and pins
- Show pins with default connections defined
- Show the hierarchy below the current context -- If checked, all pins at or below the current hierarchy are shown instead of just those at the current level of hierarchy. This enables making connections between levels of hierarchy without manually exporting pins to enable making connections at the same hierarchy level.

You can also specify a Filter string to match within the signal/port name.

After making a selection from the “Available Sources” or “Available Loads,” you can view options in the popup menu:

- “Show Connections” shows connection information and pin descriptions.
- “Open All” and “Close All” allow you to globally expand or collapse hierarchy.

To make connections in the dialog, select the pins and ports to be connected and then select the button corresponding to the type of connection you want to make: Connect, Export, Open, 0, 1, hex value, or “don’t care.” If you select the **Export** button, a secondary dialog appears so you can specify the names of the top-level ports to be created. Use the inverter “Glue” logic if a connection needs to be inverted.

All pins and ports must be connected to something or explicitly tied off in the Complete Connections activity. However, certain pins can be left unprocessed provided the core developer has indicated how these pins are handled. To remove the pins you do not have to process, go to the Filters Out section of the “Manual Connect” tab and uncheck (default) “Show pins with default connections defined” The pins that are removed with this filter are automatically connected when you click the Apply button for this activity.

The **-->Default<--** button connects all selected sources and loads to their default if they have one. The “Default” is defined by the “IfUnconnected” attribute.

The **“Tie low-->”** and **“Tie high-->”** buttons operate on loads, and allow you to tie selected loads to either a ‘0’ or ‘1’, respectively. The **“Tie to hex value... -->”** provides a field where you can enter a multi-bit value in either hex or binary to tie a multi-signal load. The **“Don’t Care -->”** button connects selected signals to a value of ‘X’.

The **<--Glue...-->** button brings up a dialog for creation of a glue logic gate. If no pins are selected, any gate can be instantiated and its pins then show up for connection. If pins are selected, then the available gates are potentially restricted based on the width and number of selected pins. In these cases, the gate is created and then the connections are made. The dialog box lets you choose from the following generic gate types: AND, NAND, OR, NOR, buffer, inverter, XOR and multiplexer.

The “**Optional signal name and connection comment**” fields allow you to specify the **Signal Name** and a **Connection Comment** for a connection. The description is written into the workspace along with the signal declaration, and also to the IP-XACT design file.

See the component databooks for the functionality of each of the pins requiring manual connection to determine how to connect individual pins. The simplest approach to completing this process is to export all of the unconnected pins and then connect them when the subsystem is instantiated.

The **Manual Disconnect** tab can be used to undo connections. This is not typically needed, but can be used to undo a mistake or to make other manual connection updates. You can filter by component, filter string, show auto-connections, or keep disconnected ports; you must click “Update Connection List” after making all filter changes in order to see the updated list.

A **Remove Glue Logic** button on the Manual Disconnect tab view allows you to remove any Glue Logic that you added in the Complete Connections activity.

The **Manual Connections Schematic** tab gives you a graphical interface in which to make connections. With this view, you select appropriate ports, and then use the popup menu to make connections. You can also change views and filters using the popup menu. See “[Manually Connect Schematic](#)” for details.

Clicking the **Apply** button automatically makes any default connections/disconnections and then, provided all required connections are complete, and views the “Report” tab where you can generate and shows the connections report.

In hierarchical subsystems, you can use the Component drop-down list to navigate between hierarchies while making connections. Use the “Go” button at the top to automatically navigate to the next level of hierarchy requiring connections to be made.

3.2.1 Manually Connect Schematic

The existing Manual Connections activity includes an alternate “Schematic” view in which manual connections can be made. This view is based on a schematic view of the pins and ports currently selected for viewing. The filtering mechanisms available in the Manual Connections dialog (tab) are utilized within the schematic reduce schematic clutter. Follow these steps to make manual “schematic” connections.

1. Set the appropriate filters from the “Manual Connect” tab. You can also choose filter options from the popup schematic menu.
2. Click the “Manual Connect Schematic” tab. The manual connection schematic view opens.
3. Select a pin/port you want to make connection to. Use CTRL-click to add additional pins/ports to the selection group (for making port to port connections). Also, CTRL-click a selected port unselects that port from the selection group.
4. Click the Schematic menu in the menu bar. Choose from the following connection options:
 - a. Export - an Export Connections dialog allows you to control connection name and parameters.
 - b. Connect | Open - make a connection or open (unconnect) an existing connection
 - c. Connect 0 | 1 | X - connect to one of these sources (inputs only)
 - d. Connect hex value - make multi-signal connection to a hex value



Note

If you are selecting/connecting only one item at a time, you can right-click that item to select it and display the Schematic menu with a single click.

3.2.2 Hierarchical Connection Support

Hierarchical components are shown in the Manual Connection Schematic if there are components beneath them that have missing connections.

Example: Consider a component U1 with interface 'Slave' and port 'clkEnable' and component U2 with interface 'Master' and port 'clkEnableOut'. If component U1 is instantiated inside a hierarchical component named H1 and component U2 is instantiated inside a hierarchical component named H2, then you can connect the two interfaces and pins as shown.

```
connect_interface -hier -from_component H1/U1 -from_interface Slave \  
                 -to_component H2/U2 -to_interface Master  
create_connection -hier {H1/U1/clkEnable H2/U2/clkEnableOut}
```

3.2.3 Complete Connections Accelerator Keys

Accelerator “hot” keys are available within the Complete Connections environment to speed up the process of making connections. For example, <alt>-r exports selected signals. The <alt> key together with the underlined letter on the button text performs the “hot” key function.

3.3 Generate Subsystem RTL

This section describes the procedure and outputs of the Generate Subsystem RTL activity.



Note

This activity creates top-level RTL. The RTL for each of the individual components is generated at the completion of the Specify Subsystem activity.

To create the top-level structural RTL code for the subsystem, click the Generate Subsystem RTL task in the Activity List. [Table 3-4](#) describes the Generate Subsystem RTL parameters.

Table 3-4 Generate Subsystem RTL Parameters

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Output language (LanguageChoice)	Language to be used when generating the top-level designs of the subsystem. The possible values are: VHDL, Verilog (default), SystemVerilog, and Verilog-1995.
Specify VHDL Libraries and VLNv Data (VHDLDesignLibrary)	This spreadsheet lets you specify a VHDL library for each hierarchical design in your subsystem, and assign a Vendor, Library, Name and Version (VLNV) string to each component in the hierarchy. The default for each design is to use the library “work.” The default for Vendor and Library is “Third Party,” and the default Version is 1.0. Access is not available for Verilog output.
Preferred VHDL logic type (PreferredDataType)	Logic data type for top-level ports and Verilog-to-VHDL port conversions when creating a VHDL subsystem. This field is active when you choose VHDL as the output language. The possible values are: std_logic (default) and std_ulogic
Include testbench probe signals (IncludeTestbenchProbeSignals)	Generates an additional VHDL package that contains definitions of global signals for all design pins to support connecting monitor probes in a VHDL testbench. This field is active when you choose VHDL as the output language. If this box is not checked, in order to monitor signals in the testbench, each signal must be routed to the top level of the design. The possible values are: false (default) and true
Write System Verilog ‘include files (IncludeSVIncludeFiles)	When generating System Verilog HDL, writes include files into the files. The possible values are: false (default) and true
Use compact instance syntax (UseCompactInstanceSyntax)	When generating System Verilog HDL, uses the compact instance syntax. This excludes the signal name when the pin name and signal name are the same. The possible values are: false and true (default)

Table 3-4 Generate Subsystem RTL Parameters (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Include port descriptions in RTL (IncludePortDescriptionsInRTL)	Writes port descriptions into the generated RTL where the ports are defined (VHCL entity or Verilog module). Descriptions are specified by setting the Description attribute on the port. The possible values are: false (default) and true
Signal name format (SignalNameFormat)	Defines the format for automatically created signal names. The default format %i_%p translates to <instance name>_<port name>, as taken from the signal driver. The default format can be overridden through a preference on the 'assembly' page of the Edit > Preferences dialog. For more about preferences, see “Editing Tool Setup and User Preferences” on page 244. The format string can have additional characters in it, and supports the following substitutions: %c: Component instance name %l: Use load component instead of driver (single load only) %d: Component port direction ('i', 'o', or 'b') %D: Component port direction ('I', 'O', or 'B')
Use unique logic nets (UseUniqueLogicNets)	Use a unique logic net for each logical constant connection. The possible values are: false (default) and true
Use nets for open output pins (UseNetsForOpenOutputPins)	Use a unique, floating net for each open output pin instead of an empty connection. The possible values are: false (default) and true
Use assign for initialization (AssignForInitialization)	Use separate assignment statement for initialization of constant nets. The possible values are: false (default) and true Disabled for VHDL output.
Allow reading of output ports (AllowReadingOfOutputPorts)	Generated netlist may utilize reading of output ports to help eliminate assign statements. This will occur when a pin is driving 1 or more other pins and a single output port. If enabled, the output port name is used as the wire connected to the pins and port. Otherwise, an assign statement would be utilized to connect to the output port. The possible values are: false (default) and true
Don't Write Synopsys Header (DontWriteHeader)	Do not write the Synopsys header to Subsystem RTL files. The possible values are: false (default) and true

Table 3-4 Generate Subsystem RTL Parameters (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Allow Aggregate Assignments (AllowAggregateAssignments)	Allow aggregate assignments to pins to reduce use of assign statements. The possible values are: false and true (default)
Don't tie open inputs to logic X (DontTieOffInputs)	Do not tie open inputs to logic X. The possible values are: false (default) and true
Define Writes for all Ports (DefineWiresForAllPorts)	Define wires for all ports even if not strictly required. The possible values are: false (default) and true
Write bit vector literals in binary format (DelimitBitVector)	When generating a netlist in Verilog or SystemVerilog, bit vector literals can be reformatted to binary. When enabled, bit vector literals are written out in groups of four bits, separated by '_' as a delimiter. The possible values are: false (default) and true
Set the comment on port to occur Inline (ExportConnectionCommentsInline)	A comment on a port usually appears before the port assignment. By setting ExportConnectionCommentsInline to true, the comment appears in-line with the port assignment. The possible values are: false (default) and true
User comment inserted into each generated file (UserComment)	Comment text that will be inserted into the header of each of the generated subsystem-level RTL files.

When you click the Apply button in this dialog, coreAssembler writes the top-level HDL code to the file *workspace/src/design.v[hd]* where *workspace* is the current workspace name and *design* is the subsystem top-level design name. The Activity View pane switches to the “Report” tab that allows you to generate and view the Generate Subsystem RTL Report listing the files that coreAssembler has generated.

This activity generates the following files in the “src” directory:

For VHDL:

- *design_pkg.vhd* — package definitions.
- *design_comp.vhd* — component declaration for the subsystem design.
- *design.lst* — source files for the entire subsystem in proper analysis order.

For Verilog:

- *design.v* — RTL for the subsystem.
- *design.lst* — source files for the entire subsystem in proper analysis order.

For SystemVerilog:

- *design.sv* — SystemVerilog wrapper for the subsystem.
- *design.lst* — source files for the entire subsystem in proper analysis order.

For IP-XACT (in the export/SPIRIT directory):

The following IP-XACT XML files are not automatically generated, but can be manually generated as optional views from the “Report” tab or in the File > Generate Optional Views menu.

design.xml – IP-XACT description of the subsystem

design/design_comp.xml – IP-XACT description of components in the subsystem

design/comp_dir/src/.v* – various Verilog descriptions of components in the subsystem

**Note**

Hierarchical subsystems have one set of files for each hierarchical level.

Next, complete the gate-level view activities described in [Creating the Gate-Level Netlist for a Subsystem](#).

3.3.1 User-Defined Netlisting

You can register a user-defined netlister for use in Generate Subsystem RTL using the `register_netlister` command.

```
register_netlister -language <language> [-callback <proc
name>][-unregister]
```

This command can be utilized to register a user-defined netlister within coreAssembler. When a netlister is registered and associated with a particular language, then the language will show up as an additional choice in the list of languages available in the Generate Subsystem RTL activity. If that language is then selected, the user-defined netlister runs when the activity is completed (by clicking 'Apply' in the GUI or when the activity is completed in batch mode with the `autocomplete_activity` command).

You must define the netlister before it is invoked. The most common way is through the definition of a plug-in which defines the command.

The netlister is run exactly once for each hierarchical design within the subsystem which requires a netlist to be generated. It is invoked without any arguments and returns a list of the files it generates. The returned list can be either a simple list of files (that is, {<file 1> <file 2> ...}) or it can be a list of two element sub-lists where the first sub-element is the file name and the second sub-element is a brief abstract describing the file contents. If provided, the abstract information is included in the report for the Generate Subsystem RTL activity.

The outer wrapper of the netlister must be written in TCL but the core of the functionality can be written in either TCL or as an IP-XACT tight generator. For a tight generator, the TCL wrapper directly invokes the generator using the `invoke_generator` command. The following example shows how to connect a user-defined netlister into coreAssembler:

```
proc mySystemCnetlister {} {
    invoke_generator mySystemCgenerator
    set files [glob [get_logical_dir src] *.*]
    return $files
}
register_netlister -language SystemC -callback mySystemCnetlister
```

3.3.2 User-Defined Packages for Components

When generating the RTL for a subsystem, coreAssembler currently creates a file containing a unique VHDL package for each component contained in each level of hierarchy. In certain cases, customers can have their own packages containing the component declarations.

Two attributes specify the information required for users to provide their own packages. These attributes are set on the hierarchical design in which the packages are required. The attribute **VHDLComponentPackages** is defined to be the names of the packages containing the component declarations for a given design hierarchy. One or more packages can be specified. A VHDL 'use' clause is included in the generated architecture for each package specified. If none are specified then a package is created dynamically.

The attribute **VHDLComponentPackageFiles** is defined to be the names of the files containing the definition of the packages specified by VHDLComponentPackages. The file names can be either absolute or relative. Relative path names are assumed to be relative to the workspace sub-directory containing the hierarchical component. These files are automatically included in the file list (retrieved by `get_hdl_file_list`) associated with the given hierarchical component. Files specified through VHDLComponentPackageFiles are ignored unless **VHDLComponentPackages** is set.

The dialog associated with the Generate Subsystem RTL activity contains a spreadsheet for specifying the VHDL library to be associated with each hierarchical component. This simple spreadsheet includes a column for the 'package names' and a column for the 'package files'.

4

Creating the Gate-Level Netlist for a Subsystem

The gate-level view of a subsystem is a translation of the RTL view into a technology-specific netlist, which might include physical placement information.

**Note**

Some of the activities for this view can be disabled using the **Edit > Workspace Options** menu item. If disabled, these activities do not appear in the Activity List. For more information, see [“Setting Workspace Options”](#) on page 48.

This chapter contains the following sections:

- [“Specifying Target Technology \(TechSetup\)”](#) on page 82
- [“Technology Cell Binding”](#) on page 85
- [“Initializing Subsystem Constraints”](#) on page 89
- [“Specifying Clocks”](#) on page 93
- [“Specifying Operating Conditions and Wireloads”](#) on page 95
- [“Specifying Power Intent”](#) on page 97
- [“Specifying Timing Exceptions”](#) on page 99
- [“Specifying Port Constraints”](#) on page 100
- [“Specifying Synthesis Methodology”](#) on page 103
- [“Specifying Test Methodology”](#) on page 107
- [“Synthesizing Subsystems”](#) on page 117

4.1 Specifying Target Technology (TechSetup)

The Specify Target Technology activity specifies the technology Logical libraries for ASIC synthesis, and also the Physical libraries for Formal Verification. The collective settings are called the TechSetup.

When you select the Specify Target Technology item in the Activity List pane, the Library Setup tabs appear in the Activity View pane. Prior to showing this pane for the first time, your setup file (.synopsys_dc.setup) is read to determine initial values for the fields.

4.1.1 Logical Library Setup

The Target Technology activity contains the Logical Library search paths and libraries associated with dc_shell or fc_shell. These parameters are defined in [Table 4-1](#).

Table 4-1 Logical Library Setup Parameters

Activity Parameter <GUI label> (<parameter name>)	Description
Setting for Search Path variable (SearchPath)	Specify the paths to your technology library (db and NDM libraries) and any other libraries required for your design. The paths here work for Design Compiler and Fusion Compiler.
Setting for TargetLibrary variable (TargetLibrary)	Specify the names of your technology library files (.db) and folders (.ndm). If your target .db library uses a separate file to characterize cells for best-case operating conditions (minimum timing), specify it in the MinLibrary parameter described below. Notes: ■ If no NDM libraries are specified in this field, the Fusion_Compiler_Reference_Methodology synthesis strategy will not be available, as covered in “Choosing a Synthesis Strategy” on page 123. ■ Only one NDM library can be specified in this field. ■ An NDM library specified here should be the first of any NDM libraries specified for the LinkLibrary parameter described in this table.
Setting for LinkLibrary variable (LinkLibrary)	Specify the names of your technology library files and folders (.sldb, .db, and .ndm) and any other libraries required to link your design. For the names of specific libraries that must be linked to synthesize the core, refer to the documentation supplied with your core. Notes: ■ If an NDM library is specified for the TargetLibrary parameter, it should be the first of any NDM libraries specified here.
Setting for MinLibrary variable (MinLibrary)	If your technology library uses a separate .db file to characterize cells for best-case (minimum timing) operating conditions, enter the name of it in this field. You can also specify this parameter as a list of the form <i>{{worst-case_library_A best-case_library_A} {worst-case_library_B best-case_library_B} ...}</i> . List specification of this parameter supports multiple min-timing libraries. A min-timing library can be associated with any library currently specified in the target or link library lists.

Table 4-1 Logical Library Setup Parameters (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
	Note: ■ There is no need to specify NDM library information in this field.

4.1.2 Physical Library Setup

The Physical Library Setup is used by DC-Topographical. You must set the dc_shell tool root in order to use DC-Topographical.

Table 4-2 Physical Library Setup Parameters (DC-Topographical)

Parameters	Description
Milkyway Design Library	Path to the Milkyway design library (default to synthesis strategy value)
TechFileFormat	Select “tf” to use the techfile to create the Milkyway design library, or “plib” to use the physical library source (.plib) file.
TechFile	Technology file used to create the Milkyway design library.
CreateMwDesignOptions	Additional options used to create the Milkyway design library (see Milkyway documents)
MaxTluPlusFile	Max TLU+ file. Not required (defaults to synthesis strategy value)
MinTluPlusFile	Min TLU+ file. Not required (defaults to synthesis strategy value)
Milkyway Reference Library	Specify the full path to the Milkyway reference library (same value as the Design Compiler variable mw_reference_library. The library must exist before specifying this parameter.
Milkyway Logic 0 Net	Corresponds to the DC variable mw_logic0_net.
Milkyway Logic 1 Net	Corresponds to the DC variable mw_logic1_net.

4.1.3 Set Tool Installation Root

If the entry fields are not enabled, it means that you do not have a path defined for access to dc_shell-t. To resolve this problem, specify the tool installation areas using the Edit > Tool Installation Roots menu path. This dialog box also contains an Update Tool Roots button that sets the tool roots to the values determined from the current environment.

**Note**

Synthesis tools that are not suitable for a specific core are permanently disabled for that core and do not appear as options in coreAssembler.

**Hint**

To use 64-bit tool binaries within coreAssembler (64-bit platforms only): In the coreAssembler application, choose **Edit -> Tool Installation Roots**. Check the “64 Bit?” box for each tool to direct coreAssembler to use the 64 bit tool binaries. Most tools have a command line switch used by coreAssembler to enable 64-bit mode. Make sure that supporting tools like Vera also point to the 64-bit installation; otherwise Verification might fail. When using Vera, you must set VERA_HOME such that it picks up the 64-bit binaries (see Vera documentation for details). This automatically checks the 64-bit box for accompanying simulation tools.

4.2 Technology Cell Binding

The optional Technology Cell Binding activity appears if any cores in your subsystem include the ability to map to cells in your technology libraries.

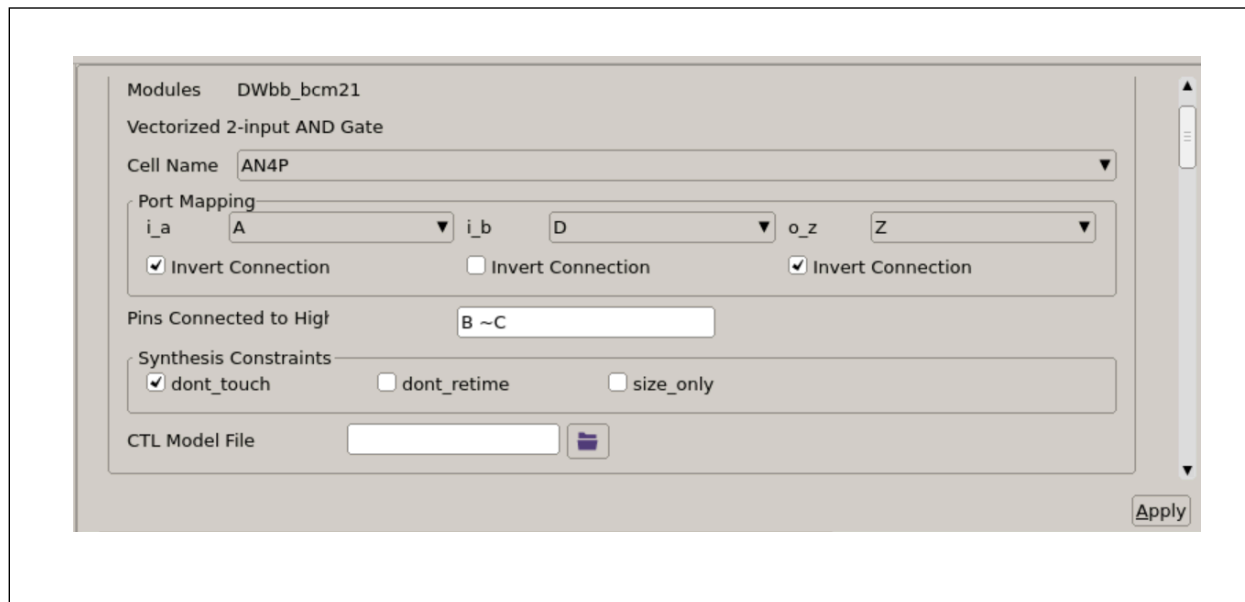
When you select the Technology Cell Binding activity, a form appears to the user to map core cells to technology-hardened cells in the target libraries identified during the Specify Target Technology activity. A set of fields appears for each core cell that can be mapped. [Table 4-3](#) describes this set of fields.

Table 4-3 Technology Cell Binding Activity Fields

Field	Description
Macro	Macro name associated with the core cell that can be mapped For a description of the core cell, hover the mouse over the macro name or right-click on it and choose "What's this".
Cell Name	Choose the library cell to map to This drop-down menu contains cells from your technology library.
Port Mapping	Map the core cell ports to the library cell ports that appear in the drop-down after you select a technology cell The ports in the drop-down match the direction of the corresponding port in the core cell.
Invert Connection	Select to negate a selected port from the dropdown list or leave it unchecked if negation is not required.
Pins Connected to High or Low	Write to the list of unconnected tech cell pins that are to be connected to logic 1 or logic 0.
Synthesis Constraints	Select the synthesis constraints (such as dont_touch) you want for the technology cell; some synthesis constraints might be checked by default.
CTL Model File	CTL model file to describe the test behavior of the technology cell. This file is read in DC/FC along with the user specified CTL model files set in the designs or the strategy parameter in FCRM/DCRM .

Figure 4-1 shows the snapshot of the TCB GUI with the parameters listed in Table 4-3.

Figure 4-1 TCB GUI Snapshot



When you complete the mapping entries and click Apply, the Technology Cell Binding report is shown in the Report tab. The reports include the following:

- **Binding Commands:** A list of the `bind_tech_cell_mapping` commands that are executed in the background. For information about the `bind_tech_cell_mapping` command, see the online command reference.
- **Files:** A table of information about the cells that were mapped. This includes a link to the RTL file where coreConsultant inserted a description and a macro line near the top that summarizes the mapping.
- **Constraint File:** A link to the constraints file that is created at `<workspace>/export/tech_cell_binding_attributes.tcl`. This file is read and used during synthesis.

If you want to define your own cells to include in this mapping activity, you can use the `create_custom_tech_cell` command. For information about this command, see the online command reference.

4.2.1 Tech Cell Mapping Use Case

Technology cell definition commands (`define_tech_cell_mapping`) are issued in coreBuilder and are packaged in the coreKit. Technology cell binding commands (`bind_tech_cell_mapping`) are issued in coreConsultant and coreAssembler before the Technology Cell Binding activity starts. This activity inserts the tech cell definitions into the RTL files (specified in the definitions) as macros within `ifdef` or `ifndef` blocks. An example of the RTL transformation is shown in Table 4-4.

Table 4-4 Commands Issued

coreBuilder	coreConsultant/coreAssembler
<pre>define_tech_cell_mapping -module BCM61 \ -macro BCM61_map \ -instance I061 \ -description "Sample tech map" \ -port_names {r1 r2 r3} \ -file "compA.v" \ -port_descriptions {{r1 port} {r2 port} {r3 port}}</pre>	<pre>bind_tech_cell_mapping -module BCM61 \ -macro BCM61_map \ -tech_cell OR2 \ -port_map {{r1 A} {r2 B} {r3 Z}}</pre>
<pre>define_tech_cell_mapping -module BCM00 \ -macro BCM00_map \ -instance I000 \ -description "Sample tech map" \ -port_names {m1 m2 m3} \ -port_descriptions {{m1 port} {m2 port} {m3 port}} \ -enable_macro_args \ -extra_macro_args "a1 a2 a3" -file "master.v" \ -width "[WIDTH-1:0]" \ -filter "Name=~ADDF" \ -output_inversion_suffix "INVERTED_OUT" -required</pre>	<pre>bind_tech_cell_mapping -module BCM00 \ -tech_cell OR2 \ -port_map {{m1 A} {m2 B} {m3 Z 1}} \ -attribute {dont_touch size_only dont_retime}</pre>

**Note**

Command reference manual should be referred for additional details about the options and usage examples.

The `bind_tech_cell_mapping` commands should be issued at the top level in a coreAssembler subsystem. An example of the modifications are shown in [Table 4-5](#).

Table 4-5 Modifications In RTL for Tech Cell Mapping

Original Component Source	Modified Component Source After tech cell Mapping
<pre>module compA(r1, r2, r3); input r1, r2; output r3;</pre>	<pre>`ifdef ENABLE_TECH_CELL_MAPPING // Technology mapping for macro: BCM61 // Module: BCM61_map // Description: Sample tech map // Technology cell used: AN2</pre>

Table 4-5 Modifications In RTL for Tech Cell Mapping (continued)

Original Component Source	Modified Component Source After tech cell Mapping
// Technology cell insertion. `ifdef BCM51_map `BCM51_map `else assign r3 = r1 & r2; `endif endmodule	`define BCM61_map AN2 I061(.A(r1),.B(r2),.Z(r3)); `endif module compA(r1, r2, r3); input r1, r2;output r3; // Technology cell insertion. `ifdef BCM51_map `BCM51_map `else assign r3 = r1 & r2; `endif endmodule // undefining the defined tech cell macros defined earlier `ifdef BCM61_map `undef BCM61_map `endif



Note

To successfully synthesize your design with the Technology cell bound RTL, you to enable the 'ENABLE_Tech_CELL_MAPPING' macro during the synthesis run.

For Synopsys IP, the macro is undefined in the undef file from the component source; for custom IP, they are undefined in the file where the macro was introduced (for custom technology cells) or the files specified with the -file option in define_tech_cell_mapping commands.

To prevent the synthesis tool from optimizing away these inserted special cells or setting timing/area specific attributes on them, a tech_cell_binding_attributes.tcl file is exported during the Technology Cell Binding activity. A sample of the contents of a tech_cell_binding_attributes.tcl file is shown below:

```
set cells [get_cells -quiet -hierarchical -filter ref_name==AN2]
if {[sizeof_collection $cells]} {
set_dont_retime $cells
}
```

4.3 Initializing Subsystem Constraints

This activity enables users to gain control over how constraints are defined. The activity defines the subsystem level clocks and derives initial subsystem level constraints from the default constraints defined in the components. This activity includes two tabs:

- Propagation Control -- includes parameters that control constraint promotion
- Manual Clock Definition -- user specification of clock naming attributes.

4.3.1 Manual Clock Definition

The subsystem connectivity is used to define standard and generated clocks at the subsystem level. Those clocks are automatically named using the driving pin or port for that clock. The virtual clocks in the subsystem are named. You are allowed to edit the names in the spreadsheet. Any virtual clocks with the same name are merged into a single subsystem level virtual clock. The columnar information for the Manual Clock Definition tab includes:

- Clock (name)
- Name of Component
- System Clock Name
- Generated Clock Pin Name

4.3.2 Propagation Control

Table 4-6 shows the controls that are available for the Propagation Control tab.

Table 4-6 Initializing Subsystem Constraints -- Propagation Controls

Activity Parameter <GUI label> (<parameter name>)	Description
Propagate all constraints (PropagateAllConstraints)	Boolean parameter indicating whether or not constraints are propagated. If true, all Synopsys design constraints are propagated upward from the leaf-level components. If false, only delay, drive, and load constraints and timing exceptions between clocks are propagated. The 'false' case is consistent with the constraints that were propagated in coreAssembler versions prior to C-2009.06. Default: 0
Implementation Context (ImplementationContext)	String parameter indicating the top-level implementation context for this subsystem. Constraints will be propagated up to the specific context. All implementation activities (Synthesis, Static Timing, Formal Verification, and Generate Test Vectors) are then run at this context. The default value is the special string '<top>', which indicates that the implementation context is the top level of the subsystem.
Propagate upward only (PropagateUpwardOnly)	If true, all Synopsys Design Constraints are propagated upward only. If constraints are manually modified after propagation, they must be manually propagated downward. This applies to all constraints except for clocks. Default: 0

Table 4-6 Initializing Subsystem Constraints -- Propagation Controls (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
Ignore clock definition or mapping errors (IgnoreClockDefinitionOrMappingErrors)	Boolean parameter indicating whether or not errors in the clock definition or clock mapping files cause the constraint propagation process to stop. Default: 0
Clock mapping file name (ClockMappingFile)	The named file containing commands to set the SystemClockName and GeneratedClockPinName attributes on component level clocks.
Clock definition file name (ClockDefinitionFileName)	File containing system level clock definition commands. This file can contain only clock related SDC commands and timing exceptions (create_clock, set_clock_latency, set_clock_uncertainty, set_clock_transition, set_false_path (from/to clocks only), set_multicycle_path (from/to clocks only), and set_max_delay (from/to clocks only).
Virtual clock naming style (VirtualClockNamingStyle)	Parameter indicates the system-level virtual clock naming style. Defaults to %h_%c where %c is replaced with the component level virtual clock name and %h is replaced with the hierarchical path to the component, using underscore “_” as a hierarchy separator. Multiple strings can be specified, where each potential virtual clock name is generated in sequence to check for a pre-existing virtual clock. If a virtual clock is found with the given name, that name is used. Otherwise a virtual clock based on first naming format is created during clock linking.
Generated clock naming style (GeneratedClockNamingStyle)	Parameter indicates the system-level virtual clock naming style. Defaults to %i_%c where %c is replaced with the component level virtual clock name and %i is replaced with the instance name of the component, using underscore “_” as a hierarchy separator. Multiple strings can be specified, separated by spaces. If multiple strings are specified, each potential virtual clock name is generated in sequence to check for a pre-existing clock. If a clock is found with the given name, that name is used. Otherwise, a clock name based on the first naming format is created during clock linking.
Real clock naming style (RealClockNamingStyle)	Parameter indicates the system-level real clock naming style. Defaults to %p where %p is replaced with the system level input port name. The key %i can also be used in the format string and is replaced with the instance name of the component whose clock is being propagated upward. If multiple clocks are connected together, the component with an output port is favored, otherwise the first component (alphabetically) is chosen. If the %i key is used with the %p key, then the port name used is from the chosen component instead of the upper level port. Multiple format strings can be specified.
Link virtual clocks using real clocks (LinkVirtualClocksUsingRealClocks)	Boolean parameter indicating if the linking of component-level virtual clocks be attempted by first matching component level real and virtual clocks and system level real and virtual clocks. If a link is found at both levels, then the corresponding virtual clocks are linked across levels. Links at a common level use the format strings specified in the “LinkVirtualClocksUsingRealClocksNamingStyle” parameter. This methodology for clock linking is only used when a format strings is specified

Table 4-6 Initializing Subsystem Constraints -- Propagation Controls (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
	in the LinkVirtualClocksUsingRealClockNamingStyle parameter, and the clock definition file name was specified to pre-define system level clocks. Default: 0 (false)
Link virtual clocks using real clocks Naming style (LinkVirtualClocksUsingRealClocksNamingStyle)	Specifies the format strings used in the linking of component-level virtual clocks. Defaults to “%c_%p” where %c is replaced with the component-level clock name and %p is replaced with the component level port name associated with the clock.

4.3.3 Constraint Initialization Flow

The following are the steps that coreAssembler uses to propagate constraints:

1. Read clock definition file (if specified)
2. Read clock mapping file (if specified)
3. Link clocks within each non-leaf component, starting from the lowest levels and working upward (in other words, a level of hierarchy is never processed until each of its non-leaf children have been processed)
4. Propagate constraints within each non-leaf component, starting from the lowest levels and working upward.
5. Generate constraint propagation report

The clock linking algorithm follows these guidelines:

- The name of the clock at the next level of hierarchy is based on values of the SystemClockName attribute on clocks at the current level. If SystemClockName is not set, the clock name is based on the format strings specified in the VirtualClockNamingStyle or RealClockNamingStyle parameters.
- When a generated clock is propagated which has the GeneratedClockPinName attribute set on it, the value of the attribute is propagated (after prepending the next level of instance name), unless the higher level clock already has the attribute set on it, which can occur if the higher level clock is predefined and includes mapping information.

4.3.4 Constraint Propagation

4.3.4.1 Non-Delay Constraint Propagation

The non-delay constraints can be broken down into two different categories. Those that are modeled in the coreTools data model, and those that are passed through the data model “as is.” Constraints that are modeled are fully propagated, and the “pass through” constraints are changing to be modeled, and then propagated as “pass through” constraints.

In addition to all input/output delay constraints, the following constraints are propagated:

- HighFanout
- IsTriState

- IdealPort
- Drive
- PinLoadCount
- PinLoadType
- DrivingCell
- IfUnconnected
- MinCap
- Capacitance
- CriticalTiming
- InputResistance
- MaxCap
- MaxFanout

For more information on these constraints, see the Design Compiler documentation.

4.3.4.2 Timing Exceptions

The only timing exceptions that are currently propagated are those where both the “from” and “to” path specifiers refer to clocks.

For example, timing exception “set_false_path -from [get_clocks hclk]” is propagated, while “set_false_path -from [get_ports hclk]” is not. This means that the path elements must be specified using the appropriate “get_<item type>” command (for example, get_pins, get_clocks, get_ports) or the all_registers command, with the -clock option. When propagating a timing exception, path specification elements (-from, -to, -through) are translated as follows:

- get_clocks <clock name> --> get_clocks <system clock name>
- get_pins <pin name> --> get_pins <instance name>/<pin name>, where <instance name> is the name of the instance in which the timing exception was originally defined
- get_ports <port name> --> get_pins <instance name>/<port name>, unless the given pin is directly connected to a top-level port, in which case the path specifier translates to get_ports <top level port name>
- all_registers -clock <clock name> --> all_registers -clock <system clock name>

Any references to pins (after propagation) in -from or -to path elements are changed to -through path elements. In addition, translated path elements referencing ports or pins are removed from timing exceptions if they are tied to a constant value.

4.4 Specifying Clocks

The clocking functionality is enhanced so that clocks can be defined on bits of a bus in addition to top-level ports. The ClockName attribute can be set on a bit (but not on a port that contains bits). This allows bit-level clocks to be defined in coreBuilder just like any other clock. The Add Clocks dialog includes individual bits in the list of potential clock sources. The synthesis constraints define clocks when specified at the bit-level. The clock linking functionality in coreAssembler propagates bit-level clocks.



Note

It is not possible to represent clocks on bits of a bus in IP-XACT versions P1685-2009 and earlier. Bit-level clock definitions, and timing constraints associated with bit-level clocks are not included in the generated IP-XACT component file if writing a schema version older than P1685-2014.

Individual coreKits identify and name all clocks in their designs. As a core integrator, you need to specify the clock attributes according to:

- Actual clock period and waveform
- Clock network delays

When you select the Specify Clock(s) activity on the Activity List, the corresponding Activity View pane appears.

On the Real and Virtual Clocks tab, specify values for each of the attributes listed in [Table 4-7](#). Do this for each clock in the design. The coreAssembler online reference provides detailed descriptions of these attributes.

Table 4-7 Clock Attributes

Attribute	Description
CycleTime	Clock period time.
Waveform	Rise and fall edge times of the clock over an entire clock period. (By default, it uses library time units). The default value for Waveform is {0 CycleTime/2}.
TestClock	This clock is to be used in test mode.
TestClockCycleTime	The cycle time to be used for a clock in test mode.
TestClockWaveform	Waveform for this clock in test mode.
FixHold	Whether or not to fix hold time violations for the selected clock. If FixHold is true, a set_fix_hold command is generated, which causes Design Compiler to insert a delay, if necessary, to fix hold time violations at registers driven by the selected clock.
ClockRiseLatency/ ClockFallLatency	Clock signal propagation from the clock definition point to the clock pin of a register, for a rising or falling transition, respectively, at the clock definition point.
ClockSourceRiseLatency/ ClockSourceFallLatency	Clock signal propagation from its actual ideal waveform origin point to the clock definition point, for a rising or falling transition, respectively, at the waveform origin point.

Table 4-7 Clock Attributes (continued)

Attribute	Description
ClockSetupUncertainty/ ClockHoldUncertainty	Setup and hold uncertainties, respectively, applied to all endpoints of a specified clock.
MinTransitionDelay/ MaxTransitionDelay	Minimum and maximum transition times, respectively, for an ideal clock.

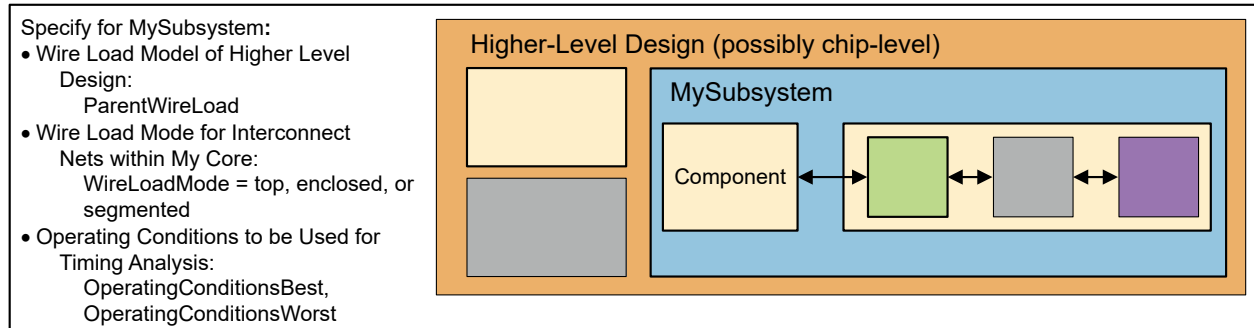
Numeric values can be entered as fixed numeric values, as fixed numeric values with a time unit, or as a formula related to the cycle time. When time units are not specified, the units of the main target library are used. For example, assuming the technology library units are 'ns' and assuming a 10 ns cycle time, then each of the following entries specifies the same 10 nano-second value: 10, 10ns, 10000ps, =percent_of_period 10.

On the Generated Clocks tab of the Specify Clock(s) activity, you specify attributes associated with each of the generated clocks in your design. At a minimum, you must define CycleTime or WaveForm.

4.5 Specifying Operating Conditions and Wireloads

You must supply values for the attributes that describe the chip environment in which you are synthesizing each core, because they are specific to your application and target technology. You only need to specify the operating condition and wireload variables on the top-level design, as shown in [Setting Operating Condition and Wireload Attributes \(on page 95\)](#).

Figure 4-2 Setting Operating Condition and Wireload Attributes



When you select the Specify Operating Conditions and Wireloads activity on the Activity List, the corresponding Activity View pane appears. Set values according to your knowledge of the environment in which the cores are to be synthesized and integrated. At the very least, you must specify a value for OperatingConditionsWorst.

[Table 4-8](#) defines the operating condition and wireload attributes. For more detailed information about these attributes, see the coreAssembler online reference.

Table 4-8 Operating Conditions and Wireload Attributes

Attribute	Description
ParentWireLoad	Wire load model to be used for the design that is one level above the subsystem.
WireLoadModel	Wire load mode for the subsystem. Valid values are top, enclosed, and segmented. If you do not specify WireLoadModel, the library's default wire load mode is used (if one exists).
WireloadGroup	Wire load model selection group for automatic wire load model selection. If you do not specify a WireLoadGroup, and none exists to inherit from the design at the next higher level, the library's default value is used (if one exists).
WireloadMinBlock Size	Minimum block size for automatic wire load selection.
Wireload	The wire load model used for synthesis. If you do not specify a value explicitly, a wire load model is automatically selected as follows: If AreaEstimate is set, the wire load for the initial compile is based on the values for AreaEstimate (if specified). It then switches to Design Compiler's automatic wire load selection for subsequent, incremental compiles. If AreaEstimate is not set, Design Compiler's automatic wire load selection is used for all compiles.

Table 4-8 Operating Conditions and Wireload Attributes (continued)

Attribute	Description
OperatingConditionsBest	Best-case operating conditions to use for minimum delay (hold time) timing analysis. Your choice impacts both optimization and timing analysis.
OperatingConditionsWorst	Worst-case operating conditions to use for maximum delay (setup time) timing analysis. Your choice impacts both optimization and timing analysis.

4.6 Specifying Power Intent

The Specify Power Intent activity in coreAssembler enables you to customize Unified Power Format (UPF) objects present in each component of your subsystem. The default values come from the coreKit and are based on the assumed target environment.

This activity is only visible when at least one component has any UPF objects in them.

When you select the Specify Power constraint activity on the Activity List, the corresponding Activity View pane appears. There are three tabs in the pane: Cell Specification, Voltage Specification, and SupplyVoltage Specification. The last tab is only visible when there is any `set_voltage` command present in any of the component's packaged UPF files. You can select a component from the component drop down option to view the UPF items associated with it.

[Table 4-9](#) provides brief descriptions of the UPF attributes.

Table 4-9 UPF Attributes

Attribute	Description
UPFStrategy	Valid on isolation, level_shifter, and retention UPF items
UPFDomain	Valid on isolation, level_shifter, and retention UPF items
UPFLibraryCells	Valid on isolation, level_shifter, and retention UPF items; you can select the library cells from a list of cells available in the current technology library
UPFSupply	Valid on port_state and power_state
UPFSupplyExpr	Valid on power_state
UPFState	Valid on port_state and power_state
UPFStateValue	Valid on port_state
UPFVoltageType	Valid on all UPF items. You can select one from the list: {nom min/max nom/min/max}
UPFMin	Float value valid on all UPF items
UPFMax	Float value valid on all UPF items
UPFNom	Float value valid on all UPF items
UPFVoltage	Float value valid on UPF supply voltage items

Upon completion of the activity, power intent files packaged with each component are updated and written in the respective component folder `<workspace>/components/<component_name>/upf`.

A `UPF_supply_<type>_<mode>` item is created for every combination of voltage type and voltage mode coming from the `get_supply_voltage` command in the intent file. Voltage type can be either power or ground. Similarly, voltage mode can be either best or worst. In batch mode, you can modify the `set_voltage` value using the `set_supply_voltage` command as follows:

```
set_supply_voltage UPF_supply_power_worst 3.14
```

To complete the Specify Power Intent activity in command line mode, use the `set_current_component`, `set_upf_attribute` and `autocomplete_activity` command as follows:

```
set_current_component <component name>
set_upf_attribute <UPF item name> <UPF attribute name> <value>
autocomplete_activity SpecifyPowerIntent
```

4.7 Specifying Timing Exceptions

In some designs, certain timing paths cannot be characterized by single-cycle timing specifications. Such timing paths are commonly referred to as timing exceptions. coreAssembler enables you to capture timing exceptions as needed. You can use the coreAssembler timing exception capability to add new timing exceptions, report the timing exceptions currently set on a design, and remove timing exceptions.

Even if your design has no timing exceptions, you must complete the Specify Timing Exceptions activity to enable the subsequent activities.

Select the Specify Timing Exceptions activity from the Activity List. The Specify Timing Exceptions dialog appears in the Activity View pane, where you can view, edit, add, or delete timing exceptions. The currently existing timing exceptions appear in the spreadsheet at the top of the dialog. [Table 4-10](#) provides a brief description of the available timing exceptions.

Table 4-10 Timing Exception Commands

Command	Description
report_timing_exceptions	Lists currently specified timing exceptions, set_false_path and false paths.
set_false_path	Removes timing constraints from particular paths.
set_multicycle_path	Modifies the single-cycle timing relationship of a constrained path.
set_max_delay	Specifies a maximum delay target for paths in the current design.
set_min_delay	Specifies a minimum delay target for paths in the current design.
reset_path	Resets the specified paths to single-cycle timing.
set_disable_timing	Disables timing arcs between two defined pins in the current cell.

To edit a timing exception:

1. Select the timing exception command in the spreadsheet.
2. Modify the options for the command in the Edit Timing Exception area.
3. Click the Update button at the bottom of the dialog to complete the modification.

To add a timing exception:

1. In the Edit Timing Exception area, select a timing exception command from the Timing Exception drop-down menu.
2. Enter the required To, From, and Through information.
3. Select other options for the timing exception command.
4. Click New to add the timing exception.

To delete a timing exception:

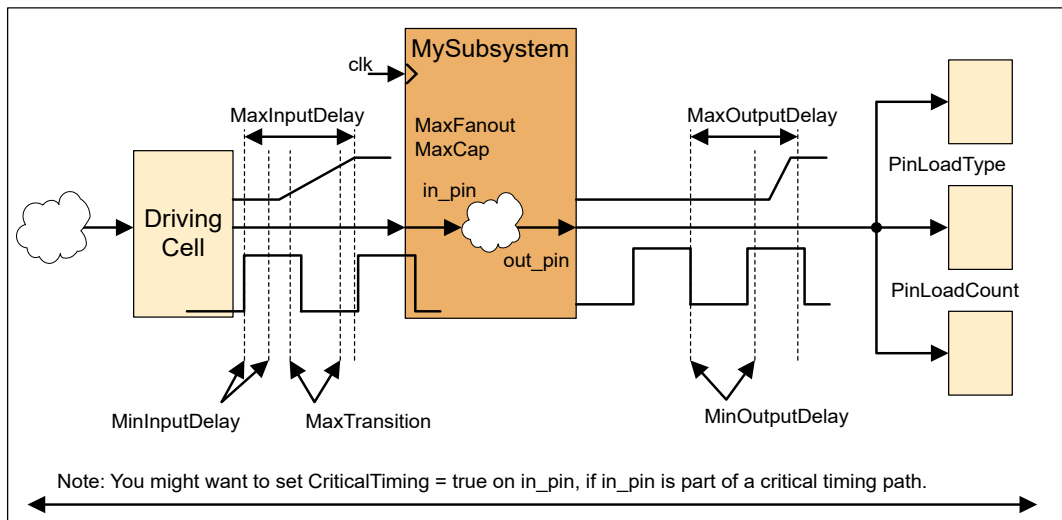
1. Select the timing exception command in the spreadsheet.
2. Click Delete in the Edit Timing Exception area.

4.8 Specifying Port Constraints

coreAssembler uses the port attributes to determine synthesis constraints for each subsystem port. The coreKits specify default values for these attributes based on the timing performance of the core in its assumed target environment. The component cores export their port constraints to the subsystem level if they are connected to exported ports. Conflicts between different components connected to the same subsystem port are resolved using the more conservative value.

[Port Constraints Attributes \(on page 100\)](#) illustrates Port Constraints attribute usage. For more information about individual attributes, see the online command reference.

Figure 4-3 Port Constraints Attributes



When you select the Specify Port Constraints activity on the Activity List, the corresponding Activity View pane appears. [Table 4-11](#) provides brief descriptions of the port attributes.

Table 4-11 Port Attributes

Attribute	Description
MinRisingInputDelay, MaxRisingInputDelay, MinRisingOutputDelay, MaxRisingOutputDelay	Delay for rising signal. Core developers specify delay values based on core timing performance and assumed chip environment. You can modify delay values to match the actual design environment in which you are synthesizing the component cores.
MinFallingInputDelay, MaxFallingInputDelay, MinFallingOutputDelay, MaxFallingOutputDelay	Delay for falling signal. Core developers specify delay values based on core timing performance and assumed chip environment. You can modify delay values to match the actual design environment in which you are synthesizing the component cores.
MinInputDelayFallingEdge[], MaxInputDelayFallingEdge [], MinOutputDelayFallingEdge[], MaxOutputDelayFallingEdge[]	The minimum and maximum input delays, respectively, for the input port with respect to the falling edge of the specified clock. If only one clock exists in the design, you do not need to name it through the subscript.

Table 4-11 Port Attributes (continued)

Attribute	Description
PinLoadType, PinLoadCount	Core developers specify technology-independent values based on the expected chip environment. If you have more specific values derived from your design and target technology, specify them before your initial synthesis.
DrivingCell	Core developers specify technology-independent values based on the expected chip environment. If you have more specific values derived from your design and target technology, specify them before your initial synthesis.
IdealPort	When true, this attribute implies infinite drive with no resistance. Therefore, directly attached nets are not checked for transition or capacitance violations. Set IdealPort on ports such as reset and test ports that you plan to buffer using post-synthesis tools.
DontTouchNetwork	If true, a Design Compiler set_dont_touch_network command is generated for the port. This prevents the modification or replacement of cells and nets in the transitive fanout of the port during optimization.
CriticalTiming	Core developers set CriticalTiming to true if the input port is likely to be part of a critical timing path. Depending on your initial synthesis results, you can change the value.
InputResistance	Specifies the resistance on the net driven by an input port.
MaxFanout, MaxCap, MaxTransition	Core developers do not specify values for these attributes, because they are specific to your design and target technology. If you do not specify values for these attributes, Design Compiler uses the design rules defined in your technology library. You specify these values when you want to impose stricter design rules than those specified by your technology library.
DftExistingSignalType	Test signal to be used for the 'existing_dft' view of set_dft_signal. These are signals that describe structures which already exist that must be understood for the design to pass DRC (dft_drc): clocks, resets, constants, existing scan chains. Default: none
DftExistingSignalActiveState	Active logic state for the signal. Default: none
DftExistingSignalTiming	Waveform to be used for the test signal. Default: none
DftExistingSignalTestMode	The name of the test mode the dft signal specification applies to. Default: none
DftSpecSignalType	Test signal to be used for the 'spec' view of set_dft_signal. These are signals that describe structures that DFT Compiler uses for insert_dft: scan in, scan out, scan enable, and so on. Default: none

Table 4-11 Port Attributes (continued)

Attribute	Description
DftSpecSignalActiveState	Active logic state for the signal. Default: none
DftSpecSignalTestMode	The name of the test mode the dft signal specification applies to. Default:
TestHold	Obsolete. See DFT section
ScanSignalType	Obsolete. See DFT section
DftSignalType	Obsolete. See DFT section

Make sure that the intent specifications for the top-level design of the subsystem are consistent with the chip environment in which the subsystem is to be synthesized. In particular, the first time you synthesize the subsystem, check that the following attributes are suitable for the target environment:

- Input Ports – MinInputDelay, MaxInputDelay, DrivingCell
- Output Ports – MinOutputDelay, MaxOutputDelay, PinLoadCount, PinLoadTypeTo view and possibly modify attribute values for individual bits of a bus, select the port in the port list and left-click the Set Bit Level Intent button.

4.9 Specifying Synthesis Methodology

Individual coreKits specify the values for the attributes that describe the design characteristics of each core. Then coreAssembler uses this information to develop synthesis strategies optimized for each core.



Note

Typically, you do not need to modify synthesis methodology attribute values unless the synthesis results fail to meet your design goals.

When you select the Specify Synthesis Methodology activity on the Activity List, the corresponding Activity View pane appears. The Synthesis Methodology Activity View contains the following tabs:

- [“General Synthesis Methodology Attributes”](#)
- [“Advanced Synthesis Methodology Attributes”](#)

4.9.1 General Synthesis Methodology Attributes

These are the attributes which you are most likely to want to change if synthesis results do not meet your design requirements. [Table 4-12](#) describes these attributes in detail.

Table 4-12 General Synthesis Attributes

Attribute	Description
MapBlockIndividually	Determines whether the selected design is to be compiled as an individual unit.
Optimization\ Priorities	Specifies the optimization priorities for the selected design: ■ timing: Focus on achieving the best timing, giving minimal consideration to area impact. ■ timing_area: Achieve timing objectives first, then try to reduce area. This is the default. ■ area_timing: The design easily meets timing objectives, so focus optimization on reducing area. ■ area: Focus on achieving the smallest design that meets timing constraints. (In some cases, area tradeoffs make it impossible to meet timing goals).
ScanStyle	Indicates the scan style to use for the selected design. The specified value is used directly as the value for the Design Compiler set_scan_configuration command. The value of ScanStyle is relevant only when you set the Test Ready Compile synthesis strategy parameter to true.
ScanExclude	Specifies a list of items that are not to be replaced by scan cells. A “set_scan_element false” command is generated for each specified item. You must set ScanExclude on the design on the implementation frontier that contains the specified item. The value of ScanExclude is relevant only when you set the Test Ready Compile synthesis strategy parameter to true.
Simulation Model	If set to “true” indicates the associated design is for simulation only and is not to be synthesized. This attribute cannot be changed in coreAssembler

4.9.2 Advanced Synthesis Methodology Attributes

These are more advanced synthesis attributes that do not generally require modification. The most common circumstances for modifying these attributes is to add to your own auxiliary script. [Table 4-13](#) describes these attributes in detail.

Table 4-13 Advanced Synthesis Attributes

Attribute	Description
Wireload	The wire load model used for synthesis. If you do not specify a value explicitly, a wire load model is automatically selected as follows: ■ If AreaEstimate is set, the wire load for the initial compile is based on the values for AreaEstimate (if specified). It then switches to Design Compiler's automatic wire load selection for subsequent, incremental compiles. ■ If AreaEstimate is not set, Design Compiler's automatic wire load selection is used for all compiles.
Uniquify	When true, indicates that Design Compiler uniquifies the selected design. If you know of a multiply-instantiated design that should not be uniquified, set Uniquify to false on that design. If Uniquify is false, a synthesis strategy is generated that compiles one instance of the design prior to the rest of the circuitry, then sets dont_touch on the design during the initial mapping phase. The RepresentativeContext attribute is used to decide which instance of the design to compile: If RepresentativeContext has been applied, constraints for the non-uniquified design are based on the specific instance of the design indicated by RepresentativeContext. If no RepresentativeContext attribute has been applied to the design, the highest instance of the design in the design hierarchy is located and used as the representative context for the non-uniquified design.
RepresentativeContext	If Uniquify = false on a selected design, then setting RepresentativeContext selects the instance of the non-uniquified design that is compiled to represent all other instances. This value is a cell name relative to the design.
TclAuxSynthesisScript[cleanup]	Executes the Design Compiler Tcl-mode script after all processing is completed, just prior to quitting.
TclAuxSynthesisScript[constrain]	Executes the Design Compiler Tcl-mode script before setting constraints.
TclAuxSynthesisScript[elab]	Executes the Design Compiler Tcl-mode script before elaboration.
TclAuxSynthesisScript[incr1]	Executes the Design Compiler Tcl-mode script before the first incremental compile.
TclAuxSynthesisScript[incr2]	Executes the Design Compiler Tcl-mode script before the second incremental compile.
TclAuxSynthesisScript[incrStrategy]	Executes the Design Compiler Tcl-mode script which applies to incremental strategies.

Table 4-13 Advanced Synthesis Attributes (continued)

Attribute	Description
TclAuxSynthesis Script[initial]	Executes the Design Compiler Tcl-mode script before the initial compile.
TclAuxSynthesis Script[setup]	Executes the Design Compiler Tcl-mode script before any other processing.
TclAuxSynthesis Script[report]	Executes the Design Compiler Tcl-mode script which applies to report generation.
TclAuxSynthesis Script[dftBistReady]	Executes the Design Compiler Tcl-mode script which applies DFT BIST mode.
TclAuxSynthesis Script[dftInsertion]	Executes the Design Compiler Tcl-mode script which applies to DFT insertion.
TclAuxSynthesis Script[dftCleanup]	Executes the Design Compiler Tcl-mode script which applies DFT cleanup.
AuxSynthesis Script[setup]	Executes the Design Compiler script before any other processing.
AuxSynthesis Script[elab]	Executes the Design Compiler script before elaboration.
AuxSynthesis Script[constrain]	Executes the Design Compiler script before setting constraints.
AuxSynthesis Script[initial]	Executes the Design Compiler script before the initial compile.
AuxSynthesis Script[incr1]	Executes the Design Compiler script before the first incremental compile.
AuxSynthesis Script[incr2]	Executes the Design Compiler script before the second incremental compile.
AuxSynthesis Script[cleanup]	Executes the Design Compiler script after all processing is completed, just prior to quitting.
AuxSynthesis Script[incrStrategy]	Executes the Design Compiler script which applies to incremental strategies.
AuxSynthesis Script[report]	Executes the Design Compiler script which applies to report generation.
CompileOrder	The order in which to compile the subblocks of this design.
CompileMapEffort [all]	Value to use for -map_effort for all compile phases for this design.
CompileMapEffort [initial]	Value to use for -map_effort for the initial compile phase for this design.

Table 4-13 Advanced Synthesis Attributes (continued)

Attribute	Description
CompileMapEffort [incr1]	Value to use for -map_effort for the incr1 compile phase for this design.
CompileMapEffort [incr2]	Value to use for -map_effort for the incr2 compile phase for this design.
CompileMapEffort [incrStrategy]	Value to use for -map_effort for the compile phases for this design.
CompileAreaEffort [all]	Value to use for -area_effort for all compile phases.
CompileAreaEffort [initial]	Value to use for -area_effort for the initial compile phase.
CompileAreaEffort [incr1]	Value to use for -area_effort for the incr1 compile phase.
CompileAreaEffort [incr2]	Value to use for -area_effort for the incr2 compile phase.
CompileAreaEffort [incrStrategy]	Value to use for -area_effort for the compile phase.
FormalVerificationAuxScript	Specifies the script to be run in Formality after the reference and implementation designs have been loaded, but just before the verify command.
StaticTimingAuxScript	Points to an auxiliary script to be executed after the design has been loaded and constrained in PrimeTime, but before the update_timing command runs the analysis.
AtpgTclAuxScript [setup]	Specifies the TestMAX TCL-mode scripts to be run just before the setup phase of test vector generation.
AtpgTclAuxScript [run]	Specifies the TestMAX TCL-mode scripts to be run just before the run phase of test vector generation.
AtpgTclAuxScript [cleanup]	Specifies the TestMAX TCL-mode scripts to be run just before the cleanup phase of test vector generation.
UPFFile	Points to the UPF file.

4.10 Specifying Test Methodology

In this activity you set/review Design for Test attributes on designs.

When you select the Specify Test Methodology activity on the Activity List, the corresponding Activity View pane appears. The Test Methodology Activity View contains the following tabs:

- [“General Test Methodology Attributes”](#)
- [“Scan Insertion”](#)
- [“Auto Fix”](#)
- [“Test Wrappers”](#)
- [“Wrapper Integration”](#)
- [“Built-in Self Test”](#)
- [“Testpoint Insertion”](#)

4.10.1 General Test Methodology Attributes

[Table 4-14](#) provides information on common scan test attributes.

Table 4-14 General Test Attributes

Attribute	Description
DftAuxSynthesisScript	Specifies the name of the script to use to perform insert_dft. The script might do the setup for DFT and run all of the commands, in which case set InsertDft to false. The script might also be used to issue more setup commands in addition to those selected in the DFT options below. If InsertDft is set to true and a script is specified, the script runs after all of the setup commands have been issued, but just before running insert_dft.
ScanSubblocksIndividually	Causes scan insertion to be performed on the subdesigns of this design. Default: 1 for top design, 0 elsewhere.
ScanBlockIndividually	Causes scan insertion to be performed on this design.
TestClockDefaultCycleTime (ns)	Default cycle time for test clocks. Default: 100

4.10.2 Scan Insertion

[Table 4-15](#) provides information on scan insertion attributes.

Table 4-15 Scan Insertion Attributes

Attribute	Description
ScanMethodology	Scan style to be used. Enum values = full_scan, none. Default: InheritValue up full_scan

Table 4-15 Scan Insertion Attributes (continued)

Attribute	Description
ScanStyle	Sets the test scan implementation style for this design. Enum values = multiplexed_flip_flop, clocked_scan, lssd, aux_clock_lssd, combinational, none. Default: multiplexed_flip_flop
ScanExclude	List of sequential cells, hierarchical components containing sequential elements, references, library cells, or designs that must not be replaced by scan cells.
IdentifyShiftRegisters	When true, insert_dft/compile -scan identifies shift-registers in the design and keeps only the first flop as a scan flop. In preview_dft/insert_dft, Scan Architecting groups the flops that form shift registers as a single scan group that is named shift_register_identified_??. The preview_dft -show all command shows the shift registers that are identified. preview_dft -script writes out the shift-registers as scan groups. If the scan style is not multiplexed_flip_flop, insert_dft/compile -scan ignores this option.
EnableScanCompression	When true, enables scan compression. For details on DFTMAX compression options, refer to the following in the DFTMAX User Guide: ■ Using DFTMAX Compression ■ Managing X Values in Scan Compression For related information, see also “ScanCompressionConfiguration”.
Show ‘advanced’ attribute columns (check box enabled)	
NumberOfScanChains	The number of scan chains to be created. Setting the value to 0 allows DFT Compiler to determine the number of scan chains. Default: 0
MaximumScanChainLength	The maximum length of a scan chain. Setting the value to 0 allows DFT Compiler to determine the maximum scan chain length. Default: 0
ClockMixing	Specifies whether cells from different clock domains can be put in the same scan chain. Enumerated Values: ■ no_mix: Cells must be clocked by the same edge of the same clock. ■ mix_edges: Cells must be clocked by the same clock, but the clock edges can be different. ■ mix_clocks_not_edges: Cells must be clocked by the same clock edge, but the clocks can be different. ■ mix_clocks: Cells can be clocked by different clocks and different clock edges.

Table 4-15 Scan Insertion Attributes (continued)

Attribute	Description
	Default: InheritValue up no_mix
TestClocksFollowSystemDomains	When true, dedicated test clocks (for example, test_scan_clock_a) are created according to different system clocks. Thus, scan cells that have different system clocks have different test clocks. When false (the default), test clocks are created without considering system clocks. Default: InheritValue up 0
AddLockUpLatch	When true (1) lockup latches are inserted between clock domain boundaries on scan chains. Default: InheritValue up 1
InsertEndOfChainLockupLatch	When true (1) lockup latches are inserted at the end of scan chains. Default: InheritValue up 0
DedicatedScanPorts	Determines whether dedicated scan out ports are created, or whether the functional ports are used as scan ports. Default: InheritValue up 0
BidirectionalMode	Determines whether bidirectional ports are treated as inputs, outputs or left untouched during scan shift. Enumerated Values = input, output, no_disabling Default: InheritValue up input
ExternalTristates	Determines the disabling option during scan shift for all tristate nets that drive output ports of a design. disable_all disables all drivers. enable_one disables all but one driver. no_disabling specifies not to insert disabling logic. Enumerated Values = disable_all, enable_one, no_disabling Default: InheritValue up disable_all
InternalTristates	Determines the disabling option during scan shift for all tristate nets that do not drive design output ports. Enumerated Values: ■ disable_all: Disables all drivers ■ enable_one: Disables all but one driver ■ no_disabling: Specifies not to insert disabling logic Default: InheritValue up disable_all
HierarchicalIsolation	When true, hierarchical isolation logic is built, so that dedicated subdesign scan-out signals are gated by the design scan enable signal. Default: 0
ScanInternalClocks	When true, internal clocks are treated as separate clocks for creating scan chains. Setting this attribute to false is equivalent to '-internal_clocks none'. When this attribute is set to true it is used in conjunction with ScanJumpBuffersInverters.

Table 4-15 Scan Insertion Attributes (continued)

Attribute	Description
	Default: false
ScanJumpBuffersInverters	Only valid when ScanInternalClocks = true. The value of this attribute is the value of the -internal_clocks option to set_scan_configuration. Default: single
ScanReplace	Replace non-scan registers by scannable registers? When true, insert_scan replaces sequential cells with scan cells, if the sequential cells are not violated by scan design rule checking. Default: 0
ReuseMVCells	Controls the reuse of existing MV isolation and level-shifter cells.
PowerDomainMixing	Specifies that insert_dft can mix scan elements from different power domains in the same scan chain. Default: 0
VoltageMixing	Allow insert_dft to insert scan elements from different voltage domains into the same scan chain.
ScanCompressionConfiguration	The value of this parameter is passed directly to the 'set_scan_compression_configuration' command. For details on DFTMAX compression options, refer to the following in the DFTMAX User Guide: ■ Using DFTMAX Compression ■ Managing X Values in Scan Compression Note that when using set_design_attribute, a blank space must be the first character. When using set_attribute, a leading blank space is not necessary.

4.10.3 Auto Fix

Table 4-16 provides information on auto fix test attributes.

Table 4-16 Auto Fix Attributes

Attribute	Description
EnableDftAutoFix	Enables the DFT Compiler Auto Fix utility. Default: InheritValue up false
AutoFixClocks	Enables/disables automatic fixing of uncontrollable clock violations. Default: InheritValue up true
AutoFixBidirectional	Enables/Disables automatic bidirectional disabling. Default: InheritValue up true
AutoFixAsync	Enables/disables automatic fixing of asynchronous preset/clear violations.

Table 4-16 Auto Fix Attributes (continued)

Attribute	Description
	Default: InheritValue up true
AutoFixAsyncWithScanEnable	When true, the test scan enable signal is used to autofix asynchronous set and reset violations. Default: InheritValue up false
AutoFixAsyncLogicGate	When set to gate, specifies that the asynchronous sets and resets are autofixed with a gate driven by the test_mode signal. Enumerated Values = mux, gate Default: InheritValue up MUX
EnableDftShadowLogic	Enables the DFT Compiler Shadow LogicDFT for this design. Default: InheritValue up false
AutoFixAsyncTestData	Port to be used as TestData for fixing set/reset violations. The port must have been defined with a DftExistingSignalType of Reset. Default: none
AutoFixClockTestData	Port to be used as TestData for fixing clock violations. The port must be associated with a clock that has TestClock==1 or have been defined with a DftExistingSignalType of a clock type. Default: none

4.10.4 Test Wrappers

Table 4-17 provides information on test wrapper attributes.

Table 4-17 Test Wrapper Attributes

Attribute	Description
WrapBlockIndividually	Causes test wrapper insertion to be performed on this design.
WrapSubblocksIndividually	Causes test wrapper insertion to be performed on the subdesigns of this design. Default: 0
WrapperDefaultSafeState	Specifies the default safe state for the wrapper cells. Enumerated Values = 0, 1, none Default: none
Show 'advanced' attribute columns (check box enabled)	
WrapperUseRegisterIO	Enables shared wrapper cells to be used for register-bound ports. The registers are reused in core wrapper chains. Default: 0

Table 4-17 Test Wrapper Attributes (continued)

Attribute	Description
WrapperRegisterImplementation	If this attribute is set to 'swap,' registers connected to ports of the design are replaced with equivalent shared wrapper cells. If the value is set to 'in_place,' additional glue logic is added to registers attached to ports of the design to reuse these registers in core wrapper chains. Enumerated Values = swap, in_place Default: swap
DedicatedWrapperCell	Specifies the default dedicated wrapper cell type. Enumerated Values = WC_D1, none Default: WC_D1
SharedWrapperCell	Specifies the default shared wrapper cell type. Enumerated Values = WC_S1, none Default: WC_S1
OneWrapperClock	If set to true (1), a single wrapper clock is used for all wrapper cells in the design. If shared wrapper cells are used in wrapping the design, the functional clocks of the shared wrapper cells are muxed with the wrapper clock in wrapper test modes by the core wrapper application. If the option is set to false (0), a dedicated wrapper clock is used for all dedicated wrapper cells of the design and functional clocks are used for shared wrapper cells of the design. Default: 0
WrapperDefaultSafeState	Specifies the default safe state for the wrapper cells. Default: none
WrapperExcludePorts	List of ports that must not be wrapped. Since this reduces test coverage, exclude output ports only when necessary. Never exclude input ports other than test signals, test holds, clocks and other asynchronous signals from being wrapped. Values = list, <port_1>, <port_2>,...<port_n>

4.10.5 Wrapper Integration

Table 4-18 provides information on wrapper integration attributes.

Table 4-18 Wrapper Integration Attributes

Attribute	Description
IntegrateTestWrappers	Specifies that all wrapped subdesigns of this design be integrated using SOC methodology. If set to true, all wrapper control signals of integration-ready subdesigns are hooked to the top-level ports during the DFT insertion process. Also, a test controller is implemented to allow testing of integration-ready subdesigns in various test modes.

Table 4-18 Wrapper Integration Attributes (continued)

Attribute	Description
	Default: 0
IntegrationModeDecodingStyle	Specifies the type of port decoding to be used for the core integration test controller. Enumerated Values = default, one-hot Default: default

4.10.6 Built-in Self Test

Table 4-19 provides information on built-in self test (BIST) attributes.

Table 4-19 Built-in Self Test Attributes

Attribute	Description
BistBlockIndividually	Causes insertion of BIST logic into this design Default: InheritValue up -attr BistSubblocksIndividually
BistSubblocksIndividually	Causes BIST logic to be inserted into the subdesigns of this design Default: 0
BistMode	BIST operations to be performed. Choosing “all” causes the design to be made bist-ready and the code to be inserted and integrated with the controller. For hierarchical BIST insertion, choose codec_insertion for the lower levels of hierarchy and integration at the level where the controller is to be integrated with the codecs. Choose bist_ready to make many BIST scan chains or channels within the design without X-propagation and without wrapping the design. Enumerated Values = all, bist_ready, codec_insertion, integration Default: all
BistType	Type of BIST to implement Enumerated Values = dbist, xbist, sbist Default: dbist
Show ‘advanced’ attribute columns (check box enabled)	
BistCodecCount	Number of bist CODECs Default: 1
BistMaxChainLength	Controls the size of the counter which increases the default upper limit on the length of the reconfigured scan chains Setting BistMaxChainLength to 0 allows the DFT Compiler to determine the size of the counter. Default: 0
BalanceBistSegments	By default, SocBist rebalances the scan channels in BIST mode

Table 4-19 Built-in Self Test Attributes (continued)

Attribute	Description
	Setting BalanceBistSegments to 0 (false) overrides this, and SocBist does not concatenate the scan segments. Default: 1
BistInvertPrpgClock	Invert the BIST clock that feeds the PRPG Registers Default: 0
BistPrpgShadowSi	By default, DFT Compiler creates from one to twelve shadow scan chains for the PRPG shadow The number of shadow scan chains created is optimized based on the length of the longest scan chain in the design. You can stipulate that DFT Compiler create a specific number of scan chains instead of setting BistPrpgShadowSi to 'n' where n is an integer from 1 to 12. Setting BistPrpgShadowSi to 'default' allows DFT Compiler to determine the number of shadow scan chains. BistPrpgShadowSi does not affect the length of the longest scan chain, but only the number of shadow scan chains created. Enumerated Values = default, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 Default: default
BistSelectorShadowSi	The number of selector shadow chains Setting BistSelectorShadowSi allows DFT Compiler to set the number of selector shadow chains based on the length of the longest scan chain. Enumerated Values = default, 1, 2, 3, 4 Default: default
BistObserveOutputs	Controls the number of scan output pins that are inserted into the design for BistMode == xdbist By default, the number of scan output pins created depends on the number of scan chains: $\text{scan_outputs} = (\text{scan_chains} / 32)$. Setting this attribute to '0' lets DFT Compiler use the default value. Default: 0
BistPrpgLength	By default, SocBist creates a 478 bit PRPG You can override this by setting BistPrpgLength to 257, creating a 257 bit PRPG. With DBIST, you can use a maximum of 512 scan chains with the 257 bit PRPG. For X-tolerant DBIST, you can use a maximum of 2048 scan chains with the 257 bit PRPG. Enumerated Values = 257, 479 Default: 479
BistDiagOutputs	The number of diagnostic outputs for DBIST Enumerated Values = 1, 2, 4, 8, 16 Default: 1
BistUseTristateMux	Specifies that tri-state muxes are used to construct the XDBIST codec

Table 4-19 Built-in Self Test Attributes (continued)

Attribute	Description
	Default: 1
BistAutoFixBusses	Indicated floating bus and bus contention issues are fixed automatically Default: 1
BistAutoFixXprop	Indicates that fixing of x-propagation are handled automatically Default: 1 if (BistType=xbist dbist)

4.10.7 Testpoint Insertion

Table 4-20 provides information on testpoint insertion attributes.

Table 4-20 Testpoint Insertion Attributes

Attribute	Description
InsertTestPoints	Controls whether observe and control points are inserted into this design Default: 0
TestabilityMethod	Testability method to be used Note that the tdvr and observe methods are identical. The tdvr setting is for backward compatibility. Default: control_and_observe
TestabilityControlSignal	Specifies the control signal to be used for test points The signal is predefined as having the TestMode signal type. If no control signal is specified, the test points are controlled by any signal with the TestMode signal type. If no TestMode signal is found, DFT Compiler creates a new TestMode port. Default: none
TestabilityClockSignal	Specifies the clock to be used for control or observe registers Default: none
Show 'advanced' attribute columns (check box enabled)	
MaxControlPoints	Maximum number of control points to be inserted to enhance testability When TestabilityMethod==control_and_observe, the larger of MaxObservePoints and MaxControlPoints are used for the -max_test_points value for set_testability_configuration. Default: 1000
MaxObservePoints	Maximum number of observe points to be inserted to enhance testability When TestabilityMethod==control_and_observe, the larger of MaxObservePoints and MaxControlPoints are used for the -max_test_points value for set_testability_configuration.

Table 4-20 Testpoint Insertion Attributes (continued)

Attribute	Description
	Default: 1000
TestabilityMaxArea	Specifies the maximum percentage of area overhead due to the observe test point structure insertion in the design Setting this attribute to 0 leaves this as unspecified in DFT Compiler. Default: 0
ControlPointsPerScanCell	Specifies the number of control points that can be shared per scan cell When TestabilityMethod==control_and_observe for XG mode the larger of ObservePointsPerScanCell and ControlPointsPerScanCell are used for the -test_points_per_scan_cell value for set_testability_configuration. Default: 8
ObservePointsPerScanCell	Specifies the number of observe points that can be shared per scan cell When TestabilityMethod==control_and_observe, the larger of ObservePointsPerScanCell and ControlPointsPerScanCell are used for the -test_points_per_scan_cell value for set_testability_configuration. Default: 8
TestpointPowerSavingOn	Adds logic and gate structure for power saving on the XOR tree Default: 0
TestabilityClockType	Specifies the clock type to be used for testability Enumerated Values = dominant, dedicated Default: dominant
ShareTestpointsAcrossHierarchy	Specifies that testpoint scan cells can be shared across hierarchy Default: 0

4.11 Synthesizing Subsystems

The “[Performing ASIC Synthesis](#)” on page 120 activity allows you to set the synthesis options, generate, and optionally run synthesis scripts.

5

Performing ASIC Synthesis

This chapter explains how to synthesize the subsystem and analyze the results to make sure the design goals are met. It also explains how to refine your synthesis intent when the most recent synthesis results fail to meet your design requirements.

This chapter contains the following sections:

- [“Performing ASIC Synthesis”](#) on page 120
- [“Generating Test Vectors”](#) on page 164

5.1 Performing ASIC Synthesis

This activity uses Design Compiler or Fusion Compiler to synthesize the subsystem assembled by coreAssembler. To begin, select the Perform ASIC Synthesis activity from the Activity List. The corresponding Synthesize Activity View appears. In this Activity View you can select a variety of synthesis options, including a synthesis strategy. After you select your options and click Apply, the coreAssembler invokes the required synthesis tools to execute your selected strategy. You can also choose to generate the scripts for the selected strategy without actually initiating the synthesis job. If so, you can run the synthesis job later by invoking the top-level run script.

The options (parameters) for the Perform ASIC Synthesis activity fall into four general categories:

- **Strategy** – Enables you to choose from a list of available synthesis strategies. Each strategy requires you to select values for a set of options (strategy parameters), which are used to control different aspects of the synthesis process. See [“Choosing a Synthesis Strategy”](#) on page 123.
- **Options** – The options related to synthesis execution, such as generating scripts only, and input/output file names. See [“Specifying ASIC Synthesis Execution Options”](#) on page 149.
- **Licenses** – Enables selection of licenses to be used during synthesis. Selecting a license means that coreAssembler can invoke synthesis tool features that require that license. See [“Viewing Licenses for Implementation Tools”](#) on page 150.
- **Reports** – Enables you to select to generate optional synthesis reports, in addition to the default set of reports that coreAssembler extracts from Design Compiler. See [“Selecting Synthesis Reports”](#) on page 151.

The following section presents information on using the available synthesis strategies to achieve the desired results.

**Note**

To use a project-specific Design Compiler setup file (.synopsys_dc.setup), place it in your *workspace/syn* directory.

When the synthesis run is complete, the results can be found in the *workspace/syn/stage* directory, where *stage* is the last completed synthesis stage. The stage names used depend on the strategy and strategy options. For example, a standard invocation based on the Design Compiler Reference Methodology will leave results in the 'initial' directory. In all cases, a symbolic link named 'final' will link to the last generated stage directory.

5.1.1 Synthesis and Integration Methodologies

The methodologies for using the available synthesis strategies to achieve your desired results depend on the synthesis tools you have available. The following sections provide methodologies for:

- [“Working with Design Compiler Strategies”](#)
- [“Working with Fusion Compiler Strategies”](#)
- [“Enabling Physical Synthesis”](#)
- [“Generating Timing Models”](#)
- [“Integrating Generated Scripts”](#)

5.1.1.1 Working with Design Compiler Strategies

The following list shows a typical coreAssembler activity flow for synthesizing and integrating a subsystem, using Design Compiler as the synthesis tool.

1. Start with a configured subsystem for which the synthesis intent has been specified.
2. Initially, use the “[Design Compiler Reference Methodology](#)” and the default QoR effort to establish a baseline set of results.
3. Synthesize the subsystem.
4. Analyze the synthesis results (see [Checking Synthesis Results](#)).

If results do not meet design objectives:

- a. Modify intent or strategy values in the following order and re-synthesize:
 - b. Try setting QoR Effort to high.
 - c. Try one of the incremental strategies in [Table 5-1](#) on page 123.
 - d. Try relaxing overly pessimistic constraints.
 - e. Try over-constraining.
 - f. Try a newer version of Design Compiler (if applicable) to use the latest methodology and tool enhancements.
 - g. Iterate from [step 3](#) as necessary (see [Refining Your Synthesis Intent](#))
5. Results meet design objectives
 6. Integrate the subsystem into the application design (see [Verifying Subsystems and Components](#)).
 7. Perform chip-level design verification and timing analysis (see “[Verifying a Subsystem](#)” on page 175).

5.1.1.2 Working with Fusion Compiler Strategies

The following list shows a typical coreAssembler activity flow for synthesizing and integrating a subsystem, using Fusion Compiler as the synthesis tool.

1. Start with a configured subsystem for which the synthesis intent has been specified.
2. Initially, use the “[Fusion Compiler Reference Methodology](#)” and the default QoR effort to establish a baseline set of results.
3. Synthesize the subsystem.
4. Analyze the synthesis results (see [Checking Synthesis Results](#) on page 161).

If results do not meet design objectives:

- a. Modify intent or strategy values in the following order and re-synthesize:
- b. Try setting QoR Effort to high.
- c. Try one of the incremental strategies in [Table 5-1](#) on page 123.
- d. Try relaxing overly pessimistic constraints.
- e. Try over-constraining.

- f. Try a newer version of Fusion Compiler (if applicable) to use the latest methodology and tool enhancements.
 - g. Iterate from step 3 as necessary (see [Refining Your Synthesis Intent](#) on page 162)
5. Results meet design objectives
 6. Integrate the subsystem into the application design (see [Verifying Subsystems and Components](#) on page 171).
 7. Perform chip-level design verification and timing analysis (see “[Verifying a Subsystem](#)” on page 175).

5.1.1.3 Enabling Physical Synthesis

coreAssembler interfaces with Synopsys DC Graphical to implement a variety of physical synthesis strategies.

5.1.1.3.1 Setting Up for Physical Synthesis

To enable physical synthesis within coreAssembler, first you need to set up your library paths in the “Specify Target Technology” activity. These library paths include:

- Logical library (.db file) – The Physical Synthesis logical library is the same as the Design Compiler target library that you specify when you execute the Specify Target Technology activity. coreAssembler automatically uses your Design Compiler target library as your Physical Synthesis logical library.
- Physical library database – You specify a Milkyway Reference Library and a Milkyway Technology File when you enable Physical Synthesis during the Specify Target Technology activity (Physical Library set-up tab), or the Physical Synthesis option of the Perform ASIC Synthesis activity. This can be a .tf or a .plib file. You can also provide optional files (TLUPlus) which allow the tools to model deep submicron effects with greater accuracy.
- Floorplan file – Floorplan information is optional and can be in DEF or TCL syntax.

Then, to enable Physical Synthesis, check the “Physical Synthesis” check-box in the Physical Synthesis tab of the strategy options (for Design_Compiler_Reference_Methodology).

To enable post-route optimization during physical synthesis, you must set the Routed attribute on the design and supply the following data:

- An SDF file (.sdf) that contains detailed, post-floorplanning delay information for the design. You specify the name of the SDF file as the value of the SynthesisSdfFile attribute on the design.
- A post-floorplanning parasitics file that contains commands to annotate resistance and capacitance values onto the design. You specify the name of the parasitics file as the value of the DesignParasiticsFile attribute on the design.



Note

Fusion Compiler is a physically aware synthesis tool. If the required physical files have been provided, the tool runs physical synthesis. No additional settings are needed or needs to be enabled.

5.1.1.4 Generating Timing Models

Some synthesis strategies include an option to generate a timing model from the final synthesis results. coreAssembler uses Synopsys PrimeTime (license required) to generate the timing model. There are several options available to select how the model is generated. The modeling options are available on the Modeling tab of the Strategy Parameters dialog for the currently selected synthesis strategy (if supported by the strategy).

5.1.1.5 Integrating Generated Scripts

The typical approach for integration of a synthesized subsystem into a chip is to synthesize in coreAssembler and then link the resulting database into the chip-level design. If desired, you can also integrate the subsystem at the script level. In this case, you integrate the scripts generated by coreAssembler into your chip-level scripts. For details, see the “Synthesis_API” man page.

5.1.2 Choosing Synthesis Activities in the GUI

The Perform ASIC Synthesis activity generates and optionally runs synthesis scripts. When you activate the activity by clicking on it in the activity list pane, the corresponding Activity View pane appears. The Synthesize Activity View pane contains the following tabs:

- **Strategy** (for details, see [“Choosing a Synthesis Strategy”](#))
- **Options** (for details, see [Specifying ASIC Synthesis Execution Options](#) on page 149)
- **Licenses** (for details, see [Viewing Licenses for Implementation Tools](#) on page 150)
- **Reports** (for details, see [Selecting Synthesis Reports](#) on page 151)

5.1.2.1 Choosing a Synthesis Strategy

On the Strategy tab, you choose from a list of available synthesis strategies. The selected strategy can be configured to your needs by selecting the “Options” button to the right of the strategy name.

When you select a strategy from the drop-down list, a detailed description of that strategy appears in the Description field to help you select the best strategy for your subsystem. [Table 5-1](#) summarizes the available strategies and the remainder of this section contains a detailed review of each strategy.

Table 5-1 Summary of Synthesis Strategies

Strategy	Description
“Design_Compiler_Reference_Methodology” on page 124	Performs a one-pass, top-down compile from RTL using compile_ultra. This flow implements the Design Compiler Reference Methodology associated with the version of Design Compiler being used. Default.
“Fusion_Compiler_Reference_Methodology” on page 136	Performs a one-pass, top-down compile from RTL using compile_fusion. This flow implements the Fusion Compiler Reference Methodology associated with the version of Fusion Compiler being used.
“DCTCL_quick_map_strategy” on page 147	Performs a quick, low-effort compile to generate a gate-level netlist of the subsystem for analysis purposes only.

The specific set of strategies that are available to you depends on:

- The tools you enabled in the **Edit > Tool Installation Roots** dialog.
- Whether the core developer disabled the use of any tools that are not supported for the subsystem.

Select a synthesis strategy from the pull-down menu and click the Options button to specify values for the strategy parameters. The following sections describe the available strategies (and their associated parameters), as well as the scenarios in which you might use them.

5.1.2.1.1 Design_Compiler_Reference_Methodology

This is the default strategy. It performs a one pass, top-down compile from RTL using compile_ultra. This flow implements the Design Compiler Reference Methodology associated with the version of Design Compiler being used.

Click the Options button to specify values for the Design_Compiler_Reference_Methodology parameters. These parameters are grouped as follows:

- **Basic**
Parameters that control the QoR effort, Initial and Incremental DC Ultra compile options, and Design for Test options (see [Table 5-2](#) on page 124).
- **Netlist Generation**
Parameters that control the Netlist Generation options (see [Table 5-3](#) on page 126).
- **Power**
Parameters that enable Power Optimization, set power constraints, and report power information. (see [Table 5-4](#) on page 126)
- **Physical Synthesis**
Parameters that enable and control a Physical Synthesis run, enable Milkyway, and specify libraries and resources for that run (see [Table 5-6](#) on page 129).
- **Clock Gating**
Parameters that enable and control the use of clock gating during the design elaboration phase of the synthesis run (see [Table 5-7](#) on page 132). By default, clock gating is disabled.
- **Automotive**
Parameters that enable and control the use of TMR and DCLS logic for automotive applications (see [Table 5-9](#) on page 135).



The coreAssembler tool uses the correct Design Compiler Reference Methodology based on the version of Design Compiler used.

Table 5-2 Design_Compiler_Reference_Methodology Basic Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
QoR Effort (QorEffort)	Specifies how much relative CPU time Design Compiler uses for synthesis. (Higher QoR Effort implies longer compile runtime.) The possible values are low, medium, and high. The default is high.

Strategy Parameter <GUI label> (<parameter name>)	Description
Optimization flow (OptimizationFlow)	Specifies the optimization flow, which is disabled by default. Specify one of the following: ☐ <code>hplp</code> : High Performance Low Power ☐ <code>hc</code> : High Connectivity ☐ <code>rtm_exp</code> : Runtime Exploration ☐ <code>tim</code> : Timing Optimization
Reduced Effort Optimization Flow (ReducedEffortOptimizationFlow)	This parameter when enabled, adds <code>-reduced_effort</code> option to the <code>set_qor_strategy</code> command. The <code>set_qor_strategy -reduced_effort</code> option adjusts selected Optimization Flow (<code>hplp</code> or <code>timing</code>) for reduced runtime at the expense of timing QOR. Applicable when Physical Synthesis is enabled.
Compile Ultra	
Enable Adaptive Retiming (AdaptiveRetiming)	Use adaptive retiming timing option <code>'-retime'</code> with the <code>compile_ultra</code> command. This option allows Design Compiler to move registers and latches to improve timing.
Additional Initial Options (CompileUltraInitOptions)	Additional options to pass to an initial <code>compile_ultra</code> command.
Additional Incremental Options (CompileUltraIncrOptions)	Additional options to pass to an incremental <code>compile_ultra</code> command.
SDC Generation	
Options for write_sdc (WriteSDCArgs)	Defines arguments to be passed to the <code>write_sdc</code> command when writing SDC constraints for synthesis.
Technology Node	
Technology Node (TechnologyNode)	Specifies the technology node to be used in <code>set_technology</code> command in Design Compiler NXT. This helps in setting up the Design Compiler NXT for technology-node specific synthesis. The Design Compiler performs validation and coreTools does not validate the technology node value.
SDF Generation	
Write SDF after synthesis (WriteSDF)	Indicates whether an SDF file is generated when synthesis finishes. If enabled, the SDF file is written to <code><workspace>/syn/<stage>/db/<design>.sdf</code> .
ICC2 Write Files	
Write ICC2 files after synthesis (WriteICC2Files)	Indicates whether an ICC2 file is generated when synthesis finishes. If enabled, the ICC2 file is written to <code><workspace>/syn/<stage>/db/ICC2_files</code> .

Table 5-3 Design_Compiler_Reference_Methodology Netlist Generation Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
Netlist Generation	
Netlist Format (NetlistFormat)	Causes synthesis checkpoints to be written in the specified netlist format. The netlist is located in the “db” directory for the checkpoint.
change_names Name Rules (ChangeNamesRuleName)	Indicates the name of a change_names naming rule. Prior to writing the netlist in the specified format, change_names is called using the specified rules.
Remove unconnected ports (RemoveUnconnectedPorts)	Removes unconnected ports from the final netlist at the conclusion of synthesis before the design is written out. Default: 0
set_fix_multiple_port_nets	
Fix Multiple Port Nets Default (FixMultiplePortNetsDefault)	This parameter causes ‘set_fix_multiple_port_nets -default’ to be used in Design Compiler. No buffers are inserted to fix multiple port nets. Default: 1. Unchecking this box enables Feedthroughs, Outputs, Constants and Buffer Constants to be set independently.
Fix Multiple Port Nets Feedthroughs (FixMultiplePortNetsFeedthroughs)	Causes the “set_fix_multiple_port_nets -feedthrough’ to be used in Design Compiler. Default: 0
Fix Multiple Port Nets Outputs (FixMultiplePortNetsOutputs)	Causes the “set_fix_multiple_port_nets -outputs’ to be used in Design Compiler. Default: 0
Fix Multiple Port Nets Constants (FixMultiplePortNetsConstants)	Causes the “set_fix_multiple_port_nets -constants’ to be used in Design Compiler. Default: 0
Fix Multiple Port Nets BufferConstants (FixMultiplePortNetsBufferConstants)	Causes the “set_fix_multiple_port_nets -buffer_constants’ to be used in Design Compiler. Default: 0

Table 5-4 Design_Compiler_Reference_Methodology Power Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
Power Optimization	
Enable Power Optimization (EnablePower)	Enables the power optimization within DC Ultra

Strategy Parameter <GUI label> (<parameter name>)	Description
Power Optimization Constraint (PowerOptimizationConstraint)	These enumerated options are: “set_max_leakage_power” which sets the max_leakage_power constraint to 0, “set_max_dynamic_power”, which sets the max_dynamic_power constraint to 0 or “set_max_total_power” which does both of the above.
Enable SAIF annotation (EnableSaifAnnotation)	Enable the annotation of switching activity from a SAIF file. Switching activity from SAIF is required to enable power optimization. Providing switching activity data also helps increase the accuracy of power analysis with PrimeTime-PX.
Saif File Path (SaifFilePath)	SAIF file to be used for power optimization
Saif Instance Name (SaifInstanceName)	This is the path to the Device Under Test (DUT) in the testbench which generates the switching activity.
Report Power Options (ReportPowerOptions)	Dash options to be passed to the report_power command in DC Ultra
UPF	
Enable UPF Flow (EnableUpf)	When Enable UPF Flow is set to true, the Unified Power Format (UPF) and power constraints files are used during synthesis. This feature is enabled for DC versions 2008.09 and later. In coreAssembler DCTCL_opto_strategy, all subdesigns to be compiled separately must have a UPFFile design attribute.
UPF Flow Type (UPFFlowType)	Use the default UPF flow (UPFPrime) or Golden UPF Flow (GoldenUPF) with golden UPF file and supplemental UPF file written after synthesis.
Incomplete UPF Allowed (IncompleteUPFAllowed)	Allow UPF synthesis flow with a partially configured UPF file.
UPF File (UpfFile)	The name of the UPF file to be used with the Design Compiler load_upf command. This field is initialized with the value of the UpfFile attribute of packaged components, if specified by coreBuilder.
Power Constraints File (PowerConstraintsFile)	The power constraints file is sourced in Design Compiler after loading the UPF file. This file can be used to issue ‘set_voltage’ commands and remove ‘dont_touch’ elements on library cells, as needed.

Table 5-5 Design_Compiler_Reference_Methodology Design for Test Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
Test Ready Compile (TestReadyCompile)	Enables test-ready compile. Primary effect of this parameter is to cause the -scan option to be added to the initial compile call.
Insert DFT (InsertDft)	Enables design-for-test circuitry to be inserted based on the settings from the Specify Test Methodology activity.

Strategy Parameter <GUI label> (<parameter name>)	Description
Internal_scan mode chain count. (NumberOfScanChains)	The number of scan chains to be created. This value is passed to the set_scan_configuration command with the -chain_count option. For Internal_scan, this is also the number of scan IO ports to be created. The default value is the design attribute NumberOfScanChains set on the implementation context.
DFT Synthesis Optimization (DFTSynthesisOptimization)	Specify the value to perform optimizations during DFT insertion. 'all' to turn on all optimizations, and 'none' to turn off all optimizations. The attribute specifies the argument to 'set_dft_insertion_configuration -synthesis_optimization'. In the topographical mode, the only allowed value is none and it cannot be changed.
CTL Models (CTLModels)	CTL Model files are used to describe the test model required to generate test vectors for the design. This strategy parameter is read in the context of top-level synthesis (during both flat or bottom-up synthesis) whereas, the design attribute CTLModels is only used during bottom-up synthesis runs. This value set here is passed onto the DCRM variable DCRM_DFT_CTL_FOR_ABSTRACT_BLOCKS along with the user specified CTLModels in the design attributes and the TechnologyCellBinding activity.
Scan Compression Settings	
Enable scan compression (EnableScanCompression)	Enables ScanCompression_mode in DFT implementation. The default value is the design attribute EnableScanCompression set on the implementation context. For more information about scan compression, refer to DFTMax and DFTMax Ultra user guides.
ScanCompression_mode IO count. (ScanCompressionIOCount)	The number of chains for Internal_scan and ScanCompression_mode may be different. E.g. hybrid flow during bottom-up synthesis. This strategy parameter sets the number of scan IOs for compression mode. The default value is the design attribute ScanCompressionIOCount set on the implementation context.
Scan compression chain count. (NumberOfScanCompressionChains)	In ScanCompression_mode, this parameter sets the number of scan chains to be created internally. The default value is the design attribute NumberOfScanCompressionChains set on the implementation context.
Scan compression usage (ScanCompressionUsage)	Sets the type of compression used. The default value is the design attribute ScanCompressionUsage set on the implementation context.

Strategy Parameter <GUI label> (<parameter name>)	Description
Scan compression test clock (ScanCompressionTestClock)	Sets a test clock for scan compression from the list of test clocks available.
Scan compression configuration (ScanCompressionConfiguration)	The value of this parameter is passed directly to the 'set_scan_compression_configuration' command. For details on DFTMAX compression options, refer to the DFTMAX User Guide, Chapter 2, 'Using DFTMAX Compression' and Chapter 4, 'Managing X Values in Scan Compression'.

Note that most of the Physical Synthesis parameter settings in [Table 5-6](#) are also accessible from the “Physical Library setup” tab of the Specify Target Technology activity.

Table 5-6 Design_Compiler_Reference_Methodology Physical Synthesis Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
Physical Synthesis (PhysicalSynthesis)	Use DC Topographical. Setting this parameter to true causes the physical synthesis placement engine to be used during optimization. Net loads and delays are estimated by placement, and wire load models are not used. Note that no placement data is saved with the ddc or Milkyway database.
Enable SPG (EnableSPG)	Enables the Synopsys Physical Guidance flow in DC-Topographical.
Floorplan Data Format (FloorplanFileDataFormat)	Adds support for the read_floorplan command. Legal values for FloorplanFileDataFormat are 'floorplan', 'def' and 'tcl'. - floorplan: (Default) floorplan data written by the 'write_floorplan' command. - def: file formats covered by the old 'DefPhysicalConstraintFile' parameters. - tcl: file formats covered by the old 'TclPhysicalConstraintFile' parameters.
Physical Constraint File (PhysicalConstraintFile)	Adds support for the read_floorplan command. String provides the path to the Physical Constraint File. The FloorplanFileDataFormat parameter sets the format for this file.
Enable Port Side Constraint (EnablePortSideConstraint)	Generates a port side file which is inserted into the synthesis flow for better pin placement. This file can only be generated when there is no floorplan file given and the SideConstraint attribute is set on the design port by the user. This can be used only in topographical mode. This feature is enabled only when the DC version is >= 2018.06.
Utilization Ratio (UtilizationRatio)	Specifies the utilization ratio as part of the synthesis intent. The default value is 0.7 and the values range from 0.0 to 1.0. The utilization ratio is applicable only when the floorplan data is not provided.

Table 5-6 Design_Compiler_Reference_Methodology Physical Synthesis Parameters (continued)

Strategy Parameter <GUI label> (<parameter name>)	Description
Min Routing Layer Name (MinRoutingLayer)	Specifies the minimum routing layer for the design. The router respects this setting. To ensure that RC estimation and congestion analysis stay synchronized with the router, when you set this option, the tool marks all routing layers below the specified layer as fully blocked for RC and congestion estimation.
Max Routing Layer Name (MaxRoutingLayer)	Specifies the maximum routing layer for the design. The router respects this setting. To ensure that RC estimation and congestion analysis stay synchronized with the router, when you set this option, the tool marks all routing layers above the specified layer as fully blocked for RC and congestion estimation.
Congestion Optimization Factor (CongestionOptimizationFactor)	Enables user to specify the maximum utilization factor for congestion optimization. This specifies how the Design Compiler can densely pack the cells in the uncongested regions. The default value is set to 0.9 in Design_Compiler_Reference_Methodology. This feature can only be configured when Physical Synthesis is enabled along with "OptimizationFlow" set to High Performance Low Power (hplp) or Timing Optimization (Tim).
Use Milkyway (UseMilkyway)	Use the Milkyway database format. Note that the Milkyway design library must have been created prior to specifying the Milkyway parameters. See Milkyway documentation for more information on the Milkyway format.
Milkyway Design Library (MilkywayDesignLibrary)	This parameter corresponds to the Design Compiler variable <code>mw_design_library</code> . Specify the full path to the Milkyway design library. The Milkyway design library must exist before specifying this parameter. <code>coreTools</code> does not create it.
Create Milkyway Design (CreateMilkywayDesign)	If true, <code>coreAssembler</code> creates the Milkyway design library. Default: 1 (true)
Tech File Format (TechFileFormat)	Choose "tf" to create the tech file to create the Milkyway design library. Choose "plib" to use the physical library source file (.plib). Default: tf
Tech File (TechFile)	Technology file used to create the Milkyway design library. The file must match the format specified in "Tech File Format."
Milkyway Reference Library (MilkywayReferenceLibrary)	This parameter corresponds to Design Compiler variable <code>mw_reference_library</code> . Specify the full path to the Milkyway reference library. The Milkyway reference library must exist before specifying this parameter. <code>coreTools</code> does not create it.
Max Tlu Plus File (MaxTluPlusFile)	The Max TLU+ file.
Min Tlu Plus File (MinTluPlusFile)	The Min TLU+ file.
Tech file to ITF Mapping File (Tech2ItfMap)	TECH2ITF mapping file name.

Table 5-6 Design_Compiler_Reference_Methodology Physical Synthesis Parameters (continued)

Strategy Parameter <GUI label> (<parameter name>)	Description
Extend MW Layers (ExtendMWLayers)	Inserts the extend_mw_layers command into DC topo scripts. Enable this parameter when the synthesis library requires more than the standard 255 layers in DC-Topographical synthesis. Default: 0 (FALSE)
Max Emulation TLU+ file name (MaxEmulationTluPlus)	Used for the -max_emulation_tluplus option to set_tlu_plus_files command. See the Design Compiler documentation for information about the set_tlu_plus_files command.
Min Emulation TLU+ file name (MinEmulationTluPlus)	Used for the -min_emulation_tluplus option to set_tlu_plus_files command. See the Design Compiler documentation for information about the set_tlu_plus_files command.
Create Mw Design Options (CreateMwDesignOptions)	Additional options for the create_mw_design command. See the Design Compiler or Milkyway documentation for options.
Milkyway Logic 0 Net (MilkywayLogic0Net)	This parameter corresponds to the Design Compiler variable mw_logic0_net. Contains the Milkyway logic0(1'b0) net for the design. Default: VSS
Milkyway Logic 1 Net (MilkywayLogic1Net)	This parameter corresponds to the Design Compiler variable mw_logic1_net. Contains the Milkyway logic1(1'b1) net for the design. Default: VDD
Save design in Milkyway Cell format (WriteMilkywayCell)	Milkyway Cell representation of the design is written out into the directory specified by the MilkywayDesignLibrary parameter at the end of synthesis. Default: False (no write occurs)
Create Congestion Map (CreateCongestionMap)	Create graphical congestion map when synthesis ends. To use this feature, you must synthesize with DC-Topographical 2010.03 or newer. This feature also requires DC-Graphical with a DC-Extension license. Your DISPLAY variable must be set to use this feature. Default: False
NDM Library	
Use NDM (UseNDM)	This switch enables NDM view to extract the physical information. NDM view is only supported in the DCRM version S-2021.06-SP4 or greater. Milkyway view is supported in the older versions of DCRM templates.
NDM Library Name (NDMLibraryName)	Specifies the name of the new NDM library. The path to the NDM library can be a relative, or an absolute path. If a relative path is given, the directory will be created in the workspace 'syn' directory. The name must not contain spaces or the colon (:) character, which is used to delimit the library and design name.

Table 5-6 Design_Compiler_Reference_Methodology Physical Synthesis Parameters (continued)

Strategy Parameter <GUI label> (<parameter name>)	Description
NDM Reference Library (NDMReferenceLibrary)	Specifies the reference libraries for the NDM library 'NDMLibraryName'. You can specify the reference libraries and physical source data using absolute or relative paths. If you specify the libraries with a relative path or with no path, the tool uses the TechSetup's activity parameter 'SearchPath' to locate the libraries. The NDM reference library must exist before specifying this parameter. coreTools will not create it.
Create Lib Design Options (CreateLibDesignOptions)	Additional options to the create_lib command which creates NDM library. Please see Design Compiler NXT or NDM library documentation for details.
Multi Vth Constraint	
Available Threshold Voltage Groups (AvailableVthGroups)	The voltage groups that are available to the user.
Low Threshold Voltage Groups (LowThresholdVoltageGroups)	Specifies the threshold voltage groups considered to be low-threshold voltage groups. The groups argument is a list of threshold voltage group identifiers, which are names defined by attributes in the Liberty (.lib) descriptions of the cells and cell librarian.
Low Threshold Voltage Percentage (LowThresholdVoltagePercentage)	Specifies the maximum allowable percentage of low-threshold voltage cells for the current design. The percent_value argument is a floating-point number between 0 and 100.
Low Threshold Voltage Constraint Type (LowThresholdVoltageConstraintType)	Specifies the constraint type, which can be either hard or soft The tool meets this constraint only if it does not adversely affect the timing constraint. Default: soft

Table 5-7 Design_Compiler_Reference_Methodology Clock Gating Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
Insert Clock Gating (ClockGating)	Add clock gating logic using the command insert_clock gating, as defined by the other clock gating parameters.
Insert Self Gating (SelfGating)	Self-gating is a clock gating technique that optimizes the dynamic power by gating the clock signal in those cycles in which the data saved in a register remains unchanged. An enable condition is computed by comparing the stored data with the new data arriving at the data pin, and that signal is used to drive the inserted self-gating cell. Enabling this adds the -self_gating option to the compile_ultra command. This option is available only when Physical Synthesis and Clock Gating are enabled. This parameter updates the control variable

Table 5-7 Design_Compiler_Reference_Methodology Clock Gating Parameters (continued)

Strategy Parameter <GUI label> (<parameter name>)	Description
	INSERT_SELF_GATING in ct_variable_setup.tcl present in the rm_dc_setup folder.
Gating Minimum Bit Width (GatingMinimumBitWidth)	Specifies the minimum bit-width of a register bank for which to gate the clock. (Corresponds to the -minimum_bitwidth option for Design Compiler's set_clock_gating_style command.) Default: 3
Latch Style Clock Gating or Combinational Clock Gating (GatingLatchCellName)	Provides a choice between latch-style and combinational gating. If you choose latch, you can enter the name of a library cell for latching the gated clock enable signal in the Gating Latch Cell Name text box. If you choose latch but do not enter a cell name, Design Compiler chooses a latch cell from the target library. (Corresponds to the -sequential_cell option for Design Compiler's set_clock_gating_style command.) If you select Combinational, GatingLatchCellName is set to none and generates a -sequential_cell none option.
Gating And Function (GatingAndFunction)	Specifies the circuitry to use to implement AND/NAND gating. The valid enum values are: integrated, and, buf and, and buf, buf and buf, nand inv, buf nand inv, inv or inv, inv nor, inv nor buf, and inv, buf and inv, nand, buf nand, nand buf, buf nand buf, inv or, inv or buf, and inv nor inv. The default is and. Your selected AND function and your choice of Latch Style or Combinational Clock Gating are used to select the AND function value for the -positive_edge_logic and -negative_edge_logic options to Design Compiler's set_clock_gating_style command. For latch-style gating, Design Compiler inserts the selected AND circuitry into the clock network for positive edge-triggered registers. For combinational-style gating, Design Compiler inserts the selected AND circuitry into the clock network for negative edge-triggered registers. If the appropriate ICG cells are available in the library, this value defaults to 'integrated'.
Integrated And Cell (IntegratedAndCell)	When the Gating And Function is set to the value "integrated," the IntegratedAndCell parameter is enabled. This parameter can be set to the name of the cell to use from the technology library.
Gating Or Function (GatingOrFunction)	Specifies the circuitry to use to implement OR/NOR gating. The valid enum values are: integrated, or, buf or, or buf, buf or buf, nor inv, buf or inv, inv and inv, inv nand, inv and buf, or inv, buf or inv, nor, buf nor, nor buf, buf nor buf, inv and, inv and buf, and inv nand inv. The default is or. Your selected OR function and your choice of Latch Style or Combinational Clock Gating are used to select the OR function value for the -positive_edge_logic and -negative_edge_logic options to Design Compiler's set_clock_gating_style command. For latch-style gating, Design Compiler inserts the selected OR circuitry into the clock network for negative edge-triggered registers. For combinational-style gating, Design Compiler inserts the selected OR circuitry into the clock network for positive edge-triggered registers. If the appropriate ICG cells are available in the library, this value will default to 'integrated'.

Table 5-7 Design_Compiler_Reference_Methodology Clock Gating Parameters (continued)

Strategy Parameter <GUI label> (<parameter name>)	Description
Integrated Or Cell (IntegratedOrCell)	When the Gating Or Function is set to the value “integrated,” the IntegratedOrCell parameter is enabled. This parameter can be set to the name of the cell to use from the technology library.
Gating Setup Time (GatingSetupTime)	Specifies the setup time constraint at the enable input of the 2-input clock gate in the clock gating circuitry. (Corresponds to the -setup option for Design Compiler's set_clock_gating_style command.) Default: 0
Gating Hold Time (GatingHoldTime)	Specifies the hold time constraint at the enable input of the 2-input clock gate in the clock gating circuitry. (Corresponds to the -hold option for Design Compiler's set_clock_gating_style command.) The default is 0.
Gating Control Point (GatingControlPoint)	Specifies if and where to insert an Or gate to control the gating signal for testability. (Corresponds to the -control_point option for Design Compiler's set_clock_gating_style command.) The allowed values are none, before, and after. Default: none
Gating Control Signal (GatingControlSignal)	Specifies the control signal for the gating control point during testing (ignored when Gating Control Point is set to none). The choices are scan_enable and test_mode. (Corresponds to the -control_signal option for Design Compiler's set_clock_gating_style command.)
Implement Observation Logic (GatingObservationLogicDepth)	Specifies whether to implement the circuitry necessary to observe gating behavior within the design. The default is unchecked (no value for GatingObservationLogicDepth; no observation logic implemented). (Corresponds to the -observation_point option for Design Compiler's set_clock_gating_style command.)
Gating Observation Logic Depth (GatingObservationLogicDepth)	Specifies the maximum depth of the XOR observability tree. (Corresponds to the -observation_logic_depth option for Design Compiler's set_clock_gating_style command.) Gating Observation Logic Depth is enabled only when Implement Observation Logic is checked.
Specify Clock Gating Cell Max Fanout (SpecifyClockGatingMaxFanout)	Enables specifying the maximum fanout of a single clock-gating cell. If not specified, or no value is provided, Design Compiler uses its default. Default: 1
Gating Cell Max Fanout Value (ClockGatingMaxFanoutValue)	Value used for the maximum fanout of a single clock-gating cell. If no value is provided, Design Compiler uses the default. Default: none

Table 5-8 Design_Compiler_Reference_Methodology Hierarchical Synthesis Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
Enable Hierarchical Synthesis (EnableHierarchicalSynthesis)	Setting the parameter to true enables the hierarchical flow. When enabled, the DDC for the sub-block is used in the Subsystem level synthesis. The flow is only enabled in coreAssembler. Default: 0
Perform Incremental Compile (IncrementalCompileForHierarchical Synthesis)	Setting the parameter to true enables compiler_ultra to be called with -incremental. The flow is only enabled in coreAssembler. The flow is based on creating block abstractions. When compile_ultra -incremental is issued, the dont_touch attribute on the sub-blocks are set to false. Default: 0

Table 5-9 Design_Compiler_Reference_Methodology Automotive Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
AutomotiveSynthesis Flow Type (AutomotiveSynthesisFlowType)	Specifies an automotive synthesis flows, such as triple module redundancy (TMR), dual core lockstep (DCLS), or TMR_and_DCLS The list of valid values depends on which intents are packaged with the IP. To enable this parameter when using DC, you must set up your synthesis run to be “physical aware” as follows: ■ Enable the PhysicalSynthesis synthesis option (described in Table 5-6) ■ Enable the UseDCNXT synthesis option (described in Table 5-24)
TMR Separation Distance (TMRSeparationDistance)	Specifies the distance values for repelling bound constraint for redundancy group This entry can be either an {x y} pair (for example, 1 1) or a single number denoting radial distance.
DCLS Distance (DCLSDistance)	Specify the DCLS distance as a pair of {width height}. This creates a group bound with the specified dimensions.

Table 5-10 Design_Compiler_Reference_Methodology Reports Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
Generate supplementary Reports (GenerateRMReports)	Enables report generation commands specified in the Reference Methodology. Default: 0
Skip Congestion Report (DisableCongestionReport)	Disables the congestion report generation. It can be used for runtime improvements.

Table 5-10 Design_Compiler_Reference_Methodology Reports Parameters (continued)

Strategy Parameter <GUI label> (<parameter name>)	Description
	Default: 1

5.1.2.1.2 Fusion_Compiler_Reference_Methodology

The Fusion_Compiler_Reference_Methodology strategy performs a one-pass, top-down compile from RTL using compile_fusion. This strategy is available in coreConsultant under the following conditions:

- Your core IP supports Fusion Compiler.
- You have specified an NDM library as explained in “[Logical Library Setup](#)” on page 82.
- You have specified a supported version of Fusion Compiler using the **Edit > Tool Installation Roots** menu, as explained in “[Set Tool Installation Root](#)” on page 83 .

When you have selected the Fusion_Compiler_Reference_Methodology strategy, click the Options button to specify values for the Fusion_Compiler_Reference_Methodology parameters. These parameters are grouped as follows:

- **Basic** (see [Table 5-11](#) on page 136)
Parameters that control the flow, technology, Design for Test, and UPF options.
- **Constraints** (see [Table 5-13](#) on page 141)
Parameters to specify the Multi Vth constraint.
- **Automotive** (see [Table 5-21](#) on page 146)
Parameters that control the use of TMR and DCLS logic for automotive applications.

Table 5-11 Fusion_Compiler_Reference_Methodology Basic Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
Flow	
Enable QIK for DesignWare Interface IP (EnableQIK)	Enables the Quick Implementation Kit (QIK) for DesignWare Interface IP. This flow is a version of the Fusion Compiler Reference Methodology which has been tuned for Synopsys DesignWare Interface IP. Requires Fusion Compiler 2022.12 or later to be enabled. Default: 0
Final FCRM Flow Step (FinalFCRMFlowStep)	Indicates how far the FCRM flow should run. The specified step will be the last one run in the FCRM flow. The legal values are: ☐ {place_opt (classic flow to final_opto)} ☐ {compile (unified flow to final_opto)} ☐ {route_opt (unified flow to final_opto + clock_opt and route_opt)}

Strategy Parameter <GUI label> (<parameter name>)	Description
	Default: Compile
QoR	
QoR Strategy (QorStrategy)	The selected strategy is passed into the set_qor_strategy -metric <strategy> Appropriate application options are set to optimize the specified target strategy. The legal values are: ☐ Timing (timing) ☐ Leakage Poser (leakage_power) ☐ Total Power (total_power) Default: Timing
Enable Reduced Effort (EnableReducedEffort)	Sets the -reduced_effort option for the set_qor_strategy command Default: 0 (false)
FloorPlan	
Floorplan Data Format (FloorplanFileDataFormat)	Format of file containing floorplan constraints The legal values are: ☐ def: DEF file formats ☐ tcl: TCL file formats
Physical Constraint File (TCLPhysicalConstraintFile)	Specifies the path to the Physical Constraint File ☐ For the DEF format, populates the 'DEF_FLOORPLAN_FILES' setup parameter. ☐ For the TCL format, populates the 'TCL_FLOORPLAN_FILE' setup parameter.
Enable Port Side Constraint (EnablePortSideConstraint)	Generates a port side file which is inserted into the synthesis flow for better pin placement. This file can only be generated when there is no floorplan file given and the SideConstraint attribute is set on the design port by the user.
Utilization Ratio (UtilizationRatio)	Specifies the utilization ratio as part of the synthesis intent. The default value is 0.7 and the values range from 0.0 to 1.0. The utilization ratio is applicable only when the floorplan data is not provided.
SDC Generation	
Options for write_sdc (WriteSDCArgs)	Defines arguments to be passed to the write_sdc command when writing SDC constraints for synthesis
TechFile	
Use Technology File (UseTechFile)	Use technology file to specify technology information

Strategy Parameter <GUI label> (<parameter name>)	Description
	When enabled, the target library (TECH_LIB) setup is not used. Default: 0 (false)
TechFile (TechFile)	Technology file used to create the Milkyway design library (read-only) You can set the file path in the Specify Target Technology activity (TechSetup), as described in “Physical Library Setup” on page 83.
Enable Library Configuration Flow (EnableLibraryConfigurationFlow)	Use the library configuration flow, which calls the library manager under the hood to generate .nlibs Default: 0 (false)
Technology Node	
Technology Node (TechnologyNode)	Specifies the technology node to be used in set_technology command in Fusion Compiler. This helps in setting up the Fusion Compiler for technology-node specific synthesis. Fusion Compiler performs validation and coreTools does not validate the technology node value.
Technology Reference Methodology Scripts (TechnologyRMScripts)	This parameter captures the path to the rm_tech_scripts delivered by Synopsys. When using QIK, it is recommended for the user to specify the path to the rm_tech_scripts.
Set technology after floorplan (SetTechnologyAfterFloorplan)	Setting of technology for nodes to be performed after floorplanning. The feature is available if Fusion Compiler version is 2022.03-SP1 or greater when TechnologyNode has been specified and either PhysicalConstraintFile is specified or Design Plan Flat Flow (EnableDesignPlanFlatFlow) is enabled. Default: 0 (false)
Technology Side File	
Technology Sidefile Settings (TechnologySideFileSettings)	Sets the Reference Methodology control variables. The parameter accepts a list of control variables and values. The information is exported in the 'syn/rm_fc_snps_ip_scripts/snps_ip_technology_sidefile_setup.tcl' file. The parameter is disabled when the QIK flow is enabled (EnableQIK is set to true). Default: Not Applicable
TLUPlus	
Early parasitic tech specification name (MinTluPlusName)	Specifies the early parasitic technology specification name
Min TLUPlus file name (MinTluPlusFile)	TLU+ files to read for Min specification If TechSetup's MinTluPlusFile activity parameter has been specified, the same file path is used for this parameter.

Strategy Parameter <GUI label> (<parameter name>)	Description
Min TLUPlus Layer map file (MinTluPlusLayerMapFile)	Layer mapping file between the ITF and Min technology specification
Late parasitic tech specification name (MaxTluPlusName)	Specifies the late parasitic technology specification name
Max TLUPlus file name (MaxTluPlusFile)	TLU+ files to read for Max specification If the TechSetup MaxTluPlusFile activity parameter has been specified, the same file path is used for this parameter.
Max TLUPlus Layer map file (MaxTluPlusLayerMapFile)	Layer mapping file between ITF and Max technology specification
UPF	
Enable UPF Flow (EnableUPFFlow)	When set to true, the UPF and power constraints files are used during synthesis Default: False
UPF File (UPFFile)	Specifies the name of the Unified Power Format (UPF) file to be used with the Fusion Compiler
Power Constraints File (PowerConstraintsFile)	Specifies the power constraints file The power constraints file is sourced in Fusion Compiler after loading the UPF file. This file can be used to issue 'set_voltage' commands and remove do_touch elements on library cells, as needed.
Multi Voltage Setup File (MVSetupFile)	Tcl script placeholder for your multi-voltage setup file, which contains details such as create_voltage_area, placement bound, power switch creation, level shifter insertion, and so on.

Table 5-12 Fusion_Compiler_Reference_Methodology_Design for Test Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
Insert DFT (InsertDft)	Enables design-for-test circuitry to be inserted based on the settings from the Specify Test Methodology activity. Default: False
DFT Flow Type (DFTFlowType)	Sets the DFT_Flow control variable to in_compile or pre_compile Default: pre_compile
Internal_scan mode chain count. (NumberOfScanChains)	The number of scan chains to be created. This value is passed to the set_scan_configuration command with the -chain_count option.

Strategy Parameter <GUI label> (<parameter name>)	Description
	For Internal_scan, this is also the number of scan IO ports to be created. The default value is the design attribute NumberOfScanChains set on the implementation context.
CTL Models (CTLModels)	CTL Model files are used to describe the test model required to generate the test vectors for the design. This strategy parameter is read in the context of top-level synthesis (during both flat or bottom-up synthesis) whereas, the design attribute CTLModels is only used during the bottom-up synthesis runs. This value set here is passed onto the FCRM variable CTL_FOR_ABSTRACT_BLOCKS along with the user specified CTLModels in the design attributes and the TechnologyCellBinding activity. Default: Null
Scan Compression Settings	
Enable scan compression (EnableScanCompression)	Enables ScanCompression_mode in DFT implementation. The default value is the design attribute EnableScanCompression set on the implementation context. For more information about scan compression, refer to DFTMax and DFTMax Ultra user guides.
Enable Distributed Codec (EnableDistributedCodec)	Enables Distributed Codec insertion in design. Can only be enabled when both EnableScanCompression and EnableTestMaxManagerFlow strategy parameters are enabled. For more information on scan compression, see the DFTMax and DFTMax Ultra User Guides. Default: 0
ScanCompression_mode IO count. (ScanCompressionIOCount)	The number of chains for Internal_scan and ScanCompression_mode may be different. E.g. hybrid flow during bottom-up synthesis. This strategy parameter sets the number of scan IOs for compression mode. The default value is the design attribute ScanCompressionIOCount set on the implementation context.
Scan compression chain count. (NumberOfScanCompressionChains)	In ScanCompression_mode, this parameter sets the number of scan chains to be created internally. The default value is the design attribute NumberOfScanCompressionChains set on the implementation context.
Scan compression usage (ScanCompressionUsage)	The value passed along with -usage option for ScanCompression_mode test mode in the define_test_mode command. See DFTMAX, DFTMAX Ultra and DFT SEQ Compression sections in the TestMAX DFT User Guide for more details. The default value for this strategy parameter is the value of the design attribute 'ScanCompressionUsage' set on the implementation context.
SEQ Compression Inputs (SeqCompressionInputs)	Total number of SEQ Compression input ports. Default: 7
SEQ Compression Outputs	Total number of SEQ Compression output ports.

Strategy Parameter <GUI label> (<parameter name>)	Description
(SeqCompressionOutputs)	Default: 1
Scan compression test clock (ScanCompressionTestClock)	Sets a test clock for scan compression from the list of test clocks available.
Scan compression configuration (ScanCompressionConfiguration)	The value of this parameter is passed directly to the 'set_scan_compression_configuration' command. For details on DFTMAX compression options, refer to the DFTMAX User Guide, Chapter 2, 'Using DFTMAX Compression' and Chapter 4, 'Managing X Values in Scan Compression'.
TestMAX Manager Settings	
Enable TestMAX Manager Flow (EnableTestMaxManagerFlow)	Enables TestMax Manager to introduce DFT infrastructure within the Designware RTL prior to synthesis. Such RTL with DFT can then be processed by the Fusion Compiler. To enable this feature the following conditions are to be met: ☐ InsertDft set to 1 ☐ The leaf level components must be TestMax Manager ready ☐ Fusion Compiler version must be 2022.03 or greater ☐ ScanCompressionUsage is not set to scan_compression ☐ TestMAX Manager version must be 2022.12-SP3 or greater ☐ Spyglass version must be 2023.03 or greater ☐ Not supported for encrypted license.
Enable Test point insertion (EnableTPI)	Enables the TestMax Manager to introduce the test points in the design RTL prior to synthesis. Such RTL with DFT can then be processed by the Fusion Compiler. The feature can only be enabled when ScanCompressionUsage is not set to scan_compression.

Table 5-13 Fusion_Compiler_Reference_Methodology_Constraints_Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
Routing Layer	
Min Routing Layer Name (MinRoutingLayer)	Specifies the minimum routing layer for the design. The router respects this setting. To ensure that RC estimation and congestion analysis stay synchronized with the router, when you set this option, the tool marks all routing layers below the specified layer as fully blocked for RC and congestion estimation.

Strategy Parameter <GUI label> (<parameter name>)	Description
Max Routing Layer Name (MaxRoutingLayer)	Specifies the maximum routing layer for the design. The router respects this setting. To ensure that RC estimation and congestion analysis stay synchronized with the router, when you set this option, the tool marks all routing layers above the specified layer as fully blocked for RC and congestion estimation.
Multi Vth Constraint	
Available Vth Groups (AvailableVthGroups)	The Voltage Groups available to the user
Low Threshold Voltage Groups (LowThresholdVoltageGroups)	A list of threshold voltage group names which are considered as low-threshold voltage groups, stored as threshold_voltage_group attributes present in the liberty or NDM description of the cell
Low Threshold Voltage Percentage (LowThresholdVoltagePercentage)	Specifies the allowable low Vth cell percentage limit on the current design, based on cost type This is a required setting. Default: 100
Low Threshold Voltage Percentage (LowThresholdVoltageCost)	Specifies how low Vth percentage limit is calculated The low Vth percentage can be based on cell count or cell area. Default: cell_count

Table 5-14 Fusion_Compiler_Reference_Methodology Power Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
Power Optimization	
Saif File Path (SaifFile)	List of SAIF files for power aware synthesis.
Saif Instance Name (SaifInstanceName)	Name of the instance of the current design as it appears in SAIF file. This argument is passed on to the read_saif command with the -strip_path option
UPF	
Enable UPF Flow (EnableUPFFlow)	When set to true, the UPF and power constraints files are used during synthesis Default: False
UPF File (UPFFile)	Specifies the name of the Unified Power Format (UPF) file to be used with the Fusion Compiler
Power Constraints File (PowerConstraintsFile)	Specifies the power constraints file

Strategy Parameter <GUI label> (<parameter name>)	Description
	The power constraints file is sourced in Fusion Compiler after loading the UPF file. This file can be used to issue 'set_voltage' commands and remove do_touch elements on library cells, as needed.
Multi Voltage Setup File (MVSetupFile)	Tcl script placeholder for your multi-voltage setup file, which contains details such as create_voltage_area, placement bound, power switch creation, level shifter insertion, and so on.

Table 5-15 Fusion_Compiler_Reference_Methodology Parasitic Definition Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
Parasitic n (where, n=1/2/3/4)	
Parasitic Model Name for Model n (ParasiticModelNamen)	Specifies the unique name which identifies this parasitic model. Default: Not Applicable
TLUPlus File for Model n (TluPlusFileForParasitich)	Specifies the names of the TLUPlus file or common nxtgrd file to read for this parasitic model. Default: Not Applicable
Layer Map File for Model n (LayerMapFilen)	Specifies the names of the layer map file between the technology file and the ITF file for this parasitic model. Default: Not Applicable

Table 5-16 Fusion_Compiler_Reference_Methodology Mode Definition Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
Mode n (where, n=1/2/3/4)	
Name for Mode n (ModeNamen)	Defines a mode to be created using the create_mode command. Each mode must have a unique name. Default: Not Applicable
Mode Constraint File for Mode n (ModeConstraintFilen)	Constraint file used to set up mode n. Default: Not Applicable

Table 5-17 Fusion_Compiler_Reference_Methodology Corner Definition Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
Index of the Primary Corner	Sets application option cts.compile.primary_corner to the selected corner.

Strategy Parameter <GUI label> (<parameter name>)	Description
(PrimaryCorner)	Default: 1
Corner n (where, n=1/2/3/4/5/6/7/8/9)	
Name for Corner n	Defines a corner to be created using the create_corner command. A corner being used for set-up time analysis must have SETUP in the name. Similarly, a corner used for hold time analysis must have HOLD in the name. Default: Not Applicable
Corner Constraint File for Corner n	Specifies the corner constraint file associated with corner n. Default: Not Applicable

Table 5-18 Fusion_Compiler_Reference_Methodology Scenario Definition Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
Scenario n (where n=0/1/2/3/4/5/6/7/8/9)	
Name for Scenario n (ScenarioNamen)	Defines a scenario to be created with the create_scenario command. The entries must be named <MODE NAME>::<CORNER NAME>, where <MODE NAME> is a defined mode and <CORNER NAME> is a defined corner. Default: Not Applicable
Active Scenario List	
Active Scenario List for compile (ActiveScenarioListcompile)	Defines the set of active scenarios for the 'compile' stage. Values must match entries in the scenario definition parameters. If not specified, all scenarios are considered active. Default: Not Applicable
Active Scenario List for clock_opt_cts (ActiveScenarioListclock_opt_cts)	Defines the set of active scenarios for the 'clock_opt_cts' stage. Values must match entries in the scenario definition parameters. If not specified, active scenarios from the prior stage will be used. If not specified for any stage, all scenarios are considered active. Default: Not Applicable
Active Scenario List for clock_opt_opto (ActiveScenarioListclock_opt_opto)	Defines the set of active scenarios for the 'clock_opt_opto' stage. Values must match entries in the scenario definition parameters. If not specified, active scenarios from the prior stage will be used. If not specified for any stage, all scenarios are considered active. Default: Not Applicable

Strategy Parameter <GUI label> (<parameter name>)	Description
Active Scenario List for route_auto (ActiveScenarioListroute_auto)	Defines the set of active scenarios for the 'route_auto' stage. Values must match entries in the scenario definition parameters. If not specified, active scenarios from the prior stage will be used. If not specified for any stage, all scenarios are considered active. Default: Not Applicable
Active Scenario List for route_opt (ActiveScenarioListroute_opt)	Defines the set of active scenarios for the 'route_opt' stage. Values must match entries in the scenario definition parameters. If not specified, active scenarios from the prior stage will be used. If not specified for any stage, all scenarios are considered active. Default: Not Applicable

Table 5-19 Fusion_Compiler_Reference_Methodology Cell Pruning Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
Don't Use Library Cells (DontUseLibCells)	Defines library cells that should not be used throughout the flow. Typically consists of very small and very large drive strength cells, and non-typical use cells. This parameter updates the FCRM control variable DONT_USE_LIB_CELL_LIST. Default: No Default Value
Valid Optimization Buffers (ValidOptBuffers)	Defines buffer cells to be used during all optimization stages except for 'cts'. Typically includes low and medium strength buffers across all voltage threshold classes. Buffer cells not listed here will be blocked from use. This parameter updates the FCRM control variable VALID_OPT_BUFFER_LIST. Default: No Default Value
Valid Optimization Inverters (ValidOptInverters)	Defines inverter cells to be used during all optimization stages except for 'cts'. Typically includes low and medium strength inverters across all voltage threshold classes. Inverter cells not listed here will be blocked from use. This parameter updates the FCRM control variable VALID_OPT_INVERTER_LIST. Default: No Default Value
Valid Hold Buffers (ValidHoldBuffers)	Defines buffer cells to be use for fixing hold time violations. Typically includes delay buffers and low drive strength buffers, usually of the slowest voltage threshold class available. This parameter updates the FCRM control variable VALID_HOLD_BUFFER_LIST. Default: No Default Value
Valid ICG Cells (ValidICGCells)	Defines the integrated clock gating (ICG) cells to be used. Typically includes ICG cells of a variety of drive strengths, usually of the fastest voltage threshold class available. ICG cells not listed here will be blocked from use. This parameter updates the FCRM control variable VALID_ICG_CELL_LIST.

Strategy Parameter <GUI label> (<parameter name>)	Description
	Default: No Default Value
Valid CTS Repeaters (ValidCTSRepeaters)	Defines valid buffer/inverter cells to be used as CTS repeater cells. Typically includes buffers and inverters of medium to high drive strength, usually of the fastest voltage threshold class available. Library cells not listed here will be blocked from use for the 'cts' stage. This parameter updates the FCRM control variable VALID_CTS_REPEATER_LIST. Default: No Default Value
Valid Skew Cells (ValidSkewCells)	Defines valid skew cells that should not be used for optimization. Cells in this list will be blocked from use for all optimization stages. This parameter updates the FCRM control variable SKEW_CELL_LIST. Default: No Default Value
Valid Tie Cells (ValidTieCells)	Defines the valid tie high and low cells to be used during optimization. Typically includes tie high/low cells of the lowest voltage threshold class available. Library cells in this list have any existing dont_touch designation removed and are defined for use in all optimization stages. This parameter updates the FCRM control variable VALID_TIE_CELL_LIST. Default: No Default Value
Clock Isolation Buffer (ClockISOBuffer)	Defines a single clock tree repeater cell of medium drive strength used to isolate clock ports. This parameter updates the FCRM control variable CLOCK_ISO_BUFFER. Default: No Default Value

Table 5-20 Fusion_Compiler_Reference_Methodology Hierarchical Synthesis Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
Enable Hierarchical Synthesis (EnableHierarchicalSynthesis)	Setting the parameter to true enables the Hierarchical flow. When enabled the ndm for the sub-block is used in the Subsystem level synthesis. The flow is only enabled in coreAssembler.

Table 5-21 Fusion_Compiler_Reference_Methodology Automotive Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
AutomotiveSynthesis Flow Type (AutomotiveSynthesisFlowType)	Specifies an automotive synthesis flow, such as triple module redundancy (TMR), dual core lockstep (DCLS), or TMR_and_DCLS The list of valid values depends on which intents are packaged with the IP.
TMR Separation Distance (TMRSeparationDistance)	Specifies the distance values for repelling bound constraint for redundancy group

Strategy Parameter <GUI label> (<parameter name>)	Description
	This entry can be either an {x y} pair (for example, 1 1) or a single number denoting radial distance.
DCLS Distance (DCLSDistance)	Specify the DCLS distance as a pair of {width height}. This creates a group bound with the specified dimensions.

Table 5-22 Fusion_Compiler_Reference_Methodology Reports Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
Generate supplementary reports (GenerateRMReports)	Enables report generation commands specified in the Reference Methodology Default: 0
Generate Stage based QOR Reports (ReportQOR)	FCRM templates generate QOR data at end of each stage, enabled by this parameter. Default: 0
Generate QOR Histogram (ReportQORHistogram)	Report a histogram for QOR data using proc_histogram call in write_data_stage Default: 0
Skip Congestion Report (DisableCongestionReport)	Disables congestion report generation. It can be used for runtime improvements. Default: 1

5.1.2.1.3 DCTCL_quick_map_strategy

This strategy performs a quick, low-effort compile to generate a gate-level netlist of the subsystem for analysis purposes only. Synthesis can optionally be carried out using DC-Explorer, provided that version G-2012.06-SP2 or newer of DC-Explorer is available.

Use DCTCL_quick_map_strategy to perform a quick feasibility check on a design that you want to synthesize later using one of the other synthesis strategies. Do not use DCTCL_quick_map_strategy simply to reduce compile time on a design intended for manufacture, because DCTCL_quick_map_strategy does not perform all synthesis steps rigorously enough for production purposes.

Click the Options button to specify values for the DCTCL_quick_map_strategy parameters in the following groups.

■ Basic

Parameters that control the basic level of effort for the estimation of the gate-level netlist (see [Table 5-23](#) on page 148).

■ Netlist Generation

Parameters that control the netlist format and rules, and the fixing of multi-port nets. These parameters are the same as for the Design_Compiler_Reference_Methodology strategy (see [Table 5-3](#) on page 126).

■ Physical Synthesis

Parameters that enable and control a Physical Synthesis run, enable Milkyway and specify libraries and resources for that run. These parameters are the same as for the Design_Compiler_Reference_Methodology strategy (see [Table 5-6](#) on page 129).

■ Clock Gating

Parameters that enable and control the use of clock gating during the design elaboration phase of the synthesis run. These parameters are the same as for the Design_Compiler_Reference_Methodology strategy (see [Table 5-7](#) on page 132).

Table 5-23 DCTCL_quick_map_strategy Basic Parameters

Strategy Parameter <GUI label> (<parameter name>)	Description
Use DC-Explorer (UseDcExplorer)	This defaults to false and is enabled only if DC-Explorer G-2012.06-SP2 or newer is available.
Stop at GTECH (StopAtGtech)	Analyzes and elaborates your configured subsystem to the Synopsys technology-independent GTECH library, without mapping the design to the target technology. Generating a GTECH version of the subsystem is an easy way to create a simulation model for a subsystem delivered as encrypted HDL. The default value is false (do not stop at GTECH). Disabled if UseDcExplorer parameter is true.
Read Constraints (ReadConstraints)	When enabled, the SDC constraints are read in during the GTECH synthesis run This option is enabled only when the “Stop at GTECH” option is checked. The default value is false (do not use only simple GTECH gates). Disabled if UseDcExplorer parameter is true.
Use only simple GTECH gates (OnlySimpleGtechGates)	Specifies whether to use only a minimal set of GTECH gates to implement the netlist The cells used are: GTECH_NOR2, GTECH_NOT, GETCH_FD2, and GTECH_FD4. This option is enabled only when the “Stop at GTECH” option is checked. The default value is false (do not use only simple GTECH gates). Disabled if UseDcExplorer parameter is true.
Exclude cell set: (ExcludeGtechCellSet)	Set of GTECH cells that are not used when mapping the design. Disabled if UseDcExplorer parameter is true.

Strategy Parameter <GUI label> (<parameter name>)	Description
Include cell set: (IncludeGtechCellSet)	Set of GETCH cells to be used in addition to the cells used when OnlySimpleGtechGates is specified. Disabled if UseDcExplorer parameter is true.
Ungroup all after compile (UngroupAllAfterCompile)	Indicates that the design is totally ungrouped after the final compile stage. Checking this option results in a completely flat netlist, regardless of any PreserveHierarchy settings. Default: 0
Compile Ultra	
Additional Initial Options (CompileUltraInitOptions)	Additional options to pass to an initial compile_exploration command This is enabled only if “Use DC-Explorer” is true.
SDF Generation	
Write SDF after synthesis (WriteSDF)	Indicates whether an SDF file is generated when synthesis completes If enabled, the SDF file is written to <workspace>/syn/<stage>/db/<design>.sdf.
ICC2 Write Files	
Write ICC2 files after synthesis (WriteICC2Files)	Indicates whether an ICC2 file is generated when synthesis completes If enabled, the ICC2 file is written to <workspace>/syn/<stage>/db/ICC2_files/*.
Design for Test	
Test Ready Compile (TestReadyCompile)	Enables test-ready compile The primary effect of this parameter is to cause the -scan option to be added to the initial compile call. This is enabled only if “Use DC-Explorer” is true.

5.1.3 Specifying ASIC Synthesis Execution Options

Select your synthesis options on the Options tab of the Synthesize Activity View. Synthesis options differ from the Synthesis Strategy options in that they do not determine how coreAssembler generates commands to the synthesis tools, but rather how the coreAssembler executes the strategy. For example, you can select a remote host on which to run the synthesis job or specify the number of CPUs on which to run parallel compiles. For large designs, running parallel compiles can significantly reduce synthesis runtime because subdesigns that are to be compiled individually can be compiled at the same time.

[Table 5-24](#) on page 150 defines the available synthesis options.

Table 5-24 Synthesis Options

Activity Parameter <GUI label> (<parameter name>)	Description
Generate Scripts Only (ScriptsOnly)	Specifies whether to execute synthesis immediately or just save the scripts for later use. When Generate Scripts Only is false, the appropriate tool is automatically invoked to execute the scripts as soon as they are generated. When Generate Scripts Only is true, the synthesis scripts are generated and written but not run. You can run them later by executing run.scr at the UNIX command prompt in the <i>workspace/syn</i> directory.
Run Style (RunStyle)	Specifies whether to execute synthesis on the local host (the default), through the load sharing facility (LSF), through the Sun grid engine (GRD).
Run Style Options (RunStyleOptions)	If you select Run Style = lsf or Run Style = grd, you can enter options (such as -m <i>machine name</i> to direct the job to a particular machine) in this text box. The specified options are passed to the bsub command (for lsf) or the qsub command (for grd).
Parallel Job CPU Limit (ParallelJobCpuLimit)	Specifies the maximum number of CPUs on which to execute parallel compiles. If you select a value greater than 1, your selected strategy is implemented by performing parallel compiles of designs that are to be compiled individually. If you select 1, your selected strategy is implemented by performing individual compiles sequentially on a single CPU. Parallel Job Cpu Limit is enabled only if DC Tcl mode or Physical Synthesis is enabled.
Send e-mail (EmailAddress)	If selected, this option sends an e-mail message to the specified address when synthesis ends or is ended. The default for the text box labeled "To" is your (the current user) user name.
Skip reading \$HOME/.synopsys_dc.setup (NoHomeInit)	Force tool to not read the .synopsys_dc.setup file from \$HOME Default: 0 (false)
Use Design Compiler NXT (UseDCNXT)	Uses Design Compiler NXT for synthesis Note that Design Compiler NXT is available starting with the P-2019.03 version of Design Compiler.
Enable Autoread Flow (EnableAutoReadDC)	Uses Autoread mode in DC to analyze the RTL files. Note: To enable this option for the IP, use coreBuilder to select the Autoread Supported? option that is under the Advanced tab in the Load Designs activity.

5.1.4 Viewing Licenses for Implementation Tools

Licenses available for synthesis appear on the Licenses tab of the Synthesize Activity View. Selecting a license enables coreAssembler to invoke synthesis tool features that require that license. For example, selecting BOA-BRT enables the use of behavioral optimization of arithmetic (BOA) and behavioral retiming (BRT) algorithms during synthesis. You can specify license availability for the following tools:

- DC-Ultra-Opt
- DC-Ultra-Features

- DesignWare Foundation
- BOA-BRT
- Behavioral-Compiler
- Test-Compiler
- Test-Compiler-Plus
- Test-DFTC-TMAX
- DC-Expert
- PrimeTime

If licenses are not available for certain features, then coreAssembler does not write commands using those features in the generated scripts. In TCL mode only, you can force implementation tools to wait for licenses if they are not available using the “Wait for Licenses” parameter on the Licenses tab to indicate how many minutes to wait for licenses to become available. The default value is 0, meaning that the tool does not wait for an unavailable license. There is also a preference available from the Edit > Preferences > Synthesis options for setting the default value of this parameter. For more about preferences, see [“Editing Tool Setup and User Preferences”](#) on page 244.

5.1.5 Selecting Synthesis Reports

For some designs, the generation of certain reports consumes unacceptably long amount of time. In the Reports tab of the Perform ASIC Synthesize activity, deselect any of the optional reports listed in [Table 5-25](#) that you do not want.

Table 5-25 Selecting Synthesis Reports

Strategy Parameter <GUI label> (<parameter name>)	Description
Checker Reports	
Generate Through-Path report (DcThroughPathReport)	Specifies whether to generate a list of through paths (top-level inputs and outputs directly connected through combinational logic). Default is do not generate.
Generate registered I/O report (DcRegisteredIOReport)	Specifies whether to generate a list of unregistered ports (ports not directly connected to a register at the input or output of the design). Default is do not generate.
Generate DC setup report (DcSetupReport)	Specifies whether to generate a synthesis setup report. Default is generate this report.
Generate check design report (DcCheckDesignReport)	Specifies whether or not to generate a DC design check report. Default is generate this report.
Generate latch report (DcLatchReport)	Specifies whether or not to generate a DC latch report. Default is generate this report.
Generate loops report (DcLoopsReport)	Specifies whether or not to generate a DC combinational loops report.

Table 5-25 Selecting Synthesis Reports (continued)

Strategy Parameter <GUI label> (<parameter name>)	Description
	Default is generate this report.
Generate check test report (DcCheckTestReport)	Specifies whether or not to generate a DC check test report. Default is generate this report when doing a test-ready compile.
Timing Reports	
Generate constraint report (DcConstraintReport)	Specifies whether or not to generate a DC constraint report. Default is generate this report.
Generate internal timing report (DcInternalTimingReport)	Specifies whether or not to generate a report showing the worst register-to-register timing paths. Default is generate this report.
Generate short I/O timing report (DcShortIOTimingReport)	Specifies whether or not to generate a DC timing report showing the worst path to/from each I/O, in short path format. Default is generate this report.
Generate long I/O timing report (DcLongIOTimingReport)	Specifies whether or not to generate a DC timing report showing the worst path to/from each I/O, in long path format. Default is generate this report.
Generate Qor report (DcQorReport)	Specifies whether or not to generate a DC Qor report. Default is generate this report.
Generate clock report (DcClockReport)	Specifies whether or not to generate a DC clock report. Default is generate this report.
Generate timing exceptions report (DcTimingExceptionsReport)	Specifies whether or not to generate a DC timing exceptions report. Default is do not generate this report.
Generate Path Group Report (DcPathGroupReport)	Specifies whether or not to generate a DC path group report. Default is generate this report.
Utilization Reports	
Generate hierarchy report (DcHierarchyReport)	Specifies whether or not to generate a DC hierarchy report. Default is generate this report.
Generate Clock Gating Report (DcClockGatingReport)	Specifies whether to generate a DC clock gating report. Default: Generate this report.

Table 5-25 Selecting Synthesis Reports (continued)

Strategy Parameter <GUI label> (<parameter name>)	Description
Generate Self Gating Report (DcSelfGatingReport)	Controls whether or not a self-gating report will be generated. Only applicable for Design Compiler Reference Methodology. The report will be written out when both the SelfGating strategy parameter and DcSelfGatingReport parameters are enabled. The report gets generated as <design_name>.mapped.self_gating.rpt
Generate reference report (DcReferenceReport)	Specifies whether to generate a DC reference report. Default: Generate this report.
Generate power domains report (DcPowerDomainsReport)	Controls whether to generate a power domain report, and other reports for a multi-voltage synthesis flow using UPF files. The other reports are: power switches, supply nets, power state table, level shifters, isolation cells, and retention cells. Default: Generate this report.
Generate resources report (DcResourcesReport)	Specifies whether a resource report is generated, and whether it is for the top-level design only or for all designs. Default: Generate this report.
Generate register count report (RegisterCountReport)	Controls whether to generate a report detailing the number of registers present in the synthesized design. Default: Generate this report.
Generate fanin/fanout report (DcFaninFanoutReport)	Controls whether to generate a report detailing fan-in and fan-out. Default: Do not generate this report.
Generate area report (GenerateAreaReport)	Controls whether to generate the area report for the current design. Default: Generate this report.
Physical Reports	
Generate ignored layers report (GenerateIgnoredLayersReport)	Reports the routing layers that are ignored during congestion analysis and RC estimation. The report gets generated for DCRM synthesis (only when PhysicalSynthesis is enabled) and FCRM synthesis run.
DFT Reports	
Generate DFT coverage estimate report (DFTCoverageEstimateReport)	Controls whether to generate a DFT coverage estimate report. This applies only if the DFT flow is enabled (for settings, see Design Compiler Reference Methodology Design for Test Parameters on page 127) Default: Generate this report.
Automotive Reports	
Generate safety register rules report (ReportSafetyRegisterRules)	Controls whether to generate a safety register rules report. This applies only if the AutomotiveSynthesisFlowType parameter is enabled (for settings, see Table 5-9 on page 135 or Table 5-21 on page 146)

Table 5-25 Selecting Synthesis Reports (continued)

Strategy Parameter <GUI label> (<parameter name>)	Description
	Default: Do not generate this report.
Generate safety register groups report (ReportSafetyRegisterGroups)	Controls whether to generate a safety register groups report. This applies only if the AutomotiveSynthesisFlowType parameter is enabled (for settings, see Table 5-9 on page 135 or Table 5-21 on page 146) Default: Do not generate this report.
Perform & report safety related checks (ReportSafetyStatus)	Controls whether to perform and report checks related to safety_register_rules and safety_register_groups. . This applies only if the AutomotiveSynthesisFlowType parameter is enabled (for settings, see Table 5-9 on page 135 or Table 5-21 on page 146) Default: Do not generate this report.

5.1.6 Launching an ASIC Synthesis Run

5.1.6.1 Performing ASIC Synthesis using Design Compiler

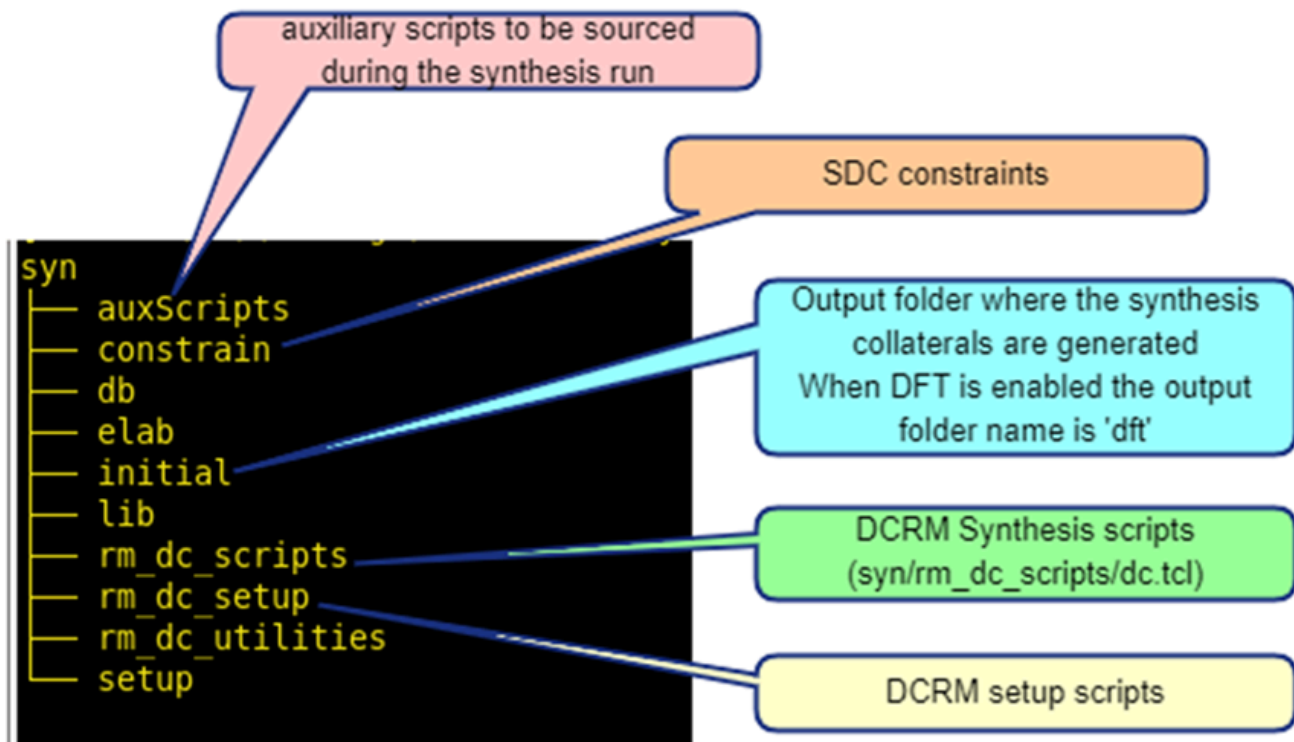
After you select your synthesis strategy, strategy parameters, and other synthesis options, click Apply in the Synthesize Activity View to complete the Perform ASIC Synthesis activity. coreAssembler generates custom, technology-specific synthesis strategy scripts to implement the reusable subsystem according to your selected synthesis strategy.

When you select Generate Scripts Only?, coreAssembler generates synthesis scripts for later use. To run synthesis using the scripts, go to your *workspace/syn* directory and invoke the run.scr script from the shell.

To complete the Perform ASIC Synthesis activity in command line mode, use the set_strategy_parameter, set_activity_parameter, and autocomplete_activity commands. For example:

```
set_strategy_parameter Design_Compiler_Reference_Methodology QorEffort
high
set_activity_parameter Synthesize StrategyName
Design_Compiler_Reference_Methodology
autocomplete_activity Synthesize
```

The synthesis folder structure post synthesis using Design Compiler is as shown in [Figure 5-1](#).

Figure 5-1 Synthesis Directory Structure – Design Compiler Synthesis

5.1.6.1.1 Synthesis Replay Scripts

Synthesis replay scripts capture the setup, variable settings, and synthesis commands used for a synthesis run. The scripts are in the directory `syn/.replay`. A shell script, `replay.sh`, residing in the `.replay` directory can be used to replay the complete synthesis process. The sole purpose of these scripts is to aid in packaging test cases to reproduce any synthesis errors using only Design Compiler or DC Graphical, not coreTools infrastructure. Synopsys does not support modification and use of the replay scripts as an implementation flow.

5.1.6.1.2 Using the `restore_checkpoint` Command

Sometimes it is desirable to directly invoke Design Compiler on one of the complete synthesis stages within your workspace. The `restore_checkpoint` command is available in the workspace for this purpose. It does the following:

- Invokes the proper version of DC
- Loads libraries and the final DDC associated with the given phase
- Returns DC control to you at the specified phase for further analysis

It essentially enables you to 'open' a synthesis stage in Design Compiler for debugging, report generation and further analysis. To use this command from a coreTools workspace:

```
cd <workspace>/syn
restore_checkpoint.scr phase name
```

The *phase name* can be any synthesis phase beginning with 'constrain' (for example, initial, incr1, final).

5.1.6.1.3 Hierarchical (Bottom-Up) Synthesis using Design Compiler

Figure 5-2 shows the hierarchical (bottom-up) synthesis flow using Design Compiler.

The flow involves the user setting a design attribute on the IP top design that identifies that the IP has to be compiled as a sub-block.

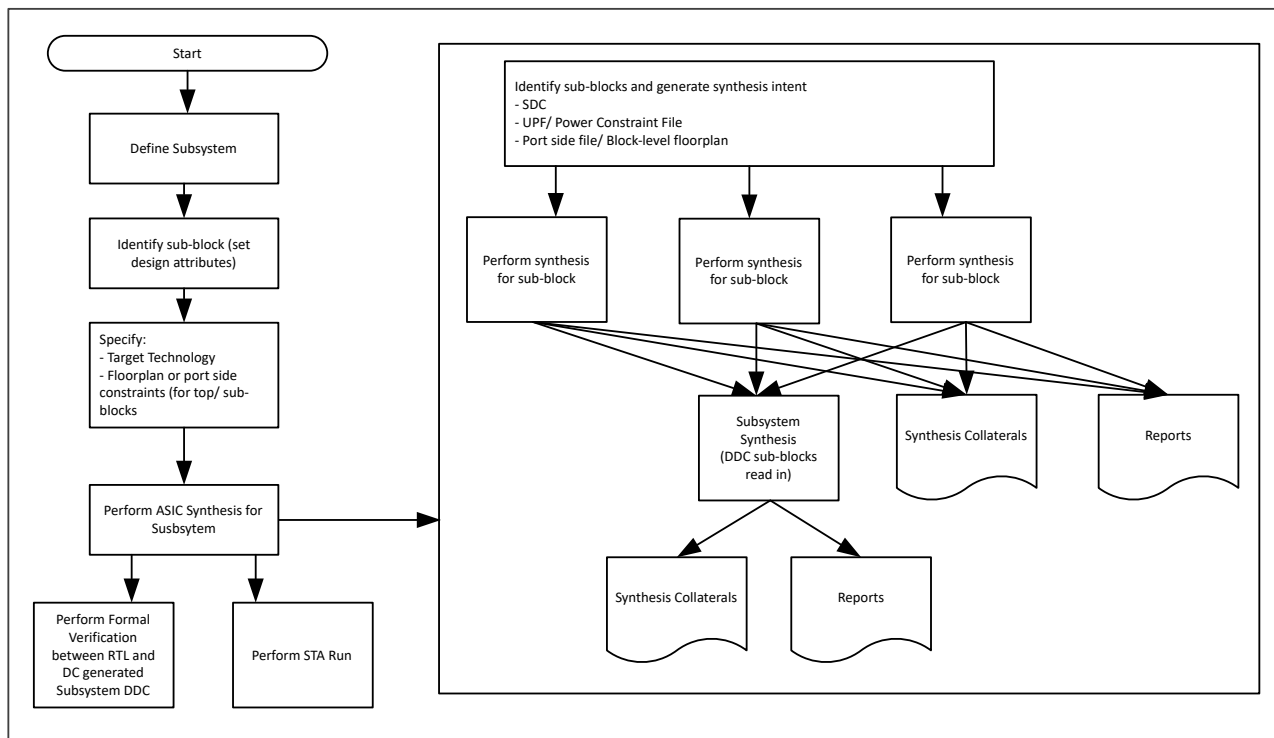
The sub-block IPs are compiled into DDC files. The DDC files are then read into the subsystem level synthesis run.

The design attribute and usage is as follows:

- Design Attribute: SynthesizeAsSubblock
- Usage:


```
set_current_component i_timers
set_design_attribute SynthesizeAsSubblock 1
```

Figure 5-2 Hierarchical (Bottom-Up) Synthesis Flow using Design Compiler



5.1.6.2 Performing ASIC Synthesis using Fusion Compiler

To enable the Fusion Compiler option for synthesis, the NDM libraries and .db files are required. They are also required to perform the formal verification activity.

The IPs used in coreAssembler are categorized into three types. They are as follows:

- coreKit components
- IPXACT
- Raw RTL

IPXACT and Raw RTL are considered Fusion Compiler ready. But for coreKit, it is to be explicitly specified that it will run Fusion Compiler.



Note

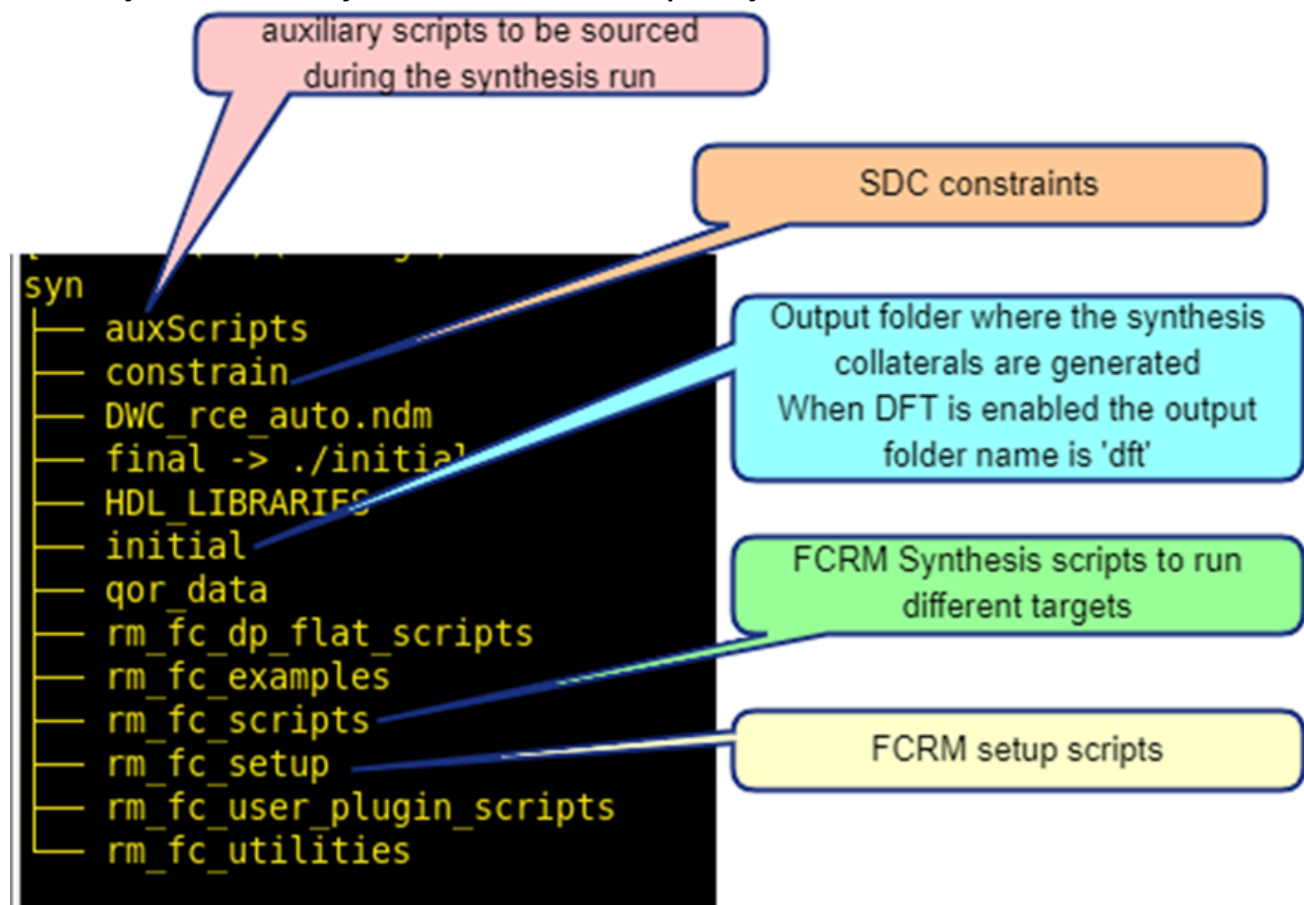
All IPs are not Fusion Compiler ready. The FCRM strategy will not show up in the drop down menu if the IP is not Fusion Compiler ready.

For the Fusion Compiler option to be enabled, there are three conditions that are to be satisfied. They are as follows:

- IPs are to be Fusion Compiler ready.
- In the Specify Target Technology/ TechSetup, the appropriate NDM libs are to be specified. The physical libraries along with the logical libraries must also be specified for Fusion Compiler.
- Fusion Compiler version should be ≥ 2020.06 .

The synthesis folder structure post synthesis using Fusion Compiler is as shown in [Figure 5-3](#).

Figure 5-3 Synthesis Directory Structure – Fusion Compiler Synthesis



5.1.6.2.1 Performing ASIC synthesis using QIK Reference Methodology for Synopsys Interface IP

The Quick Implementation Kit (QIK) for Synopsys Interface IP is a Reference Methodology for Fusion Compiler that has been designed with reference to the Synopsys Interface IP's.

QIK introduces support for Multi Corner Multi Mode (MCMM). Therefore, the setup required for QIK is different from the Fusion Compiler Reference Methodology. It also supports the ability for cell pruning (filtering the list of valid cells based on type).

Minimum Version Support:

To use QIK, the minimum version for coreTools must be greater than or equal to V-2023.09 and the Fusion Compiler version must be greater than or equal to U-2022.12.

Methodology:

To use QIK, the user needs to set the 'EnableQIK' parameter to '1' and provide definitions for the following:

- Modes
- Scenarios
- Corners
- Parasitics

Additionally, the user can specify the cells to be pruned and the list of active scenarios based on the synthesis stage.

Technology Reference Methodology Scripts:

This parameter captures the path to the rm_tech_scripts delivered by Synopsys. When using QIK its mandatory for the user to specify the path to the rm_tech_scripts.

Parasitic Definition:

The user can specify a maximum of four parasitic definitions. The information corresponding to each parasitic definition includes the following:

- Parasitic model name
- TLUPlus file name for the library
- Layer map file for the library



Note

- As a minimum, at least one Parasitic definition needs to be specified.
- If a parasitic model name is specified, then the corresponding TLUPlus file name is mandatory.
- The parasitic model name must be unique.

Mode Definition:

The user can specify a maximum of four mode definitions. The information corresponding to each mode definition includes the following:

- Name of the mode
- Mode Constraint file (The mode constraint file contains the mode specific constraints)



Note

- As a minimum, at least one mode definition must be specified.
- The mode name must be unique.

Corner Definition:

The user can specify a maximum of nine corner definitions. The information corresponding to each corner definition includes the following:

- Corner Name
- Corner Constraint File

**Note**

- The corner names must be unique and the corner constraint file must be specified by the user.
- The contents of the corner constraint file should include the PVT information that matches the library (set_voltage, set_temperature, set_process_label)

A sample Corner Constraint file content is as follows:

```
set_parasitic_parameters -corner [current_corner]
    -early_spec cworst_CCworst_T
    -late_spec cworst_CCworst_T
set_voltage 0.675 -object_list [get_supply_port VDD]
set_voltage 0.000 -object_list [get_supply_port VSS]
set_temperature -40
set_process_label ssgnp
```

Scenario Definition:

The user can specify a maximum of nine scenario definitions. The information corresponding to each corner definition includes the scenario name.

The scenario name must follow the following convention:

<MODE NAME>::<CORNER NAME>

**Note**

- The mode and corner names must be part of the Mode and Corner definition
- The scenario name must be unique.

Active Scenario List:

The user can specify the list of active scenarios as space separated values for a particular synthesis stage. If no active scenarios are specified, all scenarios will be active.

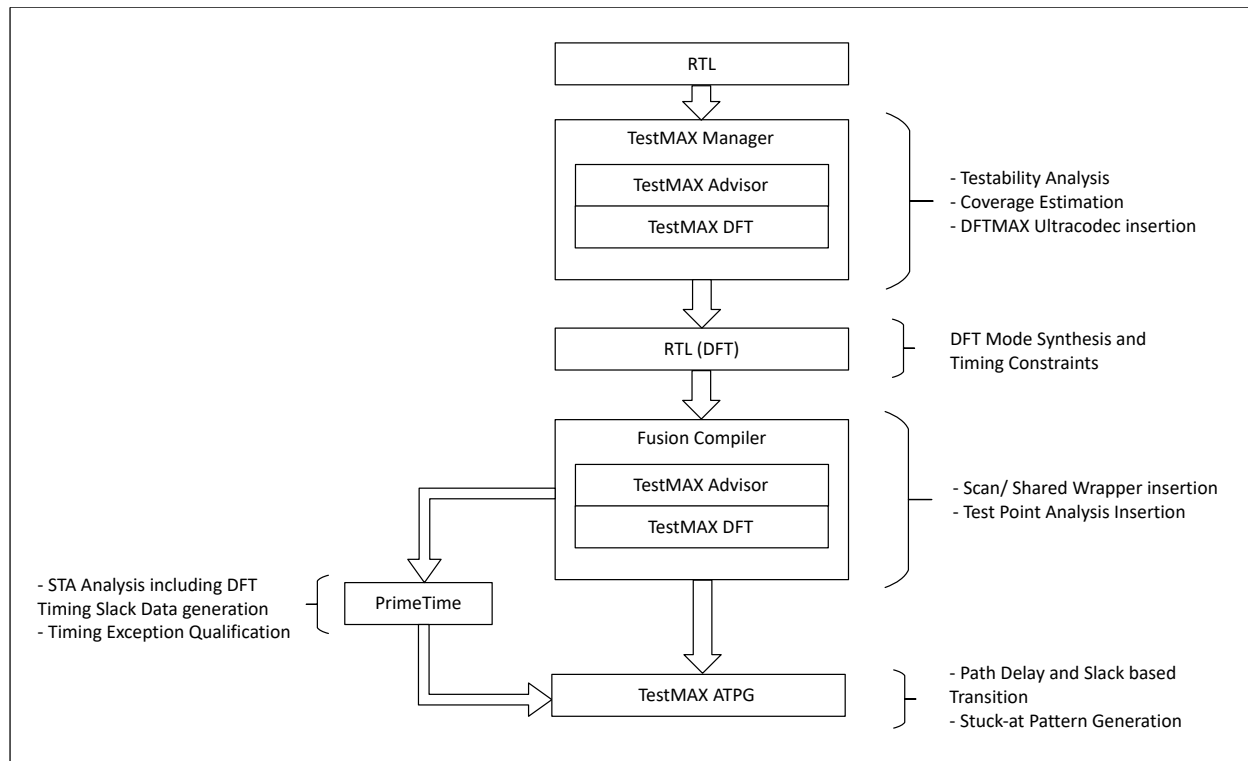
Cell Pruning List:

The user can define a list of valid cells for a given type.

5.1.6.2.2 Performing ASIC Synthesis using QIK Reference Methodology with TestMAX Manager

The TestMAX Manager tool provides a common interface for DFT implementation capabilities. Inserting the DFT feature at the RTL stage accelerates the validation of DFT with the design and provides a faster turnaround time before the physical design stage.

Figure 5-4 shows the TestMAX Manager flow chart.

Figure 5-4 TestMAX Manager Flow Chart

The input data requirements for the TestMAX Manager (TMM) flow is as follows:

- RTL – Configured RTL
- Scan Setup Definition – Scan Configuration Definition
- Scan Ports Definition – Scan Input, Scan Output, Scan Enable definition

The output data requirements for the TestMAX Manager flow that are generated by the TestMAX Manager is as follows:

- RTL with inserted DFT ports
- Updated scan configuration

5.1.6.2.3 Hierarchical (Bottom-Up) Synthesis using Fusion Compiler

Figure 5-5 shows the hierarchical (bottom-up) synthesis flow using Fusion Compiler.

The flow involves the user setting a design attribute on the IP top design that identifies that the IP has to be compiled as a sub-block. The sub-block can only be leaf level components.

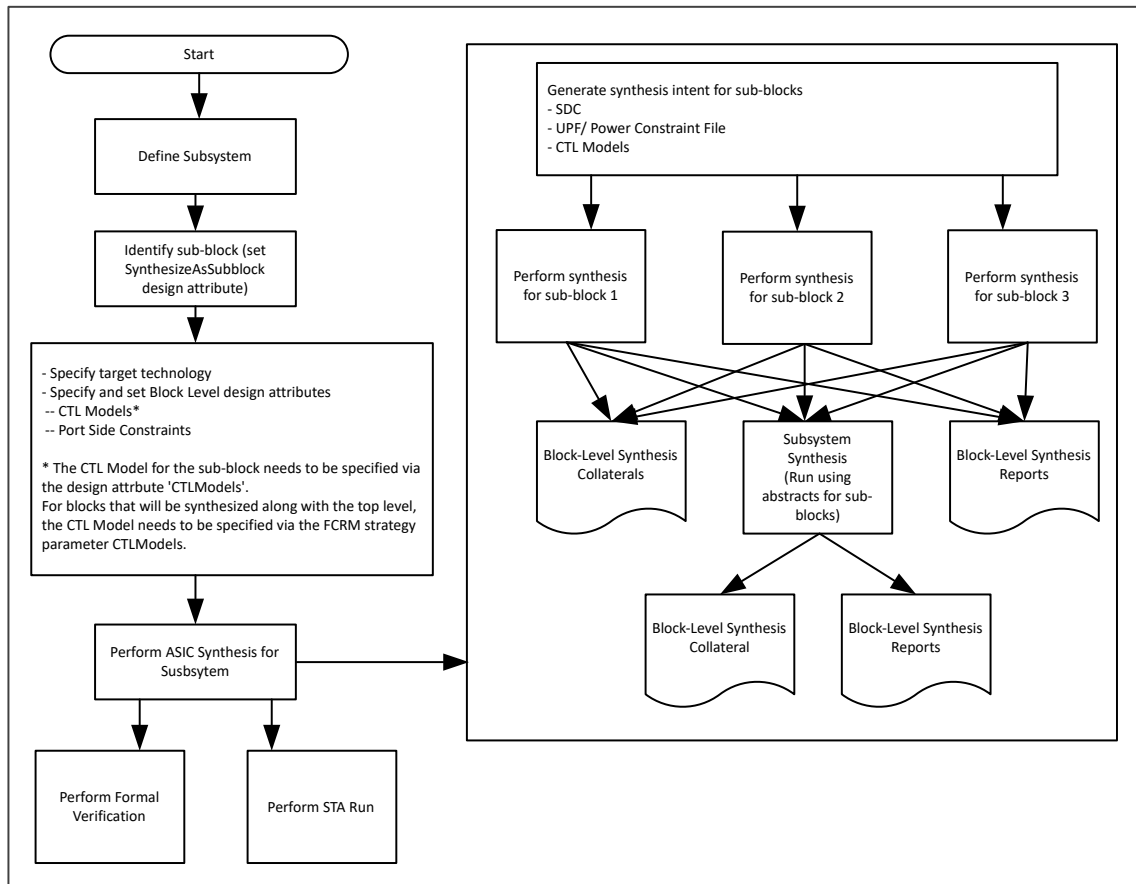
The sub-block IPs are compiled into an abstract view of the design and are passed in as the reference library as part of the subsystem level synthesis run.

The design attribute and usage is as follows:

- **Design Attribute:** SynthesizeAsSubblock
- **Usage:**

```
set_current_component i_timers
set_design_attribute SynthesizeAsSubblock 1
```

Figure 5-5 Hierarchical (Bottom-Up) Synthesis Flow using Fusion Compiler



5.1.7 Checking Synthesis Results

As part of the Perform ASIC Synthesis activity, coreAssembler extracts and analyzes several Design Compiler reports and design checks. To check synthesis status, go to the Report tab of the Synthesize Activity View pane.

The Report tab shows the synthesis status for the current subsystem, including the information listed in [Table 5-26](#). This Activity View also provides options to end the running synthesis job, to display analyzed results on completion of the job, and view Design Compiler summary reports.

Table 5-26 Activity Status Messages

Status Message	Description
Run Style	Identifies the type of synthesis run (local, lsf, or grd).
Results file	Name of the file that contains the top-level synthesis report (the summary index file).

Table 5-26 Activity Status Messages (continued)

Status Message	Description
Execution Host	Name of the machine on which the synthesis job is running (for the local or remote run styles only).
Job ID	The identifying job number for the current synthesis run.
Job Status	The state of the current synthesis run (running, done, or ended.)
Kill Job	Terminates the current synthesis job immediately. The Kill Job button is active (not greyed out) only while the synthesis run is in progress.

[Table 5-27](#) lists the DC Summary Reports available. Because these contain digested synopses of the raw information produced by Design Compiler during the current synthesis run, the coreAssembler summaries presented on the Report tab of the Synthesize Activity View provide a quick, effective means to view and analyze the results of your latest synthesis iteration (and prepare to improve on the results of the latest iteration). Wherever a hyperlink appears, you can “drill down” to progressively more detailed layers of information.

Table 5-27 DC Summary Reports

Report	Description
Compile Summary	Lists the name of the design, the target technology library, the amount of CPU time consumed, and the version of Design Compiler used.
Log File Summary	Lists the name and location of the log file, the error count, and a brief description of each error.
Completed Synthesis Stages	Lists completed stages, the error count, the time completed, and the current status.
Stage Results Summary	Lists the stage name, the critical path slack and total negative slack, the area, and the number of violating nets for each stage.

5.1.8 Refining Your Synthesis Intent

If the synthesis results do not meet your design goals (or if synthesis takes an excessive time to complete), you first consult the core-specific documentation for your design. The core documentation might contain known problem areas or suggested special procedures for improving synthesis results.

If the core developer’s tips do not yield satisfactory results, follow the guidelines for the specific problems described in the following sections. Experiment with a variety of solutions to determine the best set of synthesis intent and strategy specifications for your particular subsystem configuration, technology library, and synthesis tool version.

To implement and check your changes more rapidly, you can also use the batch mode interface to automate the process. (For more, see “[Advanced User Modes](#)” on page 237.)

5.1.8.1 Correcting Timing Problems

To correct timing problems, try setting QoR Effort to high. If that does not correct the problem, try the solutions for the specific problems described in the following sections.

5.1.8.1.1 Small Violations

For a design with a few small timing violations, try the following solutions:

Execute one of the high effort incremental strategies (for example, `DCTCL_high_effort_incr`).

5.1.8.1.2 Few Large Violations

For a design with a few large timing violations, try the following solutions:

1. Check your top-level port delay attributes for the ports on the violating paths. If possible, adjust the delay attributes and re-synthesize.
2. Execute one of the high effort incremental strategies (for example, `DCTCL_high_effort_incr`).

5.1.8.1.3 Many Large Violations

Start by trying a high effort incremental strategy. Also, check the design-specific documentation to make sure you are not specifying a clock speed that exceeds a core's advertised maximum speed. Verify that your top-level `MaxInputDelay` and `MaxOutputDelay` specifications are reasonable.

5.1.8.2 Correcting Area Problems

If the timing results meet your design goals, but the area results do not, try an incremental strategy (such as `DCTCL_area_recovery`).

5.2 Generating Test Vectors

Testability is an important issue for all IC designs. If you use DFT support to generate test models, you can automatically invoke TestMAX in order to generate test patterns. The Generate Test Vectors activity allows you to integrate the test strategy for the subsystem into the overall synthesis flow of your chip. With this activity, you can create TestMAX-generated ATPG test vectors to be used in the subsystem simulation, or incorporate a set of test vectors provided by the component vendor used in your subsystem.



Note

The **Edit > Tool Installation Roots** menu item and dialog box is where you can set the path to your TestMAX tool installation for this feature.

The *TestMAX User's Guide* provides information on how to write scripts to generate test vectors for your subsystem.

[Table 5-28](#) explains the major categories of customization for this activity.

Table 5-28 Generate Test Vector Parameters

Activity Parameter <GUI label> (<parameter name>)	Description
Strategy	
Strategy (StrategyName)	Selects the strategy for generating test vectors. The following choices are presented: ■ TMAX Reference Methodology Implements the TestMAX reference methodology flow for stuck-at, path delay, and transition delay testing. These test modes can be run in any combination or individually. The options for this strategy are described in Table 5-29 on page 166. This strategy is only enabled for TestMAX version L-2016.03 or newer. ■ ATPG basic scan strategy Creates test vectors using the Basic_scan_only engine of TestMAX. This is the recommended starting point for generating vectors, as it provides fast, good coverage for full scan designs. There are no options for this strategy. ■ ATPG user defined strategy Runs a custom Tcl script to generate test vectors with TestMAX. The libraries and design netlist are loaded, and control is turned over to the user script, which is responsible for generating and saving the test vectors. The coreTools script generates reports and exits from TestMAX if the user script does not exit from TestMAX. The option for this strategy is described in Table 5-30 on page 169. Default: Depends on version of TestMAX: ■ For version L-2016.03 or newer: TMAX Reference Methodology ■ For older supported versions: ATPG basic scan strategy

Table 5-28 Generate Test Vector Parameters (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
TestMax Command File (UserStrategyFile)	Specifies the name of the user script containing the TestMAX commands to execute for the ATPG_user_defined_strategy. Your script must assume that the models are already loaded into TestMAX before the script is invoked. Default: None
Setup	
Output Stage Name (OutputStageName)	Specifies the synthesis stage to use for test vector generation. Default: None
Input Stage Name (InputStageName)	Specifies the synthesis stage netlist to be used for generating the test vectors. Default: final synthesis stage
Input Netlist Format (InputNetlistFormat)	Specifies the format of the design netlist to be read into TestMAX. Default: verilog
Test Libraries (TestLibraries)	Specifies files containing the libraries to be used by TestMAX. The value is a list and may contain wildcards that are acceptable to TestMAX. The test libraries are the Verilog simulation models for the target libraries used during synthesis. They provide the description of the logical functionality of all the cells in the standard cell library. They are needed for the ATPG tool to comprehend the logic function implemented by each standard cell instantiated, to generate test vectors for the faults. Default: None
Libs Missing Cell Define (LibsMissingCellDefine)	If the libraries specified in TestLibraries do not have `celldefine and `endcelldefine around the module definitions, setting this parameter to 1 causes the -library option to read the libraries. Default: 0
Test Pattern Format (TestPatternFormat)	Format for saving the test patterns. Default: None
Options	
Parallel Job CPU Limit (ParallelJobCpuLimit)	Specifies the maximum number of CPUs on which to execute parallel compiles. If you select a value greater than 1, your selected strategy is implemented by performing parallel compiles of designs that are to be compiled individually. If you select 1, your selected strategy is implemented by performing individual compiles sequentially on a single CPU. Parallel Job Cpu Limit is enabled only if DC Tcl mode or Physical Synthesis is enabled.
Generate Scripts Only (ScriptsOnly)	Specifies whether to execute synthesis immediately or just save the scripts for later use. When Generate Scripts Only is false, the appropriate tool is automatically invoked to execute the scripts as soon as they are generated.

Table 5-28 Generate Test Vector Parameters (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
	When Generate Scripts Only is true, the synthesis scripts are generated and written but not run. You can run them later by executing run.scr at the UNIX command prompt in the <i>workspace/syn</i> directory.
Run Style (RunStyle)	Specifies whether to execute synthesis on the local host (the default), through the load sharing facility (LSF), through the Sun grid engine (GRD).
Run Style Options (RunStyleOptions)	If you select Run Style = lsf or Run Style = grd, you can enter options (such as <i>-m machine name</i> to direct the job to a particular machine) in this text box. The specified options are passed to the bsub command (for lsf) or the qsub command (for grd).
Send e-mail (EmailAddress)	If selected, this option sends an e-mail message to the specified address when synthesis ends or is ended. The default for the text box labeled "To" is your (the current user) user name.
Use TestMAX ATPG (UseTestMAX_ATPG)	Uses TestMAX ATPG for test vector generation. Note that TestMAX ATPG is available starting with the L-2016.03 version of TestMAX.

Table 5-29 on page 166 presents the options for the TMAX Reference Methodology strategy for generating test vectors and fault lists.

Table 5-29 Options for the TMAX Reference Methodology Strategy

Strategy Parameter <GUI label> (<parameter name>)	Description
TestSelection	
Stuck At Test (StuckAtTest)	When TRUE, stuck-at fault testing is performed. Default: TRUE
Transition Delay Test (TransitionDelayTest)	When TRUE, transition delay testing is performed. Default: FALSE
Transition Delay Coverage Goal (TransitionDelayCoverageGoal)	When TransitionDelayTest is TRUE, this parameter sets the transition delay coverage goal. Note that a coverage goal near 100% can cause significant processing time. Default: 100
At Speed Test (AtSpeedTest)	When TRUE, at-speed testing is performed. The AtSpeedScript parameter accepts a script for updating SPF waveforms for the testing. Default: 0
At Speed Script (AtSpeedScript)	Script to run in order to update the SPF waveforms for at-speed testing.

Table 5-29 Options for the TMAX Reference Methodology Strategy (continued)

Strategy Parameter <GUI label> (<parameter name>)	Description
Path Delay Test (PathDelayTest)	When TRUE, path delay testing is performed. Default: FALSE
Random Incremental Atpg (RandomIncrementalAtpg)	When TRUE, random incremental ATPG is run for all enabled tests. The IncrementalRuns parameter sets the number of incremental runs for each test. Default: FALSE
Incremental Runs (IncrementalRuns)	When RandomIncrementalAtpg is TRUE, this parameter sets the number of incremental runs for each enabled test. Default: 0
Strategy Controls	
TMAX Testbench Config (MaxTbConfigFile)	Path to a file with options for the TestMAX reference methodology scripts. Default: <synDir>/<OutputStageName>/script/tmax_maxtb_config.tcl which has the default settings from the TestMAX reference methodology scripts.
Optimize Patterns (OptimizePatterns)	When TRUE, pattern count is optimized, at the expense of runtime. This optimization applies to the stuck-at and transition delay fault models; other fault models may not get better results, so they are not supported in the reference methodology. Default: FALSE
Persistent Fault Model (PersistentFaultModel)	When TRUE, persistent fault modeling is performed, which can result in smaller overall pattern sets, but at the expense of run time. For details on the persistent fault model, see the <i>TestMAX User's Guide</i> . Default: FALSE, but automatically set to TRUE when TransitionDelayTest and StuckAtTest are both TRUE.
TMAX Debug Mode (TMAXDebugMode)	When TRUE, creates a more complete log file and additional reports for debugging. Default: FALSE
Integrate Path Delay Patterns (IntegratePathDelayPatterns)	When TRUE, path delay patterns are included with other tests, and then fault graded to reduce pattern count. When FALSE, path delay patterns are used only for characterization or speed binning, and are kept as a separate test. Default: TRUE
Rules To Downgrade (RulesToDownGrade)	A list of errors to downgrade without masking. Default: None

Table 5-29 Options for the TMAX Reference Methodology Strategy (continued)

Strategy Parameter <GUI label> (<parameter name>)	Description
Rules To Mask (RulesToMask)	A list of errors to mask. Default: None
Black Box Instances (BlackBoxInstances)	A list of instances to replace with TIEX primitives. Default: None
Exception Files (ExceptionFiles)	Paths to SDC files for timing exceptions. Each path must be relative to the synthesis directory. Default: The output from static timing analysis when the LinkToATPG option was set to TRUE in the Static Timing Analysis activity (see Static Timing Analysis -- Select and Configure Reports on page 198).
Delay Path File (DelayPathFile)	Path file for path delay testing. Default: The output from static timing analysis when the LinkToATPG option was set to TRUE in the Static Timing Analysis activity (see Static Timing Analysis -- Select and Configure Reports on page 198).
Add Pi Constraints (AddPiConstraints)	Add a constraint for a primary input The value of the parameter is a list of constraints in the form of: <port> <logic state> For example: <pre>set_strategy_parameter TMAX_Reference_Methodology AddPiConstraints [list test_se 0 rst_n 1]</pre>
Global Slack File (GlobalSlackFile)	Path file for path delay testing. Default: The output from static timing analysis when the LinkToATPG option was set to TRUE in the Static Timing Analysis activity (see Static Timing Analysis -- Select and Configure Reports on page 198).
Use the scan compression Protocol (UseScanCompressionProtocol)	When TRUE, uses the scan compression configuration SPF file If scan compression is not enabled (see “Scan Insertion” on page 107), this parameter is automatically disabled and the normal SPF file is used. Default: 0

Table 5-29 on page 166 presents the options for the ATPG user defined strategy for generating test vectors and fault lists.

Table 5-30 Option for the ATPG user defined strategy

Strategy Parameter <GUI label> (<parameter name>)	Description
TestMax Command File (UserStrategyFile)	Specifies the name of the user script containing the TestMAX commands to execute for the ATPG_user_defined_strategy. Your script must assume that the models are already loaded into TestMAX before the script is invoked. Default: None

6

Verifying Subsystems and Components

This chapter contains the following sections:

- [“Overview”](#) on page 172
- [“Verifying an Individual Component ”](#) on page 174
- [“Verifying a Subsystem ”](#) on page 175
- [“Static Checking Activity”](#) on page 176
- [“RTL Static Checking”](#) on page 180
- [“Formal Verification Activity”](#) on page 189
- [“Static Timing Analysis Activity”](#) on page 194
- [“Constraint Analysis Activity”](#) on page 202
- [“Explore RTL Architecture Activity”](#) on page 204
- [“Timing Constraints Manager”](#) on page 209
- [“Create Testbench Workspace Activity \(Subsystem Workspace\)”](#) on page 215
- [“Simulate Subsystem Testbench”](#) on page 222
- [“Integrating the Subsystem into Your Design”](#) on page 224
- [“Batch Commands”](#) on page 225
- [“Licensing and Keys”](#) on page 226

6.1 Overview

6.1.1 Individual Component Verification

The individual coreKits in the subsystem may include test suite regression environments (TREs) with which you can verify functionality at the component level. The type of testing available depends on whether the core's source files are encrypted, and on whether you have access to Synopsys simulation tools (which are the only simulators capable of reading the RTL encryption used by coreAssembler). If a coreKit did not include a TRE, or to perform component functionality tests not covered by a coreKit's TREs, you must create your own test environment for the core and run it using Design Compiler's gate-level output (or the RTL source code, if available).

6.1.1.1 Subsystem Verification

Verification of the subsystem can occur at one or more of three points in the flow:

- After configuration, but prior to synthesis, the subsystem can be tested at the RTL level. This is the fastest way to test the functionality of a subsystem, since the design has not yet been mapped to a specific technology. This option is available only if the RTL source files are not encrypted, or if you have access to one of the Synopsys simulators to read the encrypted RTL code.
- After synthesis, the functionality of the subsystem can be tested at the gate-level with the GTECH simulation model (produced by the "Create Component GTECH Simulation Model" activity) providing input for verification. Such testing is the only verification option when the core consists of encrypted RTL and you do not have the Synopsys simulators required to read the encrypted files. At this stage, you can also use formal verification to check your synthesized implementation and previously verified RTL design for functional equivalence. See [Verifying a Subsystem](#) on page 175.
- Once the configured and synthesized subsystem has been integrated into your chip-level application, you may want to test at one or both of the following levels:
 - Use a "black box" timing model of the core to check the timing of the entire application more quickly than if you had inserted the synthesized core.
 - Test the complete chip-level application (including your synthesized core) at the gate-level to make sure it functions entirely as expected.

6.1.2 Testbench Workspace Creation

The coreAssembler tool allows the creation of a testbench around one or more DUTs using packaged (and IP-XACT) verification IP components. The DUT can be a coreAssembler workspace, or any IP-XACT component or design. The testbench is constructed around the DUT without modifying its source code.

Testbench files and connections are managed in a separate testbench workspace similar to a standard workspace, but without synthesis based activities. Some of the assembly activities are optimized for working with verification IP components. In this testbench workspace, you instantiate verification IP (and possibly synthesizable IP) around a DUT that is instantiated with the `instantiate_dut` command. It is possible to have multiple DUT components in the subsystem.

The same coreAssembler facilities that automate the synthesizable component connection process are used to connect your VIP to the DUT. When connecting monitor interfaces, you can connect an interface on a monitor VIP to an interface on a component within the DUT. You can even connect to pins and monitor at any level of hierarchy in the DUT.

6.1.3 Features

- Provides a streamlined flow for working with VIP components.
- Instantiate VIP into its own level of hierarchy above the actual implementation subsystem (DUT).
- Allows comprehensive VIP model configuration using the coreAssembler user interface.
- Allows arbitrary (non-interface) connections at any level of the hierarchy.
- Allow interface connections from the testbench down into the hierarchy.
- Allow pin connections down into the hierarchy.
- Generation of a testbench that instantiates the subsystem. This allows the testbench to be used with multiple views of the IIP subsystem. For example, it is possible to simulate RTL and gates with the same testbench.

6.1.4 Limitations and Notes

- In order to build a VHDL testbench, the DUT must be generated with the “Include testbench signals” option when completing the Generate Testbench HDL activity. This limitation is because the VHDL of the top-level design must be written in a special format to allow monitoring signals in lower levels of hierarchy.
- The `update_dut` command updates to the most recent change of the Generate Subsystem RTL activity in the DUT workspace. You must always complete this activity in the DUT workspace to see DUT changes in the Testbench workspace.

6.2 Verifying an Individual Component

Component-level verification is a core-specific task and the functionality is provided as part of the individual component coreKit. If you are using components from Synopsys, there is an activity available for performing component-level verification of your particular component configuration. See the component databook for details about component-level verification of that core.

**Note**

Most Synopsys components also include customizations enabling the generation of a GTECH netlist view. This view is provided to enable a fast path to simulation for those customers who want to do RTL simulation but who do not have a simulator that can read the encrypted source code

Select **Verify Component > Setup and Run Simulations** (for *comp_name*) from the Activity List. Each component in your subsystem has an Activity List entry. In this Activity View you select the simulator, the view to be simulated, various execution options, and testbench parameters.

**Note**

The activities for component-level verification can be disabled using the **Edit > Preferences > Workspace Options** menu item. If disabled, these activities do not appear in the Activity List. For more information, see “[Setting Workspace Options](#)” on page 48.

6.3 Verifying a Subsystem

The Verify Subsystem category in the Activity List has several verification options available:

- [“Static Checking Activity”](#) on page 176
- [“RTL Static Checking”](#) on page 180
- [Formal Verification Activity](#)
- [Static Timing Analysis Activity](#)
- [“Timing Constraints Manager”](#) on page 209
- [Create Testbench Workspace Activity \(Subsystem Workspace\)](#)
- [Simulate Subsystem Testbench](#) (in the Testbench workspace)

6.4 Static Checking Activity

The Static Checking activity uses SpyGlass to perform static checking of the generated subsystem RTL.

The coreAssembler tool generates the scripts that run Spyglass on the subsystem and SpyGlass-supported components. SpyGlass actions that coreAssembler executes at the component level are shown in the Verify Component activity list. For components that do not support SpyGlass, you can specify SGDC files to generate the abstracts needed.

The generated scripts reside in the staticChecking directory in the workspace, and you can choose to either execute the scripts during the coreAssembler session or execute them separately later.

You can run SpyGlass on the subsystem using either the bottom-up or top-down flow method.

- For the bottom-up flow method, the required abstracts are generated for all SpyGlass-supported components in the subsystem. The scripts then run SpyGlass using the generated abstracts.
- For the top-down flow method, the subsystem-level SDC file, which also resides in the workspace staticChecking directory, is generated, loaded, and run. For the top-down method, abstracts are unneeded and are not generated.

You can use the Configuration tab to specify rules files, waiver files, and SGDC files. If you specify any user SGDC files when running the top-down flow, coreAssembler does not read the system-level SDC file.

You can use the Goals tab to control which SpyGlass goals are run for the subsystem. The Goals tab also displays which goals are run on the components by default. Note that abstract goals (such as cdc_abstract) are automatically checked when you select a goal that requires that abstract.

The following sections explain the activities and associated parameters for Static Checking.

6.4.1 Configuration

The parameters described here set up the static checks.

Table 6-1 Static Checking - Configuration Controls

Activity Parameter <GUI label> (<parameter name>)	Description
Setup	
Spyglass Version (SpyglassVersion)	Version of SpyGlass to use. Must be set using the Edit > Tool Installation Roots menu.
Guide Ware Version (GuideWareVersion)	Version of SpyGlass GuideWare from which the goals are sourced GuideWare is part of SpyGlass.
Subsystem Rules Files (SubsystemRulesFiles)	List of rules files to be sourced for the subsystem verification.
Subsystem Waivers Files (SubsystemWaiversFiles)	Optional user-specified waiver files.
Subsystem Sgdc Files (SubsystemSgdcFiles)	Optional user-specified SGDC files.

Table 6-1 Static Checking - Configuration Controls (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
Clean Components (CleanComponents)	Optional list of user-specified components for which to regenerate abstracts.
Flow	
Bottom Up Flow (BottomUpFlow)	Run subsystem SpyGlass bottom-up flow. Default: true

6.4.2 Execution Options

The parameters described here control of the execution of the static checks.

Table 6-2 Static Checking - Execution Options

Activity Parameter <GUI label> (<parameter name>)	Description
Generate scripts only (ScriptsOnly)	Generates the run.scr script in the work directory, which can be executed at another time. To retrieve the path to this script, use the following command: <code>get_logical_dir spyglass/run.scr</code> Default: 0
Run style (RunStyle)	When Generate scripts only is 0, this parameter allows you to select how the command is run. Values include: ■ local (default) ■ lsf ■ grd
Run style options (RunOptions)	Allows additional options for several of Run style selections. ■ When Run style = lsf or grd, you can specify options to the bsub and qsub commands, respectively. ☐ For bsub, you can specify the queue name, such as “-q normal”. ☐ For qsub, you can specify a project and other options, such as “-p normal -l arch=glinux”. Note that the -V option for qsub is included automatically.
Parallel job CPU limit (ParallelJobCpuLimit)	Specifies the number of parallel compile jobs that may be run. Default: 1
Send e-mail (EmailAddress)	If selected, this option sends an e-mail message to the specified address when the command script completes or is ended. The default for the text box labeled “To” is your (the current user) user name.

6.4.3 Goals

The Goals tab shows you a set of check boxes that allow you to select which SpyGlass goals are used.

Table 6-3 Static Checking - Goals Options

Activity Parameter <GUI label>	Description
Other Goals	Allows you to specify a path to other goals not used in the subsystem components.
GuideWare Version	Read-only fields that show the GuideWare version for the subsystem and components.
Spyglass Goal	Read-only list of available SpyGlass goals.
Run Goal	A set of check boxes that control whether a goal is run; for the parameter name that corresponds to each goal, see Table 6-4 .

Table 6-4 Run Goal Sample

Spyglass Goal	Run Goal	Parameter Name	Default
cdc/cdc_abstract	✓	SpyglassGoalCdcAbstract	1
cdc/cdc_verify	✓	SpyglassGoalCdcVerify	0
cdc/cdc_verify_struct	✓	SpyglassGoalCdcVerifyStruct	0
cdc/clock_reset_integrity	-	SpyglassGoalCdcClockResetIntegrity	0
dft/dft_abstract	-	SpyglassGoalDftAbstract	0
dft/dft_best_practice	-	SpyglassGoalDftBestPractice	0
dft/dft_dsm_best_practice	-	SpyglassGoalDftDsmBestPractice	0
dft/dft_scan_ready	-	SpyglassGoalDftScanReady	0
dft/dft_setup	-	SpyglassGoalDftSetup	0
lint/lint_abstract	✓	SpyglassGoalLintAbstract	1
lint/lint_lint_rtl	✓	SpyglassGoalLintRtl	0
rdc/rdc_verify_struct	-	SpyglassGoalRdcVerifyStruct	0
txv_verification/fp_mcp_verification	-	SpyglassGoalTxvFpMcpVerification	0

6.4.4 Clock Domains

The Clock Domains tab allows you to relate a clock domain with each clock. Specifying a clock domain sets the ClockDomain attribute on the clock and is then used by SpyGlass.

The clock and clock domain details are captured in the SDC file, and in SGDC files (one for the subsystem in the staticChecking directory, and one for each component in the workspace component hierarchy).

6.5 RTL Static Checking

The RTL Static Checking activity uses VC SpyGlass to perform the static checking of the generated subsystem RTL. This activity is enabled only when the VC SpyGlass version is 2022.06 or greater. It also requires the Design Compiler module to be loaded due to the DesignWare pre-compiled modules in the design used during verification.

The coreAssembler tool generates the scripts that runs VC Spyglass on the subsystem. The constraints in this activity is divided into four file components. They are as follows:

- SDC Files: Contains information related to clock definitions and delay informations.
- Reset Files: Used for defining the rest port definition.
- Constraints Files: Usually contains quasi-static signal information, specific to specific verification checks (lint, CDC, and RDC).
- Waiver Files: Contains waivers definition used during verification runs.

A top-level SDC file generated by the coreAssembler can be found in `vcspyglass/design_constraints` folder. Additional SDC Files can also be given by the user. Other Constraints are to be provided by the user. You can use the Constraints tab to specify additional SDC, waiver files, reset files, and constraint files.

The IPs having BCM constraints are consumed in the run automatically.

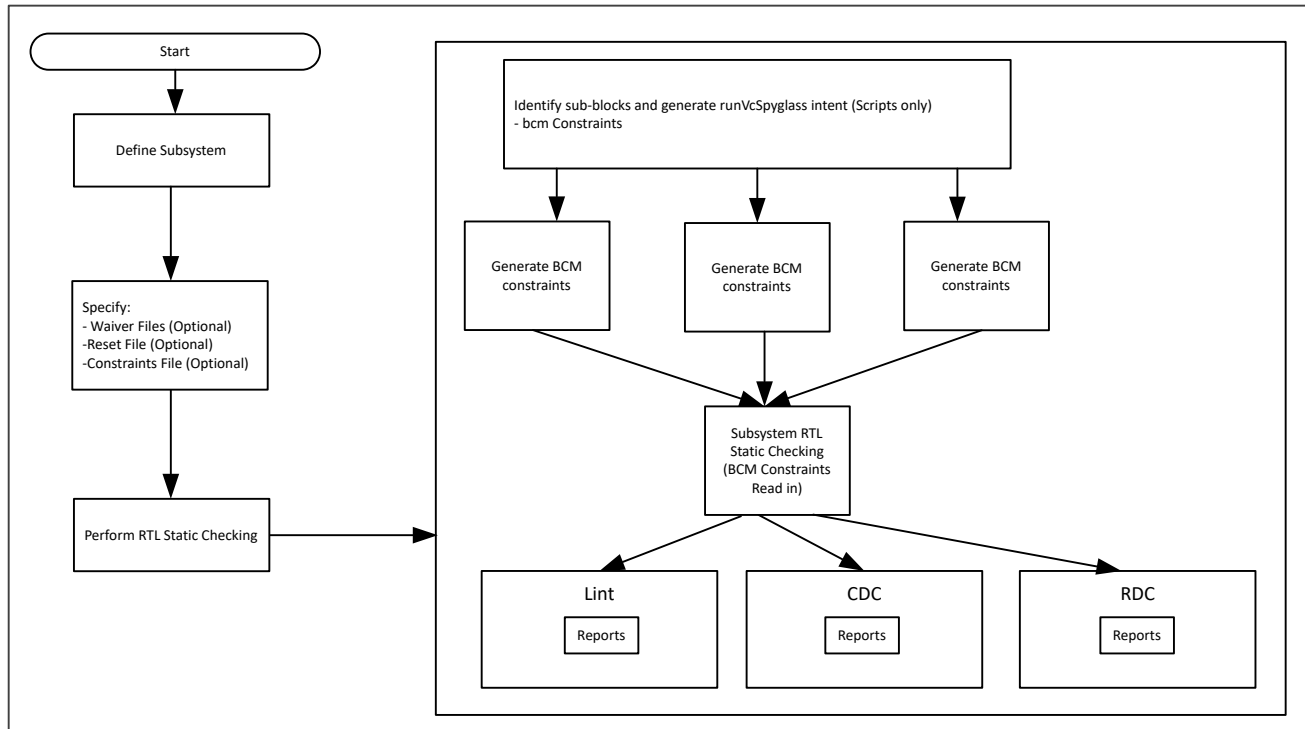
The generated scripts reside in the `vcspyglass/scripts` directory in the workspace. You can choose to either execute the scripts during the coreAssembler session or execute them separately later.

You can use the Verification tab to control which VC SpyGlass verifications are run for the subsystem. For each verification run (CDC, lint, and RDC), a separate directory is created. The reports generated during the runs can be found in the respective verification folders.

After the run is complete, the user can open the `run_vcspyglass.log` and can also see the consolidated report file in `vcspyglass.html`.

Figure 6-1 shows the top-down flow of the RTL static checking activity.

Figure 6-1 RTL Static Checking – Top-Down Flow



The coreAssembler can also run a bottom-up flow for RTL verification. Due to increase in design size, the top-down flow requires increased time in design verification. The bottom-up flow approach uses the ability of VC Static tool to generate abstracts at lower level designs and read at the implementation context level. The bottom-up flow can be enabled when the activity parameter “EnableBottomUpFlow” is set.

Figure 6-2 shows the bottom-up flow of the RTL static checking activity.

Figure 6-2 RTL Static Checking – Bottom-Up Flow

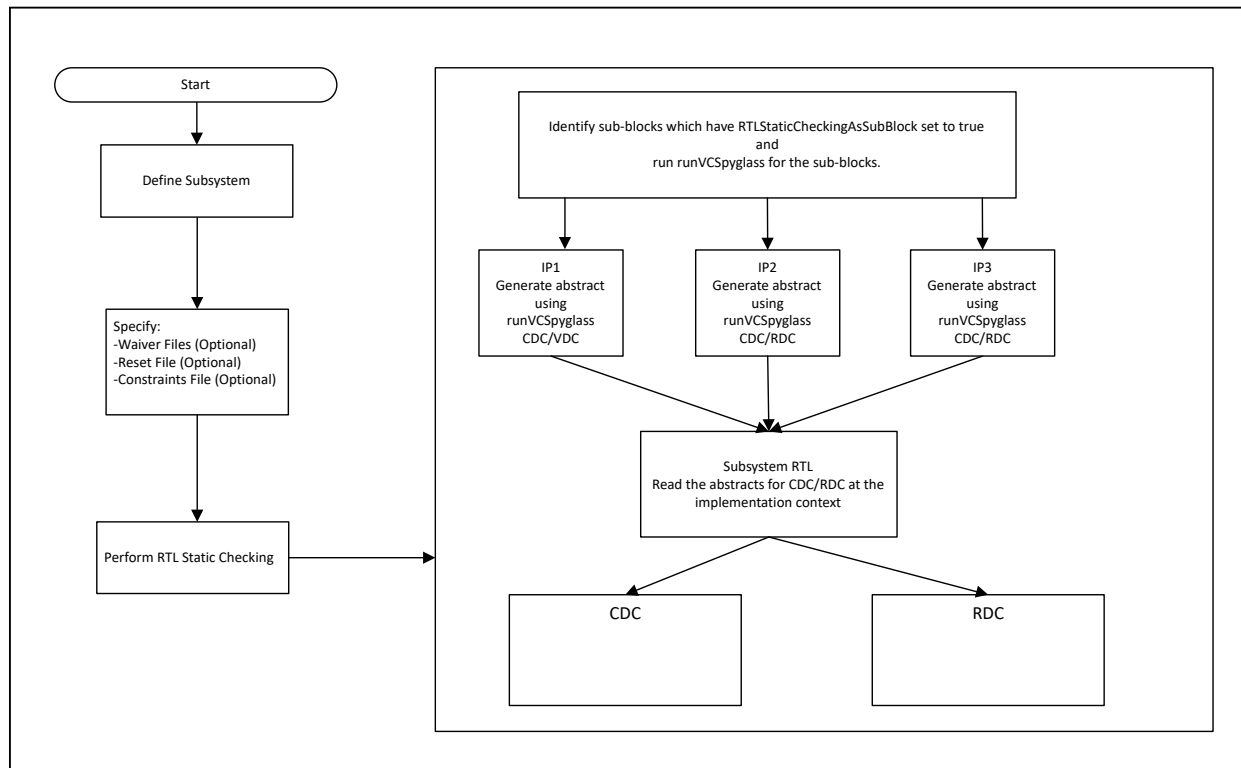
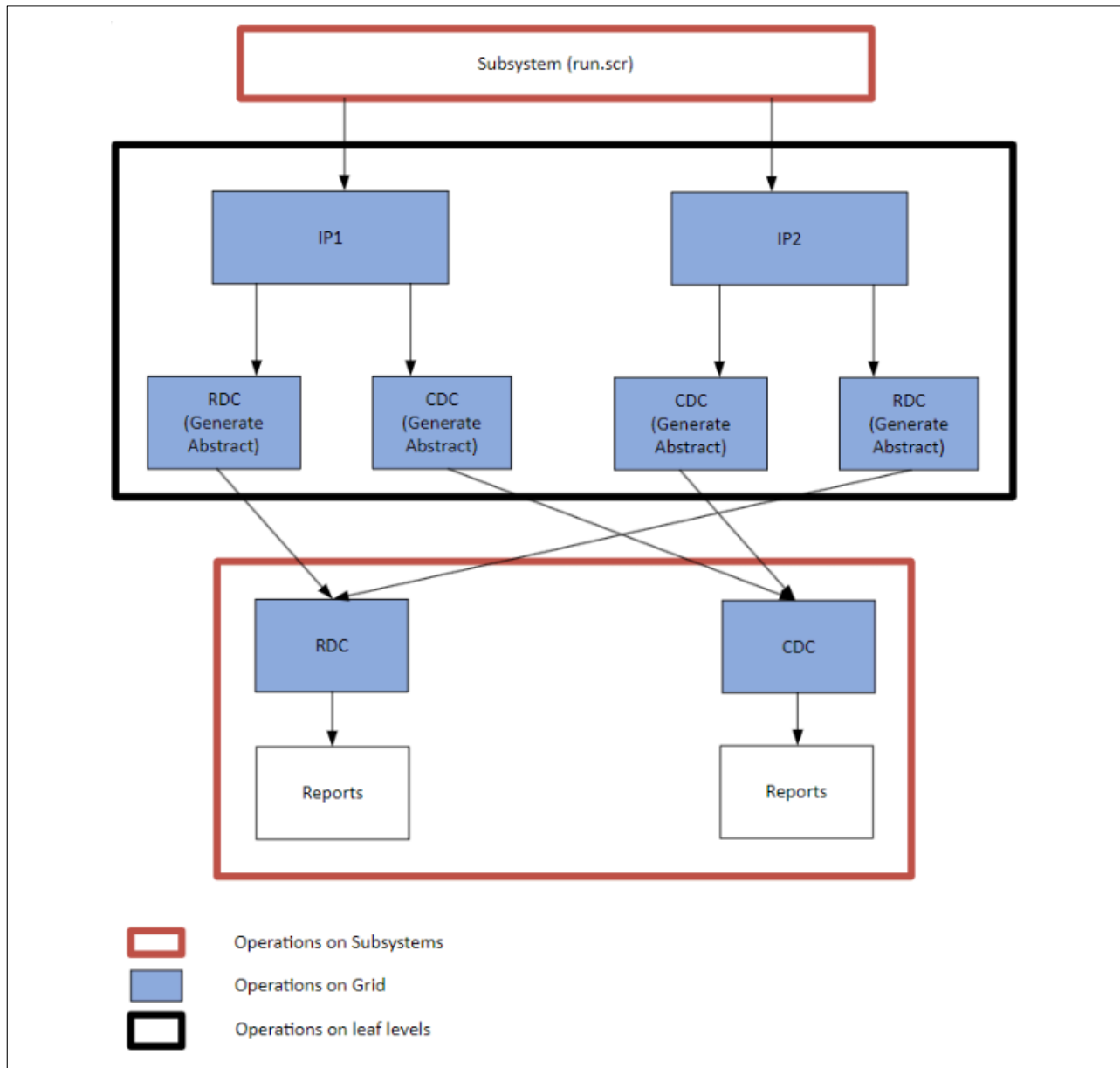


Figure 6-3 shows the bottom-up flow in the grid operation of the RTL static checking activity.

Figure 6-3 RTL Static Checking – Bottom-Up Flow (Grid Operation)



The following are the limitations for the bottom-up flow:

- Lint verification is supported, only when all the IPs are packaged with Engineering Council version as 2023.09 and VC Static version as 2022.12-SP2 and greater.
- If an IP is packaged with the Engineering Council version less than 2021.11, the LSF jobs are not fired at the leaf level for each verification modes. It rather runs the verification modes in sequence.

The design attribute description to be used for bottom-up flow is as follows:

- Design Attribute: RTLStaticCheckingAsSubblock
- Description: The attribute indicates that the design needs abstract generation and is to be verified using the bottom-up flow using VC Spyglass in coreAssembler.

The conditions to set the RTLStaticCheckingAsSubblock design attribute is as follows:

- The design must be a leaf level design
- It must not be a hierarchy
- runVcSpyglass plugin must be present for the design

6.5.1 Setup

The parameters listed in [Table 6-5](#) setup the RTL static checks.

Table 6-5 RTL Static Checking - Setup

Activity Parameter <GUI label> (<parameter name>)	Description
VC Static	
VC_STATIC_HOME (VcStaticHome)	Tool root for vc_static_shell specified in the tool root settings. Default Value: get_tool_root vcstatic
Engineering Council Rule	
Engineering Council Rule File (EngineeringCouncilRule)	VC SpyGlass Engineering Council Rule File. The Default Engineering Council File is calculated using the following algorithm: 1. Traverse through all the DesignWare components to get the Engineering Council version 2. Find the latest version among the versions collected in step (1) 3. Check if the Version identified is in the tool image, a. if yes, use it b. if no, find the Engineering Council rule file from the tool image that is most recent (closest) as compared to the version identified in step (2)
Verification	
Enable Bottom Up Verification (EnableBottomUpFlow)	Enables the bottom-up verification. This allows the abstract generation at the sub block in a hierarchy and read-in at the top. This activity parameter can be only enabled when the design attribute RTLStaticCheckingAsSubblock is set to at least one sub block. Default Value: false
Lint Verification (LintEnable)	Run Lint Verification. Verification is performed in the workspace folder vcspyglass/lint.

Table 6-5 RTL Static Checking - Setup (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
	The top level report (vcspyglass/vcspyglass.html) has links to the associated log and report files. Default Value: 0
CDC Verification (CdcEnable)	Run CDC Verification. Verification is performed in the workspace folder, vcspyglass/cdc. The top level report vcspyglass/vcspyglass.html has links to the associated log and report files. Default Value: 1
RDC Verification (RdcEnable)	Run RDC Verification. Verification is performed in the workspace folder vcspyglass/rdc. The top level report vcspyglass/vcspyglass.html has links to the associated log and report files. Default Value: 1
Enable Root Cause Analysis (RcaEnable)	Enable Root Cause Analysis. This is a feature of VC SpyGlass. For more information, see the VC SpyGlass documentation. Default Value: 0
Enable Abstract (AbstractEnable)	Enable Abstract Generation. When enabled, CDC and RDC verification generates an abstract model in the verification subfolder abstract. See the VC SpyGlass documentation for more information. Default Value: 0
Constraints	
Constraints Files (ConstraintFiles)	List of constraints files containing the quasi-static signals information. There can be multiple set of files provided as {FileA FileB FileC}.
Additional SDC Files (TopConstraintFiles)	List of top-level constraints files containing the clock definition and other delay related information. There can be multiple set of files and specified as {FileA FileB FileC}. coreTools by default generates a top-level SDC which is placed in the vcspyglass/design_constraint folder. The files which will be specified by the user, gets appended along with the coreTools generated SDC when it is read during verification.
Resets Files (ResetsFiles)	List of top-level resets files. There can be multiple set of files specified as {FileA FileB FileC}.
Waiver Files	List of waiver files.

Table 6-5 RTL Static Checking - Setup (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
(WaiverFiles)	There can be multiple set of files and specified as {FileA FileB FileC}.
Write SDC Args (WriteSdcArgs)	Arguments to the write_sdc call.
TechSetup	
Use Technology Libraries (UseTechLibraries)	Use Technology Libraries defined in the activity TechSetup. This activity must be completed. Default Value: 0
DW Components	
Use Pre-compiled (DwComponentPreCompileEnable)	Use the pre-compiled DesignWare components that are part of the VC SpyGlass release. The versions of the components are aligned with the latest release of DC at the time of the VC SpyGlass release. Default Value: 0
Compile Dir (DwComponentCompileDir)	Folder with pre-compiled DesignWare Components. You can pre-compile the components to the folder myDwComponentDir with the following command: <code>\$VC_STATIC_HOME/auxx/monet/scripts/vsi_dw2vcslib.csh -full64 \$SYNOPSIS myDwComponentDir</code> This feature is optionally enabled in the coreKit. It is disabled if the pre-compiled libraries are enabled.
Other	
Analyze VCS Args (AnalyzeVcsArgs)	Extra arguments to VCS as part of the VC SpyGlass command analyze. See VC SpyGlass documentation for more information.
Elaborate Args (ElaborateArgs)	Extra arguments to the VC SpyGlass command elaborate. See VC SpyGlass documentation for more information.
Additional RTL Files (AdditionalRtlFiles)	List of additional RTL files.
Additional Include Directories (AdditionalIncludeDirs)	List of include directories to be used for searching files specified in the AdditionalRtlFiles parameter.
Verify Threads (VerifyThreads)	The check_<verif> supports multiple threads. Default Values: ■ cdc 1 ■ lint 2

Table 6-5 RTL Static Checking - Setup (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
	■ rdc 1 ■ constraints 1

6.5.2 Execution Options

The parameters listed in [Table 6-6](#) describe the execution options of the RTL static checks.

Table 6-6 RTL Static Checking - Execution Options

Activity Parameter <GUI label> (<parameter name>)	Description
Generate scripts only? (ScriptsOnly)	Setting this causes the run.scr script to be generated in the work directory, but the script does not get executed. The user can run the script at a later time. The path to the run.scr script can be retrieved using the 'get_logical_dir syn/run.scr' command. Default Value: 0
Run style (RunStyle)	If 'Generate scripts only' is unchecked, then this selects how to run the command, which can be run locally via LSF, GRD. Default Value: gui_get_pref_value -key run_style_preference Enum Values: local lsf uge grd
Run Options (RunOptions)	Additional options for the given run style. For LSF and UGE/GRD, this specifies options to the bsub and qsub commands, respectively. For bsub, you need to specify the queue name (for example, '-q normal'). For qsub you can specify a project or other options (for example, '-P bnormal -l arch=glinux'). The -V option is included automatically for qsub. Default Value: gui_get_pref_value -key run_style_options
Parallel job CPU limit (ParallelJobCpuLimit)	Specifies the number of parallel compile jobs that can be run. Default Value: 1
Send e-mail (EmailAddress)	If this parameter is set to a value other than ' ', the email is sent when the command script completes or is ended. The default value is the current user's name.
Report generation waits for grd/lsf jobs (UseBlockingHoldJob)	If the verification jobs uses RunStyle = grd lsf, they are launched with qsub/bsub.

Table 6-6 RTL Static Checking - Execution Options (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
	The report generation job holds for the verification jobs to complete but can be run in 2 ways. They are as follows: ■ A non blocking hold job. Use qsub/bsub to schedule a job that will hold for the verification jobs to complete. This means the activity will return as soon as the verification job is created. ■ A blocking hold job. Use qrsh/bsub -K to create a job that will hold for the verification jobs to complete. This means the activity will wait until the report job is complete before returning. Default Value: 0

6.6 Formal Verification Activity

The Formal Verification activity uses Synopsys Formality (fm_shell) to check two designs for functional equivalence. You can check the gate-level design from a selected phase of a previously executed synthesis strategy against either the RTL version of the design or the gate-level design from another stage of synthesis. Running formal verification through coreAssembler involves:

- [Executing the Formal Verification Activity](#)
- [UPF Support in Formality](#)
- [Checking Formal Verification Results](#)

6.6.1 Executing the Formal Verification Activity

To execute the Formal Verification activity, select Verify Subsystem > Formal Verification in the Activity List pane. The corresponding Activity View pane appears. In this Activity View you specify the Activity Controls and the Execution Options, defined in [Table 6-7](#) and [Table 6-10](#), respectively.

Table 6-7 Formal Verification - Activity Controls

Activity Parameter <GUI label> (<parameter name>)	Description
Activity Controls	
Reference Stage Name (ReferenceStageName)	Specify the design stage to use as the golden reference design. The legal values are RTL (use the RTL version of design as reference design) or any completed synthesis stage beginning with initial (You cannot specify constrain or elab as the reference design stage).
Use Reference Stage Netlist (UseReferenceStageNetlist)	Specify that the verification use the netlist from the reference stage instead of the database. Legal values are 1 (true) and 0 (false). Default: 0
Reference Stage Netlist (ReferenceStageNetlist)	Specify the name of the reference stage netlist.
Target Stage Name (TargetStageName)	Specify the stage of the design that you want to verify. Legal values are any completed synthesis stage beginning with initial. (You cannot specify constrain or elab as the target design stage.) coreAssembler uses Synopsys Formality to formally verify the Target Stage Name design using the Reference Stage Name design as the golden reference.
Use Target Stage Netlist (UseTargetStageNetlist)	Specify that the verification use the netlist from the target stage instead of the database. Legal values are 1 (true) and 0 (false). Default: 0
Use UPF PG Netlist (UseUPFPGNetlist)	Use netlist with PG (power/ground) for Golden UPF flow. When set to false, PG info is in the UPF files. Default: false
Target Stage Netlist (TargetStageNetlist)	Specify the name of the target stage netlist. Default: subsystem.v

Table 6-7 Formal Verification - Activity Controls (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
UPF Auxiliary Script Name (UpfAuxiliaryScriptName)	Defines the name of a script to be sourced when using a UPF flow. This script is sourced after the implementation design is loaded, but before issuing the 'set_top' command. It can be used for specification of retention register models that match in both the reference and implementation designs.
Auxiliary Script Name (AuxiliaryScriptName)	Specify the name of any auxiliary fm_shell script that you want to execute before the fm_shell verify command. For example, you may want to insert commands to set or remove compare points.
Use Generated Svf (UseGeneratedSvf)	Controls whether the SVF generated during synthesis is used in Formality. Default: 1

Table 6-8 Formal Verification - Alternate Strategies

Activity Parameter <GUI label> (<parameter name>)	Description
Alternate Strategies	
Valid alternate strategies names (ValidAlternateStrategies)	List of valid alternate strategy names for Formality
Alternate strategies list (AlternateStrategies)	Run the Formality command run_alternate_strategies using the specified strategies. For a list of allowed strategies, refer to the Valid Alternate Strategies field above. Use whitespace to separate each strategy in the list. To avoid running the run_alternate_strategies command, leave this field blank.
Run all valid alternate strategies (RunAllValidAlternateStrategies)	Run all valid alternate strategies defined in the version of Formality being used. This parameter is not related to the -run_all witch used in the run_alternate_strategies command. Default: 0 (false)
Parallel alternate strategies (ParallelAlternateStrategies)	Number of alternate strategies run in parallel. Default: 4
Alternate strategies extra arguments (AlternateStrategiesExtraArguments)	Extra arguments appended to the run_alternate_strategies Formality command.
Alternate strategies run style (AlternateStrategiesRunStyle)	Defines the protocol for launching worker processes on remote .The protocol corresponds to the type of compute farm manager you are submitting to.

Table 6-8 Formal Verification - Alternate Strategies (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
	When RunStyle is 'local' then AlternateStrategiesRunStyle defaults to 'grd' otherwise AlternateStrategiesRunStyle is set to the RunStyle.
Alternate strategies run options (AlternateStrategiesRunOptions)	Grid options for launching distributed jobs This attribute value is passed to the set_host_options command with the -submit_command argument.

Table 6-9 Formal Verification – DPX Flow

Activity Parameter <GUI label> (<parameter name>)	Description
DPX Flow	
Enable DPX Flow (EnableDPXFlow)	DPX allows more partitions to be verified in parallel compared to the standard Formality equivalence checking solution. The DPX Manager (the main Formality process you interact with) saves a session file that contains the entire design space and verification context. Each worker process reads the session file and accepts tasks from the manager that consists of instructions to verify a partition using a specific solver strategy. Workers can accept one task per core and can run N number of tasks in parallel. When a task completes a specific effort level, the status is returned to the DPX Manager. The feature is can be enabled when the Formal Verification RunStyle activity parameter is set to 'grd' and the license check for 'Formality-DPX' is successful. Additional information on the DPX flow can be found in the Formality User Guide.
DPX run style (DPXRunStyle)	Defines the protocol for launching worker processes on remote or local machines. The protocol corresponds to the type of compute farm manager you are submitting to or, when not using a compute farm manager, to the method used to access the compute hosts. When RunStyle is 'local' then DPXRunStyle defaults to 'grd' otherwise DPXRunStyle is set to the RunStyle.
DPX run options (DPXRunOptions)	Grid options for launching distributed jobs. This attribute value is passed to set_dpx_options command with the -submit_command argument.
Specifies the number of machines (MaxWorkers)	Specifies the maximum number of machines requested for processing distributed tasks. The default is four machines.

Table 6-9 Formal Verification – DPX Flow (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
Number of parallel tasks per machine (MaxCores)	Specifies the maximum number of compute cores per machine assumed to be available for processing distributed tasks. The default is four cores per machine.

Table 6-10 Formal Verification - Execution Options

Activity Parameter <GUI label> (<parameter name>)	Description
Execution Options	
Generate Scripts Only (ScriptsOnly)	When this box is checked, coreAssembler generates and writes the formal verification scripts but doesn't run them. If the box is not checked (the default setting), coreAssembler automatically invokes fm_shell to execute the scripts as soon as they are generated.
Run Style (RunStyle)	Specifies whether to execute synthesis on the local host (the default), through the load sharing facility (LSF), through the grid engine (GRD).
Run Style Options (RunStyleOptions)	If you select Run Style = lsf or Run Style = grd, you can enter options (such as -m <i>machine name</i> to direct the job to a particular machine) in this text box. coreAssembler passes the specified options to the bsub command (for lsf) or qsub command (for grd).
Send e-mail (EmailAddress)	If selected, this option sends an e-mail message to the specified address when formal verification completes or is ended. The default for the text box labeled "To" is your (current user) user name.

Clicking Apply causes a script to be generated to run Formality comparing the synthesis results from the two stages (or possibly from one stage and the RTL). If the Scripts Only option is not chosen, Formality is launched and you can switch to the Report tab to see the job status and view results.

At the completion of the fm_shell execution, the fm_shell reports are in your workspace/syn/stage/report directory, where stage is your selected Target Stage Name. coreAssembler also generates an fm_shell save_session command. The resulting .fss file is in your workspace/syn/stage/db directory.

6.6.2 UPF Support in Formality

If the UPF flow was enabled for synthesis, then the appropriate UPF files are automatically loaded into Formality for both the reference and implementation designs. If the reference design is the RTL, then the original UPF file is loaded. In all other cases, the UPF file written at the completion of the synthesis stage is loaded into Formality. For more information, see the "UPF Auxiliary Script Name" entry in [Table 6-10](#).

6.6.3 Checking Formal Verification Results

coreAssembler automatically generates the following commands to extract reports from fm_shell:

- `report_libraries -short`
- `report_failing_points`
- `report_compare_points` (generated only if you specify an Auxiliary Script)

If the formal verification fails, coreAssembler also generates the following fm_shell commands:

- `diagnose`
- `report_error_candidates`

The output for each of the fm_shell report commands is in your *workspace/syn/stage/report* directory and is viewable on the Report tab of the Formal Verification Activity View. To check your formal verification results:

- For more information about diagnosing and fixing failing points, see the *Formality User Guide*. If you need to work with Formality directly to diagnose failing points, you can load the saved session file (*workspace/syn/stage/final/db/Design.fss*) directly into Formality.
- To complete the Formal Verification activity in command line mode, use the `set_activity_parameter` and `autocomplete_activity` commands. For example:

```
set_activity_parameter FormalVerification ReferenceStageName RTL
set_activity_parameter FormalVerification TargetStageName incr1
autocomplete_activity FormalVerification
```

6.7 Static Timing Analysis Activity

The **Static Timing Analysis** activity provides an interface to PrimeTime for performing static timing analysis. The activity is within the “Verification” group of activities, is not visible in testbench workspaces, and requires that a tool root is defined for PrimeTime.

Within this activity, you can choose to generate the following reports:

- Paths crossing clock domains
- Standard STA reports (as defined by the PrimeTime reference methodology)

The report for clock domain crossing paths are based on the SolvNetPlus article on that topic. The standard STA reports come from integration of support for the PrimeTime Reference Methodology. In both cases, HTML reports are available from the “Reports” tab and also referenced with HTML links.

In addition to specifying report generation, you can specify the input data required to run the static timing analysis. This consists of either a synthesis stage name (using coreTools synthesis), or a list of netlist files (using a non-coreTools based synthesis process). You can also provide your own SDC file (optional) to override the one generated by Design Compiler, an SDF or SPEF file (optional), and a PrimeTime script file (optional). For some of the more configurable reports, the ability to specify command line options is provided. You can also generate but not run the PrimeTime scripts, and launch scripts using load sharing programs.

Table 6-11 Static Timing Analysis -- Netlist Setup

Activity Parameter <GUI label> (<parameter name>)	Description
Use externally generated netlist (UseExternalNetlist)	Check this box to specify a netlist from a source outside this workspace. Leave unchecked to choose a netlist in this workspace.
Use UPF PG Netlist (UseUPFPGNetlist)	Use netlist with PG (power/ground) for Golden UPF flow. When set to false, PG info is in the UPF files. Default: false.
Input stage name (InputStageName)	Name of the stage that contains the design to be analyzed. Default: Last <incr2>
External netlist file (NetlistFile)	Path to the target netlist, required if an externally-generated netlist is to be analyzed.
Constraints File (ConstraintsFile)	SDC Constraints File to be applied to the netlist. Default: (=sPt::getDefaultConstraintsFile)
Back-annotated data (BackannotatedData)	Format of back-annotated data to apply to the netlist. Default: None
Back-annotated data file (BackAnnotatedDataFile)	File containing data to back-annotate onto the netlist. Default: (=sPt::getDefaultBackAnnotationData)
Back-annotated command string (BackAnnotationOptions)	Specify any switches and options to pass to “read_sdf” or “read_spef” to guide the back annotation.

Table 6-12 Static Timing Analysis -- Analysis Setup

Activity Parameter <GUI label> (<parameter name>)	Description
Timing Analysis Setup	
Analysis Mode (AnalysisType)	Choose the type of analysis to be performed on the netlist. Default: On-chip Variation
Set Delay Derating Factors (TimingDerateValue)	Set delay derating factors for either the design or a specified list of instances. Default: No derating
Timing Derate Options (TimingDerateOptions)	Arguments to be passed to the 'set_timing_derate' command in addition to '-early' or '-late'.
Timing Derate Factor (TimingDerateValue)	Timing Derate value to be applied. Default: 1
Use Propagated Clocks (UsePropagatedClocks)	Setting this parameter to 'true' specifies that delays be propagated through the clock network to determine latency at register clock pins. A value of 'false' means that ideal clocking is assumed. Default: 0
Analyze netlist in mission mode (AnalyzeInMissionMode)	Perform timing analysis with the netlist in 'mission' mode, that is, with scan mode disabled. In this mode, any scan-enabled pin inserted by DC is driven to a value which disables scan mode. Default: 1
Enable ECO Flow (EnableECOFlow)	Enables the ECO Flow in PrimeTime. Enabling it, PrimeTime would run the command 'fix_eco_timing'. This fixes the setup timing violations, set as default, through Engineering Change Order (ECO) changes.
Options for fix_eco_timing (FixECOTimingArgs)	Defines arguments to be passed to the fix_eco_timing when writing the Static Timing Analysis intent file.
Auxiliary script name: (AuxiliaryScriptName)	Path to optional Static Timing script. The script is executed after the design has been loaded and constrained in PrimeTime, before the update_timing command which performs the timing analysis.
Custom Script	
Custom PrimeTime Script (PTScriptFile)	Path to optional PrimeTime script. PrimeTime executes this script at the end of its run, after all selected reports have been generated.
Power Analysis Setup	
Power Analysis Flow (PowerAnalysisFlow)	Power Analysis flow to use: select 'VCD/VCD+/FSDB' for analysis using waves, 'SAIF' for analysis based on a SAIF file, or 'Vector-free' to estimate power without simulation data.

Table 6-12 Static Timing Analysis -- Analysis Setup (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
	Default: Vector-free
Power Analysis Mode (PowerAnalysisMode)	Specify Power Analysis mode. Use this parameter to choose either averaged or time-based analysis. This feature is supported in PrimeTime-PX 2008.12 or newer. Default: Averaged
Switching Activity File (SwitchingActivityFile)	Switching activity files to be used by PrimeTime-PX for power analysis. For vector-based analysis, this is either a VCD/VCD+/FSDB or SAIF file. Important: If multiple files are given, all of the files specified for this option must be of the same type (so you cannot mix VPD and SAIF switching activity files in the same session). For the vector-free flow, this is an optional file containing toggle and switching commands for PrimeTime-PX.
Simulation Type (SimulationType)	Use this parameter to specify whether your VCD (or VCD+) wave file was generated from: ■ “RTL” simulation ■ “gate” - gate-level simulation with SDF ■ “zero_delay” - gate-level without SDF Default: RTL Simulation
SAIF Name Mapping File (SaifNameMapFile)	For Power Analysis using data (waves or SAIF) captured from an RTL simulation, this option file can be used to specify mapping between RTL and gate-level object names. As the names of RTL objects may change after synthesis, providing a mapping file may help to increase the accuracy of power calculations by helping PrimeTime-PX to annotate switching activity accurately to the gate-level netlist. Default: (=sPt::getDefaultSaifMapFile)
Testbench Path to design inst. (StripPath)	If the current design is instantiated as an instance in a testbench used to generate a VCD/VCD+ or SAIF file, this parameter is used to specify the instance of the current design as specified in the testbench, for example, test_toplevel/U_instance.
Start Time for Power Analysis (StartTime)	Start time of time window for power calculation. A value of zero means that the calculation will start from the beginning of the wave file. The 'StartTime' parameter is now a list. If you specify one value in each box, that start time will be applied to all of the switching activity files listed. If you want to have different start times for different switching activity files, just enter a list (the number of entries in the list must correspond to the number of switching activity files specified). Default: 0

Table 6-12 Static Timing Analysis -- Analysis Setup (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
End Time for Power Analysis (EndTime)	End time of time window for power calculation. A value of -1 means that the calculation will cover the time window from the specified start time to the end of the wave file. The 'EndTime' parameter is now a list. If you specify one value in each box, that end time will be applied to all of the switching activity files listed. If you want to have different start and end times for different switching activity files, just enter a list (the number of entries in the list must correspond to the number of switching activity files specified). Default: -1

Table 6-13 Static Timing Analysis -- Override Tech Setup

Activity Parameter <GUI label> (<parameter name>)	Description
Override Tech Setup	
Override Search Paths (OverrideSearchPath)	A separate group of search paths for libraries which are required in Static Timing Analysis. These search paths are set in the variable ::search_path reflect in the <StageName>_pt.ptsh file. Default: null
Override Link Libraries (OverrideLinkLibraryMapping)	A separate group of link libraries which are specifically used for Static Timing Analysis. This attribute when enabled will override the previous Techsetup link libraries and setup the new libraries. These libraries will be set in the variable ::link_libraries reflected in the <StageName>_pt.ptsh file. This parameter uses a mapping technique which is used to run Static Timing Analysis with the updated SDC. To provide the inputs to this parameter, you must set it up in a list of list sequence. For example, {{Link_Library_Tech_Setup_1 Override_Link_Library_1} {Link_Library_Tech_Setup_2 Override_Link_Library_2}} When the Static Timing Analysis is fired, coreTools checks for the Link_Library_Tech_Setup_* attributes in the SDC and updates it to the parameters of Override_Link_Library_*. Only Override_Link_Library_1 will be used to extract the Operating Condition and update the Link_Library_Tech_Setup_1 OPCON in the SDC and pt intent file. Default: null
Override Default Operating Condition (OverrideDefaultOperatingCondition)	An optional parameter when using different sets of libraries in Static Timing Analysis. It is used to change the Operating Condition during STA. If not provided, the tool will automatically extract the corresponding Operating Condition from the provided reference library given in OverrideLinkLibraryMapping.

Table 6-13 Static Timing Analysis -- Override Tech Setup (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
	Default: null

Table 6-14 Static Timing Analysis -- Select and Configure Reports

Activity Parameter <GUI label> (<parameter name>)	Description
Report Design Attributes (ReportDesign)	Setting this parameter to 'true' runs the 'report_design' command. Default: 1
Report Clocks (ReportClocks)	Setting this parameter to 'true' runs the 'report_clocks' command. Default: 1
Report Register Information (ReportRegisterInfo)	Creates a report which lists the number of registers in the IP. The number of positive- and negative-edge flops per domain, and the number of latches per domain, along with a list of all such registers found. Default: 0 (false)
Report Synchronization Flops (ReportSyncPaths)	Creates a report showing the synchronization flops in the design. Default: 0 (false)
Report Case Analysis (ReportCaseAnalysis)	Runs the 'report_case_analysis' command. Default: 0
Report Timing Exceptions (ReportExceptions)	Runs the 'report_exceptions' command. Default: 1
Report Timing Exceptions Options (ReportExceptionsOptions)	Options and switches to pass to the 'report_exceptions' command.
Report Analysis Coverage Check Type (ReportAnalysisCoverageCheckType)	Specifies arguments to the -check_type option for the report_analysis_coverage command for PrimeTime. Default: setup recovery removal min_pulse_width clock_gating_setup If the synthesis strategy is Fusion_Compiler_Reference_Methodology and the FCRM Strategy parameter (EnableQIK) is set, the DefaultValue list is setup recovery removal min_pulse_width clock_gating_setup hold
Report Timing (Max Delay) (ReportMaxTiming)	Runs the 'report_timing' command to report 'max' path delays. Default: 1
Report Timing (Max Delay) Options (ReportMaxTimingOptions)	Additional options and switches to pass to 'report_iming -delay max'

Table 6-14 Static Timing Analysis -- Select and Configure Reports (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
	Default: -delay_type max -nworst 10 -max_paths 100 -input -net -trans -cap -nosplit -cross -path full -sig 5
Report Timing (Min Delay) (ReportMinTiming)	Runs the 'report_timing' command to report 'min' path delays. Default: 0
Report Timing (Min Delay) Options (ReportMinTimingOptions)	Additional options and switches to pass to 'report_timing -delay min' Default: -delay_type min -nworst 10 -max_paths 100 -input -net -trans -cap -nosplit -cross -path full -sig 5
Report Constraints (ReportConstraints)	Runs the 'report_constraints' command. Default: 1
Report Constraints Options (ReportConstraintsOptions)	Options to pass to the 'report_constraints' command. Default: -all_violators -max_delay -sig 5
Report Cross-domain Paths (ReportCrossDomainPaths)	Report paths which cross clock domains. Default: 0
Number of paths to report (NumCrossDomainPaths)	Maximum number of paths to report (from clock group to clock group) Default: 100
Report Format (CrossDomainReportFormat)	Format of cross-domain report. 'Summary' returns the start and end points of each path, 'short' reports show the cells at the start and end of each path, 'Full (detailed) paths' reports show each cross-domain path in its entirety. Default: Full (detailed) paths
Suppress Duplicate Bus Paths (SuppressDuplicateBusPaths)	Affects the reporting of cross-clock-domain paths. When enabled, duplicate paths on buses going between clock domains are not reported. Default: true
Run Bottleneck Analysis (ReportBottleneck)	Report timing bottlenecks. Default: 0
Bottleneck Analysis Options (ReportBottleneckOptions)	Specify the arguments to pass to the report_bottleneck command.
Run Clock Timing Report (ReportClockTiming)	Run the report_clock_timing report. Default: 0
Run Minimum Pulse Width rpt (ReportMinPulseWidth)	Run the report_min_pulse_width report. Default: 0

Table 6-14 Static Timing Analysis -- Select and Configure Reports (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
Minimum Pulse Width Report Options (ReportMinPulseWidthOptions)	Arguments to pass to the report_min_pulse_width command.
Run clock-gating report (ReportClockGatingCheck)	Run the report_clock_gating_check report. Default: 0
Clock Gating Report Options (ReportClockGatingCheckOptions)	Arguments to pass to the report_clock_gating_check report. Default: -sig 5
Run Power Analysis (ReportPower)	Run Power Analysis using PrimeTime-PX. Default: 0
Power Analysis Options (PowerAnalysisOptions)	Options to pass to the set_power_analysis_options command in PrimeTime-PX. This parameter is only supported in PrimeTime-PX 2008.12 or newer.
Power Report Options (ReportPowerOptions)	Options to be passed to the report_power command in PrimeTime-PX Default: -hierarchy
Report Switching Activity (ReportSwitchingActivity)	Run the report_switching_activity command to generate a report of annotated switching activity on the design. Default: 1
Switching Activity Report Options (ReportSwitchingActivityOptions)	Options and switches to pass to the report_switching_activity command.
Report Clock Gate Savings (ReportClockGateSavings)	Generate a set of reports using the report_clock_gate_savings report command. Default: 0 (false)
Link To TestMax (LinkToATPG)	When true, creates timing files needed by TestMAX for the TMAX Reference Methodology strategy. For more information, see Options for the TMAX Reference Methodology Strategy on page 166. Default: 1 (true)

Table 6-15 Static Timing Analysis -- Execution Options

Activity Parameter <GUI label> (<parameter name>)	Description
Generate scripts only? (ScriptsOnly)	Setting this causes the script run.scr to be generated in the <workspace>/syn directory, but the script is not executed. You can then run the script at a later time. The path to the run.scr script is available using the "get_logical_dir syn/run.scr" command.

Table 6-15 Static Timing Analysis -- Execution Options (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
	Default: 0
Run Style (RunStyle)	If “Generate scripts only?” is not checked, this item selects how to run the script file, which can be run locally, using LSF, GRD. Default: local
Run style options (LsfOptions)	Additional options can be provided for a given run style. For LSF and GRD, this specifies options to the bsub and qsub commands, respectively (for example ‘-q normal’). For qsub, you can specify a project or other options (for example ‘-P normal -l arch=glinux’). The -V option is included automatically for qsub. Default: (=get_activity_parameter Synthesize RunOptions)
Send e-mail / To: (EmailAddress)	If this parameter is set to a value other than null, then email is sent when the command script completes or is ended. Default: <current username>

6.7.1 Checking Static Timing Analysis Results

To complete the Static Timing Analysis activity in command line mode, use the `set_activity_parameter` and `autocomplete_activity` commands. For example:

```
% autocomplete_activity StaticTimingAnalysis
```

If you need to work directly with PrimeTime after running the activity, you can load the saved session directory *workspace/syn/stage/final/db/design_pt_session* into PrimeTime.

6.8 Constraint Analysis Activity

Choose this activity to generate and, optionally, run ptc_shell (PrimeTime) synthesis constraint analysis scripts.

Table 6-16 Constraint Analysis -- Netlist Setup Options

Activity Parameter <GUI label> (<parameter name>)	Description
Use externally-generated netlist (UseExternalNetlist)	Check this option to specify a netlist that is outside the open workspace. Leave unchecked to choose a netlist in the open workspace. Default: 0
Input stage name (InputStageName)	Name of the stage that contains the design to be analyzed. This option applies only for netlists from the open workspace (UseExternalNetlist is not checked). Default: Last <initial>
External netlist file (NetlistFile)	Path to an external netlist. Use this when UseExternalNetlist is checked.
Constraints File (ConstraintsFile)	Specify the path to an SDC constraints file to be applied to the netlist.

Table 6-17 Constraint Analysis -- Control Files

Activity Parameter <GUI label> (<parameter name>)	Description
Incremental constraints file (IncrementalConstraintsFile)	Incremental constraints file to be sourced before reading SDC.
Waivers File (WaiversFile)	Waivers file to be sourced before generating reports.

Table 6-18 Constraint Analysis -- Execution Options

Activity Parameter <GUI label> (<parameter name>)	Description
Generate scripts only? (ScriptsOnly)	Setting this causes the script run.scr to be generated in the <workspace>/syn directory, but the script is not executed. You can then run the script at a later time. The path to the run.scr script is available using the "get_logical_dir syn/run.scr" command. Default: 0
Run style (RunStyle)	If "Generate scripts only?" is not checked, this item selects how to run the script file, which can be run locally, using LSF, GRD. Default: local

Table 6-18 Constraint Analysis -- Execution Options (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
Run style options (RunOptions)	Additional options can be provided for a given run style. For LSF and GRD, this specifies options to the bsub and qsub commands, respectively (for example '-q normal'). For qsub, you can specify a project or other options (for example '-P normal -l arch=glinux'). The -V option is included automatically for qsub. Default: (=get_activity_parameter Synthesize RunOptions)
Send e-mail (EmailAddress)	If selected, this option sends an e-mail message to the specified address when formal verification completes or is ended. The default for the text box labeled "To" is your (current user) user name.

6.9 Explore RTL Architecture Activity

The Explore RTL Architecture activity incorporates the Synopsys RTL Architect™ tool. The RTL Architect tool enables early, physically aware RTL exploration to achieve a better starting point for RTL synthesis and better convergence during physical implementation.

In coreAssembler, you can use this activity to perform what-if analyses of various design configurations to speed the development of RTL and design constraints without waiting for feedback from the physical (back-end) design team. This activity produces numerous reports that show PPA and timing estimations. For multiple configuration runs (FlowType = explore), comparison reports are included. For more about the RTL Architecture tool, see the RTL Architecture product documentation.

Before running this activity, you must set paths to the following tools in Edit > Tool Installation Roots:

- PrimeTime (pt_shell)
- RTL Architect (rtl_shell)

 **Note**

All components instantiated in the subsystem need to be synthesizable in RTL Architect.

The major settings for this activity are described in the following tables:

- [Table 6-19](#)
- [Table 6-20](#)
- [Table 6-21](#)
- [Table 6-22](#)

[Table 6-19](#) describes the activity control settings for the Explore RTL Architecture activity.

Table 6-19 Explore RTL Architecture -- Activity Controls

Activity Parameter <GUI label> (<parameter name>)	Description
Flow Settings	
RTL Architect Flow Type (FlowType)	Selects the RTL Architect flow type for this block. ■ flat: Run fast synthesis (optionally with auto floorplanning) and generate congestion, timing, and power metrics ■ explore: Run data exploration based on utilization, aspect ratio, and max routing layers Default: flat
RTL Architect Design Style (DesignStyle)	Sets the DESIGN_STYLE variable for this block. ■ flat: Flat place and route flow ■ hier: Hierarchical place and route flow Default: flat
Insert DFT (InsertDft)	Enables design for test circuitry to be inserted based on the settings from the Specify Test Methodology activity.

Table 6-19 Explore RTL Architecture -- Activity Controls (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
	Default: 0 (false)
DFT Flow Type (DFTFlowType)	Sets the RTL Architect DFT_FLOW control variable to in_compile or pre_compile. Default: in_compile
Use Technology File (UseTechFile)	Use technology file to specify technology information. When enabled, the target library (TECH_LIB) setup is not used. Default: 0 (false)
Floorplan	
Floorplan Data Format (FloorplanFileDataFormat)	Format of file containing floorplan constraints. ■ DEF: Design exchange format file Default: DEF
Physical Constraint File (PhysicalConstraintFile)	Path to the floorplan file; if unspecified, a default set of: ■ A path to a TCL floorplan file populates the TCL_FLOORPLAN_FILE setup parameter. ■ A path to a DEF floorplan file populates the DEF_FLOORPLAN_FILE setup parameter.
Constraint Settings	
Options for write_sdc (WriteSDCArgs)	Defines arguments to be passed to the write_sdc command when writing SDC constraints for synthesis.
TLUPlus	
Early parasitic tech specification name (MinTluPlusName)	Specifies the early parasitic tech specification name. Default: minTLUP
Min TLUPlus file name (MinTluPlusFile)	Specifies the TLUPlus files to read for Min specification.
Min TLUPlus Layer map file (MinTluPlusLayerMapFile)	Path to the layer mapping file between ITF and Min tech specification.
Late parasitic tech specification name (MaxTluPlusName)	Specifies the late parasitic tech specification name. Default: maxTLUP
Max TLUPlus file name (MaxTluPlusFile)	Specifies the TLUPlus files to read for Max specification.
MAX TLUPlus Layer map file (MaxTluPlusLayerMapFile)	Layer mapping file between ITF and Max tech specification.

Table 6-19 Explore RTL Architecture -- Activity Controls (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
UPF	
Enable UPF Flow (EnableUPFFlow)	When Enable UPF Flow is set to true (1), the UPF and power constraints files are used during synthesis. Default: 0 (false)
UPF Flow Type (UPFFlowType)	Use the default UPF flow. Default: UPF
UPF File (UPFFile)	The name of the Unified Power Format (UPF) file to be used with the Fusion Compiler.
Power Constraints File (PowerConstraintsFile)	Power constraints file that is sourced in Fusion Compiler after loading the UPFFile. This file can be used to issue 'set_voltage' commands and remove ouch elements on library cells, as needed.
Utilization Ratio	
Utilization Ratio	Specifies the utilization of the core area. The utilization is the total area of the core occupied by all standard cells and macro cells divided by the total core area. You can specify a value between 0 and 1. For example, a core utilization of 0.8 specifies that 80 percent of the core area is used for cell placement at this stage. By default, the core utilization is 0.7. The value specified is used to update the control variable 'AUTO_FLOORPLAN_UTILIZATION' in the RTL Architect reference methodology scripts. Enabled when FlowType = flat Default: 0.7

Table 6-20 describes the flat flow settings for the Explore RTL Architecture activity.

Table 6-20 Explore RTL Architecture -- Flat Flow Settings

Activity Parameter <GUI label> (<parameter name>)	Description
Enable Congestion Metrics (EnableFlatFlowCongestionMetrics)	Enable congestion metrics in flat flow. Default: 1 (true)
Enable Timing Metrics (EnableFlatFlowTimingMetrics)	Enable timing metrics in flat flow. Default: 1 (true)

Table 6-20 Explore RTL Architecture -- Flat Flow Settings (continued)

Activity Parameter <GUI label> (<parameter name>)	Description
Enable Power Metrics (EnableFlatFlowPowerMetrics)	Enable power metrics in flat flow. Default: 0 (false)

[Table 6-21](#) describes the explore flow settings for the Explore RTL Architecture activity.

Table 6-21 Explore RTL Architecture -- Explore Flow Settings

Activity Parameter <GUI label> (<parameter name>)	Description
Utilization values (ExploreUtilizationList)	Set the EXPLORE_UTILIZATION_LIST values for the explore flow.
Aspect ratio values (ExploreAspectRatioList)	Set the EXPLORE_ASPECT_RATIO_LIST values for the explore flow.
Max routing layer value(s) (ExploreMaxRoutingLayerList)	Set the EXPLORE_MAX_ROUTING_LAYER_LIST/MAX_ROUTING_LAYER variables for the explore flow. ■ Specifying more than one value sets the EXPLORE_MAX_ROUTING_LAYER_LIST variable in the rm scripts. ■ Specifying one value sets the MAX_ROUTING_LAYER variable in the rm scripts.
Min routing layer value (ExploreMinRoutingLayer)	Set the MIN_ROUTING_LAYER value for the explore flow.
Explore Flow Run Style (ExploreFlowRunStyle)	Defines the protocol for launching worker processes on remote .The protocol corresponds to the type of compute farm manager you are submitting to. When RunStyle is 'local' then ExploreFlowRunStyle defaults to 'grd' otherwise ExploreFlowRunStyle is set to the RunStyle.
Explore Flow Grid Options (ExploreFlowGrdOptions)	Grid options for launching distributed jobs. Distributed run is set to true for explore flow and false for flat flow. This attribute value is passed to the set_host_options command with the -submit_command argument. The set_host_options command is used when the 'DISTRIBUTED' variable is set to true in rtl_setup.tcl.

[Table 6-22](#) describes the execution options for the Explore RTL Architecture activity.

Table 6-22 Explore RTL Architecture -- Execution Options

Activity Parameter <GUI label> (<parameter name>)	Description
Generate scripts only? (ScriptsOnly)	Setting this causes the script run.scr to be generated in the <workspace>/syn directory, but the script is not executed. You can then run the script at a later time. The path to the run.scr script is available using the “get_logical_dir syn/run.scr” command. Default: 0
Run style (RunStyle)	If “Generate scripts only?” is not checked, this item selects how to run the script file, which can be run locally, using LSF, GRD. Default: local
Run style options (RunOptions)	Additional options can be provided for a given run style. For LSF and GRD, this specifies options to the bsub and qsub commands, respectively (for example ‘-q normal’). For qsub, you can specify a project or other options (for example ‘-P normal -l arch=glinux’). The -V option is included automatically for qsub. Default: (=get_activity_parameter Synthesize RunOptions)
Send e-mail (EmailAddress)	If selected, this option sends an e-mail message to the specified address when formal verification completes or is ended. The default for the text box labeled “To” is your (current user) user name.

6.10 Timing Constraints Manager

Constraint verification refers to the verification of the contents of an SDC/ TCL file to flag situations where the specified constraints are either incorrect or incomplete, both of which if not addressed could result in silicon failure.

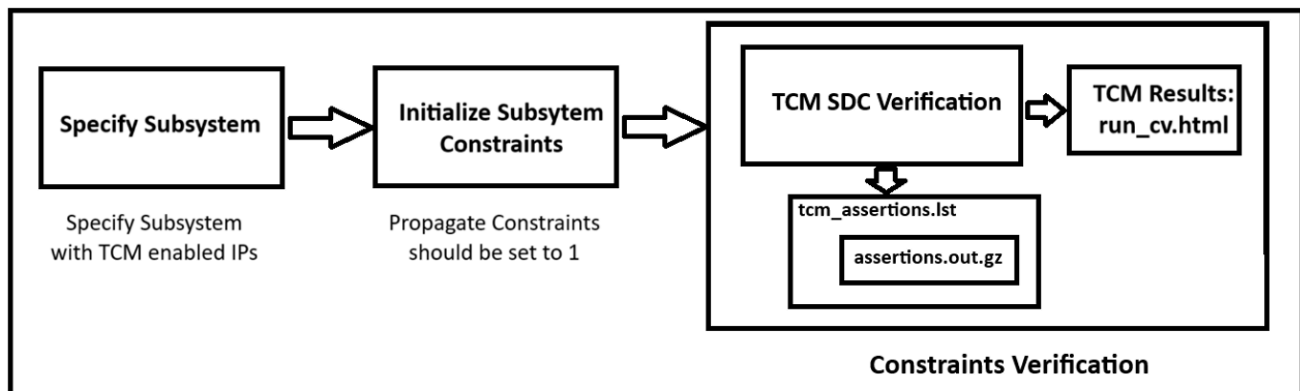
Running lint checks on the RTL is recommended for identifying and resolving problems before simulation or synthesis. Similarly, linting SDC constraints before synthesis or STA helps to quickly isolate and correct issues avoiding unnecessary delays in timing closure constraints before they emerge in the time consuming implementation analysis. The best constraint verification flow is the one which has a high ability to detect issues with minimum noise, that is, issues that upon the user's review, results in no change to the design or constraints.

The Timing Constraint Manager (TCM) is an effective tool while performing the constraint verification. It enables the user in applying minimal effort due to minimal noise and detects the constraint syntax, mapping, and verification issues efficiently. The tool performs lint checks to verify a certain group of constraints, whereas the timing exception related constraints are checked formally.

The TCM flow is performed with the Constraint Verification (CV) Activity.

Figure 6-4 shows the TCM flow diagram.

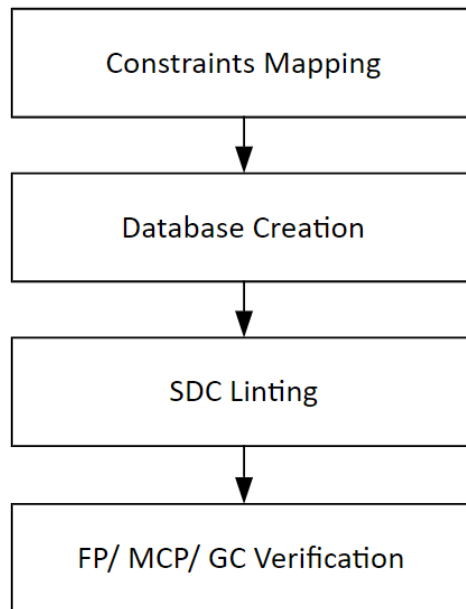
Figure 6-4 TCM Flow Diagram



Constraints Verification in coreAssembler Flow

6.10.1 Constraints Verification

The TCM flow supports the stages as shown in Figure 6-5 to perform the constraints verification.

Figure 6-5 Constraint Verification Stages

6.10.1.1 Constraints Mapping

This stage reads the RTL/ constraints and applies the constraints to the design. The constraint syntax/ semantic and RTL parsing errors are also reported.

The stage has the ability to auto handle the synthesis transformation such as “_” to “/” conversion. For example, the netlist hierarchy a_b_c can be mapped to a/b/c.

At the mapping stage, the main files are shown in [Figure 6-6](#) which can be found at `<workspace>/constraintsverification/tcm_rtl_verification`.

Figure 6-6 Constraint Mapping Stage Files

```
- setup
  ├── files.f
  ├── interactive.tcl
  ├── map_sdc_and_check_setup
  ├── map_sdc.tcl
  ├── refocus_interactive.tcl
  ├── setup.tcl
  ├── waive.sdc
  └── waive.tcl
```

6.10.1.2 Database Creation

In this stage, the database model is created. The design.fdb is the model that can be reused if the design is unchanged in the future runs. Reusing the design.fdb file also helps in reducing the runtime. [Figure 6-7](#) lists the database creation stage files.

Figure 6-7 Database Creation Stage Files

```

-- ver_db
  |-- create_ver_db
  |-- create_ver_db.tcl

```

6.10.1.3 SDC Linting

The linting stage performs the formal SDC linting. The SDC linting stage files are as shown in [Figure 6-8](#).

Figure 6-8 SDC Linting Stage Files

```

-- setup
  |-- files.f
  |-- interactive.tcl
  |-- map_sdc_and_check_setup
  |-- map_sdc.tcl
  |-- refocus_interactive.tcl
  |-- setup.tcl
  |-- waive.sdc
  |-- waive.tcl

```

6.10.1.4 FP/ MCP/ GC Verification

The FP/ MCP/ GC verification run validates the timing exceptions and generated clocks. If there are assertion failures, they are written in the same directories with the file name 'assertions.out.gz'. [Figure 6-9](#) shows the FP/ MCP/ GC verification stage files.

Figure 6-9 FP/ MCP/ GC Verification Stage Files

```

-- fp_verification
  |-- generate_html
  |-- generate_html.tcl
  |-- incremental.tcl
  |-- interactive.tcl
  |-- run_confirm.tcl
  |-- verify_fp
-- gc_verification
  |-- generate_html
  |-- generate_html.tcl
  |-- incremental.tcl
  |-- interactive.tcl
  |-- run_confirm.tcl
  |-- verify_gc
-- generate_all
-- generate_summary
-- mcp_verification
  |-- generate_html
  |-- generate_html.tcl
  |-- incremental.tcl
  |-- interactive.tcl
  |-- run_confirm.tcl
  |-- verify_mcp

```

6.10.2 Constraints Verification (CV) Activity

In coreConsultant, the TCM Constraints Verification is run with the set of activity parameters as shown in Table 6-23.



Note

- The ConstraintVerification activity in coreAssembler is specifically designed to work with subsystems that incorporate Synopsys components with TCM (Timing Constraint Manager) enabled, imported RTL components, or IP-XACT components.
- A necessary condition for running the ConstraintVerification activity in coreAssembler is that the 'PropagateAllConstraints' parameter must be set in the InitializeSubsystemConstraints activity prior to the execution.
- Constraints Verification (CV) also accepts a global waivers file, that can be supplied as an input through the WaiverFile activity parameter.

Table 6-23 Constraints Verification Activity Parameters

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Setup	
Verify All (VerifyAll)	When VerifyAll is selected, other parameters are disabled. VerifyAll runs all the constraint verification activities.
Verify False Paths (VerifyFalsePath)	Performs False Path verification
Verify Generated Clocks (VerifyGeneratedClocks)	Performs Generated Clocks verification
Verify Multi Cycle Path (VerifyMultiCyclePath)	Performs Multicycle Path verification
Global Waiver File (WaiverFile)	Specifies the name of the global waiver tcl file. The activity parameter is disabled in coreConsultant as the waiver file is provided by the IP provider.
Execution Options	
Generate Scripts Only (ScriptsOnly)	Setting this causes the run_cv.scr script to be generated in the work directory, but the script does not get executed. Users can then run the script at a later time. The path to the run_cv.scr script can be obtained by using the following command: "get_logical_dir syn/run_cv.scr" Default: 0
Run Style (RunStyle)	If "Generate scripts only" is unchecked, then this selects how to run the command, which can be run locally, through LSF, through GRD.

Table 6-23 Constraints Verification Activity Parameters (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Run Options (RunOptions)	Additional options can be provided for a given run style. For LSF and GRD, this specifies options to the bsub and qsub commands, respectively (for example '-q normal'). For qsub, you can specify a project or other options (for example '-P normal -l arch=glinux'). The -V option is included automatically for qsub.
Parallel Job CPU Limit (ParallelJobCpuLimit)	Specifies the maximum number of CPUs to be allowed in TCM. It is set on the TCM variable max_parallel_processes which determines the maximum number of processes that are allowed to run the verification. If you select a value greater than 1, TCM will split up the exceptions to be verified into separate groups, depending on the value of max_parallel_processes and is verified on a separate processor.
Send e-mail (EmailAddress)	If selected, this option sends an e-mail message to the specified user when synthesis completes or is ended. The default for the text box labeled "To" is your (the current user's) user name.

6.10.3 Running the TCM Flow

For the Synopsys components that are TCM enabled, imported, and SPIRIT components, the following are the steps to be implemented to run the TCM flow:

1. The TCM is to be loaded by setting the tool root path as follows:

```
set_tool_root tcm -root <PATH to TCM tool root or FISHTAIL_HOME>
```
2. Perform and complete the Specify Subsystem activity.
3. Set propagate constraints to '1' in the initialize subsystem constraints.
4. Perform the verification by selecting any of the goals listed in [Table 6-23](#) on page 212 in the "Constraints Verification (CV) Activity".
5. View the results (run_cv.html) under the reports tab post completion of the "Constraints Verification (CV) Activity".
6. A tcm_assertions.lst list file is generated if there are unverified exceptions. The following is the path for the list file:

```
<workspace>/constraintsverification/tcm_assertions.lst
```
7. View the results (run_cv.html) under the reports tab post completion of the "Constraints Verification (CV) Activity".

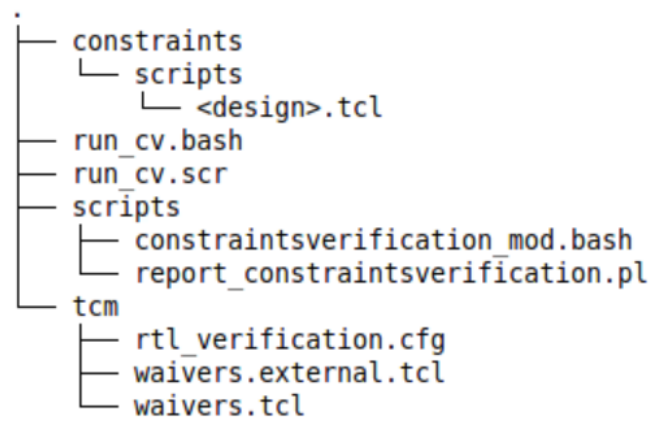


Note

The prerequisite for running the Timing Constraint Manager is that the minimum version for TCM must be U-2022.12-SP5.

6.10.4 Constraints Verification Directory Tree Structure

[Figure 6-10](#) shows the list of the main files generated by coreConsultant for the TCM flow. Additionally, the TCM generates its own set of files in the `<workspace>/tcm/tcm_rtl_verification` folder.

Figure 6-10 Constraints Verification Directory Tree Structure

6.11 Create Testbench Workspace Activity (Subsystem Workspace)

To create a testbench and simulate your subsystem, you must first complete all of the activities in the Create RTL activity group in order to use your DUT in the testbench. If there are still activities in the Create RTL activity group that have not been completed, coreAssembler asks if you want to “auto-complete” those tasks before moving on to the Create Testbench Workspace activity. If you click Yes, coreAssembler completes these activities with the default settings.

The Create Testbench dialog includes the following options:

- Testbench Workspace Name - name used for the “top” level of your testbench
- Testbench Workspace Directory - the directory in which to create your testbench. The default is to use *current_dir/tb_workspace_name* for the new directory
- Testbench Strategy and Language - allows you to choose between two approaches to simulation:
 - UVM RAL Subsystem (default) / UVM Subsystem - when selected, the testbench generated will follow the UVM methodology and will perform register testing using UVM RAL. For more information, see [Test Coverage \(UVM RAL Method\)](#) on page 216.
 - UVM RAL Subsystem Ping Test - when selected, the testbench generated follows the UVM methodology and performs register testing using UVM RAL. The intent of this activity is to ensure the connectivity and integrity of the subsystem.
- Automatically Instantiate DesignWare VIP - connects available VIP to corresponding interfaces available on your DUT.
- Close DUT workspace - you can choose to close the subsystem workspace or leave it open. If you leave it open, a second window opens the Testbench workspace.

**Note**

Verification IP can only connect to interfaces exported at the top of your DUT.

Clicking Apply for the Create Testbench Workspace activity:

- Saves (and close, if you specify) the subsystem workspace
- Creates a Testbench Workspace of the specified name at the specified path
- Automatically instantiates VIP components and monitors (if you enabled these items)
- Creates an activity flow to allow you to create Testbench RTL and Simulate Subsystem Testbench

Once you have entered the testbench workspace, you are allowed to modify and customize the testbench to meet your needs. This testbench flow is the same as for the manual testbench creation process.

This is the same activity that you can enter manually by creating a new workspace with the “Testbench” workspace box checked.

**Note**

The **Edit > Tool Installation Roots...** menu item and dialog box is where you can set the path to your simulator for the Simulate Subsystem Testbench activity.

6.11.1 Testbench Strategy and Language

Create Testbench Workspace activity gives you the choice between creating a UVM RAL Subsystem (default)/UVM Subsystem or UVM RAL Subsystem Ping Test. This choice determines the types of VIP components automatically instantiated in the testbench.

- The top-level testbench file for the UVM testbench, `simulation_top.v`, is written out. The testbench workspace that is instantiated in `simulation_top.v` is `tb_workspace_name.v`. For UVM components, interfaces are instantiated instead of components; netlisting is identical.
- The initialization/configuration of the VIP is written out to the `simulation_top.v` file. The text comes from extensions in the IP-XACT packaging of the VIP components and generated by coreAssembler.
- The UVM SystemVerilog files are written into the workspace/sim directory. The 'sim' folder contains the following structure:
 - misc files - `run.scr` (and so on) to run simulation and generate reports
 - Makefile - Contains all make targets
 - tests - Contains all tests - UVM - `hw_reset`, `bit_bash`, `reg_access`
 - env - Contains environment related files
 - logs - Contains all the log files
 - src - `simulation_top.v`: Instantiates the testbench top (which includes Subsystem and VIP module instantiations through IPXACT)
 - reports - Contains `report.html` and other supporting HTML files

6.11.2 Test Coverage (UVM RAL Method)

The UVM RAL option performs RAL testing over all the IP components present in the subsystem.



Note

As this flow targets only the IP components, no AMBA Slave VIPs are connected in the UVM RAL subsystem flows.

The UVM RAL testbench creates a UVM RAL model of each peripheral in the subsystem (more correctly, every instance in the subsystem that contains one or more registers). The RAL-based testbench may run one or both of the following tests on each register field in the subsystem:

- hardware-reset: Verifies that a register field can be reset correctly into a determined state by applying a system reset, removing it and then reading the register field.
- bit-bash: Verifies that a known value can be written to and read back from a register field.
- reg-access: Performs a backdoor testing of a register. The register must have a `UVMRALBackdoorName` attribute created in `memory_map.tcl`. If an IP core does not have a backdoor attribute, then no testing is performed -- only simulation is run.

This attribute contains the 'path' below the IP core to the register, as in this example:

```
set_attribute $reg -attr RALBackdoorName -value U_reg.pause_reg
```

Here, `U_reg` is an internal module instance.

You can control which (if any) of these tests are run on each register field. It's not uncommon to suppress one or both, depending on the functionality of the register field. You may choose not to run a test if the R/W access attributes on a register field dictates so.

6.11.2.1 UVM RAL Makefile Options

By default, `make -all` is run in the Simulate Subsystem activity with the following options:

```

***** User Defined Target *****
    default : provide a RAL_SEQUENCE="<seq1> <seq2>" from
hw_reset_tests and bit_bash_tests
    all: runs all sequences in hw_reset_tests and bit_bash_tests
    all_hw_reset_tests: runs all sequences in hw_reset_tests only
    all_bit_bash_tests: runs all sequences in hw_reset_tests only
***** simv Targets *****
    run_vlog_vcs: runs simv of RAL_SEQUENCE="<seq1> <seq2>"
    run_vlog_vcs_all_hw_reset_tests: runs simv of all_hw_reset_tests
    run_vlog_vcs_all_bit_bash_tests: runs simv of all_bit_bash_tests
***** Miscellaneous *****
    dw_vip_setup_force: Forcefully installs vips
    clean: Clean object and temporary files, logs, waves
    deepclean: Removes vip installations and performs make clean
    help: This message

```

The UVM RAL Subsystem supports ping test format. In this format, only one R/W register is tested for reading and writing. For more, see [UVM RAL Subsystem Ping Test](#) on page 217.

6.11.2.2 UVMTestText Feature

The UVMTestText feature provides you the ability to add user specific steps in the UVM Test phases of RAL tests in the UVM RAL Subsystem flow. UVMTestText is a design attribute used with any IP Core that needs text insertion at various UVM phases. You configure, drive, and control core signals using this feature. For details and usage, see the UVMTestText man page.

Example using UVMTestText:

```

eval_in_component i_rap {
    set_design_attribute UVMTestText\[reset_phase-hw_reset\] {repeat
(10) @ (posedge %I%.i_rap_clkgen.clk);}
}
    OUTPUT: hw_reset_test.sv in reset_phase
    ....
    repeat (10) @ (posedge
simulation_top.tb_Subsystem.DUT.i_rap.i_rap_clkgen.clk);
    ...

```

6.11.3 UVM RAL Subsystem Ping Test

The UVM RAL-based ping test performs RAL-based testing on all the slave components IP present in the subsystem. The intent of this activity is ensure the connectivity and integrity of the subsystem.

The test iterates over the memory map of the slave and identifies and tests a single register according to the following criteria:

- If a read-writable register is found with no attributes set on it, then the bit-bash, hardware-reset, and reg-access tests are performed on it.
- If no read-writable register is found, then a read-only register with no attributes set on it is searched for and, if found, the hardware-reset and reg-access tests are performed on it.
- If no register meets the above criteria, the tool reports an error and the testbench workspace is not created.

For additional details, see [Test Coverage \(UVM RAL Method\)](#) on page 216.

6.11.4 Testbench Workspace Creation Methodology Overview

Before creating the testbench, you must first describe your subsystem DUT by completing the “Create RTL” grouping of activities. Then you save and close the current workspace (your DUT workspace). Then, to create a testbench for simulation, perform the following activities.

1. Open a new workspace:
 - Specify a workspace name
 - Choose Testbench for the workspace type
2. Instantiate the DUT (automatically performed if entering from a subsystem workspace)
3. You can do the following within the Create Testbench activity (testbench workspace):
 - Add VIP components, clock generators and bus monitors (can be automatically added using the **Create Testbench Workspace** activity from within the Subsystem workspace):
 - Specify connection if the VIP connections are ambiguous
 - Connect to interfaces located within the hierarchy of the DUT
 - Instantiate a verification master to the exported AHB_Master in this subsystem
 - Configure all the VIP components added to the subsystem (Configure Components tab)
 - You cannot configure components located within a DUT from the testbench workspace
4. Select **Complete Connections** activity:
 - Connect side-band signals (non-interface) needed for verification
 - Select pins anywhere in the subsystem to connect to monitor signals
5. To complete the assembled testbench, apply the **Generate Testbench HDL** activity. This activity must complete successfully before you can enter the **Simulate Subsystem Testbench** activity.

Once you have completed the Create Testbench activity group (Specify Testbench, Complete Connections, and Generate Testbench HDL), move to the Simulate Subsystem Testbench activity.
6. Specify the Simulate Subsystem Testbench options, then **Apply** to launch the simulation.

6.11.5 Create Testbench GUI Functionality

The graphical user interface for creating a testbench is quite similar to the synthesizable IP flow. The schematic view (Specify Testbench > Add Components) allows you to place and connect components, as well as peeking into the synthesizable IP (DUT) subsystem. The tree view lets you view the testbench components and interfaces in textual mode. The DUT subsystem is not modifiable within the testbench

workspace, as is indicated by all synthesizable IP elements being grayed out. You must open the subsystem workspace to edit the DUT subsystem, then save and update the DUT within the testbench workspace in order to make changes to the DUT Subsystem within the testbench. Both subsystem and testbench workspaces can be open at the same time, but the testbench workspace must always reload an updated and saved DUT workspace to see these subsystem changes.

Connections from the synthesizable IP in the DUT to a verification IP component in the testbench are shown similar to an exported interface and are not the same color as the synthesizable IP components so that the verification IP is clearly identified.

For example, an AHB monitor VIP might have AHB_Master and AHB_Slave interface monitors, and HCLK and HRESETn consumer monitors. The change connection dialog for the AHB monitor component shows the components that support the connections required.

The Configure Components tab is the same as in the synthesizable IP flow, but only the verification IP components are allowed to be configured.

The Manual Connections activity shows the entire hierarchy in the selector spreadsheets, allowing access to any level within the DUT.

Once top-level HDL has been written for the testbench, the Simulate Subsystem Testbench allows you to configure the tests and launch a subsystem simulation.

6.11.5.1 Testbench Menu

The Testbench menu is located in the menu bar above the Activity window. This menu helps you build your testbench using the automatic features of coreAssembler. It provides choices similar to “Automatically Instantiate DesignWare VIP” to **Create Testbench Workspace** activity in the Subsystem workspace. The items in this menu include:

Add UVM-based Application VIP

This Testbench menu choice instantiates UVM type Verification IP (system-verilog) that corresponds to the interfaces available on your DUT. A master VIP for each of the AXI/AHB interfaces is added. This VIP does not include APB peripheral ports, such as SIO.

The TB_clkgen module is also added to drive the clocking of your testbench. Unused clock ports on the TB_clkgen module are handled, so that you do not need to set them to “unused” in the testbench workspace before generating the testbench.

6.11.5.2 Specify Testbench Activity

In this activity (similar to “Specify Subsystem” in the Subsystem workspace), you define your testbench and the verification IP and monitors you want to use. This includes instantiation of VIP components, connecting, and configuring interfaces.

**Note**

Similar to the subsystem workspace, the testbench workspace schematic and tree views show the interface-to-interface connections and not the pin-to-pin connections.

The pop-up menu items used to complete this activity are:

- **Add New Component**
- **Add DUT** -- The DUT can be either from a workspace directory or a IP-XACT subsystem definition. Note that **Duplicate Component** is not available for adding multiple instances of a DUT to test. You are also not allowed to rename the DUT, or **Edit Interface Parameters** for the DUT.
- **Update DUT** -- This allows you to update the DUT to the latest version. This is useful when you have made changes to the top-level of your DUT, and want to use those change in the testbench. Update DUT first checks to see if you have the latest version, and if you do, lets you know by a dialog box.

6.11.5.3 Configure Interfaces Testbench Tab

The Configure Interfaces dialog view allows you to configure interfaces between VIP components and the DUT. A spreadsheet gives you the existing interface values, where you can choose from other valid values for the interface parameter.



Note

Internal DUT interface configuration is not accessible with the Testbench Workspace. In order to change interface configuration within the DUT, you must return to the Implementation Workspace, use the Configure Interfaces tab within the Specify Subsystem

6.11.5.4 Configure Components Testbench Tab

The Configure Components tab is VIP component-specific and provides one hierarchical entry per component in a tree dialog. Some VIP configuration parameters are set automatically based on values of interface parameters. These parameters appear disabled in the component configuration dialogs. For details of specific VIP configuration requirements, see the VIP databook or switch to the Help tab to see a table of parameter descriptions.

The TB_clkgen module is configurable, and allows you to control the PCLK, HCLK, ACLK, and PCIe clock cycle times and the reset durations.

The DUT is not configurable in the testbench workspace. To change configuration of the DUT, you must return to the DUT workspace, make the changes, and “Update DUT” into the testbench workspace.

6.11.5.5 Complete Connections Activity

See the DUT specification and the VIP databooks for the functionality of each of the pins requiring manual connection to determine how to connect individual pins. The simplest approach to completing this process is to export all of the unconnected pins and then connect them when the subsystem is instantiated.

The Manual Disconnect tab can be used to undo connections. This is not typically needed, but can be used to undo a mistake or to make other manual connection updates.

Clicking the Apply button automatically makes any default connections and then, provided all pins are connected, and shows the “Report” tab where you can generate and shows the connections report.

6.11.5.6 Generate Testbench HDL Activity

This section describes the procedure and outputs of the Generate Testbench HDL activity for the Create Testbench activity group.



Note

This activity creates the top-level testbench. The HDL for each of the individual VIP components is generated at the completion of the Specify Testbench activity.

To create the top-level VHDL or Verilog code for the subsystem, click the Generate Testbench HDL task in the Activity List. In the dialog that appears in the Activity View pane you choose the output language, VHDL or Verilog. If you choose VHDL as the output language, you can specify `std_logic` or `std_ulogic` as the preferred logic type for any Verilog components in the subsystem.

The “Specify VHDL Libraries” spreadsheet lets you specify a VHDL library for each hierarchical design in your subsystem. The default for each design is to use the library “work.” In batch mode, a VHDL library can be associated with a level of design hierarchy using the `VHDLDesignLibrary` attribute. Access is not available for Verilog output.

You can also specify a user comment to be inserted into the generated RTL. Enter any text into this field, and it is added as a comment to the generated RTL.

When you click the Apply button in this dialog, coreAssembler writes the top-level HDL code to the file `<tb_workspace>/src/<design>.v[hd]` where `<tb_workspace>` is the current testbench workspace name and `<design>` is the testbench top-level design name. The Activity View pane switches to the Report tab that contains allows you to generate and view the Subsystem Source Code report listing the files that coreAssembler has generated. You can also generate IP-XACT, System-level Parameters, and Example Instantiation views within the “Reports” tab.

6.12 Simulate Subsystem Testbench

The Simulate Subsystem Testbench dialog includes three tabs, each with the following options:

6.12.1 Simulator Setup

In the Simulator Setup tab, you can specify the following options:

- Simulator - choose the simulator you want to run the simulation. Note: for UVM testbenches, only the VCS simulator is available.
- Generate waves file - outputs a Value Change Dump (VCD) file
- UVM RAL Coverage - Generates the coverage model for each of the tests
- Enable UVM Test List - Provides an option to control the UVM test execution



Note The **Edit > Tool Installation Roots** menu item and dialog box is where you can also set the path to your simulator. The initial value of these paths comes from the simulator environment variables.

Table 6-24 lists the parameters for the Enable UVM Test List section of the Simulator Setup tab.

Table 6-24 Enable UVM Test List Parameters

Activity Parameters	Description
RunRegAccessTests	Run UVM Register Access tests for all components in the testbench workspace. Default: True
RunHWResetTests	Run UVM Hardware Reset tests for all components in the testbench workspace. Default: True
RunBitBashTests	Run UVM Bit Bash tests for all components in the testbench workspace. Default: True

6.12.2 Testbench Definition

The Testbench Definition tab lets you specify how components in your subsystem are tested. The information that appears in this dialog depends on how many components (AHB masters and slaves) you have in the design.

The Test Master column shows which VIP master is used to test the slave referenced in that row of the spreadsheet. This master defaults to be the farthest away from the slave. However, by using the Test Master drop-down list for a particular slave, then you can change the Test Master to any AHB or AXI VIP component which can communicate with the slave. If you do not want a particular slave to be tested, select Do Not Test from the drop-down list in the Test Master column.

6.12.3 Execution Options

In the Execution Options tab, you specify the following options:

- Generate Scripts only - generates simulation scripts but does not launch the run.
- Run Style - local, lsf, or grd

- Run Style Options - specify a style if a non-local run style is used
- Send e-mail - check box if you want e-mail sent
- To - where you want the e-mail sent
- Clean up files when repeating simulation - If selected (default), this option deletes existing simulation files before redoing the Simulate Subsystem Testbench activity.

6.12.4 Generating the Simulation run.scr Script

Clicking Apply generates a simulation sim/run.scr script which launches the specified simulation (default). If launched, the simulation runs in the background and the report tab is displayed, showing you status as the simulation progresses. This page updates automatically, or you can click the “refresh” button to update immediately. When the simulation has completed, you can view the various simulation reports and files.



Note

These reports and some of the optional view files must be created and updated manually. Use the **File > Generate Reports** and **File > Generate Optional Views** menu items to generate reports and optional views.

6.13 Integrating the Subsystem into Your Design

After completing the coreAssembler design flow, you have a customized implementation of a subsystem that has been optimized to meet your design objectives. You must now integrate the subsystem into your application design and verify the subsystem in the context of your application, according to your verification strategy.

6.13.1 Connecting to Surrounding Logic

For descriptions of each core's functionality and the behavior and timing of its ports, see the core-specific documentation provided by core developers.

Connect the subsystem's I/O ports to your application design according to your chip-level specifications. You can use the sample instantiation at `export/<design>_inst.v[hd]` as a starting point.

6.13.2 Verifying Functionality

In most design flows, you have already verified the functionality of the RTL version of each core. To verify the functionality of the optimized gate-level version, use any or all of the following methods:

- Perform formal verification (for example, using Synopsys Formality) to verify the gate-level implementation against the already-verified RTL version. Formal verification checks that the two versions of the design are functionally identical.
- Simulate the gate-level design in the verification environment supplied by the core developer.

6.13.3 Verifying Timing

For chip-level timing analysis, you can choose to generate a timing model for the subsystem as a by-product of each coreAssembler synthesis strategy. To do so, go to the Modeling tab of the Strategy Parameters dialog by clicking the Options button next to whichever synthesis strategy you have selected from the Synthesize Activity View. Click Generate Timing Model and specify values for the other parameters on the tab. You can use the resulting timing model for timing analysis with Synopsys PrimeTime or some other chip-level timing analysis tool.

6.14 Batch Commands

When a workspace is created with the `-testbench` option, the following commands have modified behavior:

<code>create_workspace</code>	The <code>-testbench</code> option has been added to this command.
<code>instantiate_dut</code>	This command instantiates a coreAssembler subsystem (workspace) or a IP-XACT component into a testbench workspace as the DUT.
<code>update_dut</code>	This command brings in the latest changes from the DUT workspace. Any changes to the exported interfaces of the DUT that modify the top-level HDL require that the Specify Subsystem activity again be completed.
<code>connect_interface</code>	The <code>-from_component</code> and <code>-to_component</code> arguments accept a relative path to component interface to connect to (for example <code>"dut/ahb_sys/i_ahb"</code>). One of the components specified must be in the testbench level of hierarchy.
<code>create_connection</code>	This command accepts a full path to a pin name. The check for a source is disabled if one of the connectors is within the DUT. One of the connectors specified must be in the testbench level of hierarchy.
<code>export_pin</code>	This command is disabled in the testbench-level of hierarchy. The activity flow mirrors the current IIP assembly flow. See the use cases for an example of that flow.

6.15 Licensing and Keys

If you are using only Synopsys Library components within your testbench, only a DesignWare key is needed to create the testbench; if non-Synopsys Library packaged components or IP-XACT components are used the coreAssembler license key is required. The license is checked out when the testbench workspace is created or opened, and released when it is closed.

Additionally, the simulation process may also pull a DesignWare key if your testbench contains Synopsys IP components. To prevent the use of two DesignWare keys simultaneously, you can:

1. Choose “Generate scripts only” in the Execution Options for Simulate Subsystem Testbench
2. Exit coreAssembler
3. Run the “run.scr script in the *tb_workspace/sim/run.scr*

The script invokes the simulation, and create a “run.log” file when complete.

7

Exporting a Subsystem for Reuse

This chapter contains the following sections:

- [“Export Subsystem”](#) on page 228

7.1 Export Subsystem

The following sections describe methods you can use to export your design subsystem.

7.1.1 Create Subsystem Template

A subsystem template is a fixed starting point for a new subsystem. A template consists of one or more partially configured components. The components and configuration values in a template can be “locked” by the creator of the template, meaning that they cannot be changed when the template is used by someone else. The template is intended to be used as a starting point in defining a specific implementation of a general subsystem type. The template is stored in <creator_name>.template and can be handed off to another person for use in defining a subsystem.

If you want to add plugins to this template, see the *coreBuilder User Guide* for details of the `add_verify_subsystem_hook` command.

To create a subsystem template:

1. Complete the necessary configuration and verification steps to define the general template that you want to make available for specific implementations of a subsystem type.
2. Select the **Export Subsystem > Create Subsystem Template** activity from the coreAssembler Activity List.
3. Select the Components tab and specify which of the template components are to be locked to further changes.
4. Select the Exported Interfaces tab and specify which of the template exported interfaces are to be locked to further changes.
5. Select the Interface Parameters tab and specify which of the template interface parameters are to be locked to further changes.
6. Select the Configuration Parameters tab and specify which of the template configuration parameters are to be locked to further changes.
7. Select the Create Subsystem Template tab and specify the template name and top design name for the template.
8. Click the Apply button. The template is saved as a single file named <Template_Name>.template in the ./export directory of the current workspace.

7.1.2 Export IP-XACT Compliant XML

To export (write) an IP-XACT compliant design subsystem, see “[Exporting a IP-XACT Design \(coreAssembler only\)](#)” on page 285.

**Note**

IP-XACT design file “views” are not automatically created. Use the File > Generate Optional Views menu item to globally generate these view.

7.1.3 Build Subsystem coreKit

If you want to deliver your preconfigured subsystem as a component that can be instantiated in coreAssembler or reused in coreConsultant, you need to either package the subsystem in coreKit format using the Build Subsystem coreKit activity, or export the subsystem in the IP-XACT compliant design format.

7.1.3.1 Build Subsystem coreKit Activity

To complete the Build Subsystem coreKit activity:

1. Select the **Export Subsystem > Build Subsystem coreKit** activity from the coreAssembler Activity List.
2. In the Basic tab of the Build Subsystem coreKit dialog which appears in the Activity View pane, enter data for basic options.
3. In the Implementation Tools tab of the Build Subsystem coreKit dialog, enter data for the implementation options.
4. Click Apply to invoke coreBuilder and package the subsystem coreKit.

coreBuilder builds the coreKit (*name.coreKit*) and places it in the install directory of your coreAssembler workspace. This component will be seen as a black box when instantiated (that is, it is not configurable).

8

Unpackaged Component Import

The coreAssembler tool allows you to quickly create subsystems from components packaged using the coreBuilder tool. In certain cases, some of your subsystem components might not benefit from the value added by packaging in the coreBuilder tool. You can use these simpler, less flexible components in your coreAssembler subsystem. This chapter guides you through the process of adding such unpackaged components to a coreAssembler subsystem.

This chapter contains the following sections:

- [“Locating a Design for Import”](#) on page 232
- [“Interfacing your Imported Component to Other Subsystem Components”](#) on page 234
- [“Recommendations and Limitations”](#) on page 236

8.1 Locating a Design for Import

The first step in importing an unpackaged core is specifying the location of all associated data (source code, netlists, and intent scripts).

An imported core can be a mixture of hard, soft and firm IP. However, the top-level description of any imported core must be in either HDL or IP-XACT compliant XML form.

8.1.1 Importing the Component Data

The `import_component` command is used to specify the source of all data associated with the core you want to import. You can select this option during the Specify Subsystem activity (Add Component tab) by choosing the <Import Component> item in the component list, or use the command directly as follows.

```
import_component
  -name <name>
  -design <name>
  -language <language>
  -files <fileList>
  [-library <libName>]
  [-include <dirList>]
  [-intent <dirList>]
  [-linklibs <dbList>]
```

For details of all the switches associated with this command, see the `import_component` man page. Remember to specify the file list in analysis order (with the top-level file last in the list).

8.1.2 Importing Soft IP

Soft IP (that is, IP completely described in RTL) can be imported using the `import_component` command as shown in the following example:

```
import_component -name i_sampledesign \
  -design sampledesign -language verilog \
  -files { sampledesign.v alu.v controller.v } \
  -intent {sdc}
```

In this example, the core 'sampledesign' is described in three files; `sampledesign.v`, `alu.v` and `controller.v`, and associated constraints (in SDC format in this case) are found in the 'sdc' directory.

8.1.3 Importing Hard IP

In certain circumstances you can import a hard (or firm) core, for which there is no RTL available. This can be achieved by writing an HDL 'wrapper' for the core to be imported. Then, when executing the `import_component` command, the '-files' option simply specifies the top level HDL wrapper, and the '-linklibs' argument specifies the compiled netlist.

The following example demonstrates how you could import a pre-compiled core called 'my_design', by instantiating it in a wrapper called 'my_wrapped_design'.

```
module my_wrapped_design
```

```

        // outputs
        A_top;
        // Inputs
        X_top);
output A_top;
input X_top;
my_design i_my_design
    (//(I/O association list for 'wrapped' design)
     .A (A_top),
     .X (X_top));
endmodule

```

If the design 'my_design' is available in DB format, then to import this pre-compiled core simply use the `import_component` command as follows;

```

import_component -name i_my_wrapped_design \
                -design my_wrapped_design \
                -language verilog \
                -files src/my_wrapped_design.v \
                -linklibs db/my_design.db

```

8.1.4 Importing IP-XACT Components and Designs

For information on importing IP-XACT components and designs into coreAssembler, see [“Importing an IP-XACT Component \(coreAssembler\)”](#) on page 284 and [“Importing a IP-XACT Design \(coreAssembler\)”](#) on page 285.

8.1.5 Importing Synthesis Intent

If synthesis intent is available for a given core (in coreTools format), you can associate it with an imported core in the same way that you associate intent with a core during packaging in coreBuilder. Simply specify the search path containing one or more directories where intent can be found. Any intent file of the form <design_file_name>.tcl is imported with the core.

In addition, any SDC constraint file of the format <design_file_name>.sdc is imported with the core.

SDC files generated by Design Compiler are always bit-blasted, and can be quite large. The following command compresses these SDC files so that common bit-level constraints are specified at a bus-level instead:

```
compress_sdc in_filename out_filename
```

8.2 Interfacing your Imported Component to Other Subsystem Components

Once you have imported an unpackaged core, you can attach it (where appropriate) to any other subsystem component through one or more available interfaces.

8.2.1 Attaching an Available Interface

In GUI mode, you can select the imported core and choose the 'Attach Interface' button in the Add Component tab of the Specify Subsystem activity. This launches a dialog entitled 'Attach Interfaces to a Component'. Clicking the 'Attach' button presents a list of available interfaces in the subsystem.

You can then simply choose an appropriate interface to add to your imported core. This effectively allows you to connect your imported core to another subsystem component or to an already exported interface. For example, you can connect your imported core to a bus, or to a subsystem clock or reset signal.

To attach an interface to an imported component in batch mode, use the 'attach_interface' command -

```
attach_interface -to <componentName>
               -name <newName> \
               [-component <componentName>] \
               -interface <instanceName> [-monitor]
```

8.2.2 Port/Parameter Association

To complete the process, you need to associate each interface port with the appropriate port on the imported core to which you have attached an interface. To do this, select the interface whose ports/parameters you want to associate, and then choose 'Port + Parameter Setup' button in the attach interfaces dialog. In batch mode, use the 'set_interface_port_attribute' command as follows:

```
set instance [attach_interface -to <comp. inst. name> \
                        -name <intf. inst. name> \
                        -interface <intf. name>]
set_interface_port_attribute $instance \
                        <intf. port name> InterfaceLink \
                        <imported comp. port name>
```

So, for example, if you have an imported component instance 'i_risc' you could connect it to an available reset interface port as follows -

```
# attach the APB_Reset interface to the imported
# i_risc instance
set instance [attach_interface -to i_risc \
                        -name APB_Reset_risc \
                        -interface APB_Reset]
# 'connect' the i_risc/rst_n port to the APB_Reset/present
# port
set_interface_port_attribute $instance \
                        present InterfaceLink rst_n
```

In this example, the APB_Reset interface was exported to the top-level of the subsystem from an available (packaged) component with this interface. However, you can also connect any imported component port internally within the subsystem.

8.2.3 Loading of Undefined Interfaces

The following command enables loading files containing interface definitions directly into coreAssembler:

```
load_interface_definitions <list_of_interface_definition_files>
```

Files ending with .tcl are assumed to contain TCL interface definitions. Files ending with .xml are assumed to be IP-XACT bus definitions or abstraction definitions.

The Load Interface Definitions menu item within the Add Components schematic window brings up a dialog in which TCL and XML files can be specified. When you OK the dialog box, the files specified are loaded by a call to load_interface_definitions.

If there are any errors loading a given interface definition file, then all interfaces loaded from that file are removed.



Caution

If a file is loaded which attempts to redefine existing interface definitions, the loading of that file fails.

8.2.4 Adding Attached Interfaces

Follow these steps to add an attached interface:

- Select a component, and select the **Attach Interfaces** menu item to view the “Attach Interface Instance to a Component” dialog.
- Select the component and interface to be attached (from drop-down controls), and specify the name for the attached interface instance.
- Click “OK” to close the “Attach Interface Instance to a Component” dialog and cause the interface instance to be created. After the interface is created, the “Associate and Setup Interface Ports and Parameters” dialog appears.
- Enter values for the InterfaceLink attribute for each interface port and parameter in the interface instance.
- Click “OK” to close the dialog and complete the process.



Note

The **Attach Interfaces** menu item is no longer available to update or debug existing interface instances (by selecting the menu, an interface, and then hitting the Port and Parameter Setup button). In order to enable this functionality, the **Port Associations** menu item is enabled on non-exported interfaces and takes you directly to the “Port Association” dialog for the selected interface.

8.3 Recommendations and Limitations

Although this capability gives you more flexibility when creating subsystems, note that an imported component differs in a number of ways from a component packaged using coreBuilder.

In some cases, simply importing a component directly can offer sufficient flexibility. In other cases, it is preferable to package the core using coreBuilder. This section outlines some typical scenarios where you are be faced with this choice.

8.3.1 One-time Use Model

The ability to add an unpackaged core to a coreAssembler subsystem is principally provided for IP cores which are typically not designed for reuse. As such, there is no ability to associate elements such as a verification environment, documentation. Configurability is limited. This can only be achieved by first packaging the core with coreBuilder.

It is possible to have more than one instance of an imported component, provided the additional instances are created using the `duplicate_component` command within the same level of hierarchy (and not with multiple import calls), or imported into a different level of hierarchy.

8.3.2 Limited Configuration Capability

An imported component is only user-configurable from its top-level parameters (Verilog) or generics (VHDL). Other forms of design parameter are not directly user-configurable.

If you want to make the configuration interface for a given imported component more usable, it is recommended that you add some configuration setup commands to an intent file for that component as follows:

For a Verilog parameter defined as

```
parameter ADDR_SIZE = 4;
```

the configuration parameter label is normally 'ADDR_SIZE'. To create a different label, and thus increase usability, you can add the following to the intent script:

```
set_parameter_attribute ADDR_SIZE Label "Address Size"
```

To automatically import this intent script, simply define the path to the directory where it is located when importing an unpackaged component. As with intent scripts and packaged components, as long as the base name of the intent script matches the base name of one of the source files, it is automatically imported.

If you want to have a complex configuration setup for your imported component, we recommend that you first package it using coreBuilder.

8.3.3 Connecting an Imported Core

An imported core can only be automatically connected to another subsystem component or external port through an interface which already exists in the subsystem (either on an existing component or exported from an existing component). This means that two or more imported cores cannot be connected together by a unique interface.

If an imported core requires unique subsystem connections, you need to use the manual connection capability. Alternatively, using coreBuilder to package a core with built-in interface definitions gives you the advantage of connectivity automation/flexibility and configuration consistency.

9

Advanced User Modes

The preceding chapters explained how to implement a reusable core or a subsystem using the coreTools GUI mode. This chapter covers advanced user modes and customization techniques, most of which require you to write batch files or use the command prompt. These modes lack a graphical interface, but they compensate by facilitating rapid, powerful changes to your subsystem's configuration and synthesis specifications. Typically, users take increasing advantage of these modes as they become experienced with the tool in general, and its various forms of command syntax, in particular.

This chapter contains the following sections:

- [“Command Line Interface”](#) on page 238
- [“Batch Mode”](#) on page 241
- [“Editing Tool Setup and User Preferences”](#) on page 244
- [“Working with Verification IP”](#) on page 259
- [“Working with Hard Macros”](#) on page 260
- [“Dynamically Adding Files to the HDL File List”](#) on page 261
- [“Signal Naming Control in Generated Netlist”](#) on page 262
- [“Replace Component Mapping File Format”](#) on page 263
- [“Dumping Debug Information”](#) on page 265
- [“Documentation Generation”](#) on page 266
- [“Table Driven Data Import/Export”](#) on page 267

9.1 Command Line Interface

The coreTools applications provide a command-line interface so that you can enter commands directly from the command line, or write the commands to a file for batch-mode execution.

9.1.1 Accessing the Command Line Interface

You can invoke the command line interface in two ways:

- Enter commands at the tool prompt in the coreTool console.
- Invoke the coreTool in shell mode by entering the following command at the UNIX prompt:

```
$SYNOPSIS/bin/<coreToolName> -shell
```

where <coreToolName> is either coreConsultant or coreAssembler.

The following sections show how to use the command line interface to perform a variety of activity-related tasks. See the online command reference for complete descriptions of all of the available commands.



Note

You can switch between modes at any time using the `gui_start` and `gui_stop` commands.

9.1.2 Setting a Design Configuration Parameter

To set a new value on a top-level design configuration parameter, use the following format of the `set_configuration_parameter` command:

```
set_configuration_parameter -component component param value
```

For example, to set the value of the parameter `extd_intr` to 1 or on the instance CPU, enter the following command:

```
set_configuration_parameter -component CPU extd_intr 1
```

Note that when you use `set_activity_parameter` to change a parameter, the status of the associated activity is automatically changed to incomplete. When the activity is completed, the coreTool, writes out the configuration-dependent files for your new design configuration and enables the activities that depend on Specify Configuration.

To get the value of a top-level configuration parameter, use the `get_configuration_parameter` command:

```
get_configuration_parameter extd_intr
```

9.1.3 Setting an Activity Parameter

To change the value of a parameter on an activity, use the `set_activity_parameter` command:

```
set_activity_parameter activity_name parameter_name parameter_value
```

For example, to select `DC_opto_strategy` as the synthesis strategy:

```
set_activity_parameter Synthesize StrategyName DC_opto_strategy
```

Note that when you use `set_activity_parameter` to change a parameter, the status of the associated activity is automatically changed to incomplete.

9.1.4 Reporting Activity Parameters

When you select values for activity parameters through the GUI, the coreTool applies the information to a parameter associated with the activity. For example, when you select DC_opto_strategy as your synthesis strategy, coreAssembler or coreConsultant applies the value DC_opto_strategy to the StrategyName parameter on the Perform ASIC Synthesis activity.

To list the names and a brief description of the parameters associated with an activity, use the report_activity_parameters command:

```
report_activity_parameters ActivityName
```

For example:

```
<coreToolName>> report_activity_parameters GenerateSubsystemRTL
```

```
GenerateSubsystemRTL
```

```
LanguageChoice
```

```
HelpText          Name:          LanguageChoice
                  Default:      Verilog (<functionof>)
                  Values:      VHDL, Verilog, SystemVerilog,
```

```
Verilog-1995
```

```
                  Indicates the language to be used when generating the
top-level designs of the subsystem.
```

```
Value              Verilog
```

```
PreferredDataType
```

```
HelpText          Name:          PreferredDataType
                  Default:      std_logic
                  Values:      std_logic, std_ulogic
                  Enabled:      LanguageChoice == "VHDL"
```

9.1.5 Setting the Value of a Synthesis Strategy Parameter

To set the value of a synthesis strategy parameter, use the set_strategy_parameter command:

```
set_strategy_parameter strategy_name parameter_name parameter_value
```

For example, to set the DCTCL_opto_strategy QorEffort parameter to high:

```
<coreToolName>> set_strategy_parameter DCTCL_opto_strategy QorEffort
high
```

When you use set_strategy_parameter to set the value of a synthesis strategy parameter, the coreTool automatically sets the status of the Perform ASIC Synthesis activity to incomplete.

To get the current value of a synthesis strategy parameter, use the get_strategy_parameter command:

```
get_strategy_parameter strategy_name parameter_name
```

9.1.6 Abbreviating Commands

You can abbreviate commands and options to the shortest unambiguous string. For example, you can abbreviate the set_port_attribute command to "set_por". You can also abbreviate options. For example you can abbreviate "all_outputs -no_primary" to "all_o -no_p".



Caution You cannot abbreviate built-in Tcl commands.

The command abbreviation is included as a convenience for users of the command line mode. *Do not use command abbreviation in script files.* Remember that existing command abbreviations can become ambiguous if and when future versions of the coreTools add to or change the list of commands.

If you try to enter a command abbreviation that is not unique, coreAssembler lists as many as three commands that match the ambiguous abbreviation:

```
coreAssembler> cur
Error: ambiguous command 'cur' matched 3 commands:
  (current_design, current_instance, current_kb) (CD-006)
coreAssembler>
```

For a list of all commands that match the ambiguous abbreviation, use Help with wildcard pattern matching:

```
coreAssembler> help cur*
current_design      # Set/get the current design
current_instance    # Set/get the instance
current_kb          # Make the named knowledge base the current one.
coreAssembler >
```

9.1.7 Re-Executing the coreTool Design Flow

When you modify one or more activity parameters, you need to re-execute the coreAssembler design flow to implement those changes. The command below automatically completes various sets of activities, depending on the options specified:

```
autocomplete_activity <activity name>
```

For example, if you change specifications for the Perform ASIC Synthesis activity, use `autocomplete_activity` to re-execute Synthesize automatically, this time using the new specifications:

```
coreAssembler > autocomplete_activity Synthesize
```

As previously indicated, using the `set_activity_parameter` to change the value of an activity automatically sets the state of that activity to not completed. However, changes that you make to synthesis intent by executing `set_design_attribute` and `set_port_attribute` do not cause any changes to the current state of the coreAssembler activities.

9.2 Batch Mode

You can execute coreAssembler commands in batch mode by writing the required sequence of commands to a file using standard Tcl syntax. This is useful for:

- Regression testing
- Experimentation with different design alternatives

9.2.1 Batch Command File Examples

A common use for batch mode in the coreAssembler is to change values for one or more parameters for an activity, then re-execute that and all subsequent activities. This scenario assumes that you have already used the GUI to complete the design flow at least once, but this is not a strict requirement.

[Example 9-1](#) changes a configuration parameter on a core, then re-executes configuration and all subsequent steps in the design flow, using the currently stored specifications for the subsequent activities.

[Example 9-2](#) changes a set of parameters for an activity, then re-executes that and all subsequent activities in the design flow, using the currently stored specifications.

Example 9-1 Batch File – Parameter Configuration Change

```
#Change parameter value
set_interface_parameter CPU/Master MasterSlot 3
#Re-execute configuration and subsequent activities
autocomplete_activity Synthesize
#Quit coreAssembler
quit
```

Example 9-2 Batch File – Activity Specification Change

```
#Change activity specifications
set_activity_parameter Synthesize StrategyName DC_opto_strategy
#Re-execute this activity and all subsequent activities
autocomplete_activity Synthesize
#Quit coreAssembler
quit
```

For details about the commands used in these examples, see the command reference for coreTools.

9.2.2 Automatically Generating a Batch File

The coreTools provide a command to automatically write a batch script file that contains the commands required to re-create the current state of your workspace. You can then use the resulting batch script in either of the following ways:

- As an example of how to write a batch script
- To create another identical workspace in batch mode
- To modify the batch file and create a new workspace with different options

To automatically generate a batch script in command line mode use the `write_batch_script` command. For example:

```
coreAssembler> write_batch_script my_script.tcl
```

See the online command reference for the complete of `write_batch_script` options.

To automatically generate a batch script in GUI mode select **File > Write Batch Script** in the coreTool menu bar. The GUI method only supports writing the entire subsystem.

9.2.3 Batch File Options

The script generator has all interfaces connected explicitly, and not rely on automatic interface connection. In addition, the following list describes the `write_batch_script` options and conditions:

- When generating a batch script using the GUI, checking is first performed to prevent overwriting an existing batch file.
- If the workspace already exists, you can answer “Yes” to overwrite the existing workspace, or provide a “path” where to create the new workspace.
- Selecting the “Include Design Constraints” option writes constraints into the batch file. Otherwise, these constraints are not written.

9.2.4 Executing a Batch Command File

To invoke a coreTool and automatically execute a command file, specify your user workspace name and use the `-f` option to specify the coreTool startup command, for example:

```
<coreToolName> -shell workspace_name -f batch.tcl
```

The example previously shown performs the following sequence:

1. Starts coreAssembler in shell mode.
2. Loads your user workspace (`workspace_name`).
3. Executes the batch mode command file `batch.tcl`.

You can also execute a batch command file from the command line interface by using the Tcl source command, for example:

```
coreAssembler> source batch.tcl
```

9.2.5 Batch Mode coreKit Installation

If the installation procedure specified by a core developer does not require a core-specific license, you can install that coreKit in batch mode by using the coreTool `batch_install` command:

```
batch_install coreKit_filename installationAreaDir
```

coreKit_filename is the name of the coreKit file (typically `<name>.coreKit`) provided by the core developer. *installationAreaDir* is the target directory into which you want to install the coreKit.

For example, to invoke coreAssembler in shell mode and automatically install a coreKit in batch mode, execute the following command:

```
coreAssembler -shell -x "batch_install coreKit_filename  
installationAreaDir; quit"
```

The previous example performs the following sequence:

1. Starts the specified coreTool in shell mode.
2. Installs the specified coreKit into installationAreaDir.
3. Exits the coreTool.

9.3 Editing Tool Setup and User Preferences

You can control certain tool aspects that are set automatically when the tool opens. This is done using:

- “Setup Variables”
- “User Preferences”

9.3.1 Setup Variables

To establish a custom environment, you can set tool variables and define tcl commands in .synopsys_rt.setup files. When launched, coreAssembler looks for three .synopsys_rt.setup files in the following order:

When launched, the specified coreTool looks for three .synopsys_rt.setup files in the following order:

1. *Site-wide* coreAssembler variable settings in:

`<install_dir>/admin/setup/.synopsys_rt.setup`

You can view the contents of this file to see the variables that are most commonly set.

2. *Personal* settings in:

`$HOME/.synopsys_rt.setup`

For a short sample of a personal setup file, see `<install_dir>/admin/setup/user.synopsys_rt.setup`.

3. *Project-specific* settings can be placed in a .synopsys_rt.setup that resides in the directory where you invoked the coreAssembler tool.

When you invoke coreAssembler, it reads and executes the setup files before executing any command line arguments.

The variable settings and tcl command definitions in the most recently read file take precedence over those in previously read files. In other words, the setup file in your \$HOME directory overrides the tool setup in `<install_dir>/admin/setup/.synopsys_rt.setup`, and a project-specific setup file overrides all others.

To define useful tcl commands in the setup file, use the ‘proc’ statement. Several examples are as follows:

```
proc fpa { args } { return [eval find_item -type param $args] }
proc fpi { args } { return [eval find_item -type pin $args] }
proc fn { args } { return [eval find_item -type net $args] }
proc fc { args } { return [eval find_item -type cell $args] }
```



Note

All coreTools applications use the same setup file.

Table 9-1 describes some of the variables that you can set in .synopsys_rt.setup files. For other variables you can set, see the `<install_dir>/admin/setup/.synopsys_rt.setup` file.

Table 9-1 Setup Variables

Variable	Description	Default
component_search_path	List of directories which contain installed coreKits, used to determine the initial list of available components for coreAssembler	[getenv\ DESIGNWARE_\ HOME]/iip/latest
default_browser_command	Specifies which browser for coreTools to use when accessing reports. Note: PDF files open from either a direct \$PATH to acroread, or as a plug-in to your default browser.	firefox %s mozilla %s netscape %s
search_path	The list of directories to be searched when the coreTool looks for workspace-specific files.	"\$synopsys_root/ \ libraries/dware/rt."
scratch_dir	The directory used as a staging area during installation of a coreKit and for the storage of temporary files.	/tmp
collection_print_item_limit	The maximum number of item names to print as part of a command result.	10

9.3.2 User Preferences

The coreAssembler tool reads a set of tool interface preferences (if they exist) when it opens in either GUI or command-line mode. When set, the preferences reside in the following file:

\$HOME/.synopsys_rt_prefs.tcl

All preferences are user-specific and are used (as appropriate) for each coreTools application you run.

To set your preferences, choose **Edit > Preferences** from the toolbar menu. Note that the settings are saved when coreAssembler is closed.



Note

All coreTools applications use the same preferences file.

Table 9-2 describes the available preferences.

Table 9-2 GUI Preferences

Preference	Description
Appearance > Global > Fonts	
GUI: Default font Name: app_default_font_family Default: Bitstream Vera Serif, 10	Specifies display font family and size used for most application text at startup. This font must be proportional.
GUI: Console font Name: app_fixed_width_font_family Default: Courier, 10	Font family and size used for console text

Table 9-2 GUI Preferences (continued)

Preference	Description
Appearance > Global > Spreadsheet	
GUI: Show spreadsheet Boolean values as a check box Name: show_spreadsheet_bool_as_checkboxes Default: 1 (true)	Show spreadsheet Boolean values as a check box when true; otherwise, display as a combination box
GUI: Show all spreadsheet columns Name: show_all_grid_columns Default: 0 (false)	Show all attributes in spreadsheets, not just the most common set.
Appearance > External Browser	
GUI: External browser launch command Name: default_browser_command Default: firefox %s	The default browser invocation command string replaces %s with the URL.
GUI: External PDF viewer: Name: default_pdf_viewer Default: acroread	The default viewer for PDF files. The PDF file name is appended to whatever string is specified for the viewer.
Appearance > Windows	
Dialog Window GUI: Show the main application dialog tabwindow on tool startup Name: show_tabwindow_on_guistart Default: 1 (true)	Show the main application dialog tabwindow on tool startup
Console Window GUI: Initial console height (in percent) Name: initial_console_height Values: 15, ..., 50 Default: 19	Relative height of console at tool startup. This value is a percentage of window space that the console should occupy. The value 0 means do not show the console, which can then be shown using View > Console Window.
Activity Window GUI: Initial activity window width (in percent) Name: initial_activity_window_width Values: 10, ..., 50 Default: 25	Relative width of activity window at tool startup. This value is a percentage of window space that the activity window should occupy. The value 0 means do not show the activity window, which can then be shown using View > Activity Window.

Table 9-2 GUI Preferences (continued)

Preference	Description
Appearance > Schematic > Net	
Lines GUI: Net line color Name: schematic_net_line_color Default: cyan	Preference for net line colors in schematics
Text GUI: Net name text color Name: schematic_net_name_text_color Default: pink	Preference for net name text colors in schematics
Text GUI: Show net name text Name: schematic_show_net_name_text Default: 0 (false)	Preference for showing net name text in schematics
Appearance > Schematic > Pin	
Lines GUI: Pin line color Name: schematic_pin_line_color Default: green	Preference for pin line colors in schematics
Text GUI: Pin name text color Name: schematic_pin_name_text_color Default: pink	Preference for pin name text colors in schematics
Text GUI: Show pin name text Name: schematic_show_pin_name_text Default: 1 (true)	Preference for showing pin name text in schematics
Manual Connection Colors GUI: Logic open color Name: schematic_highlight_logic_open_color Default: light_green(8)	Preference for logic open colors in schematics. Values: yellow(0) orange(1) red(2) green(3) blue(4) purple(5) light_orange(6) light_red(7) light_blue(9) light_purple(10)
Manual Connection Colors GUI: Logic one color Name: schematic_highlight_logic_one_color	Preference for logic one colors in schematics. Values: yellow(0) orange(1) red(2) green(3) blue(4) purple(5)

Table 9-2 GUI Preferences (continued)

Preference	Description
Default: light_blue(9)	light_orange(6) light_red(7) light_blue(9) light_purple(10)
Manual Connection Colors GUI: Logic zero color Name: schematic_highlight_logic_zero_color Default: light_blue(9)	Preference for logic zero colors in schematics. Values: yellow(0) orange(1) red(2) green(3) blue(4) purple(5) light_orange(6) light_red(7) light_blue(9) light_purple(10)
Manual Connection Colors GUI: Logic dc color Name: schematic_highlight_logic_dc_color Default: light_orange(6)	Preference for logic dc colors in schematics. Values: yellow(0) orange(1) red(2) green(3) blue(4) purple(5) light_orange(6) light_red(7) light_blue(9) light_purple(10)
Manual Connection Colors GUI: Multiple connection color Name: schematic_highlight_logic_open_color Default: light_green (8)	Preference for logic open colors in schematics. Values: yellow(0) orange(1) red(2) green(3) blue(4) purple(5) light_orange(6) light_red(7) light_blue(9) light_purple(10)
Appearance > Schematic > Port	
Lines GUI: Port line color Name: schematic_pin_line_color Default: green	Preference for port line colors in schematics.
Text GUI: Port name text color Name: schematic_pin_name_text_color Default: pink	Preference for port name text colors in schematics.
Text GUI: Show port name text Name: schematic_show_pin_name_text Default: 1 (true)	Preference for showing port name text in schematics.
Appearance > Schematic > Cell	
Lines GUI: Cell line color Name: schematic_cell_line_color Default: green	Preference for cell line colors in schematics.
Text GUI: Cell name text color	Preference for cell name text colors in schematics.

Table 9-2 GUI Preferences (continued)

Preference	Description
Name: schematic_cell_name_text_color Default: pink	
Text GUI: Show cell name text Name: schematic_show_cell_name_text Default: 1 (true)	Preference for showing cell name text in schematics.
Hierarchical Colors GUI: Hierarchical cell line color Name: schematic_highlight_hierarchical_cell_line_color Default: orange	Preference for hierarchical cell line colors in schematics. Values: yellow(0) orange(1) red(2) green(3) blue(4) purple(5) light_orange(6) light_red(7) light_blue(9) light_purple(10)
Bus Colors GUI: Hierarchical cell line color Name: schematic_highlight_bus_cell_line_color Default: yellow	Preference for bus cell line colors in schematics. Values: yellow(0) orange(1) red(2) green(3) blue(4) purple(5) light_orange(6) light_red(7) light_blue(9) light_purple(10)
VIP Colors GUI: Vip cell line color Name: schematic_highlight_vip_cell_line_color Default: purple	Preference for VIP cell line colors in schematics. Values: yellow(0) orange(1) red(2) green(3) blue(4) purple(5) light_orange(6) light_red(7) light_blue(9) light_purple(10)
Appearance > Schematic > Other	
Schematic Colors GUI: Connection error color Name: schematic_highlight_connection_error_color Default: red	Preference for connection error colors in schematics. Values: yellow(0) orange(1) red(2) green(3) blue(4) purple(5) light_orange(6) light_red(7) light_blue(9) light_purple(10)
Schematic Colors GUI: Interface unused color Name: schematic_highlight_interface_unused_color Default: light_green	Preference for interface unused colors in schematics. Values: yellow(0) orange(1) red(2) green(3) blue(4) purple(5) light_orange(6) light_red(7) light_blue(9) light_purple(10)
Synopsys IP Updates (see also, “Component Update Checking” on page 45)	
GUI: Enable automatic Synopsys IP update checks Name: allow_automatic_update_checks Default: 1 (true)	Enable automatic update checks for used Synopsys IP. If disabled, you can perform a manual IP update check anytime using the Help > Check For IP Updates menu item.

Table 9-2 GUI Preferences (continued)

Preference	Description
GUI: Synopsys IP update check frequency in days Name: automatic_update_check_frequency Default: -1	The frequency to check for available updates of the tool and the Synopsys IP being used. The default setting of -1 causes coreAssembler to automatically prompt you to set this preference when you open a workspace that contains Synopsys IP.
GUI: Inform Synopsys about opting out of automatic IP update checks Name: inform_optingout_automatic_update_checks Default: 1 (true)	Inform Synopsys about opting out of automatic IP update checks.
Workspace Options (see also, “Setting Workspace Options” on page 48)	
GUI: Include synthesis related activities Name: ws_options_synthesis Default: 1 (true)	Include activities in the activity list which relate to the generation of a gate-level implementation of the component or subsystem.
GUI: Write batch script after each assembly-related activity Name: ws_options_batch Default: 0 (false)	Write a batch script for recreating the subsystem as each activity in the ‘Create RTL’ group completes. The batch script is written to <workspace>/export/batch.tcl.
GUI: Include component specific unit-level test activities Name: ws_options_unittest Default: 1 (true)	Include activities associated with unit-level testing of DesignWare implementation IP. This includes GTECH model generation and unit-level simulation. Note: This option applies only in coreAssembler.
GUI: Show “Memory Map Specification” activity Name: ws_options_memorymapspecification Default: 1 (true)	Shows the Memory Map Specification activity and its associated dialog in the workspace activity flow. Note: This option applies only in coreAssembler.
Document Generation (see also, Documentation Generation on page 266)	
GUI: Generated Documentation Format Name: documentation_format Default: DocBook	Generate reports/documentation in the specified format: ■ DocBook: User configurable reports generated in DocBook XML and then automatically translated to HTML. ■ HTML: Generate non-configurable reports directly in HTML format.
GUI: Compress Documentation Name: docbook_compress_documentation Default: 1 (true)	When set to true, items that are grouped with DocGroup will not be reported individually. For more about DocGroup and DocMaster, see the man pages.

Table 9-2 GUI Preferences (continued)

Preference	Description
GUI: Separate Files Per Parameter Group Name: docbook_file_per_param_group Default: 0 (false)	■ 0 (false): All parameters are written to a single table. ■ 1 (true): Each page from the configuration dialog has a separate table, and each table is written to a different file.
GUI: Separate Files Per Address Block Name: docbook_separate_address_block_files Default: 1 (true)	■ 0 (false): All register and register field information for an address block is written to a single file. ■ 1 (true): Information for each register is written to a file and register field information is written to a separate file.
GUI: Separate Files Per Port Group Name: docbook_separate_file_per_port_group Default: 1 (true)	■ 0 (false): All ports are written to a single file. ■ 1 (true): Each port group is written to a a separate file.
GUI: Port Documentation Attributes Name: docbook_port_attrs Default: PortDirection *DocGeneratelf *DocSynchronousTo *DocRegisteredBy *DocPowerDomain *DocDescriptionField *DocDescription	Indicates which attributes appear in DocBook documentation generated for top-level ports (signals) of a component. Attributes whose names are preceded by '*' appear in a single column in the generated table. Attributes beginning with 'Doc' are replaced with the corresponding non-Doc version if the Doc* version of the attribute is unset. If the DocGeneratelf attribute is included, it is excluded from reports generated for configured designs.
GUI: Parameter Documentation Attributes Name: doc_param_attrs Default: Value *Name *Values *DocDefaultValue *DocEnabled *DocDescriptionField *DocDescription	Indicates which attributes appear in DocBook documentation generated for top-level parameters of a component. Attributes whose names are preceded by '*' appear in a single column in the generated table. Attributes beginning with 'Doc' are replaced with the corresponding non-Doc version if the Doc* version of the attribute is unset. If the Value attribute is included, it is excluded from reports generated for unconfigured designs. Note that the name attribute is included by default because the first column of the parameter tables uses the GUI label instead of the parameter name.
GUI: Register Documentation Attributes Name: docbook_register_attrs Default: DocGlobalAddressOffset *DocShortDescription	Indicates which attributes appear in DocBook documentation generated for registers within a memory map of a component. Attributes whose names are preceded by '*' appear in a single column in the generated table. Attributes beginning with 'Doc' are replaced with the corresponding non-Doc version if the Doc* version of the attribute is unset.
GUI: Register Field Documentation Attributes Name: docbook_field_attra Default: Name DocMemoryAccess *DisplayName *Reserved *DocRegisterResetValue *DocTestable *DocRegisterResetMask *DocVolatileMemory *WriteConstraint *MinWriteConstraint	Indicates which attributes appear in DocBook documentation generated for fields within a register. Attributes whose names are preceded by '*' all appear in a single column in the generated table. Attributes beginning with 'Doc' are replaced with the corresponding non-Doc version if the Doc* version of the attribute is unset.

Table 9-2 GUI Preferences (continued)

Preference	Description
*MaxWriteConstraint *DocDescription *DocDescriptionField *BitRangeVariables *MemoryAccessDescription *ReadActionDescription *WriteBehaviorDescription	Note that the Name attribute is included by default because the first column of the field tables uses the bit range as a descriptor.
Formal Verification	
GUI: Target stage netlist Name: formver_use_target_stage_netlist Values: Use db (false), Use netlist (true) Default: Use db	Sets the default value for formal verification activity parameter UseTargetStageNetlist.
GUI: Reference stage netlist Name: formver_use_reference_stage_netlist Values: Use db (false), Use netlist (true) Default: Use db	Sets the default value for formal verification activity parameter UseReferenceStageNetlist.
Tech Setup	
GUI: Run DC to init variables Name: run_dc_to_init_variables Default: 0 (false)	Forces coreTools to run Design Compiler to determine the values of synthesis setup variables.
Assembly	
GUI: Default Port Name Format (via exported interface) Name: auto_port_name_format Default: %e_%p	The format to be used for generating port names for exported interfaces. Port name format usage is as follows: ■ %c: Component instance name ■ %p: Interface instance port name ■ %e: Exported interface name ■ %i: Interface instance name ■ %d: Component port direction (i, o, b) ■ %D: Component port direction (I, O, B)
GUI: Default Port Name Format (via manual export) Name: manual_port_name_format Default: %c_%p	The format to be used for generating port names for manually exported ports. Port name format usage is as follows: ■ %c: Component instance name ■ %p: Component port name ■ %d: Component port direction (i, o, b) ■ %D: Component port direction (I, O, B)

Table 9-2 GUI Preferences (continued)

Preference	Description
GUI: Default Signal Name Format Name: signal_name_format Default: %c_%p	The format to be used for generating signal names when writing RTL. Signal name format usage is as follows: ■ %c: Component instance name ■ %l: Use load component name instead of driver (single load only) ■ %p: Component port name ■ %d: Component port direction (i, o, b) ■ %D: Component port direction (I, O, B)
GUI: Automatically apply when leaving a dialog or page Name: auto_apply Default: 0 (false)	Controls how the tool handles changes to dialog boxes or pages that have not been applied. ■ 1 (true): Automatically apply changes in the dialog or page ■ 0 (false): Query the user whether to apply or cancel the changes
GUI: Allow Incomplete Subsystem Name: allow_incomplete_subsystem Default: 0 (false)	Controls activity completion and RTL generation for an incomplete subsystem.
GUI: Allow Undriven Pins (see help) Name: allow_undriven_pins Default: 0 (false)	When true, errors related to undriven pins are downgraded to warnings. Note: The resulting netlist will not include these connections because they are effectively undriven, so not including the connections is functionally equivalent to including an undriven wire.
GUI: Remove Unneeded Ports Name: remove_unneeded_ports Default: 0 (false)	When true, ports created as part of the interface-based connection process that are not needed, are removed. Ports are not needed in the following scenario: When an interface is exported out of a lower-level of hierarchy and the connection on one side or the other is missing. This results in exported ports that are either 'open' or connected to a constant (if the interface definition provided a default value). If connected to open, the ports will be removed. If connected to a constant, the port is removed and the constant is connected to any required pins in the lower-level hierarchy.
GUI: Use Merged Glue Logic Source Name: use_merged_glue_logic_source Default: 0 (false)	When true, the generated file, <subsystem_name>_sim.lst, will not reference individual Synopsys glue logic component source. The glue logic will be compiled from the shared glue logic source in <coreTools image root>/auxx/dware/glue.

Table 9-2 GUI Preferences (continued)

Preference	Description
GUI: Retain Multi-Dimension ports when exporting Interface Name: retain_multidim_ports Default: 0 (false)	Retain the Multi-Dimensional ports when ports are exported via interface. The user can set this option using the batch script as follows:gui_set_pref_value -category Globals -key {retain_multidim_ports} -value "true".
GUI: Retain Port range for static ports while exporting Name: retain_static_port_range Default: 0 (false)	Retain the single bit wide vector range if specified. For example, for input [0:0]in1, the range must be retained while in1 is exported.
GUI: Don't Auto-connect Interfaces Name: disable_autoconnect Default: 0 (false)	When true, interfaces will not auto-connect in GUI mode.
Process Execution	
GUI: Run style preference Name: run_style_preference Values: local, lsf, grd Default: local	Controls how to run external jobs. Jobs can be run locally, through LSF, or through GRD.
GUI: Run style options Name: run_style_options Default: N/A	Specifies additional options for a given run style (run_style_preference). ■ For lsf and grd, this specifies options for the bsub and qsub commands.
Synthesis	
GUI: Bus naming style Name: bus_naming_style Default: N/A	Sets the bus_naming_style variable for synthesis.
GUI: Show rise & fall delays separately Name: synth_show_rise_and_fall_delays Default: 0 (false)	■ 1 (true): Separate columns are shown for rise and fall delay constraints. ■ 0 (false): Constraints are shown merged with minimum value shown for minimum delays and the maximum value shown for maximum delays.
GUI: Wait for licenses Name: syn_wait_for_licenses Default: 0 (minutes)	Sets the number of minutes to wait for Design Compiler licenses. Causes Design Compiler to be invoked using: '-checkout Design-Compiler -wait <minutes>'/ In Design Compiler, scripts also wait before giving up when checking out needed licenses, such as Ultra, Test-Compiler, and so on.

Table 9-2 GUI Preferences (continued)

Preference	Description
GUI: Alib library analysis path Name: alib_library_analysis_path Default: ./	Sets the alib_library_analysis_path variable for synthesis. For more information, see the man page in Design Compiler.
GUI: Alib library analysis cache Name: alib_library_analysis_cache Default: ./	Sets the alib_library_analysis_cache variable for synthesis. For more information, see the man page in Design Compiler.
GUI: Include params in top design name Name: include_params_in_top_design_name Default: 1 (true)	■ 1 (true): Parameters on the top-level design cause the design name to be modified as specified in the Design Compiler variable, template_parameter_style. ■ 0 (false): The top-level design name is unmodified by its parameter settings.
Multiple Port Nets GUI: Do not fix multiple port nets Name: fix_multiple_port_nets_default Default: 1 (true)	When true, causes “set_fix_multiple_port_nets -default” to be used in Design Compiler, so no buffers will be inserted to fix multiple port nets.
Multiple Port Nets GUI: Buffer feedthroughs Name: fix_multiple_port_nets_feedthroughs Default: 0 (false)	When true, causes “set_fix_multiple_port_nets -feedthroughs” to be used in Design Compiler.
Multiple Port Nets GUI: Buffer pins driving multiple outputs Name: fix_multiple_port_nets_outputs Default: 0 (false)	When true, causes “set_fix_multiple_port_nets -outputs” to be used in Design Compiler.
Multiple Port Nets GUI: Duplicate logic constants Name: fix_multiple_port_nets_constants Default: 0 (false)	When true, causes “set_fix_multiple_port_nets -constants” to be used in Design Compiler.
IP-XACT	
Output Controls GUI: Schema version for IP-XACT output files Name: xml_schema_version Default: 1685-2014	Indicates SPIRIT IP-XACT schema version against which the files are written. Possible values are as follows: ■ 1.0: Uses SPIRIT 1.0 ■ 1.1: Uses SPIRIT 1.1 ■ 1.2: Uses IP-XACT 1.2

Table 9-2 GUI Preferences (continued)

Preference	Description
	■ 1.4: Uses IP-XACT 1.4 ■ 1.5: Uses IP-XACT 1.5 ■ 1685-2009: Uses IEEE 1685-2009 ■ 1685-2014: Uses IEEE 1685-2014
Output Controls GUI: Include component synthesis constraints Name: include_synthesis_constraints_in_components Default: 0 (false)	When false, synthesis constraints are not included in generated component files.
Output Controls GUI: Include defaulted parameters in design files Name: include_defaulted_params_in_design Default: 0 (false)	■ 1 (true): Parameters that are set to default values are written to the appropriate IP-XACT design file. ■ 0 (false): Only parameters that are set to non-default values are written to the appropriate IP-XACT design file.
Output Controls GUI: Include unused bus interfaces Name: include_unused_interfaces Default: 0 (false)	When true, unused (unconnected) bus interfaces are included in the component definition.
Output Controls GUI: Allow Broadcast Interface Connections Name: allow_broadcast_connections Default: 0 (false)	When true, allow one-to-many interface connections in the IP-XACT design files, where the pins on the one-side are all outputs.
Output Controls GUI: Include fileSet information Name: include_fileset_information Default: 0 (false)	When true, fileSet information is included in the component definition.
Output Controls GUI: Include unused signals in bus interface Name: include_unused_ports_in_interface Default: 0 (false)	When true, unused bus interface signals (signals with no corresponding component signal) are included in the signalMap of the corresponding busInterface element. These appear as signalName elements that have no value.
Output Controls GUI: Write IP-XACT components Name: write_spirit_components Default: 0 (false)	When false, IP-XACT components are not rewritten when writing a hierarchical IP-XACT component or design file. Instead, references to these components are handled through the standard VLNV (vendor, library, name, version) mechanism.

Table 9-2 GUI Preferences (continued)

Preference	Description
Output Controls GUI: Include accessHandle information Name: include_accesshandle_information Default: 1 (true)	When true, accessHandle information is included in the component definition
Output Controls GUI: Use absolute path writing file/fileSet dependencies Name: write_absolute_dependency_path Default: 0 (false)	When true, use an absolute path writing file/fileSet ipxact:dependency.
Output Controls GUI: Include Defines in IP-XACT component Name: include_defines_in_fileset Default: 0 (false)	If the value of this preference parameter is true, the defines that are set are written out in IP-XACT. The defines can be set in the design attribute 'StaticDefineExprs' for coreBuilder, coreConsultant, and coreAssembler. The defines can also be set in the LoadDesigns activity parameter 'CompileDefines' in coreBuilder.
Output Controls GUI: Include undefined macros in IP-XACT Name: include_undefined_macros_in_ipxact Default: 0 (false)	If the value of this preference parameter is true, then undefined macros will also be written out in IP-XACT.
Output Controls GUI: Include Doc attributes Name: include_doc_attrs Default: 0 (false)	The Control variable enables exporting of Documentation attributes in the component XML. The Doc attrs are only exported for Designware IP in coreConsultant and coreAssembler. ■ DocGroup: coreBuilder (addressBlock, register, port, param), coreConsultant (addressBlock, register, port) ■ DocMaster: coreBuilder (addressBlock, register, port, param), coreConsultant (addressBlock, register, port) ■ DocOffset: coreBuilder (register), coreConsultant (register) ■ DocGenerateIf: coreBuilder (port), coreConsultant (N/A) ■ DocRegisterResetValue: coreBuilder (register, registerField), coreConsultant (register, registerField) ■ DocDescriptionField: coreBuilder (register, registerField, port, param), coreConsultant (register, registerField, port, param)

Table 9-2 GUI Preferences (continued)

Preference	Description
Schema Validation GUI: Input errors should be Name: xml_schema_validation Default: 1 (warnings)	Indicates how to deal with XML schema validation errors when reading XML files: ■ 0 (suppressed): All errors are suppressed ■ 1 (warnings): Treat errors as warnings; continue processing ■ 2 (errors): Treat errors as errors; processing stops
Schema Validation GUI: Output errors should be Name: xml_schema_output_validation Default: 0 (suppressed)	Indicates how to deal with XML schema validation errors when writing XML files: ■ 0 (suppressed): Validation is suppressed ■ 1 (warnings): Treat errors as warnings; continue processing ■ 2 (errors): Treat errors as errors; processing stops
Assembly GUI: Enable Automatic Connections for IP-XACT Buss Definitions Name: auto_connect_spirit_definitions Default: 1 (true)	When true, indicates that interfaces whose definition comes from an IP-XACT bus definition (as opposed to a coreBuilder TCL interface definition) can be connected automatically.
Assembly GUI: Use IP-XACT ad-hoc Connection Names as Signal Names Name: use_ipxact_adhoc_names_as_signal_names Default: 0 (false)	When true, indicates that signals defined through ad-hoc connections should be named based on the name specified in the ad-hoc connection element.
Assembly GUI: Treat single pin ad hoc connections as a connection to 'open' Name: ipxact_treat_single_pin_adhoc_as_open Default: 0 (false)	When true, indicates that single pin ad-hoc connections will be treated as a connection to 'open' for both reading and writing IP-XACT design files.
Assembly GUI: Treat unconnected interfaces as unused in IP-XACT designs Name: ipxact_mark_unconnected_interfaces_as_unused Default: 0 (false)	When true, unconnected interfaces on an IP-XACT component will be marked as unused when reading a design file which does not make a connection to the interface.

9.4 Working with Verification IP

A verification IP provider can use the coreBuilder tool to package the verification IP into a coreKit for use with coreConsultant or coreAssembler. The design flow for verification IP is a subset of the steps required for a synthesizable soft core. In particular, the Specify Target Technology and Synthesis activities are irrelevant.

The verification IP can be in the form of an HDL model, C model, timing model, or some other format. The packaging process in the coreBuilder tool requires that the top-level view of the IP block is in VHDL or Verilog format. Therefore, non-HDL verification IP requires a top-level VHDL or Verilog wrapper.

The coreConsultant or coreAssembler design flow for a verification IP block typically consists of the following steps:

1. Install the coreKit.

Use coreConsultant or coreAssembler to install the coreKit.

2. Create a workspace.

3. Execute the Specify Configuration activity.

If the verification IP is configurable through HDL design parameters, the Specify Configuration activity configures the HDL and writes out configuration-specific files to your *workspace/src* directory.

The verification IP can also have configuration options that are not implemented as HDL parameters. In such cases, the IP provider might provide custom coreConsultant activities to configure the IP. Complete any custom configuration activities according to the options specified by the IP provider.

4. Implement the verification IP as needed for your design verification strategy.

9.5 Working with Hard Macros

A hard macro provider can use the coreBuilder tool to package the hard macro into a coreKit for use with coreConsultant or coreAssembler. Hard macros are, by nature, not configurable and not synthesizable. Therefore, the design flow for a hard macro is a subset of the steps required for a synthesizable soft core. In particular, the Specify Target Technology and Synthesis activities are irrelevant.

The packaging process of the coreBuilder tool requires that the top-level view of the IP block is in VHDL or Verilog format. Therefore, a hard macro requires a top-level VHDL or Verilog wrapper.

The coreConsultant or coreAssembler design flow for a hard macro typically consists of the following steps:

1. Install the coreKit.

Use coreConsultant or coreAssembler to install the coreKit.

2. Create a workspace.
3. Execute the Specify Configuration activity.

Although individual hard macros are not configurable, the hard macro provider may allow you to select from a set of implementations of the hard macro. If the selection is implemented through HDL parameters, the Specify Configuration activity configures the HDL and writes out configuration-specific files to your *workspace/src* directory.

4. Execute verification activities.

The hard macro provider can provide a TRE or other custom verification-related activities for the macro. Verification can involve a verification model for the macro. Execute any design-specific verification activities as instructed by the macro provider.

5. Implement the hard macro in your application design.

9.6 Dynamically Adding Files to the HDL File List

The HDL file list is the list of files which define a given component. There are cases where the specific set of files is not known when the component is instantiated. For example, the assembly flow can include a custom step for memory cell compilation. This feature only applies after the component (or subsystem) has been configured. Additionally, the files being added must exist on disk when `get_hdl_file_list` is called or else the files are not in the returned list. The interface for specifying a user-defined file set is by the new `set_hdl_file_list` command:

```
set_hdl_file_list [<list of files>]
```

The `<list of files>` can include special entries similar to those allowed in the coreBuilder Load Designs activity to indicate the language or VHDL library if required. If the `set_hdl_file_list` command is invoked without a file list, the original HDL file list is restored.

9.7 Signal Naming Control in Generated Netlist

When generating the RTL for a subsystem, control over the naming is provided using the following two mechanisms:

1. A new variable in the system set-up file controls default signal naming:

```
set signal_naming_style "%c_%p"
```

This (default) results in a name of the form <component>_<port> (for example, i_ahb_haddr). If the generated signal name is not unique, then the tool reverts to the default naming style to generate a unique name.

2. It is also possible to set the ConnectionName attribute on the driving pin of a connection to force a specific name to be used for the associated signal. The create_connection command and the Complete Connections activity dialog support specification of the signal name for a specific connection. Specifically, the create_connection command supports two new options:

- ❑ -name to specify the name
- ❑ -description to provide a description for the connection

The description is written out along with the signal declaration, and is also written to the IP-XACT design file. The Complete Connections dialog has a check-box labeled "Provide Connection Details." If checked, a dialog asks for signal name and description as each connection is made.

9.8 Replace Component Mapping File Format

Replace Component (replace_component command) and the corresponding replace component dialog are updated to support the entry of mapping information.

```
replace_component -name <name> [-all] <component> [-mapinfo <map info  
or file name>]
```

The mapping information is used to resolve differences between the source and target components. If the mapping information is specified directly or by the new attribute, it must be provided as a TCL list where each list entry corresponds to one of the lines defined in [Table 9-3](#).

Table 9-3 Replace Component Mapping File Format

Format	Description
map component <source component VLVN> <target component VLVN>	This line is used to indicate that all following lines provide mapping information from the specified source component to the specified target component. This line is only required in mapping files which define mapping information for multiple components in the same file. All lines following this, to the next .map component. line are assumed to apply to mapping between the indicated components.
map interface <source instance name> <target instance name>	This line is used to indicate a mapping between interface instance names. The named interfaces might have different abstractions but must be of the same bus definition. Connections to the source interface instance are transferred to the target interface instance.
map parameter <source parameter name> <target parameter name>	This line is used to indicate a mapping between design parameters. Values assigned to the source parameter must be legal on the target parameter. The value of the source parameter are transferred to the target parameter unless there is a .value parameter. line for the target parameter.
map port <source port name> <target port name>	This line is used to indicate a mapping between design ports. Connections to the corresponding source pin are transferred to the corresponding target pin.
missing [interface port parameter] [source target] <name>	This line identifies a difference between the source and target components. The named objects of the specified type are assumed to not exist in the indicated component.
value parameter <target parameter name> <value>	This line indicates that the specified parameter in the target component is assigned the indicated value. The value can be any value legal for that parameter. Parameter expressions are allowed.
connect interface <target instance name> [interface port] <target name>	This line provides a directive for connecting the interface instance specified by <target instance name>. If the corresponding source interface is not connected or there was no corresponding source interface, an attempt is made to connect the indicated interface instance to the component to which the <target name> interface or port is connected.

Prior to replacing a component, the mapping syntax is validated. If the syntax is invalid, the component replacement is terminated. After swapping the components, each map element is processed to deal with the differences between the source and target components. Errors in the mapping data are flagged and ignored. If unresolved differences remain between the components, and the coreAssembler in GUI mode,

then the “Unmatched interface instances and parameters” dialog box is displayed to resolve remaining differences.

9.8.1 Memory Map Propagation

To help you generate a unified memory map for top-level exported interfaces seen as slaves from outside of the subsystem, the Propagate Memory Map item in the schematic menu is available when a valid exported interface is selected. A corresponding command is also available:

```
propagate_memory_map -interface <interface_name> -map <memory_map_name>
```

The generated memory map contains an address block for each address block seen in each memory map visible by tracing interface connections from the exported interface. Register offsets within each address block are unchanged. Base addresses of the blocks themselves are adjusted based on the start address of the slave containing the memory map. It is not legal to propagate a memory map to an exported interface that is already associated with a memory map.

Existing (propagated) memory maps can be removed using Remove Memory Map item in the schematic menu (valid when an exported interface is selected that has previously been associated with a memory map), or using the following command:

```
remove_memory_map -interface <interface name>
```

This removes the associated memory map and the link from the interface to the memory map.

9.9 Dumping Debug Information

The menu entry File > Build Debug Tarfile is used to capture debug information for the Synopsys Support Center. This menu item creates the file <working_dir>/debug.tar.gz. This debug file includes:

- A batch script for recreating the workspace
- Inclusion of the current workspace contents
- Synthesis and simulation log files (if available)
- Environment variable settings
- Inclusion of DESIGNWARE_HOME listing
- Output from the existing .debug_info. command
- The results of an environment check (if available)
- Tool version information
- Indication of whether design source files are encrypted or in “source” form
- A “Notes:” description of the problem

9.10 Documentation Generation

The “report” documentation generation process is based on DocBook instead of HTML. This allows documentation for I/O definition, parameter definition, and memory maps to be generated in HTML, RTF, and MIF formats from the DocBook source. The coreAssembler tool automatically generates HTML reports from this source, and allows users to output to other formats using standard DocBook utilities.

DocBook file generation is enabled by default. To disable this feature, choose the Edit > Preferences > “Documentation Generation” menu item and dialog to control the following (for more about preferences, see [“Editing Tool Setup and User Preferences”](#) on page 244):

Table 9-4 Documentation Generation Tool Preferences

GUI Label	Description
Generated Documentation Format Default: DocBook	If the DocBook format is chosen, documentation is generated in DocBook format and then translated to HTML for coreAssembler reports. Both file types (.html, .xml) appear in the report directory. If the HTML format is chosen, only the old non-configurable HTML-style reports are generated.
Port Documentation Attributes	Default Values: DocPortWidth PortDirection *DocDescription *Registered *SynchronousTo *DocGenerateIf *DocDescriptionField
Parameter Documentation Attributes	Default Values: Value *Name *DefaultValue *Values *DocEnabled *DocDescriptionField *DocDescription
Register Documentation Attributes	Default Values: GlobalAddressOffset RegisterSize MemoryAccess *DocDescription *RegisterResetValue *DocEnabled
Register Field Documentation Attributes	Name MemoryAccess *DisplayName *RegisterResetValue *DocEnabled *DocDescription

The attribute lists control the contents of the DocBook documentation. Attributes appearing without a leading asterisk ‘*’ get their own column in the generated tables. Attributes with ‘*’ are all included in a single “Description” column with appropriate labels, to prevent wide tables.

9.11 Table Driven Data Import/Export

The coreTools support import and export of data in a table based format to enable user flows that are driven from spreadsheets. This provides both an ease-of-use advantage (no customer conversion to coreTools commands required) as well as a performance benefit as table processing is generally faster than command processing. The table read/write commands are available in batch mode and are also available from the File > Table menu.

9.11.1 General Table Formatting

Table files can contain one or more tables. Within a table file, tables can be named or anonymous. Named tables provide two additional capabilities that are not available with anonymous tables:

- Named tables can contain blank lines
- Named tables can be extracted from multi-table files and processed without processing of other named or anonymous tables.

All tables have the following general format: `<table name><header row><data>`, where the `<table name>` is optional. When reading a table file, a table is assumed to begin at the first non-blank row. If that row contains exactly two cells, the first of which has a value of "Table", then it is assumed to be a table name. Otherwise it is assumed to be the row containing the column headers (i.e. the `<header row>`). If there is a table name, the next non-blank line is assumed to be the `<header row>`. The columns which have entries in the `<header row>` define the set of columns for the table. The `<data>` section of the table begins at the next non-blank line following the `<header row>`. The `<data>` section ends at the next blank line beyond its starting point, unless the table is named and the table is being read with the `-blanks` option specified.

All of the table reading commands locate and process tables in a consistent fashion, and share a number of common arguments:

- **<table file>**: The name of the table file to be processed.
- **-tables <table names>**: Indicates the names of the tables to be processed in the given table file. Only tables with the indicated names are processed. If this option is not specified, then all tables in the table file are processed (and must therefore be supported by the same table reading command).
- **-columns <column data>**: Defines static columns that are assumed to exist in all processed tables. The `<column data>` is a list of two-element lists where the first element is a column name and the second element is the value to appear in that column for every table row. If a column is defined in this manner and explicitly exists in the table, then a table reading error occurs.
- **-map <column map>**: Defines column header name mapping. Different tables have different requirements regarding the names of columns in the table. This option can be used to map from the names seen in the table to the names expected by the table reading command. This enables user-defined column names while still providing the table reader with known names. The `<column map>` is a list of two-element lists where the first element is the name of a column in the table and the second element is the name expected by the table reading command.
- **-separator <column separator>**: Defines the character used to separate columns in the table file. If not specified a tab character is assumed.

The table writing commands also share a number of common arguments:

- **<table file>**: The file into which the table is written.
- **-name**: The name for the table file. If specified, a `<table name>` row is included to identify the table.
- **-append**: If specified, the table is appended to the indicated file. Otherwise the file is overwritten.

- **-map** <column map>: Defines column header name mapping. Different tables have different requirements regarding the names of columns in the table. This option can be used to map from the names desired in the table to the names expected by the table writing command. This enables user-defined column names within the written tables. The <column map> is a list of two-element lists where the first element is the name of a column in the table and the second element is the name expected by the table writing command.
- **-separator** <column separator>: Defines the character used to separate columns in the table file. If not specified a tab character is assumed.
- **-columns** <column names>: Indicates a list of columns whose values are expected to be identical across all table rows and are thus excluded from the table. If the table writer determines that the values are not identical, then table writing fails.

9.11.2 **Table Commands**

9.11.2.1 **read_attribute_table**

This command provides basic attribute table read capability:

```
read_attribute_table <table filename> \  
    [-tables <table names>] \  
    [-columns <column name value pairs>] \  
    [-map <column mapped name pairs>] \  
    [-separator <column separator>] \  
    [-create | -update] \  
    [-loops]  
    [-blanks]  
    [-comment_indicator <table comment indicator>]
```

This command is used to set attributes on objects in the coreTools data model. Attribute tables cab have the following columns:

Table 9-5 Attribute Table Columns

Column Name	Optional	Description
Type	No	Indicates the object type
Name	No	Indicates the object name
<attribute name>	Yes	Indicates the name of the attribute to be set. An arbitrary number of these columns is allowed.

This command is used to set the indicated attributes on the indicated objects. Objects are located using the item type and item name. When a table contains objects of multiple types, scoping is handled automatically when switching between object types that have a parent-child relationship. For example 'port' lines appearing after a 'design' line are assumed to indicate ports within that design. Scoping is supported for the following object type transitions:

- design --> port
- memoryMap --> addressBlock
- addressBlock --> register
- addressBlock --> registerArray
- registerArray --> register
- register --> registerField

As each of these transitions occurs, the current context is changed by pushing the prior element onto the context stack. When the reverse transitions occur elements are popped from the stack as appropriate. This scoping means that names specified for items can always be simple local names without context in the name itself.

If neither -create nor -update option is specified, each line is assumed to refer to an object that already exists in the data model. If -create is specified, each object is assumed to not exist, and is created before setting any attributes on it. If -update is specified, a search is performed and the object is created only if it does not already exist.

If the -loops option is specified, each name is scanned for loop syntax. The loop syntax is {<loop var>:<start index>...<end index>}. When seen, the line is replicated for each inclusive index value. Each attribute value is scanned for \$<loop var> and that string is replaced with the loop index value. When a line with loop syntax is used for scoping, all scoped objects are created within each replicated scoping object. For example, [Table 9-6](#) results in two registers being created, each with two fields, F1 and F2.

Table 9-6 Example Creating Two Registers with Two Fields F1 and F2

Type	Name	Size	Description
register	Interrupt{i:0...1}	32	Interrupt \$i
field	F{j:0...1}	16	Field \$j in interrupt \$i

9.11.2.2 write_attribute_table

This command provides basic attribute table write capability:

```
write_attribute_table <table filename> \
    [-name <table name>] \
    [-append] \
    [-separator <column separator>] \
    [-comment_indicator <table comment indicator>]
    [-map <column name map pairs>] \
    [-columns <column name value pairs>] \
```

```
-items <list of items> \
-attributes <list of attributes> \
-defaults
```

This command writes attribute data in table format to the indicated file. The resulting table includes a row for each specified item and a column for each specified attribute, plus columns for the object name and type (unless removed with `-columns`). If there are multiple types of objects in the list to be written, only those attributes that apply to an object type are written for a given object. If the `-defaults` option is not specified, attributes whose value matches their default are not written.

9.11.2.3 read_pin_connection_table

This command is used to read a table of connection commands:

```
read_pin_connection_table <table filename> \
    [-tables <table names>] \
    [-columns <column data>] \
    [-map <column map>] \
    [-separator <column separator>] \
    [-hierarchy] \
    [-nosource]
```

The pin connection table is available for coreAssembler only and is used to define manual connections for pins and ports. Pin connection tables can be used to represent calls to the `create_connection`, `export_pin`, `create_port`, and `set_current_component` commands.

[Table 9-7](#) defines the columns that can appear in the table. Any additional columns appearing in the table are documented as warnings and ignored.

Table 9-7 Columns in Pin Connection Table

Column Name	Optional	Description
Type	No	Indicates the type of connection command defined on the table line. Legal values are 'connect', 'remove', 'export', 'port' and 'component'. These correspond to calls to the 'create_connection', 'remove_connection', 'export_pin', 'create_port' and 'set_current_component' commands, respectively. The default is 'connect' and the cell in this column can be left empty when 'create_connection' is the desired command.
Names (or multiple columns named "Name <other text>")	No	Value depends on the value of the Type cell. Multiple "Name <other text>" columns can be used to enable specifying each pin or port name in a separate column (if desired). The values of all the name columns are merged prior to processing. connect: list of ports and/or pins to be connected export: the name of the port to be created and the name of the pin being exported. If the port name does not include a range specification and the pin name either does not include a range specification or includes a range of more than 1 bit, then the <code>-auto_dim</code> option is passed to <code>export_pin</code> .

Table 9-7 Columns in Pin Connection Table (continued)

Column Name	Optional	Description
		port: name of the port to be created including any desired range specification. component: component context in which all following commands, apply to the next line containing „component. in the Command column.
Constant	Yes	Indicates the constant value to which all ports and pins in the Names column are tied. Valid only when the Type value is 'connect'. Valid values are those defined for the –constant option of the create_connection command.
Signal	Yes	Indicates the name for the wire associated with given connection. Passed unaltered by the –signal option to create_connection and export_pin.
Comment	Yes	Indicates the comment associated with the given connection. Passed unaltered by the –comment option to create_connection and export_pin.
Direction	Yes	Port direction for port creation. Valid (and required) only when the Type value is 'port'. Valid values are 'in', 'out', and 'inout'.
PortType	Yes	Port type for port creation. Optional name of the type for the port to be created. Passed unaltered by the –type option of the create_port command.

If the –hierarchy option is specified, each pin name in a 'connect' or 'export' line is scanned to see if it is hierarchical. If it is, then –hierarchy is passed to the corresponding generated command. If the –nosource option is specified, then the –nosource option is passed to every generated create_connection command.

An example pin connection table is shown below:

Table 9-8 Pin Connection Table Example

Type	Names	Constant	Signal	Comment
Connect	U1/Z U2/Z U3/Z		Zpins	
Connect	U4/Z[7:0]	0xf		Tied off for now
Export	MyPort U5/A			

9.11.2.4 write_pin_connection_table

This command is used to write a table of pin connection commands:

```
write_pin_connection_table <table filename> \
    [-name <table name>] \
    [-append] \
    [-separator <column separator>] \
    [-map <column map>] \
    [-columns <column names>] \
```

`[-recursive]`

Pin connection commands are written to the indicated <table filename>. By default, connection commands are only written for the current context. Specifying the `-recursive` option causes commands to be written for all hierarchies.

9.11.2.5 read_subsystem_table

This command is used to read a table defining a subsystem:

```
read_subsystem_table <table filename> \
    [-tables <table names>] \
    [-columns <column name value pairs>] \
    [-map <column name map pairs>] \
    [-separator <column separator>]
    [-blanks]
    [-comment_indicator <table comment indicator>]
```

A subsystem table can be used to define the contents of a subsystem in coreAssembler. The tables can contain information about components to instantiate, configuration information, and interface and pin-level connection details. [Table 9-9](#) defines the columns that can appear in the table. Any additional columns appearing in the table are documented as warnings and ignored.

Table 9-9 Columns in Subsystem Table

Column Name	Optional	Description
Type	No	Indicates the kind of information in the table line. Each kind maps to a command used to define the subsystem. Legal values are 'instance', 'interface', 'unused', 'connect', and 'configure' which correspond to the <code>instantiate_interface</code> , <code>export_interface</code> , <code>set_unused_interface</code> , <code>connect_interface</code> , and <code>set_configuration_parameter</code> or <code>set_interface_parameter</code> commands, respectively.
VLNV	Yes	Indicates the desired VLVN for a component instantiation.
TargetInstanceName	No	Indicates the desired instance name for a newly created instance or the target instance name for a command.
TargetInterfaceName	Yes	Indicates the desired interface name for a newly exported interface or the target interface name for a command.
SourceInstanceName	Yes	Indicates the source instance name for a command requiring both a source and target instance.
SourceInterfaceName	Yes	Indicates the source interface name for a command requiring both a source and target interface.
ParameterName	Yes	Indicates the name of a parameter to be configured. Either a component parameter or an interface parameter depending on whether or not the <code>TargetInterfaceName</code> column contains a value.

Table 9-9 Columns in Subsystem Table (continued)

Column Name	Optional	Description
ParameterValue	Yes	Indicates the value to be applied to the named parameter.

[Table 9-10](#) indicates which columns are required based on the line type.

Table 9-10 Required Columns Based on Line Type

Line Type	Required Table Columns
instance	VLNV, TargetInstanceName
interface	TargetInterfaceName, SourceInstanceName, SourceInterfaceName
unused	TargetInstanceName, TargetInterfaceName
connect	TargetInstanceName, TargetInterfaceName, SourceInstanceName, SourceInterfaceName
configure	TargetInstanceName, ParameterName, ParameterValue (TargetInterfaceName required if setting an interface parameter)

An example subsystem table is shown in [Table 9-11](#). Note that some of the column names do not match those expected by `read_subsystem_table`. This implies that this table must be read using the `-map` option.

Table 9-11 Subsystem Table Example

Type	VLNV	Instance	Interface	Instance2	Interface2	Parameter	Value
instance	DW_ahb	i_ahb					
instance	Synopsys DesignWareLibrary DW_apb 2.10a	i_apb					
connect		i_ahb	AHB_Slave	i_apb	AHB_Slave		
configure		i_ahb				AHB_LITE	true

This same sparse table can be separated into 3 different tables as shown in [Table 9-12](#). In this example, the tables are named so that the table file can be processed as a single file or by individual calls to the `read_subsystem_table` command to read individual tables specified using the `-tables` option. Note also that within each table the values in the 'Type' column are static. This means that with this multi-table layout, the 'Type' column could be removed if the tables were always read individually and had the `-columns` option specified to indicate the static value for the 'Type' column.

Table 9-12 Multi-table File Example

Table	Instances			
Type	VLNV	Name		
instance	DW_ahb	i_ahb		
instance	Synopsys DesignWareLibrary DW_apb 2.10a	i_apb		
Table	Connections			
Type	Instance	Interface	Instance2	Interface2
connect	i_ahb	AHB_Slave	i_apb	AHB_Slave
Table	Configuration			
Type	Instance	Parameter	Value	
configure	i_ahb	AHB_LITE	true	

9.11.2.6 write_subsystem_table

This command is used to write a table defining a subsystem.

```
write_subsystem_table <table filename> \  
    [-name <table name>] \  
    [-append] \  
    [-separator <column separator>] \  
    [-map <column map>] \  
    [-columns <column names>] \  
    [-types <line types>]
```

The subsystem table is written as indicated by the command options. The -types option can be used to restrict which types of subsystem data are written. Only those line types seen in <line types> are written. See the description of read_subsystem_table for description of the valid line types.

10

Customizing a Technology Library

Under certain circumstances, you can customize your technology library for use with coreAssembler by setting attributes on the library or its individual cells. For example, you can disable the selection of certain library cells or make a certain cell representative for its function.

Although you apply these coreAssembler-specific attributes to the technology library .db file, which is then read into Design Compiler, Design Compiler simply ignores them. Only coreAssembler uses these special attributes (to generate synthesis scripts).

To customize a technology library:

1. Invoke Design Compiler:

```
% dc_shell-t
```

2. Read the technology library .db file:

```
dc_shell-t> read <lib_file_name>.db
```

3. Apply to the library and its cells the attributes listed, respectively, in [“Library Customization Attributes”](#) on page 277 and [“Cell Customization Attributes”](#) on page 278.

Use the following syntax:

```
dc_shell-t> set_attribute lib_name attr_name attr_value
```

```
dc_shell-t> set_attribute cell_name attr_name attr_value
```

For example, to specify a library cell as the representative high drive strength combinational cell, use the following commands:

```
dc_shell-t> set_attribute cell_name representative_combinational true
```

```
dc_shell-t> set_attribute cell_name representative_drive_strength high
```

coreAssembler then selects the specified cell when evaluating the {=select_by_class “combinational” “high”} formula.

4. Write out the library to a new file:

```
% write_lib -format db new_lib_file_name.db
```

5. Perform (or repeat) the coreAssembler Specify Technology Library activity to (re)analyze the modified technology library. Make sure you specify the new library file name.

6. Re-execute any previously executed steps in the design flow.

After you set the library and cell attributes, whenever coreAssembler generates synthesis strategy scripts, it considers the attributes that you have set on the technology library. Once again, these new attributes have no direct affect on Design Compiler.

This chapter contains the following sections:

- [“Library Customization Attributes”](#) on page 277
- [“Cell Customization Attributes”](#) on page 278

10.1 Library Customization Attributes

Set the attributes listed in [Table 10-1](#) to customize your technology library:

Table 10-1 Library Customization Attributes

Attribute	Description
default_clock_uncertainty	Establishes a default clock skew uncertainty for the library: ■ aux_timing_script_during_setup ■ aux_area_script_during_setup ■ aux_timing_script_before_initial_compile ■ aux_area_script_before_initial_compile ■ aux_timing_script_before_incr_compile ■ aux_area_script_before_incr_compile These attributes allow you to execute a technology library customization script during or before a specific phase of an area or timing-driven compile: ■ During technology library analysis. ■ Before the initial compile. ■ Before an incremental compile. The coreAssembler first looks for the script in the current working directory, then in the path or paths specified in your DC search_path variable. The application then copies the script to your user workspace and executes it at the specified time. For example, if you specify aux_timing_script_before_initial_compile on a library, coreAssembler executes the specified script before using that library to execute the initial compile of a timing-driven synthesis.
ignore_design_rules_during_initial_compile	Some libraries use overly restrictive design rules that reduce QoR, especially if those rules are applied during the early phases of optimization. Setting ignore_design_rules_during_initial_compile to true defers the design rule fixing until the later compile phases. This improves QoR for some libraries.
use_fast_delay_calculation	This attribute enables you to specify exactly when to use fast delay calculation for the specified library. The legal values are: ■ Initial – The initial mapping phase only ■ Pre-layout – All phases prior to layout. ■ Never – Will never use fast delay calculation. Using fast delay calculation for early phases of optimization can decrease synthesis runtime without adversely affecting QoR. By default, this attribute disables high timing effort and uses fast delay calculation for the early phases of synthesis.

10.2 Cell Customization Attributes

Set the attributes listed in [Table 10-2](#) to customize the use of individual cells within your technology library:

Table 10-2 Cell Customization Attributes

Attribute	Description
representative_combinational	Overrides the automatic selection of a representative combinational cell when evaluating the {=select_by_class "combinational"} formula. The application instead uses this cell as the representative combinational cell to generate electrical constraints.
representative_sequential	Overrides the automatic selection of a representative sequential cell when evaluating the {=select_by_class "sequential"} formula. The application instead uses this cell as the representative sequential cell to generate electrical constraints.
representative_drive_strength	Specifies a cell as representative for the specified drive strength. Valid values are low, median, or high.
representative_cell	Specifies a cell as representative for its function. For example, setting representative_cell on a nand2 cell, selects this cell during evaluation of {=select_by_function "nand2"}. Setting representative_drive_strength on the same cell (to low, for example), selects the cell during evaluation of {=select_by_function "nand2" "low"}.
dont_use_initial_timing	Prevents Design Compiler from using this cell during the initial compile of timing-driven synthesis.
dont_use_initial_area	Prevents Design Compiler from using this cell during the initial compile of area-driven synthesis.
dont_use_incremental_timing	Prevents Design Compiler from using this cell during incremental compiles of timing-driven synthesis.
dont_use_incremental_area	Prevents Design Compiler from using this cell during incremental compiles of area-driven synthesis.

11

Using IP-XACT Components and Designs

The SPIRIT (Structure for Packaging, Integrating, and Re-using IP within Toolflows) Consortium has developed IP-XACT, an XML-based standard for describing IP and IP packaging, as well as plug-in to IP tools and flows, to enable more efficient and cost-effective integration of IP from multiple sources. This chapter describes the coreTools implementation of the IP-XACT schema.

**Note**

The latest version of IP-XACT supported by the coreTools is IP-XACT version 1.5. This version has been standardized by the IEEE through the P1685 working group. Older versions of the standard are supported for both input and output.

This chapter contains the following sections:

- [“IP-XACT Overview”](#) on page 280
- [“IP-XACT Schema Versions”](#) on page 281
- [“List of Features”](#) on page 282
- [“User Interface”](#) on page 284
- [“Batch Mode Interface”](#) on page 288
- [“Licensing and Keys”](#) on page 291
- [“IP-XACT XML Implementation”](#) on page 292
- [“Glossary of IP-XACT Terminology”](#) on page 293

11.1 IP-XACT Overview

The IP-XACT standard consists of an XML schema which defines a meta-data representation for IP and tool flows, and a set of semantics further restricting how the schema should be utilized. The schema defines a large number of XML elements, but these are key elements that users and tools typically interact with. Definition of these key elements can be found in the “[Glossary of IP-XACT Terminology](#)” on page 293.

The following are some of the IP-XACT terminologies used:

- bus definition
- abstractor
- catalog
- component
- design
- designConfiguration
- generator
- generatorChain

11.2 IP-XACT Schema Versions

coreTools supports reading and writing all publicly released versions of IP-XACT files from 1.0 through 1685-2014. As required by the IP-XACT standard, when reading files, all data is up-converted to the latest version supported (1685-2014) prior to use within the tools. By default IP-XACT files are written using the 1685-2014 version of the schema. If you want the tools to write an older version, go to the IPXACT tab of the Edit > Preferences menu and choose your desired IP-XACT schema output version.

11.3 List of Features

The following IP-XACT features are supported in the coreTools:

11.3.1 IP-XACT Packaging Features

11.3.1.1 Support Use of IP-XACT Bus Definitions in coreBuilder

The Load Interface Definitions activity is modified so that the list of interface definition files can include files containing IP-XACT bus definitions. IP-XACT bus definitions are translated into our interface definitions as is currently done in coreAssembler. These translated interface definitions can then be instantiated during Add Interfaces. The interface names are based on the VLNV of the bus definition as follows:

- V_L_N_V for a symmetric bus definition
- V_L_N_V_master for the master interface of an asymmetric bus definition
- V_L_N_V_slave for the slave interface of an asymmetric bus definition
- V_L_N_V_system_<group> for a system interface with the specified group name

11.3.1.2 Generation of an IP-XACT Component from coreBuilder

Generation of an IP-XACT component from within coreBuilder is enabled. The Build coreKit activity includes a check-box to generate the IP-XACT component.

Components generated by coreBuilder include a fileset for each filegroup in the Bill-of-Materials and parameters are set up to be configurable, unlike the case from coreConsultant where the parameter values are static. The generated IP-XACT component contains vendor extensions for some key functionality not available in IP-XACT, but it is not a complete representation of the coreKit. For example, plug-ins are not represented, conditional I/O is not supported, and parameter checking is not available.

11.3.1.3 Support Partial Import of IP-XACT Components in coreBuilder

In order to facilitate import of information from an existing IP-XACT file, an import command is defined as follows:

```
import_ipxact_data <IP-XACT component file name>
```

This command is valid any time after the Load Designs activity has been completed. It can be used to import the following information:

- **Memory maps** - memory maps in the IP-XACT file are read in and replace all pre-existing memory maps.
- **Vendor extensions** - vendor extensions on ports, model parameters, component parameters, and the component itself are imported. Extensions in the 'snps' namespace are removed and all others are saved so that they can be included in any generated IP-XACT component written by coreBuilder, coreConsultant, or coreAssembler.

11.3.2 coreAssembler

- Import IP-XACT components into coreAssembler. IP-XACT components are installed and connected using the same process as packaged components.
- Generate static hierarchical IP-XACT components from coreAssembler. Create a hierarchical component modeling the subsystem. This provides a crude netlist (connections at interface level only) and indicates exported pins. Also generates a IP-XACT component as a separate file for each of the component in the subsystem to accompany the hierarchical component description.
- Generate a static IP-XACT design from coreAssembler. Create design element showing instantiated components and their connections.
- Translate legacy HDL to IP-XACT. The `translate_netlist` command reads structural RTL and creates IP-XACT files in the specified directory.
- Import a IP-XACT design into coreAssembler. Translate a IP-XACT design element into the corresponding coreAssembler subsystem. Instantiate all components, make defined interface connections, and preserve any unused data for future export.
- Import synthesis constraints from the component description, and also write them out to any component description when writing IP-XACT components.
- Run generators explicitly within coreAssembler. These are typically system level generators used in the validation process but can do almost anything. Currently loaded generators are available through the “Generators” menu.
- IP-XACT Memory Map constructs are recognized and preserved in imported and exported IP-XACT XML files. See the *coreTools Release Notes* to determine how these Memory Map constructs are used within a coreTools application.

11.4 User Interface

All IP-XACT functionality is accessible in both the graphical user interface and batch mode (commands) as described in the following sections.

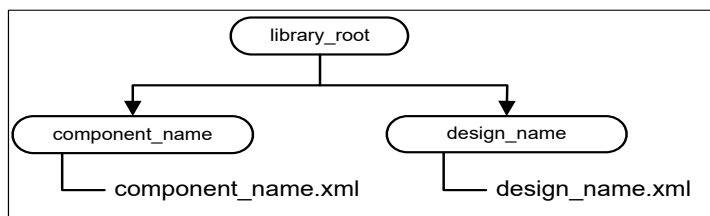
11.4.1 Creating a IP-XACT Library

For ease of use and access, it is best to collect IP-XACT components into a library. To make a component visible for instantiation, set up the following type of directory hierarchy:

```
library_root/component name/component name.xml -- IP-XACT
component description
```

[IP-XACT Library Structure \(on page 284\)](#) shows how this IP-XACT library is structured.

Figure 11-1 IP-XACT Library Structure



Note

The version is contained in the *component_name.xml* file by the *ip:version* tag.

11.4.2 Adding an IP-XACT Library to Your Search Path (coreAssembler)

You must add the IP-XACT *library_root* to your component search path in order to see the IP-XACT components in the coreAssembler component list. To define a IP-XACT search path:

1. Choose the **Edit Search Path...** menu entry while in the schematic window of coreAssembler
2. Enter the path to the *library_root* directory of your IP-XACT library in the “Available Components Search Path” field.
3. OK the dialog box

Using this method, the components show up for instantiation in the Add Component Instance to Subsystem dialog box.

11.4.3 Importing an IP-XACT Component (coreAssembler)

Once an IP-XACT Library structure has been created with the components you want to use, you can use the normal coreAssembler method of choosing and adding components. The ability to import an IP-XACT component into coreAssembler shares the existing interface for adding coreBuilder packaged IP, as follows:

1. Choose **Add New Component** from the popup menu, or click on the **Add Components** icon.
2. When the Add Component Instance to Subsystem dialog box appears, click in the “Component Names” field for each of the components (IP-XACT or other type of component) you want. If you have a large number of components in the view, you can also collapse or expand the view of components in the VLNV (vendor/library/name/version) view.

3. Once you have component instance names provide in the “Component Names” field, either click the **OK** button to add the components.

**Note**

coreAssembler must be able to find any bus definitions referenced in the imported component. See the **Edit Search Path...** menu item.

11.4.4 Importing a IP-XACT Design (coreAssembler)

You can read in an IP-XACT design into an empty workspace using the **File > Read IP-XACT File** menu item or the `read_ipxact_file` command.

To import an IP-XACT component:

1. Choose the **File > Read IP-XACT File** menu item. The IP-XACT XML Import dialog box appears.
2. For “File Type” choose “design”
3. Enter or browse to the XML file name using the “Input file name” field.
4. OK the IP-XACT XML Import dialog box to import the component.

The coreAssembler tool builds the design workspace using the IP-XACT XML file definitions, including library-referenced components.

11.4.5 Exporting an IP-XACT Component (coreAssembler, coreConsultant)

To export an IP-XACT-compliant component:

1. Choose the **File > Write IP-XACT File** menu item. The IP-XACT Export dialog box appears.
2. Click the “Export as: component” choice button.
3. Either enter the component file directory and file name in the “Output file name” field -or- browse to the `<library_root>` directory in the “Output directory name” field.
4. OK the IP-XACT Export dialog box to export the component.

This file is generated automatically as `workspace/export/design.xml` after you perform the Specify Configuration activity in coreConsultant.

11.4.6 Exporting a IP-XACT Design (coreAssembler only)

You can export an IP-XACT-compliant design from coreAssembler as follows:

1. Choose the **File > Write IP-XACT File** menu item. The IP-XACT Export dialog box appears
2. Click the “Export as: design” choice button.
3. Choose “Include components recursively” if you want the individual component instances in your design also exported.
4. Either enter the design file directory and file name in the “Output file name” field -or- browse to the `<library_root>` directory in the “Output directory name” field.
5. OK the IP-XACT Export dialog box to export the design (and possibly included components).

**Note**

coreAssembler automatically writes IP-XACT XML data (components and design) to the `<workspace>/export/SPIRIT` directory after the Generate Subsystem RTL activity, unless this step has been disabled using the **Edit > Workspace Options** menu item. See “[Setting Workspace Options](#)” on page 48.

11.4.7 Legacy HDL Translation to IP-XACT

When transitioning to an IP-XACT driven design flow for an existing (legacy) subsystem, it is desirable to be able to leverage the investment made in manually creating the subsystem netlist. This can be accomplished using the `translate_netlist` command which enables the creation of IP-XACT design and component files from an existing structural netlist. Note that the input netlist must be purely structural. Any behavioral content is lost and result in the creation of an incomplete IP-XACT design file. This is due to the fact that IP-XACT has no mechanism for representing anything but structural RTL code. The `translate_netlist` command reads the structural RTL (defined by the related list of files) and creates IP-XACT files in the output directory.

```
SYNTAX:translate_netlist -top <design_name>
        [-components <component_names>]
        -language verilog | vhdl | systemverilog
        [-xml <search_path>]
        [-out <directory_name>]
        <file_names>
```

11.4.8 Using IP-XACT Generators and Generator Chains

Generators and generators chains are added as menu entries within the Generator top-level menus. These generators are found in IP-XACT components when they are loaded into coreAssembler. Generator chains must be loaded explicitly from a file.

The **File > Read IP-XACT File...** menu item allows you to read a generator chain file into coreAssembler. Reading a IP-XACT generator chain file makes the generator chain available for use through the **Activity list**. All generators referenced by the generator chain must already be loaded into coreAssembler or the reading of the generator chain fails.

11.4.9 The TGI (Tight Generator Interface)

Implementation of the tight generator interface (TGI) allows a generator writer to connect a generator to coreAssembler and then communicate using messages based on the SOAP protocol. The set of valid messages are defined by the TGI API which is defined by the SPIRIT Consortium and supported in IEEE P1685. API calls are available to query everything in the component and design schemas, and to build up and manipulate design files. The introduction of the TGI makes the existing loose generator interface (LGI) in IP-XACT v1.2 obsolete.

The purpose of using the SOAP interface is to allow the generator author to write in any implementation language. However, provided is a sample implementation of a generator using TCL, as part of our tool image. Note that while the generator author is free to write the generator in any language, the client side (coreAssembler) launches the generator through a simple 'exec' process so it is the job of the generator author (or user) to ensure that the generator can actually run on the platform on which the client (coreAssembler) is running, as defined by the IP-XACT semantic rules.

There are potential backward compatibility issues associated with these features:

- Components are translated to the 1.4 schema as they are read in. By default they are written in the 1.4 schema. If you want output in an older schema version, you must specify the output version. Translation to older versions may result in the loss of information.
- Support for the LGI is dropped.
- Support for PMD is dropped; SPIRIT Consortium decided to drop support for it.

11.5 Batch Mode Interface

This section describes the command-line interface for working with IP-XACT-compliant designs, components and generators.

11.5.1 Importing an IP-XACT Component

Instantiation of IP-XACT-compliant XML components is accomplished using the existing `instantiate_component` command, as shown in the following example:

```
instantiate_component -name name component_name
```

11.5.2 Importing IP-XACT Designs or Generator Chains

The `read_ipxact_file` command is used to import an IP-XACT design file or generator chain file:

```
read_ipxact_file -type [design | generatorChain] file_name
```

Parameters

Table 11-1 Parameters Table - 1

Parameter	Description
-type	The top-level element type of the IP-XACT file. Values: design, generatorChain
file_name	The name of the IP-XACT file to be processed.

Description

This command is used to read (import) an IP-XACT format XML file into coreAssembler. The currently supported files types are “design” and “generatorChain.” Reading an IP-XACT design file causes the associated design to be created in the current level of hierarchy, which must be empty when the design is read. Reading an IP-XACT generatorChain file makes the generator chain available for invocation through the Activity list. All generators referenced by the generator chain must already be loaded into coreAssembler or the reading of the generator chain fails.

Examples

Create a new top-level design from myDesign.xml:

```
coreAssembler> create_workspace
```

```
coreAssembler> read_ipxact_file -type design myDesign.xml
```

Read an IP-XACT generator so that it can be used to invoke multiple generators:

```
coreAssembler> read_ipxact_file -type generatorChain myChain.xml
```

11.5.3 Exporting an IP-XACT Component

Components can be exported as IP-XACT components from coreConsultant, coreAssembler and coreBuilder using the new `write_ipxact_component` command:

```
write_ipxact_component [-hierarchy] [-file output file] [-directory directory]
```

Parameters:

Table 11-2 Parameters Table - 2

Parameter	Description
-hierarchy	Include component hierarchy. You cannot use the -file option with this option. Default is no hierarchy.
-file <i>output file</i>	Name of the output XML file. If this option is not used, the file name is design.xml where design is the workspace design name.
-directory <i>directory</i>	Directory for generated files. If this option is not used, the file is written to the current working directory.

Description

Generates an IP-XACT component format (<ipxact:component>) file from the current subsystem. By default only the top-level component is written out to file <design>.xml.

Examples

Generate a simple top-level IP-XACT “component” file:

```
coreAssembler> write_ipxact_component
```

Write the entire subsystem of components into directory H as an IP-XACT component file:

```
coreAssembler> write_ipxact_component -directory H -recursive
```

11.5.4 Exporting an IP-XACT Design (coreAssembler only)

Designs can be exported from coreAssembler using the new write_ipxact_design command:

```
write_ipxact_design [-hierarchy] [-file output_file] [-directory directory]
```

11.5.4.1 Parameters

- hierarchy

Include component hierarchy, and all components of the design are written. You cannot use the -file option when using this parameter.
- file *output file*

Name of the output XML file. If not specified, the file is named *design.xml* where *design* is the workspace design name.
- directory *directory*

Directory for generated files. If not specified, the file is written to the current working directory.

11.5.4.2 Examples

Generate a simple top-level design file:

```
coreAssembler> write_ipxact_design
```

Write the entire subsystem (design + components) into directory H:

```
coreAssembler> write_ipxact_design -directory H -recursive
```

11.5.5 IP-XACT Generators

The following command executes an IP-XACT generator:

```
invoke_generator generatorName
```

For more information on all batch-mode commands, see the specific MAN page in coreAssembler.

11.6 Licensing and Keys

All IP-XACT functionality requires the coreAssembler key. If coreAssembler is currently running off of the DesignWare key, an attempt is made to use a coreAssembler key. If none is available, the IP-XACT operation is not performed.

11.7 IP-XACT XML Implementation

The IP-XACT XML syntax that is used by or generated from coreTools falls into three categories.

- Supported constructs that conform to the SPIRIT Consortium (v1.4 and earlier) or the IEEE P1685 (1685-2009 and later) versions of IP-XACT.
- Synopsys constructs
- Standard IP-XACT constructs that are not supported in coreTools

For references, you can assume that coreTools supports all IP-XACT standard constructs unless otherwise specified. These constructs are not listed in this document. Details about the IP-XACT schema are available from the IEEE and can also be downloaded for free (compliments of Accellera) at:

<http://www.accellera.org/downloads/ieee>

11.7.1 Synopsys IP-XACT Constructs

All Synopsys extensions to the IP-XACT schema are placed inside IP-XACT vendor extension areas so that the generated components are 100% compliant with the IP-XACT schema.

11.7.2 Unsupported IP-XACT Constructs in coreTools

Not all data in the component or design file is supported by coreAssembler. See the *coreTools Release Notes* “Limitations” section for more information.

11.7.3 IP-XACT Examples

For IP-XACT examples, see the Accellera website at <http://www.accellera.org>.

11.8 Glossary of IP-XACT Terminology

abstraction definition	Defines the ports and properties of a particular abstraction (for example, RTL or TLM) of a given bus definition
abstractor	A specialized form of a component which is used to provide the details about how to translate between two interfaces when the interfaces are at different levels of abstraction
Accellera	Group of member companies with a mission to drive worldwide development of standards, which enhance a language-based design automation process. In March of 2010, Accellera and The SPIRIT Consortium completed a merger. The union improves the development of language-based and IP standards. See http://www.accellera.org/tech/ .
bus definition	Defines the properties of a bus that are consistent across all abstractions of the bus interface.
component	Defines a single IP. Contains information about I/O, register and address map requirements, bus interfaces (instances of bus definitions), available views, and so on. Components are elements that can be instantiated in a subsystem by interacting with a design environment. Components may be hierarchical.
design	Defines a subsystem. Contains information indicating how a set of components is to be connected and configured. Connections are made at the interface level. Designs can be converted into hierarchical components and then instantiated in a higher-level design.
generator	Generators define tools that are to be run to transform the design in some way. Typical examples include configuration, RTL generation, and build and running a simulation model. The generator element describes when and where the tools should be run and defines how to invoke the tools, but the actual tools are completely independent of the generator.
LGI	Loose generator interface (LGI) files -- These files are created in the first step of executing a SPIRIT generator. LGI is not supported in the v1.4 schema and has been replaced by TGI methodology.
library	A directory that contains SPIRIT-compliant components or designs in a format that can be used
IP-XACT	An XML-based standards for describing IP and IP packaging, as well as plug-in to IP tools and flows, to enable more efficient and cost-effective integration of IP from multiple sources.
SPIRIT Consortium	See Accellera. A former group of EDA vendors with the common interest to develop IP-XACT standards for describing portable IP.
TGI	Tight generator interface (TGI) files -- These files are used by applications to perform various activities on IP-XACT components and interfaces.
vendor	A provider of IP-XACT-compliant components and/or designs.
XML	Extensible Markup Language -- derived from Standard Generalized Markup Language (SGML). See http://www.w3.org/xml/ .